

SwiftUI

— ARCHITECTURE —



Patterns and Practices for
Building Scalable Applications

by Mohammad Azam

azamsharp.school

About the author	9
Introduction	10
Who This Book Is For.....	12
MVVM and the Cost of Carrying Old Patterns Forward	14
MVVM in SwiftUI	15
Apple's Intentional Design for SwiftUI.....	19
But What About Testing	20
Conclusion	21
CONTAINER/PRESENTER PATTERN	22
Implementation	22
View Communication Using @Binding.....	29
View Communication Using Closures	30
Grouping Enum Events	32
Conclusion.....	33
MV PATTERN.....	34
Understanding the View in SwiftUI	35
Observable Stores and Stateless Services.....	38
Large Scale Apps.....	42
Conclusion.....	47
COMMUNICATION PATTERNS IN OBSERVABLE STORES	49
Handling Store Events in the View	50
Handling Store Events Through Delegates	51
Handling Store Events Using Combine Publishers	53
Handling Store Events Using AsyncStream	56
withObservationTracking	59
Conclusion.....	62
DEEP Dive Into Environment	64
Observable Object Protocol	64

Re-Evaluation (Diffing) vs Re-Rendering	67
Environment Object and View Rendering	68
Environment Object and Navigation Stack.....	72
New @Observable Macro.....	73
Environment Usages in Real World Apps	75
Conclusion.....	78
SWIFTDATA ARCHITECTURE	79
Architecture	79
Business Rules	81
Should we create separate layer for business logic?	84
What if I want to use a different persistent storage in the future?	84
Queries	86
Previews	88
Handling JSON Responses	91
CloudKit	95
Conclusion.....	98
Global Sheets and Toasts	100
Basic Sheets.....	100
Global Sheets	104
Toasts	106
Conclusion	109
Validation	111
Simple Validation	111
Validation Summary.....	113
Validation Using Property Wrappers	114
Implementing Validate Property Wrapper.....	119
Conclusion	122
Navigation	123

Basic List Navigation	123
Decouple Navigation from Views	124
Dynamic/Programmatic Navigation in SwiftUI.....	125
Implementing Navigation Hook Using Environment Values	129
Multi-Tab Navigation in SwiftUI Using Multiple Arrays	131
Multi-Tab Navigation in SwiftUI Using NavigationPath and Navigate Hook	133
Pop to Root.....	135
Conclusion	136
UIKIT TO SWIFTUI.....	137
Why Migrate?.....	137
When SwiftUI May Not Be the Right Move Yet.....	138
Migration Strategy	139
Embed SwiftUI in UIKit	140
Loading UIKit Views into a SwiftUI App	147
Embedding UIKit Controllers in SwiftUI	149
Conclusion	152
Composing and Customizing Views	153
Composable List Cell Views.....	153
Generic List Row	156
View Extensions and Modifiers.....	162
Button Styles	166
Formatting	168
Conclusion	174
TESTING	176
Testing Pyramid	176
Not all tests are created equal	179
Test Behavior not implementation details.....	180
SwiftUI Previews Are Like Tests	182
Testing Presentation Logic	183

Testing SwiftData Models.....	190
Integration Testing	192
End-to-End Testing.....	194
View Model Tests Does Not Test the UI.....	200
Writing Tests Using AI	202
Conclusion.....	203
PERFORMANCE AND DEBUGGING.....	204
View Identity.....	204
Using Self._printChanges() to Understand SwiftUI Updates.....	207
Viewing the SwiftUI View Hierarchy	208
View Composition.....	210
@FetchRequest and @Query	212
Image Caching.....	214
Request Caching.....	217
Keep the Main Actor Free of Intensive Work	219
ObservableObject to @Observable	222
Conclusion.....	225
Profiling SwiftUI	227
Network Profiling	227
SwiftUI View Rendering	234
Hangs	238
Conclusion.....	243
FIREBASE AND SUPABASE.....	245
Firebase APIs in the View	245
Abstracting Firebase Through Stores	247
Running the Emulator	254
Firebase and Previews.....	256
Conclusion.....	265

Architecture in the Age of AI267

 AI As An Abstraction Layer.....267

 Vibe Coding268

 Enterprise Applications.....269

 Architecture Matters More in the Age of AI269

 Conclusion.....270

CONCLUSION272

To my parents, who still call me to change their lightbulbs.

To my beautiful wife, who has always supported my hard work and dedication.

And to my two monkeys, who are the reason this book took a little longer than planned. ❤️

ABOUT THE AUTHOR

Mohammad Azam is a software developer, educator, and independent app creator with over a decade of experience building real world applications. He is best known for his work in the Apple ecosystem, where he focuses on Swift, SwiftUI, and modern app architecture. Over the years, he has helped thousands of developers learn iOS development through workshops, courses, conference talks, and online content.

Azam is the founder of [AzamSharp School](#), a learning platform dedicated to practical, experience driven education for developers. His teaching style is direct, pragmatic, and grounded in real projects, not theoretical diagrams or framework worship. He believes good architecture should make your code easier to reason about, not harder to write.

Through building and shipping multiple production apps, Azam has developed a strong point of view on SwiftUI architecture. This book reflects that experience. It is not about chasing patterns for their own sake, but about understanding how SwiftUI wants you to think and designing systems that evolve naturally with the framework.

When he is not coding or teaching, Azam enjoys speaking at conferences, experimenting with new technologies, and sharing lessons learned from both successes and mistakes. His goal is simple: help developers build better software with more confidence and less frustration.

INTRODUCTION

Architecture has always been a hotly debated topic. Developers tend to get more fired up when discussing software architecture than almost any other aspect of building software. This becomes especially true when someone introduces a completely different way of building apps. At that point, the discussion stops being technical and starts feeling personal, because it challenges ideas people have spent countless hours learning and defending.

When I started writing about SwiftUI architecture in 2019, like many others, I followed the classical MVVM pattern. It was familiar, widely accepted, and felt like the safe choice. But between 2021 and 2022, as I started experimenting with different approaches and openly talking about alternatives, things changed quickly.

The reaction was intense.

I received countless messages telling me to stop publishing those articles. Some developers claimed the ideas were completely wrong. Others accused me of confusing or misleading junior developers. I even received direct messages from people I had never met, asking me to take the articles down from my personal website. And to be honest, this still happens to this day.

Instead of discouraging me, it did the opposite.

It pushed me to dig deeper.

So I published more.

I started examining SwiftUI from multiple angles, including validation, navigation, testing, and building global components. I realized that if I wanted simpler solutions, I needed to look beyond SwiftUI itself and study mature frameworks that had already solved many of these problems. That led me back to WPF, ASP.NET, React, and Flutter. Fortunately, I had hands-on experience with all of them, which allowed me to approach SwiftUI with a broader perspective.

Many of the ideas in this book are inspired by those frameworks. That is why you will see frequent references to React, WPF, and ASP.NET throughout the chapters. They shaped how I think about architecture, separation of concerns, and long-term maintainability.

Around 2024, roughly three years after I started publishing these ideas, something interesting happened. I began to see more and more developers independently arriving at similar conclusions and adopting comparable architectural approaches. That was incredibly rewarding. It felt like the work had finally come full circle.

This book has been in the making for the last three to four years, and it is one of the most ambitious projects I have taken on. It represents my understanding of SwiftUI and my belief that SwiftUI can be used effectively without unnecessary layers or accidental complexity.

I also strongly believe that software architecture is always about tradeoffs. There is no single perfect solution. That is why this book does not present one rigid approach. Instead, it explores multiple techniques, explains when they work well, and just as importantly, highlights the side effects and costs that come with each decision.

Thank you for purchasing this book. I truly hope you enjoy reading it as much as I enjoyed writing it.

WHO THIS BOOK IS FOR

This book is written for developers who are building real SwiftUI applications and want to make better architectural decisions without overcomplicating their code. In other words, it's for people who care about getting things right without turning their codebase into a science experiment.

If you have already spent some time with SwiftUI, you have likely felt some tension. Things that worked well in UIKit or other frameworks do not always translate cleanly. Patterns that once felt safe start to feel heavy. You may have tried forcing MVVM everywhere, adding layers just to make things testable, or reshaping your app to fit a pattern instead of letting the framework guide you. I've done all of that myself. If any of this sounds familiar, you're exactly who this book is written for.

This book is for iOS developers who:

- Are comfortable with Swift and SwiftUI basics
- Are building production apps, not just sample projects
- Want architecture that scales without becoming rigid
- Care about clarity, maintainability, and long-term change
- Prefer understanding tradeoffs over following rules

It is especially useful if you have asked questions like:

- Where should this logic actually live in a SwiftUI app?
- Do I really need a view model for this screen?
- How do I handle navigation without everything falling apart?
- What is actually worth testing, and what is not?
- How can I structure my app without fighting SwiftUI?

This book is not about memorizing patterns or chasing the “correct” architecture. Instead, it focuses on developing a mental model for how SwiftUI actually works and how to design your app around that reality. The goal is to help you write code that is easier to reason about today and easier to change tomorrow.

This book is not aimed at absolute beginners, and that is intentional. There are already great resources that teach SwiftUI fundamentals like views, state, bindings, and basic navigation. This book starts where those usually stop. Once you are comfortable with the basics, it helps you think more deeply about structure, responsibility, and long-term design.

Finally, this book is for developers who value pragmatism over dogma. There is no single architecture that fits every app. Throughout the book, you will see different approaches, along with explanations of when they make sense and when they do not. You are encouraged to adapt these ideas, not blindly copy them.

If you finish this book with a clearer understanding of SwiftUI, more confidence in your architectural decisions, and fewer unnecessary abstractions in your codebase, then it has done its job.

MVVM AND THE COST OF CARRYING OLD PATTERNS FORWARD

I first came across the MVVM pattern around 2008 or 2009 while working for a very large oil and gas company. At the time, we were building a WPF application used by oil workers on offshore rigs to order parts and equipment. Our team had six to eight developers, and the application was deployed across most of the rigs owned by the company. This was not a small internal tool. It was a real production system that people depended on every single day, often in harsh and unpredictable environments where reliability was not optional.

A few years later, I joined another company, this time in the computer retail business. It is one of the top five hardware providers in the world. Their project was an ASP.NET web application built using MVC, but it also borrowed several ideas from MVVM. I worked on some of the core parts of that system and gained a broader perspective on how MVVM could exist outside the desktop world.

Beyond those two projects, I worked on several other systems that used MVVM in different forms. I genuinely enjoyed working with the pattern, especially when it was paired with WPF. The combination of XAML views and C# view models felt natural. It encouraged cleaner code, clearer responsibilities, and systems that were easier to understand and maintain over time.

MVVM was popularized and formally named by John Gossman, a Microsoft architect working on WPF. However, the ideas behind MVVM existed even earlier. A closely related pattern called Presentation Model was described by Martin Fowler in a 2004 article.

In that article, Fowler talked about manually synchronizing view properties with a presentation model and then keeping everything in sync as the application evolved. He also shared his frustration with that approach and wrote:

“Probably the most annoying part of Presentation Model is the synchronization between Presentation Model and view. It's simple code to write, but I always like to minimize this kind of boring repetitive code. Ideally some kind of framework could handle this, which I'm hoping will happen some day with technologies like .NET's data binding.”

Fowler turned out to be right. In 2006, Microsoft released Windows Presentation Foundation, which introduced powerful data binding features between XAML views and their associated C# view models. These capabilities removed much of the repetitive synchronization code developers previously had to write by hand. For the first time, MVVM felt natural instead of forced.

Years later, in 2013, React was announced and quickly changed how many developers thought about building user interfaces on the web. Today, it is consistently ranked as one of the most loved frameworks by web developers. That was not always the case. Early on, many developers were

uncomfortable with React because it mixed HTML, CSS, and JavaScript in the same file. To them, this felt like a violation of separation of concerns.

Over time, the community began to see things differently. React still respected separation of concerns, but it organized code around behavior instead of file types. Everything related to login lived in a Login component. Everything related to photos lived in a Photos component. This shift took time, but once it clicked, it changed how many developers approached application architecture.

One of the most common patterns used in React applications is the Container and Presenter pattern, which we will discuss later in this book. While MVVM can technically be applied in React, it is rarely used in practice. After talking with many React developers over the years, I have yet to meet one who actively uses MVVM in a React codebase. The reason is simple. React components already manage state and binding directly, which makes a separate ViewModel layer unnecessary in most cases.

So why spend so much time talking about WPF and React in a chapter about MVVM and SwiftUI?

Because context matters. More importantly, understanding why a pattern exists and what problem it was designed to solve matters even more.

In the next section, I will walk through my early attempts to apply MVVM in SwiftUI and explain why it often felt like I was fighting the framework instead of working with it.

MVVM IN SWIFTUI

SwiftUI was announced at WWDC 2019, and my first attempt to apply MVVM started on day one. I honestly do not remember why I chose MVVM at the time. Most likely it was because everyone else was doing it, and I followed along without questioning it. For the next three years, I primarily used MVVM with SwiftUI.

My process was simple. For each screen, I created a separate view model. If the view model had dependencies, I passed them through the initializer. If I was building a movies app with screens like `MovieListScreen`, `AddMovieScreen`, and `MovieDetailScreen`, I would end up with `MovieListViewModel`, `AddMovieViewModel`, and `MovieDetailViewModel`.

The term **screen** is important here. A screen is technically a SwiftUI view, but it is used as a container rather than something meant for reuse. If you come from a web background, you can think of a screen as a page. Smaller sections inside that page are views. Screens are not meant to be reusable, but views inside them can be, if they are designed correctly.

You could argue that not every screen needs a view model. But if you are following classical MVVM, the relationship between a screen and its view model is typically one to one. While it is possible to share a view model across multiple screens, most of the time you end up creating a separate one per screen. The reason is that the view model is expected to host all presentation logic for that screen.

Going back to our movies example, let us look at `MovieListViewModel`, which is used by `MovieListScreen` to populate the list of movies.

```
@Observable
class MovieListViewModel {

    var movies: [Movie] = []
    let httpClient: HTTPClient

    init(httpClient: HTTPClient) {
        self.httpClient = httpClient
    }

    func loadMovies() async throws {
        movies = try await httpClient.loadMovies()
    }
}
```

This is a fairly standard view model implementation. `MovieListScreen` receives `MovieListViewModel` through initializer injection. Since `MovieListViewModel` depends on `HTTPClient`, that dependency also needs to be created.

```
struct MovieListScreen: View {

    let vm: MovieListViewModel

    var body: some View {
        List(vm.movies) { movie in
            Text(movie.name)
        }.task {
            do {
                try await vm.loadMovies()
            } catch {
                print(error.localizedDescription)
            }
        }
    }
}

#Preview {
    MovieListScreen(vm: MovieListViewModel(httpClient: HTTPClient()))
}
```

`MovieListScreen` depends on `MovieListViewModel`, and `MovieListViewModel` depends on `HTTPClient`.

If you plan to mock `HTTPClient`, introducing a protocol is a good idea.

At this point, everything works. The list loads correctly. You can add search, sorting, or any other presentation logic to `MovieListViewModel`.

Now the app needs a way to add new movies. To support this, we introduce `AddMovieScreen` and a new view model called `AddMovieViewModel`. `AddMovieViewModel` is responsible for validating user

input and sending a request to create a movie. Once the movie is created, it needs to communicate back to MovieListScreen so the list can update. This communication can be handled using closures, bindings, or other mechanisms.

Below you can find the implementation of the AddMovieViewModel:

```
@Observable
class AddMovieViewModel {

    let httpClient: HTTPClient

    var name: String = ""
    var description: String = ""

    init(httpClient: HTTPClient) {
        self.httpClient = httpClient
    }

    var isValid: Bool {
        !name.isEmptyOrWhitespace && !description.isEmptyOrWhitespace
    }

    func saveMovie() async throws -> Movie? {
        let movie = Movie(name: name, description: description)
        return try await httpClient.createMovie(movie)
    }
}
```

AddMovieViewModel also depends on HTTPClient because it performs a POST request. Once the server creates the movie, it returns a new instance with an id, which is then passed back to the caller.

```
struct AddMovieScreen: View {

    @Bindable var vm: AddMovieViewModel
    let onSave: (Movie) -> Void
    @Environment(\.dismiss) private var dismiss

    var body: some View {
        Form {
            TextField("Title", text: $vm.name)
            TextField("Description", text: $vm.description)
        }
        .toolbar {
            ToolbarItem(placement: .topBarTrailing) {
                Button("Done") {
                    Task {
                        if let movie = try await vm.saveMovie() {
                            onSave(movie)
                            dismiss()
                        }
                    }
                }.disabled(!vm.isValid)
            }
        }
    }
}
```

```

    }
}

```

AddMovieScreen binds directly to AddMovieViewModel using the Bindable property wrapper. This allows the view to connect directly to the view model's state.

Finally, MovieListScreen presents AddMovieScreen and wires up all the dependencies.

```

struct MovieListScreen: View {

    let vm: MovieListViewModel
    @State private var isPresented: Bool = false

    var body: some View {
        List(vm.movies) { movie in
            Text(movie.name)
        }.task {
            do {
                try await vm.loadMovies()
            } catch {
                print(error.localizedDescription)
            }
        }.toolbar {
            ToolbarItem(placement: .topBarTrailing) {
                Button("Add Movie") {
                    isPresented = true
                }
            }
        }.sheet(isPresented: $isPresented) {
            NavigationStack {
                AddMovieScreen(vm: AddMovieViewModel(httpClient:
vm.httpClient)) { movie in
                    vm.movies.append(movie)
                }
            }
        }
    }
}

```

This works. But this is the moment where you need to step back and ask an important question.

Right now, the app has three to five screens, and there is already a noticeable amount of code dedicated to view models, dependency passing, and coordination between screens. What happens when the app grows to thirty or fifty screens? Does that mean thirty or fifty view models? And if so, how do those view models communicate with each other in a way that remains understandable?

I can tell you from experience that this quickly becomes difficult to manage. After a while, you lose track of which view model is the actual source of truth. The code becomes harder to reason about, and small changes start to have unexpected side effects.

I have often seen developers welcome complexity instead of actively trying to remove it. Maybe it is because complexity makes software development feel more valuable. Maybe it creates a false sense of control. Whatever the reason, complexity always comes with a cost.

To work around these issues, I have seen developers try to access environment objects inside view models so they can share global state. But the environment is not designed for that. It is only available inside views. Passing environment objects into view models just brings you back to the same problem you were trying to solve.

At this point, you might wonder whether view models themselves should be injected into the environment. Technically, this works. View models are observable, so they can live in the environment. But this goes directly against the core idea of MVVM, where a view model is tied to a specific screen.

Yes, there are exceptions. Authorization state is a good example. But most screens have very different needs. You would not reuse your LoginViewModel, which contains all the presentation logic for a login screen, inside a registration screen. The responsibilities are different.

These issues do not exist because SwiftUI is limited. They exist because of how we try to use it. We bring our past experience with us and force the framework to fit those expectations. I did exactly that. I posted countless questions on Stack Overflow about views not updating even though the data had changed. Instead of understanding how SwiftUI was designed to work, I added refresh flags and toggles to force updates.

Those were not solutions. They were workarounds. It is like owning a Ferrari and insisting it should be pulled by donkeys. Let the Ferrari be fast. Learn how to drive it.

APPLE'S INTENTIONAL DESIGN FOR SWIFTUI

Apple has been investing heavily in SwiftUI since its original release in 2019. This includes new views, property wrappers, macros, and even new frameworks like SwiftData, which are built primarily for SwiftUI. When we step back and look at all these additions, a pattern starts to emerge. Most of these features only work inside a SwiftUI view.

We already know that `@State`, `@Binding`, `@Environment`, and `@Bindable` are limited to views. But the same is true for other property wrappers such as `@AppStorage`, `@FocusState`, `@FetchRequest`, `@SectionedFetchRequest`, and `@Query`.

For Core Data and SwiftData, you can often find ways to move these into classes if you really want to. But at that point, you are no longer using the platform as it was designed. You are working around it.

That raises an important question. Why did Apple make these decisions? Why not allow these features to be used anywhere?

The answer becomes clearer when you look at SwiftUI as a whole. SwiftUI is opinionated. It expects state, identity, and lifecycle to live close to the view. These property wrappers are not just storage. They

are hooks into the framework's rendering, invalidation, and update system. That only works when SwiftUI fully controls the context in which they run.

SwiftData is a good example of this thinking. The `@Query` property wrapper is only available inside a view. Yes, you can use a `FetchDescriptor` in a class and return the results to a view, but once you do that, you lose automatic tracking. You are then responsible for keeping the user interface in sync with the data yourself.

Both `@Query` and `@FetchRequest` are highly optimized for SwiftData and Core Data. They are designed to keep data and UI aligned with minimal effort. Once you move that logic outside the view, you step off the fast path.

There was a [talk](#) at NSSpain by developers from SoundCloud where they shared a similar lesson. They had built their app using VIPER along with an internal framework. The app was rejected by Apple because it was significantly slower than competing apps. After removing their custom layers and adopting SwiftUI property wrappers directly, performance improved dramatically.

Once they started using SwiftUI the way it was designed to be used, the performance problems disappeared.

So what is Apple's recommended approach?

Based on their code samples, documentation, WWDC sessions, and conversations with Apple engineers, a clear pattern emerges. **SwiftUI views are expected to host presentation logic. Observable objects and stateless services are expected to host business logic.** The framework handles the rest.

That simplicity is not accidental. It is the design.

BUT WHAT ABOUT TESTING

One of the biggest arguments for using the MVVM pattern in SwiftUI is testing. Instead of letting the view host presentation logic, we move that logic into a view model so it can be tested. On the surface, this sounds reasonable.

Unlike React and Flutter, Apple does not provide an easy way to test logic that lives inside a view. In React, you can mount a view and directly test the logic it contains. Flutter supports widget testing as well. SwiftUI, however, does not offer any out of the box solution for testing view logic.

There are third party tools such as ViewInspector that make this possible, but they come at a cost. You are adding more complexity and another dependency to your application, all for the purpose of testing something that SwiftUI was never designed to expose directly.

So how do we test the logic inside a view? To answer that, we first need to ask what kind of logic actually belongs in a SwiftUI view. As mentioned earlier and as Apple has shown many times, views are meant to host presentation logic. That means the discussion is really about testing presentation logic.

Whether presentation logic should be tested depends on the kind of behavior the view is responsible for. In many cases, this behavior can be verified using Xcode previews. Apple has even suggested in WWDC sessions that previews should be treated as tests. Previews allow you to quickly validate different states and edge cases without writing a single line of test code.

If the logic inside a view starts to grow or becomes difficult to reason about, it should be extracted. This does not automatically mean introducing a view model. Often, the logic can live in a simple struct, function, or extension and be tested in isolation. A common example is form validation. Validation rules can be moved into a String extension and tested without any dependency on SwiftUI.

But what about application flow? How do we test navigation, presenting sheets, or making sure the right view is visible at the right time? The most reliable way to verify this type of behavior is through end to end or UI testing. This is the only approach that truly confirms the application behaves correctly from the user's perspective.

A common criticism of end to end tests is that they are slow. To a degree, this is true. UI tests are slow because they exercise the application the same way a real user does. They tap buttons, navigate screens, and trigger real side effects. That work takes time. The key is to focus these tests on user stories rather than individual interface elements. As the flow is tested, the correctness of the underlying UI is validated naturally along the way.

We will explore testing strategies in much more detail in the **Testing in SwiftUI** chapter later in this book.

CONCLUSION

SwiftUI applications can be written in many different ways. As a software developer, your primary responsibility is to reduce complexity. The architecture or pattern that helps you achieve simplicity, clarity, and long term maintainability is the one that makes the most sense.

If you believe you can achieve those goals using the traditional MVVM pattern in SwiftUI, then there is no reason to stop using it.

However, if creating separate view models for each screen, moving presentation logic into those view models, and injecting view models into views while also injecting the networking layer as a dependency into the view model starts to feel like added complexity, then it may be time to look at a different approach.

In the next chapter, we will cover the container and presenter pattern, which is the default pattern used when building React applications.

CONTAINER/PRESENTER PATTERN

In React, one of the most common architectural approaches is the Container Presenter pattern. The core idea is simple. A container component is responsible for data. It fetches it from an API, prepares it, and manages state. Presenter components, on the other hand, focus entirely on rendering the user interface.

This separation makes the data flow easy to follow. Data moves in one direction from parent to child, and presentation logic stays clean and independent from data management. Over time, this clarity pays off, especially as applications grow.

Since React and SwiftUI share many conceptual similarities, it is natural to ask whether this same pattern can be applied when building SwiftUI applications.

Let's explore that idea.

IMPLEMENTATION

Even if you have never written React code before, this will likely feel familiar if you have experience with SwiftUI. We will start with the container component. Its main responsibility is to fetch data and pass it down to one or more presenter components.

For this example, we will keep things simple and retrieve data from a JSON API.

```
// UserListContainer.jsx (Container)
import { useEffect, useState } from "react";
import UserList from "../UserList";

export default function UserListContainer() {
  const [users, setUsers] = useState([]);

  useEffect(() => {
    // Simulating API call
    fetch("https://jsonplaceholder.typicode.com/users")
      .then(res => res.json())
      .then(data => setUsers(data));
  }, []);

  return <UserList users={users} />;
}
```

The `useEffect` function in React is known as a hook. You can think of it as loosely comparable to the `task` modifier in SwiftUI. Inside `useEffect`, we perform a network call using the `fetch` API. Once the data is retrieved, it is assigned to the private state variable `users` using `setUsers`.

So what happens when a state variable changes?

The body of the component is re-evaluated and re-rendered. In React, the body is whatever is returned from the component function. In our case, the `UserListContainer` function returns `<UserList users={users} />`. This means the `UserList` component is rendered, and we pass the private state `users` into the `users` property of `UserList`.

If this feels a little strange right now, that is perfectly normal. Once you see the equivalent implementation in SwiftUI, the similarities will become clear and the pattern will instantly make sense.

Although React components can also be written using classes, functional components are much more common today. They allow developers to write cleaner and more concise code to achieve the same results.

It is also worth noting that not every presentation component needs its own container. In many cases, a single container can serve multiple presentation components. The container fetches data and passes it down, while presentation components focus only on rendering UI. Communication in the opposite direction is typically handled using callbacks or global state solutions like Redux.

Before applying the container/presenter pattern in SwiftUI, it's important to understand why you might use it. Many developers assume this pattern is only useful for small or simple apps. From my experience, that's not the case. In fact, I've seen it power extremely large React applications, and it can absolutely scale to large SwiftUI projects as well.

This approach works especially well when most business logic lives on the server. Presentation logic can remain inside the view as long as it stays simple. If it starts to grow, you can extract it into a dedicated struct, making it easier to reuse and test. In this model, views usually fetch their own data, and when another view needs the same data, it is passed through initializer injection. Child views can communicate back using bindings or closures.

One downside is data sharing. When multiple views need access to the same data, especially when they need to stay in sync, passing data through initializers can become cumbersome. In those cases, a global state solution is often a better fit. We will look at that approach in a later chapter.

Sorting and searching introduce another design choice. Simple logic can live directly in the view. More complex behavior should be extracted into a dedicated struct or helper. As with most architectural decisions, context matters.

With that foundation, let's recreate the earlier example in SwiftUI. We'll start by implementing a stateless `HTTPClient`, which plays a role similar to the `fetch` API in JavaScript. Its sole responsibility is to perform HTTP requests and return the results.

Because it maintains no internal state, we treat it as a stateless service. To keep our code flexible, the `HTTPClient` will also be provided as an environment value, allowing views to access it without explicitly passing it around.

```
struct HTTPClient {  
    func fetchPosts() async throws -> [User] {
```

```

        let (data, _) = try await URLSession.shared.data(from: URL(string:
"https://jsonplaceholder.typicode.com/users")!)
        return try JSONDecoder().decode([User].self, from: data)
    }
}

extension EnvironmentValues {
    @Entry var httpClient = HTTPClient()
}

```

Using `Environment` values in SwiftUI is one of the cleanest ways to inject dependencies into views without cluttering initializers or passing them manually through multiple constructors. Some developers hesitate to use this approach because if you forget to inject a dependency, SwiftUI won't give you a compile-time error. While that's true, in practice it's rarely an issue.

During development, you'll run the app dozens of times. If an `Environment` dependency is missing, the problem will surface immediately, and you'll fix it long before shipping. Even if it slips past your manual checks, automated end-to-end tests will catch it. And if tests are missing, your QA team will still flag it during validation. In short, there are multiple safety nets in place that ensure these errors don't make it to production.

Another advantage is that `Environment` values are usually injected at a single, centralized location. If something is missing, it is often a one line fix at the app entry point rather than a long debugging session.

With that in mind, let's return to our example.

Below is a `ContentView` implementation that uses an `httpClient` from the environment to display a list of users:

```

struct ContentView: View {

    @Environment(\.httpClient) private var httpClient
    @State private var users: [User] = []

    private func fetchUsers() async {
        do {
            users = try await httpClient.fetchPosts()
        } catch {
            print(error.localizedDescription)
        }
    }

    var body: some View {
        List(users) { user in
            Text(user.name)
        }.task {
            await fetchUsers()
        }
    }
}

```

The `fetchUsers` function acts as a simple view helper that uses the `httpClient` to make a network request and load users. Keep in mind that the actual request is setup and executed inside `HTTPClient`. This way you can test the behavior in isolation. Once fetched, the results are assigned to a local state variable, which is then rendered inside a SwiftUI `List`.

This example highlights an important point: you don't need multiple layers of abstraction just to fetch data and display it on the screen. SwiftUI makes it possible to keep things lightweight and direct.

Of course, you can extend this further by introducing loading states in your `ContentView`. This adds more structure and makes it easier to handle scenarios like loading, success, and failure. Here's an updated implementation:

```
struct ContentView: View {

    @Environment(\.httpClient) private var httpClient
    @State private var loadingState: LoadingState = .loading

    private enum LoadingState {
        case loading
        case success([User])
        case failure(Error)
    }

    private func fetchUsers() async {
        do {
            let users = try await httpClient.fetchPosts()
            loadingState = .success(users)
        } catch {
            loadingState = .failure(error)
        }
    }

    var body: some View {

        switch loadingState {
        case .loading:
            ProgressView("Loading...")
                .task {
                    await fetchUsers()
                }
        case .success(let users):
            List(users) { user in
                Text(user.name)
            }
        case .failure(let error):
            Text(error.localizedDescription)
        }
    }
}
```

It's worth noting that, at this stage, the view doesn't contain any real business logic. Its responsibility is limited to presentation logic, expressed through simple `switch` statements.

That said, you can still extend the view's capabilities with lightweight features like **sorting**. For instance, you might allow users to toggle between ascending and descending order when displaying data.

Here's an implementation that demonstrates how sorting behavior can be added directly within the view:

```
private fun sortUsers(_ users: [User]) -> [User] {
    switch sortOrder {
    case .ascending:
        return users.sorted { $0.name < $1.name }
    case .descending:
        return users.sorted { $0.name > $1.name }
    }
}

var body: some View {
    switch loadingState {
    case .loading:
        ProgressView("Loading...")
        .task {
            await fetchUsers()
        }
    case .success(let users):
        List(sortUsers(users)) { user in
            Text(user.name)
        }.toolbar {
            ToolbarItem(placement: .topBarTrailing) {
                Button(sortOrder == .ascending ? "Sort Descending" :
"Sort Ascending") {
                    sortOrder = (sortOrder
== .ascending) ? .descending : .ascending
                }
            }
        }
    case .failure(let error):
        Text(error.localizedDescription)
    }
}
```

In the updated code, we introduced some useful enhancements. The `sortUsers` function handles sorting in both ascending and descending order. Since this is strictly presentation logic, it's perfectly fine to keep it inside the view. It's also straightforward enough that it doesn't really warrant its own unit tests.

However, if you want to reuse this logic across multiple views or make it easier to test in isolation, you can extract it. One approach, inspired by React hooks, is to expose reusable logic through the environment.

To achieve this, we'll implement a generic sort function packaged as a `SortAction`.

The implementation is shown below:

```

struct SortAction {

    var sortOrder: SortOrder = .ascending

    func callAsFunction<T: Comparable, U>(_ array: [U], by keyPath: KeyPath<U,
T>, _ order: SortOrder = .ascending) -> [U] {
        switch order {
        case .ascending:
            return array.sorted { $0[keyPath: keyPath] < $1[keyPath:
keyPath] }
        case .descending:
            return array.sorted { $0[keyPath: keyPath] > $1[keyPath:
keyPath] }
        }
    }
}

```

The `sort` function is designed to work with any type that conforms to the `Comparable` protocol. To use it, you provide the `keyPath` for the property you want to sort on, along with the desired sort direction.

To make this functionality reusable across your app, you can expose it as a custom environment hook. This allows any view to access the sorting logic without duplicating code. Simply add your new `SortAction` to `EnvironmentValues`, as shown below:

```

extension EnvironmentValues {
    @Entry var sort = SortAction()
}

```

Now, you can start using it in your SwiftUI views through the `Environment`.

```

struct ContentView: View {

    @Environment(\.httpClient) private var httpClient
    @Environment(\.sort) private var sort
    @State private var loadingState: LoadingState = .loading
    @State private var sortOrder: SortOrder = .ascending

    private enum LoadingState {
        // implementation here
    }

    private func fetchUsers() async {
        // implementation here.
    }

    var body: some View {

        switch loadingState {
        case .loading:
            ProgressView("Loading...")
                .task {
                    await fetchUsers()
                }
        }
    }
}

```

```

        case .success(let users):
            if users.isEmpty {
                ContentUnavailableView("No users found.", systemImage:
"heart")
            } else {
                List(sort(users, by: \.name, sortOrder)) { user in
                    Text(user.name)
                }.toolbar {
                    ToolbarItem(placement: .topBarTrailing) {
                        Button(sortOrder == .ascending ? "Sort Descending" :
"Sort Ascending") {
                            sortOrder = (sortOrder
== .ascending) ? .descending : .ascending
                        }
                    }
                }
            }
        }

        case .failure(let error):
            Text(error.localizedDescription)
    }
}

```

With this setup, you can call the `sort` function directly in your `List` to toggle between ascending and descending order.

Keep in mind that using the above code, `sort` function will be called anytime the `List` is rendered. A better approach would be to explicitly call it on `onChange` event, listening to the changes to the `sortDirection` state property.

This isn't the only way to make sorting reusable in your views. You could extend `Array` with generic sorting helpers, or extract utility functions into a dedicated file and reuse them throughout your project. For larger datasets, it's often more efficient to handle sorting on the server side and return results in a paginated format. You can also move the sorting out of the body into its own function that triggers using the `onChange` event. There are many ways of going about it and it is all based on specific requirement and scenario.

The approach shown here is inspired by React codebases, where "**hooks**" encapsulate logic and expose it through a shared context. Ultimately, the right solution depends on your app's architecture and your team's workflow.

Another interesting aspect of the Container/Presenter pattern is how views communicate with each other. Since this pattern doesn't rely heavily on `ObservableObject` (though you can use it, other patterns may be a better fit if that's your preference), communication is typically achieved using either bindings or closures.

VIEW COMMUNICATION USING @BINDING

Imagine you have a screen that displays a list of movies. At the top, there's an **"Add Movie"** button in the toolbar. When tapped, it presents a sheet that lets the user add a new movie. Once the movie is added, it needs to be sent back to the main view so it can be appended to the movie list.

In the **Container/Presenter** pattern, one way to accomplish this is by using SwiftUI's built-in `@Binding`. You pass the array of movies down to the `AddMovieScreen`, where a new movie can be appended. As soon as a movie is added, the parent view (`MovieListScreen`) will automatically re-evaluate and re-render with the updated list.

Here's the implementation of `MovieListScreen`:

```
struct MovieListScreen: View {

    @State private var isPresented: Bool = false
    @State private var movies: [Movie] = []

    var body: some View {
        List(movies) { movie in
            Text(movie.name)
        }.toolbar {
            ToolbarItem(placement: .topBarTrailing) {
                Button("Add Movie") {
                    isPresented = true
                }
            }
        }.sheet(isPresented: $isPresented) {
            AddMovieScreen(movies: $movies)
        }
    }
}
```

As you can see, when the sheet is presented, the entire `movies` list is passed down to the `AddMovieScreen`. This allows the new movie to be inserted directly into the array. SwiftUI then takes care of updating the parent view by re-rendering the list.

Here's the implementation of `AddMovieScreen`:

```
struct AddMovieScreen: View {

    @Binding var movies: [Movie]
    @Environment(\.dismiss) private var dismiss

    var body: some View {
        Button("Add Movie") {
            let movie = Movie(name: "Batman")
            movies.append(movie)
            dismiss()
        }
    }
}
```

In `AddMovieScreen`, the `movies` array is received as a `@Binding`. When a new movie is appended, the state changes in the view that owns the array, which is `MovieListScreen`. SwiftUI detects this change and automatically refreshes the list to show the newly added movie.

While this approach works correctly, I'm not a fan of passing the entire `movies` array into `AddMovieScreen`. It feels a bit off in terms of design. And while "feelings" may not be the strongest technical justification, I've found that when code doesn't feel right, it often hints at a deeper architectural issue. In those cases, I try to either avoid the approach or look for a cleaner alternative.

One such alternative is to use **closures**. Instead of passing the whole array, `AddMovieScreen` can return the new movie back to the caller via a closure, leaving `MovieListScreen` responsible for managing its own state. We'll explore this closure-based approach in the next section.

VIEW COMMUNICATION USING CLOSURES

Adding closures to your views introduces flexibility, allowing them to adapt to different scenarios. When used thoughtfully, closures can significantly improve the reusability of a view. In our case, we can add a closure to `AddMovieScreen` that is triggered when a new movie is created. This closure sends the movie back to the container view, where it is then appended to the list.

The implementation is shown below:

```
struct AddMovieScreen: View {  
  
    let onMovieAdded: (Movie) -> Void  
    @Environment(\.dismiss) private var dismiss  
  
    var body: some View {  
        Button("Add Movie") {  
            let movie = Movie(name: "Lord of the Rings")  
            onMovieAdded(movie)  
            dismiss()  
        }  
    }  
}
```

```
struct MovieListScreen: View {  
  
    @State private var isPresented: Bool = false  
    @State private var movies: [Movie] = []  
  
    var body: some View {  
        List(movies) { movie in  
            Text(movie.name)  
        }.toolbar {  
            ToolbarItem(placement: .topBarTrailing) {  
                Button("Add Movie") {  
                    isPresented = true  
                }  
            }  
        }.sheet(isPresented: $isPresented) {  
            AddMovieScreen { movie in
```

```

        movies.append(movie)
    }
}
}

```

`AddMovieScreen` now exposes a closure called `onMovieAdded`. This closure is triggered whenever a new movie is created. The movie is passed back to the calling view, `MovieListScreen`, where it is appended to the collection and the list updates automatically.

This approach works well and keeps the intent clear. `AddMovieScreen` becomes a reusable view you can drop into different places without changing its internal logic. The trouble starts when the view begins to expose too many actions through separate closures. At that point, the API starts to feel noisy, and the view becomes harder to use and maintain.

For example, imagine `AddMovieScreen` is also responsible for deleting or selecting a movie. In that case, we would need to expose separate closures for each action. While this still works, the view quickly becomes cluttered with multiple callbacks, which can make the design harder to maintain.

Here's an implementation that demonstrates this scenario:

```

struct AddMovieScreen: View {
    let onMovieAdded: (Movie) -> Void
    let onMovieDelete: (Movie) -> Void
    let onSelectMovie: (Movie) -> Void
    @Environment(\.dismiss) private var dismiss

    var body: some View {
        Button("Add Movie") {
            let movie = Movie(name: "Lord of the Rings")
            onMovieAdded(movie)
            dismiss()
        }
    }
}

```

It is already looking quite crowded. The calling site is also no longer simple as you can see the following implementation.

```

AddMovieScreen(onMovieAdded: { movie in
}, onMovieDelete: { movie in }, onSelectMovie: { movie in
})

```

One way to reduce this “**closure bloat**” is to expose **enum-based events** from `AddMovieScreen`.

Instead of having multiple closures for different actions, we can consolidate them into a single closure that accepts an enum value representing the event. This not only reduces the number of closures but also makes the communication pattern more organized and easier to extend.

We'll explore this enum-based approach in the next section.

GROUPING ENUM EVENTS

In the previous section, `AddMovieScreen` exposed three separate closures, one for each event. As more events are introduced, this approach adds complexity in two places. The view itself becomes harder to reason about, and the calling site has to deal with multiple callbacks instead of a single, clear interaction point.

A cleaner alternative is to consolidate these closures into a single `enum`. The enum, defined inside the view, can represent all possible events such as `addMovie`, `deleteMovie`, or `selectMovie`. When new functionality is needed, you simply add another case to the enum instead of creating a new closure.

Here's how the implementation looks:

```
struct AddMovieScreen: View {  
  
    let onEvent: (Event) -> Void  
    @Environment(\.dismiss) private var dismiss  
  
    enum Event {  
        case addMovie(Movie)  
        case deleteMovie(Movie)  
        case selectMovie(Movie)  
    }  
}
```

When adding a new movie, you simply trigger the appropriate enum case and pass the movie along with it.

```
Button("Add Movie") {  
    let movie = Movie(name: "Lord of the Rings")  
    onEvent(.addMovie(movie))  
    dismiss()  
}
```

The calling site becomes simpler as well, since it only needs to handle the enum's defined cases.

```
AddMovieScreen { event in  
    switch event {  
        case .addMovie(let movie):  
            print(movie.name)  
        case .deleteMovie(let movie):  
            print("deleted")  
            print(movie.name)  
        case .selectMovie(let movie):  
            print("selected")  
            print(movie.name)  
    }  
}
```

This technique not only reduces the amount of code in your views but also makes the calling site cleaner and easier to follow.

The choice between using individual closures and grouping events with an enum often comes down to personal preference and comfort. A good rule of thumb is to begin with closures for their simplicity, and if the view starts to feel cluttered or harder to manage, transition to the event-grouping approach.

CONCLUSION

The Container/Presenter pattern offers a practical way to structure SwiftUI applications by keeping data management and presentation separate. Containers focus on fetching and preparing data, while presenters are responsible only for rendering the user interface. This clear division results in simpler, more reusable views that are easier to maintain and test.

We also explored different techniques for view communication, including bindings, closures, and enum-based events. Each approach has its place, but the enum strategy in particular helps reduce complexity and makes calling sites easier to understand.

Overall, this pattern provides a strong foundation for building maintainable SwiftUI projects. It works especially well for applications where data is not heavily shared across multiple views and each view owns its data through local state. It can also scale to larger applications when paired with complementary strategies like global state management, which we will explore in the next chapter.

MV PATTERN

I first came across the MV pattern through an Apple Developer Forums [post](#) back in 2022. Like many people who participated in that discussion, I dismissed it almost immediately. At the time, I was heavily invested in MVVM. I had spent years learning it, teaching it, and applying it in SwiftUI. I had published dozens of articles, recorded videos, and spoken at multiple conferences explaining why MVVM was the pattern everyone should follow when building SwiftUI applications.

I was not ready to hear anything negative about a pattern I had spent nearly three years promoting and refining.

Over time, that mindset started to crack.

As I built more apps, I began to notice a pattern in my own code. By strictly following MVVM, I was often making things more complicated than they needed to be. Simple screens ended up with unnecessary layers. State felt fragmented. Boilerplate grew faster than the app itself.

My previous experience with React and Flutter also played a role here. Those frameworks encourage a different way of thinking about UI and state, one that felt lighter and more direct. That background pushed me to take a second look at the MV pattern being discussed on the forum.

So I went back and read the entire thread from start to finish. That took some time. There were hundreds of replies, and the original author was extremely active, answering questions in detail and sharing well thought out explanations, diagrams, and complete code examples.

Once I had a solid understanding of the pattern and how it fit into SwiftUI, I reached out to the author and asked if we could talk. We ended up spending about an hour on Zoom discussing the ideas behind the MV pattern in much more depth. We did not agree on everything, but we both walked away with a better understanding of each other's perspective and a shared respect for the tradeoffs involved.

Since then, I have seen many variations of the MV pattern in the wild. I have seen implementations that completely avoid observable objects and rely almost entirely on structs. I have seen others that still use controllers, stores, services, and even view models, but follow the core principles of MV rather than MVVM.

In this section, I will share my understanding of the MV pattern and how I personally apply it when building SwiftUI applications. Every app is different. Architecture is always contextual. If I were building a SwiftData heavy application, my approach would look very different. For the sake of clarity, we will focus on client server applications where managing global state is an important concern.

Before we dive deeper, we need to answer a fundamental question.

What is a view in SwiftUI?

UNDERSTANDING THE VIEW IN SWIFTUI

One of the core ideas behind the MV pattern is that in SwiftUI, you do not need a view model for every screen because the view itself already behaves like a view model.

This statement has caused a lot of confusion in the iOS community.

When developers hear this, they often jump to the conclusion that everything should live inside the view. That is absolutely not what this means. Unfortunately, many developers never get past that initial misunderstanding and end up fighting the SwiftUI framework instead of working with it.

Let's clear this up.

A SwiftUI view is very similar to a component in React or a widget in Flutter. I mention React often in a SwiftUI book for a reason. React, Flutter, and SwiftUI share a lot of conceptual DNA, especially React and SwiftUI. Looking at how similar problems are solved in other frameworks can often make things clearer.

Consider the following React example that fetches data from an API and displays it on the screen.

```
import React, { useEffect, useState } from "react";

// Minimal React app to display a list from an API
// This example fetches a list of users from JSONPlaceholder

export default function App() {
  const [items, setItems] = useState([]);

  useEffect(() => {
    fetch("https://jsonplaceholder.typicode.com/users")
      .then((res) => res.json())
      .then((data) => setItems(data));
  }, []);

  return (
    <div style={{ fontFamily: "sans-serif", margin: "2rem" }}>
      <h1>Users</h1>
      <UserList items={items} />    </div>
    );
}
```

This follows the Container Presenter pattern. The container, `App`, fetches the data. The presenter, `UserList`, renders it. The important detail is the code after the `return` keyword. The `<div>`, `<h1>`, and `<UserList>` elements represent the view. That and only that part is responsible for describing what should appear on the screen.

Technically even the return part is not the actual view in React because these are virtual DOM elements. They will be mapped to the real HTML DOM elements to be displayed on the screen.

Now, compare that to SwiftUI code below and think about the question that what part of the code is responsible for returning a view in SwiftUI.

```

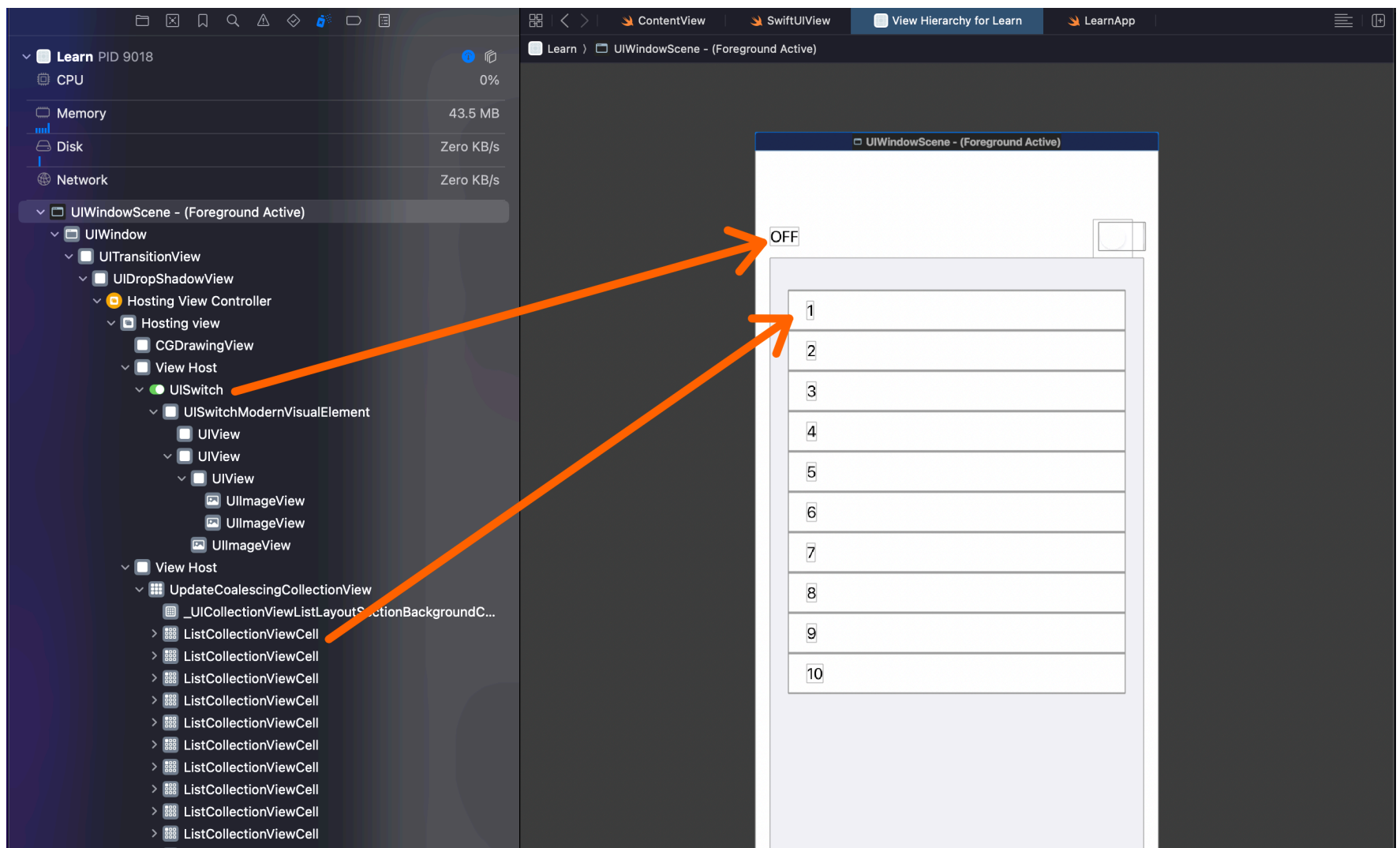
struct CounterView: View {
    @State private var isOn: Bool = false

    var body: some View {
        VStack {
            Toggle(isOn: $isOn) {
                Text(isOn ? "ON": "OFF")
            }
            List(1...10, id: \.self) { index in
                Text("\(index)")
            }
        }
        .padding()
    }
}

```

You can clearly see that the body property in SwiftUI is similar to the return from the React functional component. Both are responsible for returning the declaration of the view. I am calling it a declaration of the view and not the actual view because even in SwiftUI, the views you return are not the actual views but later they are mapped to the real views and then rendered on the screen.

You can clearly see this by clicking the **"Debug view hierarchy"** button, when the simulator is running. This is shown below:



In SwiftUI, a `Toggle` is ultimately mapped to a `UISwitch`, and a `List` is mapped to a `UICollectionView`. This is very similar to how React works, where a function returns virtual DOM elements that are later translated into their corresponding HTML elements and rendered on the screen.

The ability to debug the view hierarchy in SwiftUI gives us a practical way to diagnose user interface issues and a behind the scenes look at what is actually being rendered on the screen. It is an invaluable tool to have when a view does not behave or render the way you expect.

In addition to returning a declaration of the view through the `body` property, a SwiftUI view can also host presentation logic. It is important to separate presentation logic from business or domain logic. They are not the same thing.

The purpose of presentation logic is to support the view and improve the user experience. It is concerned with how information is displayed and how users interact with the interface. It is not driven by business rules or domain constraints.

Here are a few common examples of presentation logic in SwiftUI:

- **Formatting values for display**

Converting a `Date` into a human readable string or formatting a price with the correct currency symbol.

- **Toggling UI state**

Managing whether a sheet is presented, an alert is visible, or a section is expanded or collapsed.

- **Styling decisions**

Choosing colors, fonts, or icons based on state, such as showing red for errors or green for success.

- **Animations and transitions**

Controlling how views appear, disappear, or move in response to user actions.

- **User input validation**

Performing lightweight checks like ensuring a text field is not empty, limiting input length, or verifying that an email contains an at symbol. These checks guide the user but do not enforce business rules.

All of these behaviors exist to enhance the user experience. They shape how the interface feels and responds, but they do not change the core rules of the application.

That is why presentation logic belongs in the view.

Business or domain logic, on the other hand, should live elsewhere. Rules such as verifying that a username is unique, determining whether a customer qualifies for a loan, or enforcing application invariants should be handled by stores, models, or dedicated services. Keeping these responsibilities outside the view layer leads to clearer boundaries and a more maintainable architecture.

OBSERVABLE STORES AND STATELESS SERVICES

In the chapter on the **Container and Presenter** pattern, you learned that for certain types of applications it is perfectly acceptable to access an `HTTPClient` directly from the view. The resulting data can be stored in a private local state variable and passed down to subviews as needed. This approach works well when the data is scoped to a small part of the view hierarchy.

The downside appears when that same state needs to be accessed by multiple views, especially when those views live under different parents. At that point, passing state manually through initializers starts to feel awkward and adds unnecessary complexity.

This is where SwiftUI's Environment becomes a better tool.

The Environment allows you to expose shared state to your application. Conceptually, it is similar to Redux in React, where components can access global state without threading it through every level of the tree. The key difference is that SwiftUI gives you this capability without actions, dispatchers, reducers, or middleware.

To place something in the Environment, it must be a class and must either conform to the `ObservableObject` protocol or use the `@Observable` macro. You can name these objects however you like. In practice, you will see names such as stores, services, or controllers and even managers.

I usually use the name **Store** when I am talking about an observable object that owns state. That is my default choice most of the time.

But it is not a strict rule. Naming should reflect what the type is actually doing. For example, if I am building a Maps application and I need an observable class to coordinate map related behavior, calling it a **Manager** often makes more sense.

I use the term **Service** for stateless types, whether they are structs or classes. Things like `HTTPClient` or `PDFExporter` fall into this category. A service does not hold state. It does its job, returns the expected result, and moves on.

At this point, you might be wondering whether a class that conforms to `ObservableObject` is just a view model. If that is the case, how are stores or services any different from view models?

To answer that, we need to step back and ask why the observable object was introduced in the first place.

In a traditional MVVM setup, a new observable object is created whenever a new screen is added. Each screen gets its own view model. Since an observable object represents a source of truth, this effectively means that every screen owns its own source of truth.

But that assumption does not always make sense.

Not every screen requires a separate source of truth. In many applications, multiple screens operate on the same underlying data. A movies app is a common example. You might have a movie list screen, a movie detail screen, and an add movie screen. If you follow classical MVVM strictly, you would end up with `MovieListViewModel`, `MovieDetailViewModel`, and `AddMovieViewModel`.

In reality, all of these screens are working with the same set of movies.

What you actually need is a single observable object that acts as the source of truth for movies. In some cases, you might even have a single observable object that represents most or all of the application state.

Earlier, we established an important idea in SwiftUI: the view itself already behaves like a view model. That means presentation logic can live directly inside the view. Once you accept that, the separation becomes much clearer. Presentation logic stays in the view, while business and domain logic move into stores or services.

So instead of `MovieListViewModel`, `MovieDetailViewModel`, and `AddMovieViewModel`, you end up with a single `MovieStore`.

Some developers suggest creating a single view model, such as `MovieViewModel`, and injecting it into the Environment. While this can work technically, it no longer aligns with MVVM principles. In MVVM, view models are created per screen and are not meant to be shared across multiple screens.

There are exceptions. Small view models that exist solely to support reusable views can be shared. A common example is a `UserStatusView` paired with a `UserStatusViewModel`. But this is not the norm and should be treated as an exception rather than a rule.

Another common reason developers introduce separate view models per screen is to host presentation logic. As we discussed earlier, SwiftUI removes much of that need. Presentation logic can live directly in the view without requiring an intermediate abstraction.

This brings us to the next question. How should stores or services be structured?

I usually start with a single store named after the application itself. For example, if I am working with the Platzi fake store API, I might create a `PlatziStore`. This store can depend on an `HTTPClient` and maintain the overall state of the application.

A possible implementation looks like this:

```
@MainActor
@Observable
class PlatziStore {

    var categories: [Category] = []
    var locations: [Location] = []

    let httpClient: HTTPClientProtocol
```

```

init(httpClient: HTTPClientProtocol) {
    self.httpClient = httpClient
}

func loadCategories() async throws {
    categories = try await httpClient.loadCategories()
}
}

```

As you can see, `PlatziStore` uses the `@Observable` macro, which makes it a source of truth in the application. It is also annotated with `@MainActor`, ensuring that all of its properties, such as `categories` and `locations`, as well as its methods like `loadCategories`, are isolated to the main actor.

This is important because SwiftUI requires UI updates to happen on the main thread. Since `PlatziStore` holds state that directly affects the user interface, marking it with `@MainActor` guarantees thread safety and helps prevent subtle race conditions.

Keep in mind that with Swift 6.2 and Xcode 26, you don't need to decorate the `@Observable` class with `@MainActor` as that is the default isolated environment.

You do not always need a single observable object for the entire app, but it is almost always a good idea to start small. As the application grows or as new sources of truth emerge, you can introduce additional observable objects. The key point is that a new observable object should be introduced because you need a new source of truth, not simply because you added a new screen.

This is one of the fundamental differences between the MV pattern and MVVM. In MV, the view itself acts as the view model and can host presentation logic. In MVVM, a new view model is typically created for every screen.

To see why this matters, consider an application with thirty screens. And to reiterate, screens refer to container views, while views are presenters that are typically designed with reusability in mind. A single screen can contain many reusable views. Creating thirty separate view models, one for each screen, quickly introduces unnecessary complexity and makes the codebase harder to reason about and maintain.

Now compare that with an approach where you introduce a store or service only when a new source of truth is genuinely needed, or when a new bounded context emerges in the domain.

In practice, this results in far fewer observable objects, because most screens are reading from the same underlying state. The architecture becomes simpler, more cohesive, and easier to evolve over time.

The table below illustrates this difference more clearly.

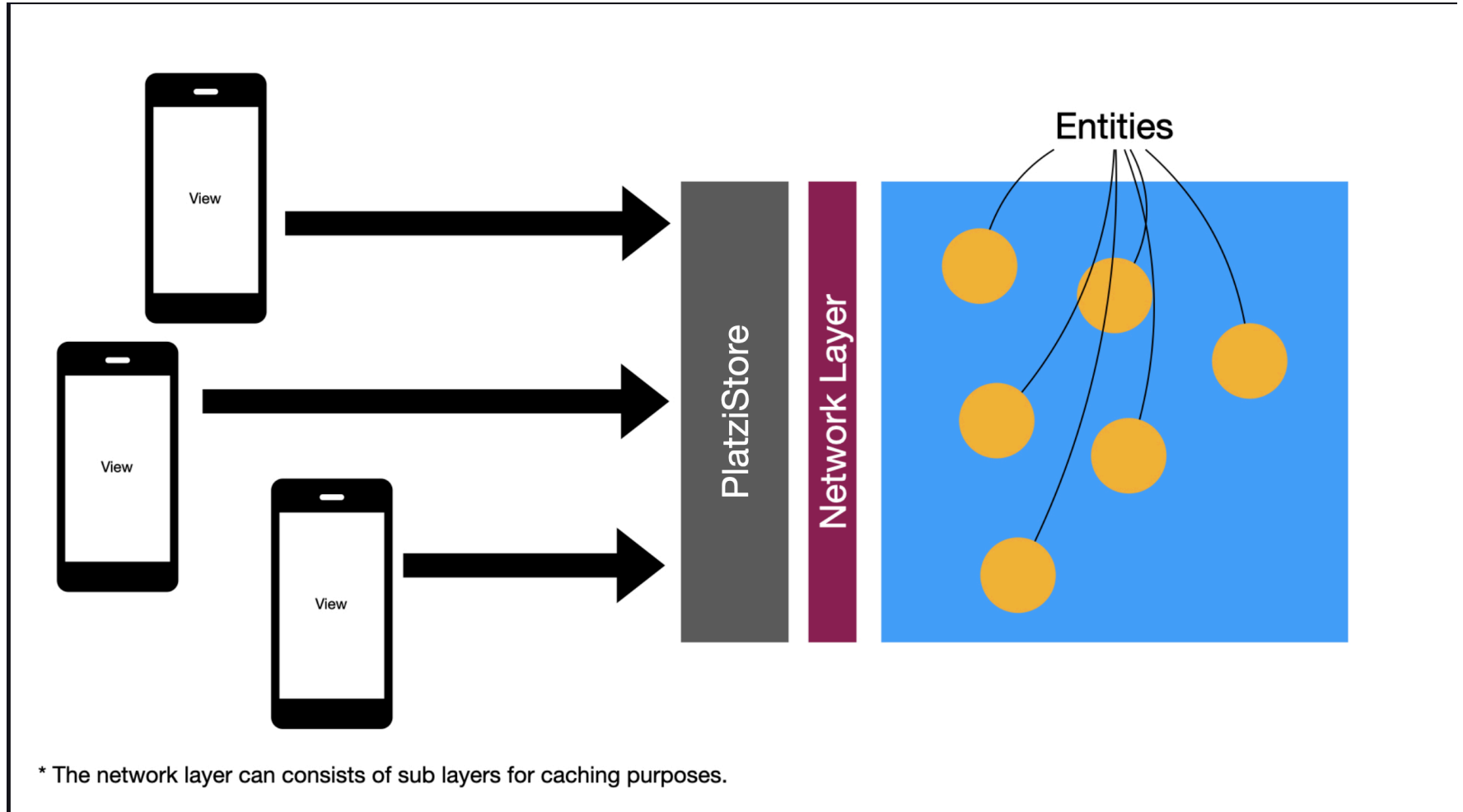
Screen	MVVM	MV
ProductListScreen	ProductListViewModel	PlatziStore
AddProductScreen	AddProductViewModel	

Screen	MVVM	MV
ProductDetailScreen	ProductDetailViewModel	
CategoryListScreen	CategoryListViewModel	
AddCategoryScreen	AddCategoryViewModel	
ReviewListScreen	ReviewListViewModel	

As you can see, if you follow the classical MVVM approach and create one view model per screen, you quickly end up with dozens of view models. This not only adds unnecessary code to the application but also makes it harder to understand and maintain over time.

By contrast, when you introduce an observable object only because you need a new source of truth, and not simply because you added another screen, you end up with far fewer observable objects. In this approach, a store like `PlatziStore` can serve multiple screens and keep the overall architecture simpler and more cohesive.

The diagram below shows how views will interact with the `PlatziStore`.



In this example `PlatziStore` is serving as a deep module. This means it is managing the entire state of the application. This technique is perfectly fine for small or even medium sized apps but once you start working on large scale applications, you want to divide a single observable object into multiple stores. In the next section, we are going to see how large scale applications can use this technique to separate concerns, reduce complexity, and improve maintainability by introducing specialized stores for different domains of the app.

Once again keep in mind that the presentation logic is part of the view itself. The store maintains the domain or business logic only. The store can also use a networking client to fetch the data from a JSON API.

LARGE SCALE APPS

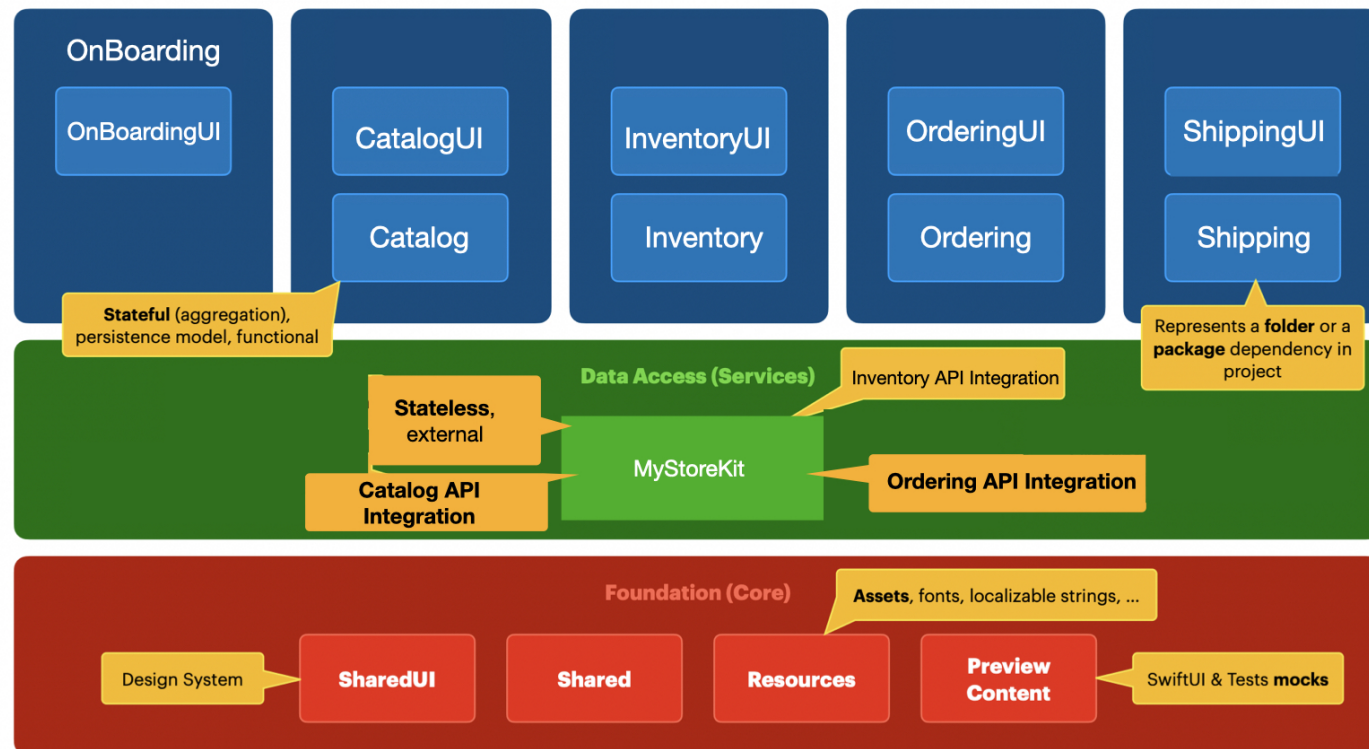
Before we dive into large-scale apps and the idea of creating dedicated stores, I want to make one thing clear: you can absolutely build a large-scale app with a single observable object, like `PlatziStore`. It's similar to having a single `index.js` or `app.js` file in Express.js that hosts all routes and endpoints. While this approach works, it can become difficult to maintain and scale as the application grows.

So, the big question is: how do we divide one large observable object into multiple smaller ones? For this, I turn to the principles of domain-driven design. One of its core ideas is the concept of a *bounded context*. A bounded context defines clear boundaries around a specific part of the domain, ensuring that its concepts, models, and rules remain consistent and independent from others. When applied to SwiftUI applications, this approach allows us to break down a single, monolithic observable object into multiple specialized stores such as `UserStore`, `ProductStore`, or `OrderStore`. The result is an architecture that is more modular, easier to test, and far simpler to understand and maintain as the app grows.

If your company has a domain expert, take the time to connect with them. Invite them for lunch, coffee, or even a casual chat. Ask thoughtful questions and explore how the business really works. The better you understand the domain, the better you can model it in code. A strong grasp of the business context gives you clarity, helps you make smarter design decisions, and leads to more maintainable and meaningful software.

The image below shows you the birds eye view of the SwiftUI architecture at scale.

Architecture



Let's take a closer look at different components of the SwiftUI architecture.

CatalogUI, InventoryUI etc:

CatalogUI or **InventoryUI** can live as separate folders or even as independent packages, each containing all of the user interface elements for a specific domain. For example, **CatalogUI** might include every screen and view related to browsing or adding catalogs, while **InventoryUI** would contain the views used to manage inventory.

This style of organization is similar to how Apple structures some of its own frameworks, such as **FinanceKitUI** and **MessageUI**. The goal is not to copy Apple, but to adopt a structure that makes the codebase easier to navigate, understand, and reason about as it grows.

Catalog, Inventory, Ordering and Shipping:

These objects either conform to the `ObservableObject` protocol or use the `@Observable` macro, and they are most often implemented as stores or services. In practice, this means **Catalog** becomes **CatalogStore** or **CatalogService**, **Inventory** becomes **InventoryStore** or **InventoryService**, and so on.

I also refer to these objects as *aggregate roots* because they act as the single point of entry into a bounded context. They encapsulate both the state and behavior of their domain and expose a controlled interface for interacting with it. All mutations, queries, and side effects related to that domain flow through the aggregate root, which makes changes easier to reason about and helps maintain consistency across the application.

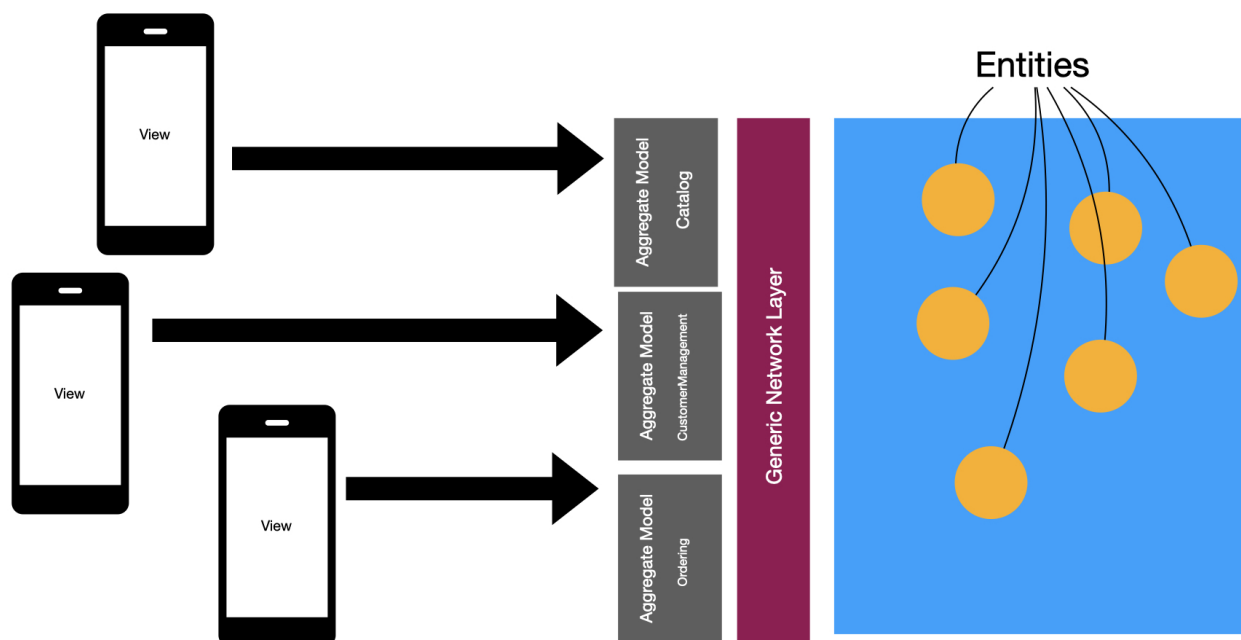
MyStoreKit

This layer contains all of the worker services that the application depends on. This typically includes networking clients, payment gateways, analytics trackers, and other infrastructure services that communicate with external systems. These services are not tied to any specific feature or domain. Instead, they provide the foundational functionality that the rest of the application builds upon.

Foundation Core

The foundation layer serves as a *shared toolkit* across the entire app. It's the place for common reusable user interface components, preview resources, utility extensions, and other building blocks that multiple features can depend on. By centralizing these cross-cutting concerns, you avoid duplication and keep your feature-specific layers focused only on their domain.

The view interaction with stores/aggregate models is shown below:



* The network layer can consists of sub layers for caching purposes.

A single screen can work with just one store, or it can pull from multiple stores, depending on what the screen needs to show. For example, a view that displays a list of orders and also shows shipping details might use both `OrderStore` and `ShippingStore`.

You can pass stores into a view through its initializer, or you can inject them through the SwiftUI environment. I usually lean toward the environment approach because it lets you inject stores at a specific point in the view hierarchy and automatically makes them available to every child view under that point.

The code below shows multiple stores injected at the root of the SwiftUI hierarchy.

```
import SwiftUI

@main
struct PlatzziApp: App {

    @State private var orderingStore = OrderingStore()
```

```

@State private var shippingStore = ShippingStore()

var body: some Scene {
    WindowGroup {
        NavigationStack {
            ContentView()
        }
        .environment(orderingStore)
        .environment(shippingStore)
    }
}

```

Once stores are injected at the root of the application, they become accessible from anywhere in the SwiftUI view hierarchy. That flexibility is powerful, but it also requires discipline. Stores should be accessed primarily from container views, not from small reusable views. Container views are responsible for loading data from stores and shaping it for presentation. Child views should receive only the data they need, usually through plain values, bindings, or simple callbacks.

This approach gives SwiftUI a clearer picture of data flow. Views become easier to reason about, updates are more predictable, and SwiftUI can do a better job tracking changes and optimizing performance.

There are rare situations where a child view is always used in a very specific context. In those cases, it can be reasonable for that view to read directly from the environment, especially if it is deeply nested and needs to update some shared or global state. Even then, it is usually worth trying bindings or closures first to see if the same result can be achieved without coupling the view directly to a store.

When we talk about aggregate models or stores, we are really talking about ownership and responsibility. A store represents a single domain. It owns the state, enforces the rules, and exposes clear commands and queries. By routing changes through the store, behavior stays consistent across the app, and the codebase remains easier to understand, reason about, and maintain over time.

`ShippingStore` is an example of such an aggregate. It represents the shipping domain and groups together everything related to shipments and carriers in one place.

```

@Observable
@MainActor
final class ShippingStore {
    private(set) var shipments: [Shipment] = []
    private(set) var carriers: [Carrier] = []
    var lastError: String?

    private let gateway: ShippingGateway

    init(gateway: ShippingGateway) {
        self.gateway = gateway
    }

    // Queries / derived state
    var activeShipments: [Shipment] {

```

```

        shipments.filter
    { [.pending, .rated, .labeled, .inTransit].contains($0.status) }
    }

    func shipment(id: UUID) -> Shipment? { shipments.first { $0.id == id } }

    // Commands

    func loadCarriers() async {
        do { carriers = try await gateway.carriers() }
        catch { lastError = "Failed to load carriers \(error)" }
    }

    func draftShipment(origin: Address, destination: Address, parcels:
[Parcel],
                        customs: CustomsDeclaration? = nil, insurance:
InsurancePolicy? = nil) {
        // draft shipment code...
    }
}

```

Depending on the complexity of your application, a store like `ShippingStore` can grow to include several properties and operations. That alone is not a reason to split it. A store should be divided only when you can clearly identify separate bounded contexts or sub-domains. If you cannot articulate a strong reason for the split, it is usually better to keep related behavior together. Before performing such a division, it can also be useful to talk to a domain expert, who may help identify natural boundaries that are not obvious from the code alone.

Once your stores are defined, you can inject them into the SwiftUI view hierarchy. Injecting them at the root of the application makes them available to all views, which is appropriate for truly global state. If a store is only relevant to a specific section of the app, you can inject it at a particular view instead. For example, in a tab-based application, each tab can inject only the stores it needs, keeping dependencies explicit and reinforcing the idea that each store owns a well-defined domain.

```

struct TabViewHome: View {

    @State private var orderingStore = OrderingStore()
    @State private var shippingStore = ShippingStore()

    var body: some View {
        TabView {
            Tab("Orders", systemImage: "house") {
                OrderingScreen()
                    .environment(orderingStore)
            }
            Tab("Shipping", systemImage: "heart") {
                ShippingScreen()
                    .environment(shippingStore)
            }
        }
    }
}

```

This technique works best when the view hierarchy for each tab is mostly interested in data from a single store. In that setup, you can inject the store at the root of the tab and everything inside that tab's navigation stack can access it.

But real apps are not always that clean. If the ordering flow, or any screen in its navigation stack, needs shipping information, then you can inject `ShippingStore` at the root of the Orders tab as well. That gives `OrderingScreen` and all of its child views access to both `OrderStore` and `ShippingStore` whenever they need it. I like to start with a narrow scope and expand only when the UI proves it needs more. In other words, give each root view and its navigation stack only the stores they need today, and add more stores later when new requirements show up.

Not every application uses tabs to create boundaries. If your app is built around a single navigation stack, it is usually simplest to inject all stores at the root of the application. Most of the time, that root is the `NavigationStack` itself. Once stores are injected there, they are available across the entire app, which avoids constantly threading dependencies through initializers as you move deeper into the navigation flow.

Conceptually, this is very similar to Redux global state in a React app. In Redux, every component can access the global state, but most components only read a small slice of it. SwiftUI works in a comparable way when you inject stores into the environment. All views can access the stores, but each view should only pull in what it actually needs.

I have been using this approach in my own apps, especially the ones that talk to a server, and it has noticeably reduced complexity. If you are building a small or medium-sized app, I would start with a single observable store and only introduce another store when you truly need a new source of truth. For larger applications, it is worth sitting down with a domain expert and defining stores around the bounded contexts of the system, so each store owns a clear piece of the domain instead of becoming a grab bag of unrelated responsibilities.

The type of application dictate the architecture. MV Pattern using stores and services is an excellent architecture for client/server applications. But if you are working with SwiftData apps then there is a separate architecture for it. I will cover that later in the book.

CONCLUSION

The MV pattern reframes how we think about SwiftUI. The view owns its presentation logic, while stores and services own domain and application state. You do not need a view model for every screen. What you need are sources of truth that align with your domain, not with your navigation tree.

Start small. A single app level store, for example `AppStore`, can comfortably power a small to medium sized app. As complexity grows, you can slice that store along clear bounded contexts such as `UserStore`, `ProductStore`, `OrderStore`, or `ShippingStore`. Each store then owns its state, enforces its rules, and exposes explicit commands and queries for that part of the domain.

Inject these stores through the environment, either at the root of the app, per tab, or at a feature root. From there, pass only what child views actually need. Presentation concerns like formatting, sorting, toggles, or lightweight validation should live in the view. Business rules such as invariants, workflows, and coordination between networking and persistence belong in stores and services. Any state that directly affects the UI should be marked with `@MainActor` so updates remain safe and predictable.

Compared to classical MVVM, this approach removes a lot of ceremony. It breaks the habit of creating one view model per view and replaces it with fewer, more meaningful sources of truth. Testing also becomes clearer. You verify presentation logic at the view layer and business rules at the store level.

Every app is different. In designs that lean heavily on `SwiftData` or other persistence driven approaches, the boundaries may shift. That is fine. The core principle stays the same: let views declare UI and interactions, and let stores enforce rules and manage state.

In the next section, we will take a closer look at how different stores can communicate with each other.

COMMUNICATION PATTERNS IN OBSERVABLE STORES

Modern SwiftUI apps almost always end up with observable stores. They hold state, enforce business rules, and quietly do the heavy lifting behind your views.

A single store feels manageable at first. Everything lives in one place and the flow is easy to follow. Then the app grows. Features overlap. Stores need to react to each other. That is when communication stops being obvious and starts becoming the hardest part of the design. This is where architecture either stays clean or slowly turns into a mess.

In this chapter, we will walk through practical and real world ways for stores to talk to each other. Not theoretical patterns. Not academic diagrams. Just approaches you can actually use as your app evolves.

We will start with the simplest option and gradually move toward more flexible, event driven solutions using Combine and Swift concurrency. Each section focuses on when a pattern works, when it starts to break down, and what problem the next pattern actually solves.

What we'll cover:

Direct view coordination

Easy to understand and quick to implement. It works well early on, but it tightly couples views to business logic and does not scale gracefully.

Delegate pattern

A familiar one to one communication mechanism. Clear and explicit, but wiring and lifecycle management become heavier as more listeners are introduced.

Combine publishers

A reactive approach where stores publish events and other stores subscribe. This decouples producers from consumers and supports multiple reactions naturally.

AsyncStream

A modern, lightweight alternative built on Swift concurrency. It provides typed event streams without bringing in Combine or managing cancellables.

withObservationTracking

A different way of thinking about communication altogether. Instead of emitting events, stores react directly to observed state changes when the relationship is simple and tightly coupled.

Throughout the article, we'll focus on organizing stores around clear responsibilities. Think `UserStore`, `InsuranceStore`, or `PaymentsStore`. When each store owns a well defined domain and uses the right communication strategy, the entire system becomes easier to reason about and easier to test.

Whether you're working on a small app with a single store or a large SwiftUI codebase with multiple domains, these patterns will help you keep your stores focused, your dependencies under control, and your views exactly where they belong. Simple, readable, and free of architectural noise.

HANDLING STORE EVENTS IN THE VIEW

Consider a healthcare or insurance app where adding a dependent immediately affects pricing. When a new dependent is added to a user's profile, insurance rates must be recalculated to reflect the updated coverage.

In this setup, responsibilities are clearly separated. The `UserStore` manages user information, including dependents, while the `InsuranceStore` owns insurance plans, premiums, and rate calculation logic. Each store focuses on its own domain, which keeps the overall design clean and predictable.

The question is not whether something should happen when a dependent is added. That part is obvious. The real question is how that change should be communicated to the rest of the system.

One straightforward approach is to let the view coordinate the interaction. The view has access to both `UserStore` and `InsuranceStore`. After a dependent is successfully added through `UserStore`, the view explicitly asks `InsuranceStore` to recalculate the rates.

There is nothing inherently wrong with this approach. The intent is clear, the flow is easy to follow, and the behavior is explicit. For small features, this can be perfectly acceptable.

The real limitation shows up when the system grows.

Today, adding a dependent might only affect insurance rates. Tomorrow, it might also impact eligibility rules, billing, notifications, analytics, or compliance checks. As more stores need to react to the same event, the view becomes responsible for notifying each one. That is where the approach starts to break down.

The real issue is not that the view makes a call. The issue is that a single action now has multiple subscribers, and the coordination logic has no clear owner. This is the point where more event driven communication patterns start to make sense.

The following implementation demonstrates this direct, view coordinated approach and sets the stage for exploring better options when multiple stores need to react to the same change.

```
struct ContentView: View {  
  
    @Environment(UserStore.self) private var userStore  
    @Environment(InsuranceStore.self) private var insuranceStore  
    @State private var insuranceRate: InsuranceRate?  
  
    let userId = 1  
  
    private func addDependent() async {  
        let dependent = Dependent(id: 2, name: "Nancy", dob: Date())
```

```

        do {
            // add dependent to user management store
            try await userStore.addDependent(dependent, to: userId)
            insuranceRate = try await
insuranceStore.calculateInsuranceRates(userId: userId)
        } catch {
            print(error.localizedDescription)
        }
    }
}

```

This example makes the tradeoffs clear. The flow is explicit and easy to follow, but the coordination burden grows as more parts of the system need to respond to the same action. Each new reaction adds another responsibility to the view, even though the view has no real ownership over those behaviors.

At this point, it becomes reasonable to ask whether the view should be the one orchestrating these updates at all. If adding a dependent is a meaningful domain event, perhaps the store that owns that action should be able to notify interested parties directly.

One way to address this is by letting stores emit events and allowing other parts of the system to subscribe to them. A familiar and simple starting point for this kind of communication is the delegate pattern.

In the next section, we'll explore how delegates can be used to handle store events, reduce coordination logic in the view, and move responsibility closer to where the state actually changes.

HANDLING STORE EVENTS THROUGH DELEGATES

Delegates are not a pattern most developers associate with SwiftUI, but they are still a useful and valid tool when applied intentionally. When used thoughtfully, delegates can provide a clear and explicit way for stores to communicate important domain events.

In the previous section, the view was responsible for coordinating multiple stores. With delegates, that responsibility moves closer to the source of the change. Instead of the view telling other stores what happened, the store that owns the action can notify interested parties directly.

To begin, define a delegate protocol that represents the event being emitted. In this case, the event is adding a dependent to a user.

```

@MainActor
protocol UserStoreDelegate: AnyObject {
    func dependentAdded(dependent: Dependent, userId: Int) async
}

```

Next, the `UserStore` is updated to hold a reference to one or more delegates. When a dependent is added, the store notifies its delegates instead of relying on the view to coordinate the response.

@Observable

```

class UserStore {

    let httpClient: HTTPClient
    weak var delegate: (any UserStoreDelegate)?

    var users: [User] = // array of users

    init(httpClient: HTTPClient) {
        self.httpClient = httpClient
    }

    // add a dependent to the user
    func addDependent(_ dependent: Dependent, to userId: Int) async throws {

        // get the user
        guard let index = users.firstIndex(where: { $0.id == userId }) else
{ throw UserError.userNotFound }

        users[index].dependents.append(dependent)
        await delegate?.dependentAdded(dependent: dependent, userId: userId)
    }

}

```

The InsuranceStore can then conform to the delegate protocol and react to the event by recalculating insurance rates.

```

@Observable
class InsuranceStore: UserStoreDelegate {

    let httpClient: HTTPClient
    var insuranceRate: InsuranceRate?

    init(httpClient: HTTPClient) {
        self.httpClient = httpClient
    }

    func dependentAdded(dependent: Dependent, userId: Int) async {
        // calculate rates
        calculateInsuranceRates(dependent: dependent, userId: userId)
        print("dependentAdded InsuranceStore")
    }

    private func calculateInsuranceRates(dependent: Dependent, userId: Int) {
        insuranceRate = InsuranceRate(
            monthlyPremium: Decimal(Double.random(in: 300...600)),
            deductible: Decimal(Double.random(in: 500...2000)),
            coverageAmount: Decimal(Double.random(in: 10_000...50_000))
        )
    }

}

```

Finally, the delegate relationship is established at composition time, usually in the app's entry point.

@main

```

struct LearnApp: App {

    @State private var userStore: UserStore
    @State private var insuranceStore: InsuranceStore

    init() {
        let http = HTTPClient()
        let userStore = UserStore(httpClient: http)
        let insurance = InsuranceStore(httpClient: http)
        userStore.delegate = insurance // wire them here
        _userStore = State(initialValue: userStore)
        _insuranceStore = State(initialValue: insurance)
    }
}

```

With this setup, the view no longer needs to be aware of insurance logic at all. The `UserStore` owns the action, emits the event, and the `InsuranceStore` reacts automatically.

Traditionally, delegates are described as supporting only a single listener. In practice, this limitation can be addressed by introducing a multicast delegate that forwards the same event to multiple listeners. This makes it possible for additional stores such as `DocumentStore`, `BillingStore`, or `NotificationStore` to respond to the same change without adding more coordination code to the view.

While multicast delegates solve the multiple listener problem, they still require explicit wiring and lifecycle management. Delegates must be registered and retained carefully, and the sender still has knowledge that delegation is being used as a communication mechanism.

Delegates work best when there is a clear one-to-one relationship, the lifecycle of both objects is well defined, and the communication path is stable. When those assumptions hold, delegates are simple, explicit, and easy to reason about. When they do not, the wiring cost grows quickly.

As applications grow and event propagation becomes more dynamic, these constraints can start to feel restrictive. This is often the point where a more decoupled, reactive approach becomes appealing.

In the next section, we'll explore how Combine publishers provide a natural way to model store events as streams, allowing multiple subscribers to react independently without tight coupling between stores.

HANDLING STORE EVENTS USING COMBINE PUBLISHERS

Another way to handle store communication is by modeling changes as events and broadcasting them using the publisher and subscriber pattern. Instead of explicitly notifying delegates, a store can expose a publisher that emits events whenever something meaningful happens. Other stores can then subscribe and react independently.

In this approach, `UserStore` does not know who is listening. Its only responsibility is to publish the fact that a dependent was added. This small shift removes the need for explicit wiring between stores and makes it easy for multiple consumers to react to the same change.

Below, `UserStore` exposes a publisher that emits an event whenever a new dependent is added. The event carries just enough information for subscribers to act on it.

```
@MainActor
@Observable
class UserStore {

    var users: [User] = []

    private let dependentAddedSubject = PassthroughSubject<(Dependent, Int),
Never>()

    var dependentAddedPublisher: AnyPublisher<(Dependent, Int), Never> {
        dependentAddedSubject.eraseToAnyPublisher()
    }

    func addUser(_ user: User) {
        users.append(user)
    }

    func addDependent(_ dependent: Dependent, to userId: Int) async {
        guard let index = users.firstIndex(where: { $0.id == userId }) else
{ return }
        users[index].dependents.append(dependent)
        dependentAddedSubject.send((dependent, userId))
    }
}
```

The key difference compared to delegates is that this event can be consumed by any number of subscribers without changing `UserStore`. The store publishes once, and the system fans out naturally.

`InsuranceStore` subscribes to this event stream during initialization. Whenever a dependent is added, it reacts by recalculating insurance rates. Errors are handled locally and do not block other subscribers or the UI.

```
@MainActor
@Observable
class InsuranceStore {

    var insuranceRate: InsuranceRate?

    private var cancellables = Set<AnyCancellable>()

    init(userStore: UserStore) {
        userStore.dependentAddedPublisher
            .sink { [weak self] dependent, userId in
                guard let self else { return }
                Task {

```

```

        let rate = await self.calculateInsurance(
            for: userId,
            dependent: dependent
        )
        self.insuranceRate = rate
        print("✅ Insurance updated for user \(userId):")
        print(rate)
    }
}
.store(in: &cancellables)
}

private func calculateInsurance(
    for userId: Int,
    dependent: Dependent
) async -> InsuranceRate {

    // Simulate async work
    try? await Task.sleep(nanoseconds: 300_000_000)

    return InsuranceRate(
        monthlyPremium: Decimal(Double.random(in: 300...600)),
        deductible: Decimal(Double.random(in: 500...2000)),
        coverageAmount: Decimal(Double.random(in: 10_000...50_000))
    )
}
}

```

Because the store is annotated with `@MainActor`, updates to observable state like `insuranceRate` are safely performed on the main thread. Capturing `self` weakly inside the subscription prevents retain cycles, and there is no need for the view to be aware of any of this wiring.

From the view's perspective, nothing changes. The UI simply adds a dependent and moves on.

```

let userStore = UserStore()
let insuranceStore = InsuranceStore(userStore: userStore)

// Add a user
let user = User(id: 1, name: "Azam")
userStore.addUser(user)

// Add a dependent (this triggers InsuranceStore automatically)
Task {
    let dependent = Dependent(id: 1, name: "Child One")
    await userStore.addDependent(dependent, to: 1)
}

```

With this setup, the view stays lean and focused on user interaction. `UserStore` emits a single, well defined event, and any interested store can subscribe and react independently. `InsuranceStore` can recalculate rates, a document related store can regenerate forms, and an analytics store can log the change, all without knowing about each other.

There are no imperative follow up calls in the view, no tight coupling between stores, and no growing chain of coordination logic. Adding a new side effect is as simple as attaching another subscriber. The producer remains unchanged, and existing consumers continue to work as before.

This pattern works especially well when a single action can trigger multiple reactions and when those reactions are likely to evolve over time. The UI remains predictable, domain logic stays inside the stores, and the system scales without pushing coordination responsibilities into the wrong layer.

This is the real strength of the Combine approach. You publish a single typed event, and the system naturally fans out to multiple independent consumers. Adding a new reaction later is just adding another subscriber.

Combine also gives you tools to shape the event stream when needed. If adding dependents can happen rapidly, operators like `removeDuplicates`, `debounce`, or `throttle` can help control how often expensive work runs.

In the next section, we'll look at `AsyncStream`, which gives you a concurrency native way to model store events without publishers, operators, or cancellables, while still supporting multiple listeners. Combine is a great fit when your app already depends on it or when you need stream shaping through operators like `debounce`, `merge`, or `removeDuplicates`. If your codebase is primarily async and await driven, however, you may want a lighter, concurrency native alternative.

HANDLING STORE EVENTS USING ASYNCSTREAM

Swift Concurrency gives us a lightweight, Combine free way to broadcast store events using `AsyncStream`. It fits naturally with `async` and `await`, keeps views thin, and supports multiple listeners without wiring delegates.

The core idea is simple. `UserStore` owns the action, so it also owns the event. When a dependent is added, `UserStore` emits a typed event and every interested store reacts on its own.

Modeling store events

We start by defining a typed `UserEvent` enum that represents all events `UserStore` can emit. This keeps events explicit, makes them easy to extend later, and avoids having a growing list of separate callbacks.

```
enum UserEvent {  
    case dependentAdded(dependent: Dependent, userId: UUID)  
    case dependentRemoved(dependent: Dependent, userId: UUID)  
}
```

Instead of exposing multiple functions or separate streams, the store exposes one stream of typed events.

Supporting multiple listeners

To support multiple subscribers, `UserStore` maintains a dictionary of continuations. Each subscriber gets its own stream, and each stream has its own continuation. The continuation is the write end of the stream, which allows the store to yield new events to listeners.

```
// one continuation per subscriber
private var continuations: [UUID: AsyncStream<UserEvent>.Continuation] =
[:]
```

When a consumer calls `events()`, the store returns a fresh `AsyncStream<UserEvent>` and registers its continuation. One important detail is that the `AsyncStream` builder closure is not `@MainActor` isolated, even if `UserStore` is. That is why the continuation registration hops to the main actor using a task.

```
// Public event stream (a new stream per subscriber)
func events() -> AsyncStream<UserEvent> {
    AsyncStream { continuation in
        let id = UUID()

        // AsyncStream builder is not MainActor-isolated, so hop to
MainActor to mutate state
        Task { @MainActor [weak self] in
            guard let self else { return }
            self.continuations[id] = continuation
        }

        continuation.onTermination = { [weak self] _ in
            Task { @MainActor in
                guard let self else { return }
                self.continuations.removeValue(forKey: id)
            }
        }
    }
}
```

The `onTermination` handler is just as important as registration. It ensures that when a subscriber stops listening, its continuation is removed from the dictionary. Without this cleanup, the store would retain continuations for subscribers that no longer exist.

Emitting events

To broadcast an event, `UserStore` simply fans out to all registered continuations and yields the same event to each subscriber. This is what gives you multicast behavior without a multicast delegate.

```
private func emit(_ event: UserEvent) {
    for continuation in continuations.values {
        continuation.yield(event)
    }
}
```

Wiring a domain action to the stream

Now `addDependent` updates state and emits a typed event. This is the only place where the event is produced, which keeps the system consistent.

```
func addDependent(_ dependent: Dependent, to userId: UUID) async throws {
    guard let index = users.firstIndex(where: { $0.id == userId }) else
    { return }
    users[index].dependents.append(dependent)
    emit(.dependentAdded(dependent: dependent, userId: userId))
}
```

Listening from another store

A subscriber store listens by consuming the stream in a task. `InsuranceStore` keeps a reference to that task so it can cancel any previous listener and avoid multiple concurrent loops.

```
func startListening(to events: AsyncStream<UserEvent>) {
    listener?.cancel()

    listener = Task { [weak self] in
        guard let self else { return }

        for await event in events {
            switch event {
            case let .dependentAdded(_, userId):
                await self.calculateInsurance(for: userId)

            case .dependentRemoved:
                break
            }
        }
    }
}
```

Because the store is `@MainActor`, updating `insuranceRate` is safe and predictable. Cleanup is also handled by canceling the listener task in `deinit`.

```
deinit {
    Task { @MainActor [weak self] in
        self?.listener?.cancel()
    }
}
```

The same pattern works for any other store. For example, `DocumentStore` can listen to the same stream and regenerate documents without knowing anything about insurance logic.

Wiring subscriptions at composition time

Subscriptions should be created at the composition root, not inside views. This keeps views focused on user interaction, ensures exactly one subscription per store, and makes teardown deterministic.

```
// Wire listeners once
insurance.startListening(to: user.events())
docs.startListening(to: user.events())
```

With this setup, views simply trigger actions like `addDependent`. Every side effect happens in subscriber stores reacting to the event stream.

This `AsyncStream` approach gives you explicit, event driven communication that fits naturally with Swift concurrency. Stores own their actions, emit well defined events, and subscribers react independently. Views remain thin, and adding new side effects does not require touching existing code.

At the same time, this is still an explicit event system. You are modeling changes as events, managing subscriptions, and deciding when and how information flows between stores. That clarity is often a benefit, but it is not always necessary.

In some cases, you do not actually need to broadcast events at all. If one store only needs to react to observed state changes in another store, SwiftUI's observation system can do that work for you automatically.

In the next section, we'll look at how `withObservationTracking` allows stores to react to changes in observed state directly, without events, streams, or subscriptions, and when that approach can be an even better fit.

WITHOBSERVATIONTRACKING

`withObservationTracking` is not an event system. It does not broadcast changes or support fan-out. It works best when one store needs to react to specific state changes in another store and that relationship is stable and tightly coupled. If the relationship is dynamic or multiple consumers need to react independently, an explicit event-based approach is usually a better fit.

In this section, you are going to learn how you can observe changes between multiple observable objects in SwiftUI.

Imagine a scenario with two stores: **UserStore** and **InsuranceStore**. The `UserStore` manages users, while the `InsuranceStore` is responsible for calculating insurance for each user and their dependents. The `User` model itself has a `name` and a collection of `dependents`.

Whenever a new dependent is added through the `UserStore`, the `InsuranceStore` must recalculate the insurance premiums. In other words, these two observable objects, `UserStore` and `InsuranceStore`, need to stay in sync and communicate with each other.

We will begin by defining our core models, `User` and `Dependent`. The implementation is shown below:

```
@Observable
class Dependent {
    let id: Int
    let name: String
```

```

    init(id: Int, name: String) {
        self.id = id
        self.name = name
    }
}

@Observable
class User {
    let id: Int
    var name: String
    var dependents: [Dependent] = []

    init(id: Int, name: String, dependents: [Dependent] = []) {
        self.id = id
        self.name = name
        self.dependents = dependents
    }
}

```

Both `Dependent` and `User` are reference types.

Next, we will implement the `UserStore`. The responsibility of `UserStore` is to maintain the state of users and perform different actions on them, such as adding a dependent.

```

@MainActor
class UserStore {

    var users: [User] = // array of users

    func addDependent(_ dependent: Dependent, to userId: Int) {

        guard let index = users.firstIndex(where: { $0.id == userId })
        else {
            return
        }

        users[index].dependents.append(dependent)
    }
}

```

Inside the function for adding a dependent, we first locate the current user using their unique id and then update that user with the new dependent. It is important to understand that we do not use array helper functions like `first` to find the user, since those return a copy of the user due to value semantics. Instead, we access the user by its index in the array and update it directly at the source.

Once the dependent has been added, the `InsuranceStore` must be notified so it can recalculate the premium rates. In other words, the `InsuranceStore` should observe changes in the `dependents` property of the `UserStore` for a particular user. Whenever a dependent is added or modified, the `InsuranceStore` can respond by recalculating the premiums.

Now we can turn our attention to the `InsuranceStore`, where we will set up observation tracking. The `UserStore` is injected into the `InsuranceStore` through its initializer, and the `startTracking`

function is responsible for initiating the tracking process. The complete implementation is shown below:

```
@MainActor
class InsuranceStore {

    let userStore: UserStore

    init(userStore: UserStore) {
        self.userStore = userStore
    }

    func startTracking(for userId: Int) throws {

        guard let index = userStore.users.firstIndex(where: { $0.id ==
userId }) else {
            throw UserError.notFound
        }

        withObservationTracking {
            print("apply")
            // what properties to track
            let _ = userStore.users[index].dependents.count

        } onChange: { [weak self] in
            Task { @MainActor in
                guard let self else { return }
                await self.calculateInsurance()
                try self.startTracking(for: userId)
            }
        }

    }

    private func calculateInsurance() async {
        print("Calculating Insurance...")
    }
}
```

The `withObservationTracking` function takes two arguments: `apply` and `onChange`. The `apply` closure specifies which properties should be tracked. In our implementation, we track the `userStore.users[index].dependents.count` property. This means the `onChange` closure will be triggered whenever the number of dependents changes for that particular user.

Inside the `onChange` closure, we create a `Task` and explicitly hop to the main actor before calling `calculateInsurance`. This is required because `InsuranceStore` is main actor-isolated, while `Task` does not inherit actor isolation. Without switching back to the main actor, Swift will report a concurrency error when a main actor-isolated method is called from a nonisolated context.

Call to main actor isolated instance method '`startTracking()`' in a synchronous nonisolated context

Another important detail about the `startTracking` function is that it calls itself at the end. By default, the `onChange` closure is invoked only once when the observed property changes. Calling

`startTracking` again effectively re-arms the observation tracking, allowing it to fire repeatedly as the property continues to change.

`startTracking()` only reads the observed value to re-arm a new tracking region, so it doesn't trigger `onChange` by itself. Wrapping that call in `Task { @MainActor in ... }` defers the re-arm to the next run-loop turn, which prevents synchronous re-entry and accidental loops.

The `onChange` event only fires when a user's dependents change—that is, when dependents are added or removed. The calling code used to test this behavior is written in Swift Playgrounds, and is shown below:

```
let userStore = UserStore()
let user = userStore.users[0]
let insuranceStore = InsuranceStore(userStore: userStore)
try insuranceStore.startTracking(for: 1)

// now change the user and add dependent
user.dependents.append(Dependent(id: 1, name: "Steven Smith"))
```

This pattern works best when there is a clear and stable relationship between stores and one store needs to stay in sync with specific state changes in another. There is no event to broadcast, no stream to manage, and no coordination logic pushed into the view.

`withObservationTracking` shines in these narrow, well defined scenarios, but it is not a general replacement for event driven communication. When multiple stores need to react independently or when the set of consumers is likely to change over time, explicit events through `Combine` or `AsyncStream` provide a more scalable and flexible model.

Used intentionally, observation tracking gives you a direct and efficient way to keep related domains in sync without adding architectural machinery where it is not needed.

CONCLUSION

Managing communication between observable stores is one of the biggest challenges in building scalable SwiftUI applications. As we have seen, there is no single approach that works for every case because each technique has strengths and limitations. Direct coordination from views works for small projects but quickly leads to tangled dependencies. Delegates provide a simple way to forward events but limit you to a single listener. `Combine` publishers and `AsyncStream` allow for decoupled, event driven architectures where multiple stores can react independently. Finally, `withObservationTracking` gives you fine grained control when you need to observe relationships between stores without adding extra burden to the view layer.

The common theme across all these patterns is keeping views lean and letting stores own their domains. By selecting the right strategy for each scenario, you can ensure that your app remains modular, testable, and easy to extend. Whether you are wiring up a small feature or coordinating

complex domains like users, dependents, and insurance rates, these patterns give you the flexibility to scale while keeping your SwiftUI codebase clean and predictable.

DEEP DIVE INTO ENVIRONMENT

In the previous chapter, we explored how communication patterns such as delegates, publishers, and async streams help coordinate data between observable stores. Each of these techniques focused on how stores talk to each other and how views respond to those changes. Yet as your application grows, not all shared information fits neatly within a single store. Some data such as user preferences, theme settings, or global services needs to be accessible throughout the app without creating tight coupling or cluttering every view with extra parameters.

This is where SwiftUI's environment system becomes essential. The environment allows data and dependencies to flow naturally through the view hierarchy, creating a clean and flexible way to share global context. Instead of manually passing objects from parent to child, SwiftUI automatically provides environment values wherever they are needed. Whether you are supplying a global store, an API client, or app wide settings, `@Environment` and `@EnvironmentObject` make it possible to build scalable architectures that remain simple and easy to understand.

In this chapter, we will take a closer look at how the environment system works, how it differs from other forms of state management, and how to apply it effectively in real world applications. You will learn when to use `@Environment` versus `@EnvironmentObject`, how to create custom environment values for dependency injection, and how these techniques align with modern SwiftUI architecture principles introduced in iOS 17 and later.

By the end of this chapter, you will understand how to use SwiftUI's environment system to manage shared state efficiently, reduce coupling between layers, and design applications that are easier to maintain, extend, and test.

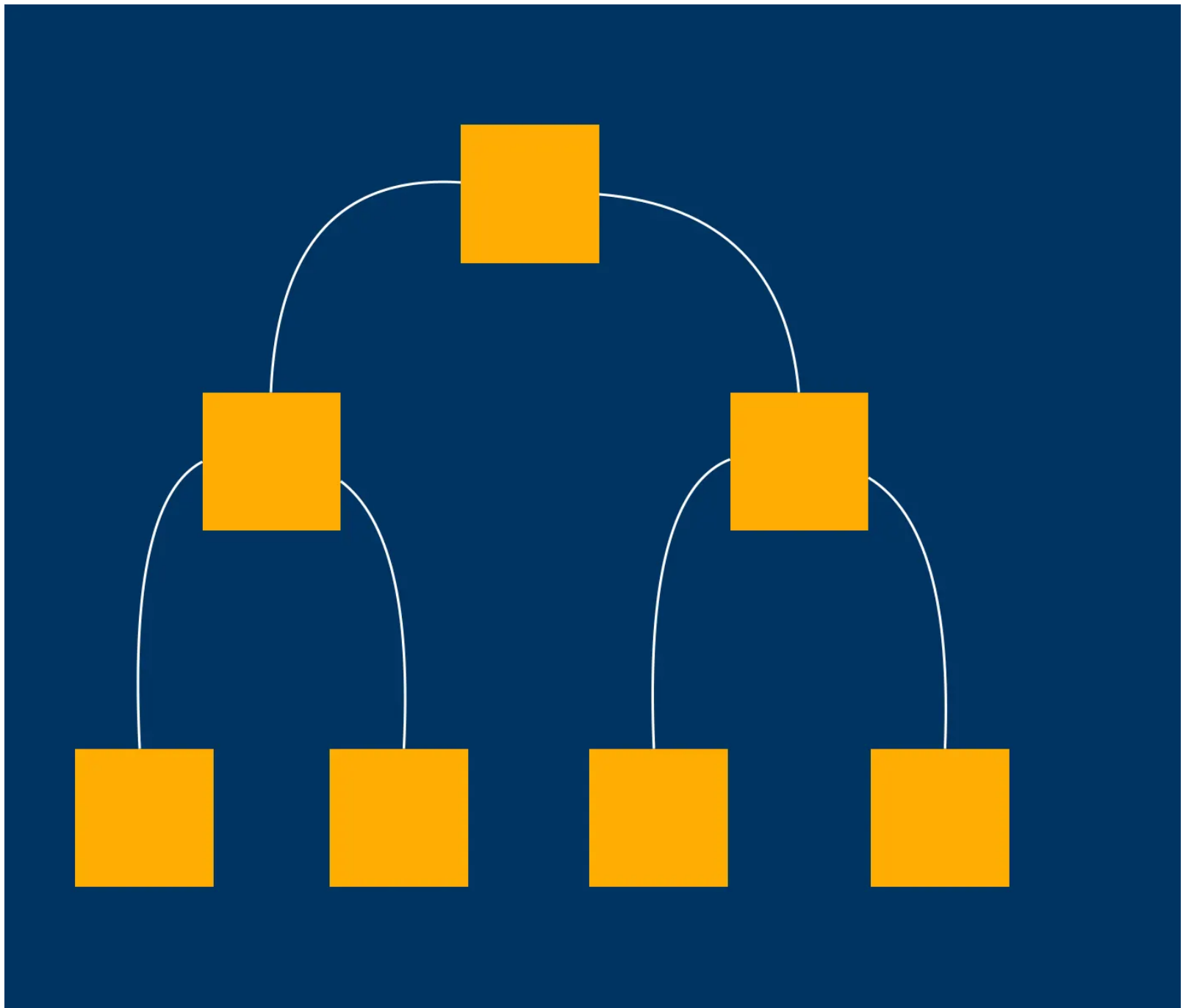
OBSERVABLE OBJECT PROTOCOL

Before the introduction of the `Observation` framework in iOS 17, SwiftUI developers primarily used the `ObservableObject` protocol to create new sources of truth for their applications.

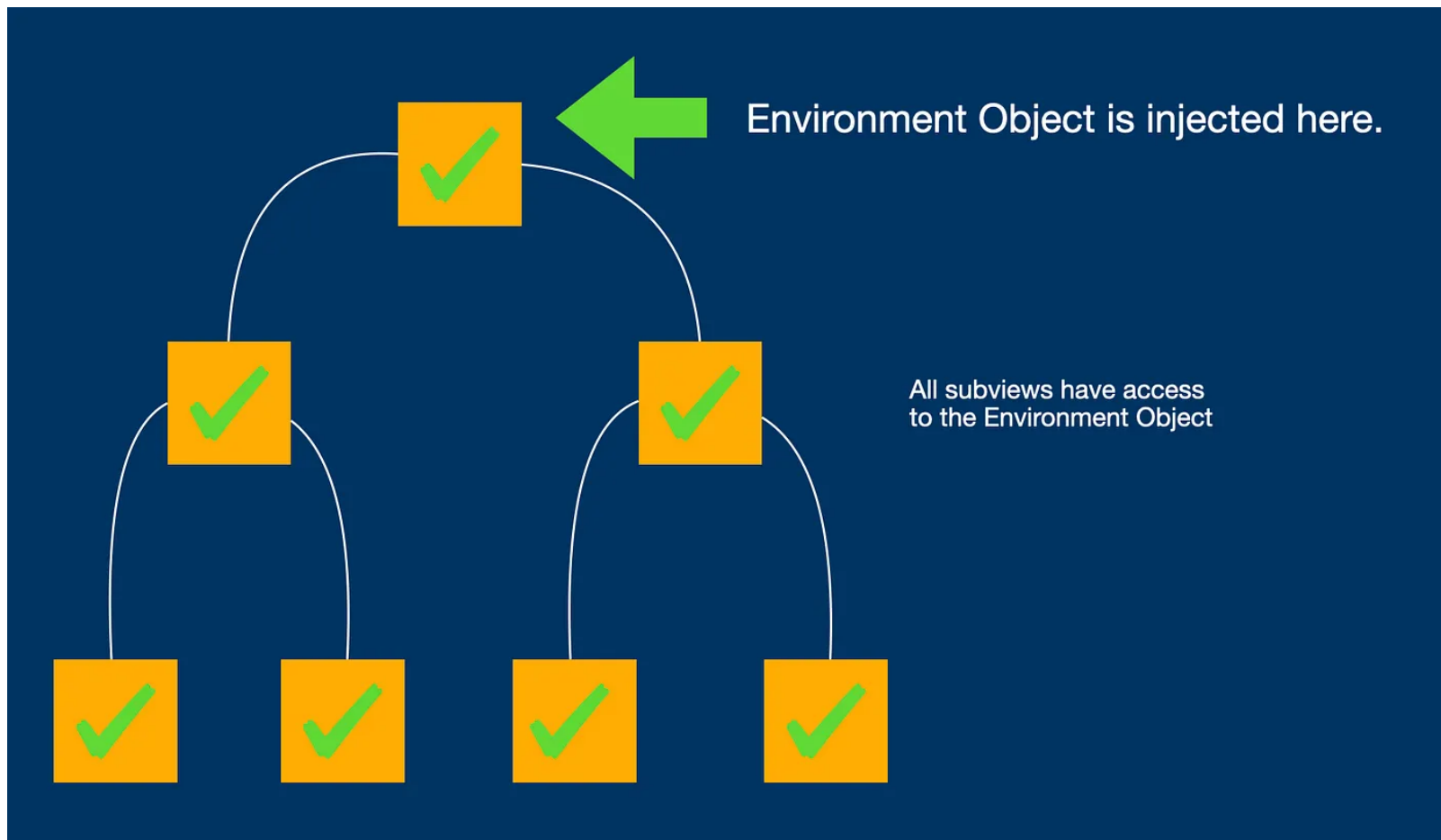
The `ObservableObject` protocol is a fundamental feature in SwiftUI used to create and manage shared data that can be observed by SwiftUI views. By conforming to `ObservableObject`, a class becomes a source of truth that SwiftUI views can automatically monitor for changes, allowing the views to update their UI dynamically when the data changes.

One of the benefits of a class conforming to `ObservableObject` protocol is that the instance of the class can be placed in the environment object, making it available globally in the views. Globally is a loaded term since the availability of the environment object depends on where in the hierarchy the object was injected.

Take a look at the following hierarchy.

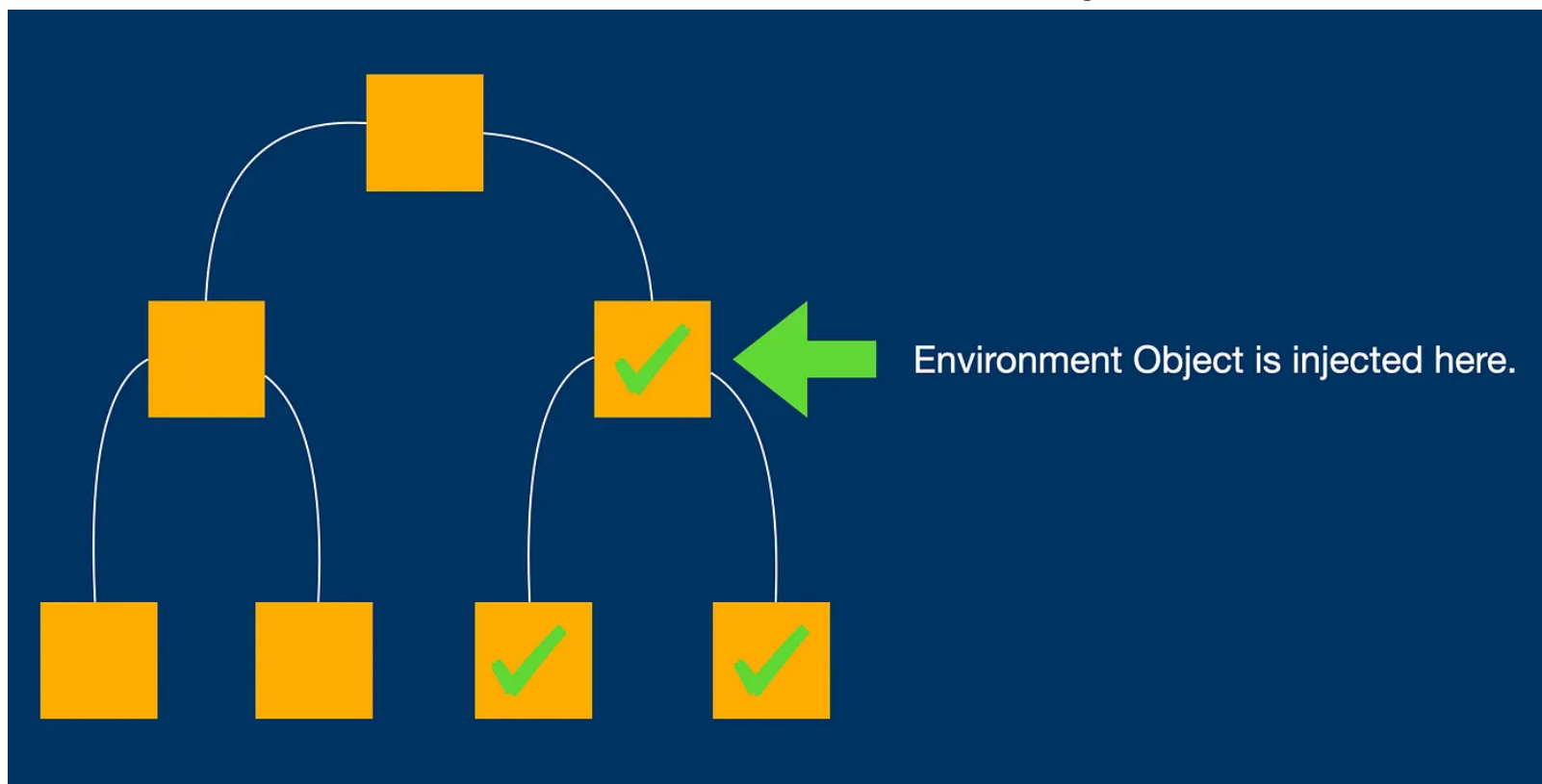


The root view has two children and each child has further two children. If we inject an environment object at the root of the application then it will be available to all the views, including the root view. This is shown in the following screenshot.



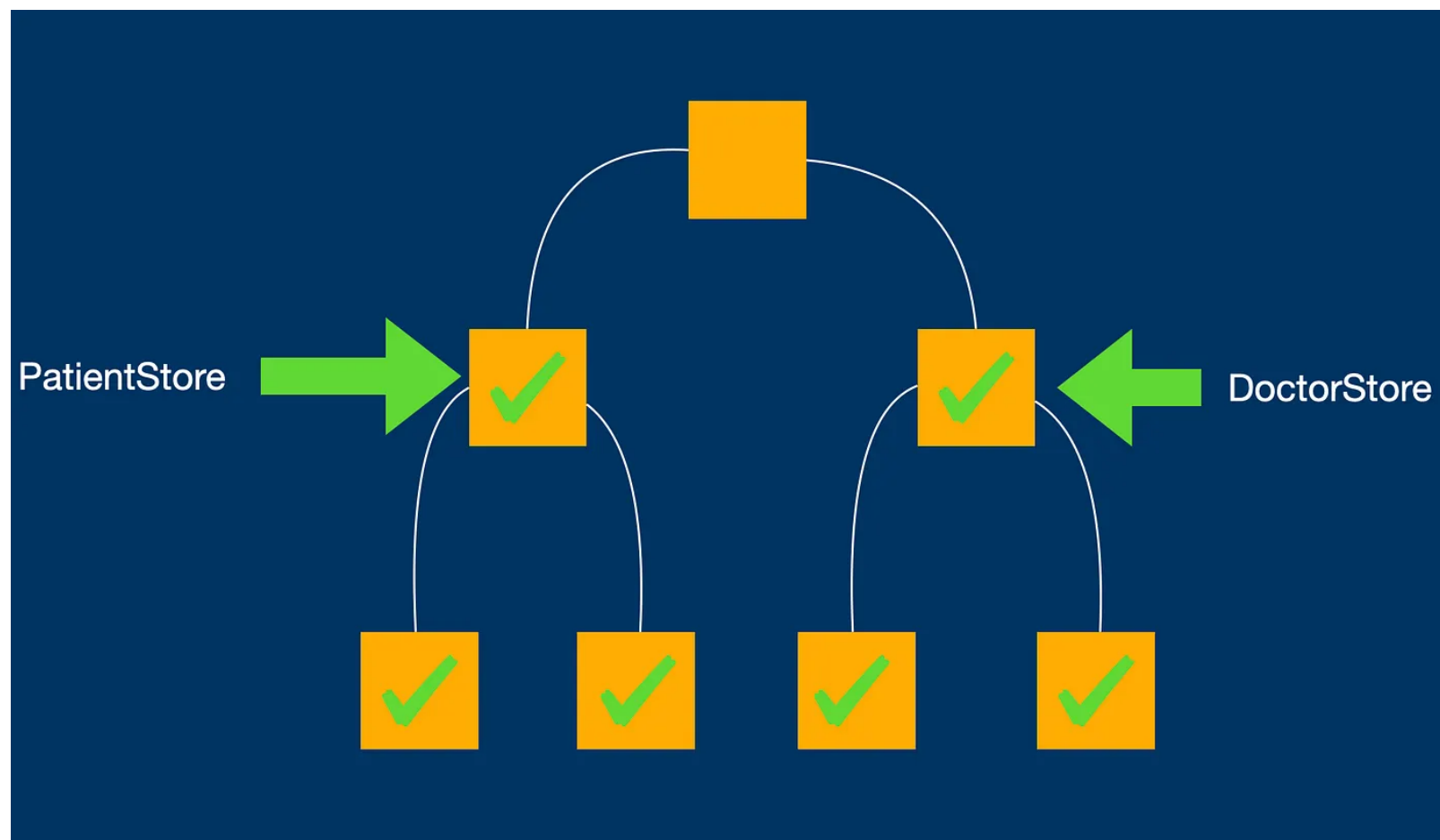
The environment object is injected at the root of the application, this means it is available in the root view and all the other views that are children of the root view.

If environment object was injected in one of the child views then it will only be available to that child and all the children of that child view. This is shown in the following screenshot.



You are also not limited to a single environment object. Based on your needs you can inject multiple environment objects at different or even same points of your application. This is shown in the

screenshot below:



Why would you choose to inject different environment objects at various points in your SwiftUI view hierarchy? Consider a scenario where you're building a hospital app with multiple tabs such as Home, Patients, and Doctors.

In this case, each tab can have its own dedicated data store. For instance, the Patients tab could use a `PatientStore`, while the Doctors tab could use a `DoctorStore`. These stores manage data specific to their respective contexts and can be injected into their corresponding parts of the view hierarchy.

Both `PatientStore` and `DoctorStore` could rely on a shared service layer to fetch the necessary data, ensuring a clean separation of concerns and efficient data handling tailored to each feature in the app. This approach promotes modularity, encapsulates dependencies, and simplifies state management within a complex SwiftUI application.

However, a common practice among developers is to inject all environment objects at the root of the application. This makes them accessible from any part of the app's view hierarchy.

RE-EVALUATION (DIFFING) VS RE-RENDERING

One of the confusion regarding using environment objects is that developers believe that when using environment object, it will automatically re-render all the views that are dependent on it. In order to understand this we first must understand the difference between re-evaluation and re-rendering.

In SwiftUI, when the state changes, the framework triggers a process called re-evaluation for the affected views. During this process, SwiftUI determines which parts of the view hierarchy need to be updated.

Re-evaluation involves diffing, where SwiftUI compares the previous and current versions of the views to identify what has changed and what has remained the same. This efficient mechanism ensures that only the necessary views are updated, optimizing performance and minimizing the impact of state changes on the user interface. This declarative approach makes UI updates seamless and reactive to state changes in your application.

Take a look at the following code:

```
struct CounterView: View {  
    @State private var count: Int = 0  
  
    var body: some View {  
        let _ = Self._printChanges()  
        VStack {  
            Text("\(count)")  
            Button("Increment Counter") {  
                count += 1  
            }  
  
            List(1...20, id: \.self) { index in  
                Text("\(index)")  
            }  
        }  
    }  
}
```

When you press the button, the counter is incremented, and the view's **body** is re-evaluated. Inside the **body**, the line `let _ = Self._printChanges()` is executed, which logs information about changes whenever the **body** is re-evaluated.

It's important to understand that re-evaluation of the body does not mean all views within it are re-rendered. and usually the process of diffing is highly optimized and is performed very quickly.

In this example, only the `Text ( "\ (count) " )` view will be re-rendered because it directly depends on the **count** property. When **count** changes, SwiftUI uses its diffing mechanism to determine that this specific **Text** view needs updating, while other views in the hierarchy remain unchanged. This targeted update ensures efficient rendering and performance optimization.

ENVIRONMENT OBJECT AND VIEW RENDERING

Let's explore a simple example of using an Environment Object (iOS 16 and earlier) to understand its behavior during view rendering. Below is the implementation of a **Store** class that conforms to the **ObservableObject** protocol. This class has a **count** property, and the **ObservableObject** conformance makes the **Store** class a source of truth, enabling SwiftUI views to observe and react to changes in its state.

```
class Store: ObservableObject {
    @Published var count: Int = 0
}
```

If we want to use **Store** as a shared global state, we can inject it into a specific view. Here's how it's done:

```
#Preview {
    ContentScreen()
        .environmentObject(Store())
}
```

This makes the **Store** instance accessible to the **ContentScreen** view and all its child views. By doing so, any view within this hierarchy can use the **Store** instance without needing to pass it explicitly as a parameter.

Although the **EnvironmentObject** is accessible to all views in the hierarchy, it's important to avoid accessing it directly from leaf views or views intended for reusability. This practice can lead to tight coupling and makes the view less flexible and harder to reuse in different contexts.

Instead, a better approach is to pass only the data required by the view as parameters and expose actions via closures. This keeps the view lightweight, self-contained, and reusable in other parts of the application without dependency on the specific environment.

To use the **Store** within a view, we can retrieve it using the **@EnvironmentObject** property wrapper. Here's an example:

```
struct ContentScreen: View {
    @EnvironmentObject private var store: Store

    var body: some View {
        let _ = Self._printChanges()

        VStack {
            Text("\(store.count)")
            Button("Increment") {
                store.count += 1
            }
        }
    }
}
```

When button is pressed, count property of **Store** is updated and displayed in a Text view.

Now, let's take a look at an example of what will happen if you add subviews to the **ContentScreen**, which also accesses **Store** environment object.

We have added two new subviews inside the `ContentScreen`. The implementation is shown below:

```
struct NumberListView: View {  
    @EnvironmentObject private var store: Store  
  
    var body: some View {  
        let _ = Self._printChanges()  
        VStack {  
            Text("NumberListView")  
        }  
    }  
}  
  
struct LightBulbView: View {  
    @EnvironmentObject private var store: Store  
  
    var body: some View {  
        let _ = Self._printChanges()  
        VStack {  
            Text("LightBulbView")  
        }  
    }  
}
```

One interesting thing to note about both the views is that even though they have a reference to the environment object, none of them utilize any properties of the `Store` class in their body.

The `ContentScreen` is updated to the following implementation:

```
struct ContentScreen: View {  
    @EnvironmentObject private var store: Store  
  
    var body: some View {  
        let _ = Self._printChanges()  
  
        VStack {  
            Text("\(store.count)")  
            Button("Increment") {  
                store.count += 1  
            }  
  
            NumberListView()  
            LightBulbView()  
        }  
    }  
}
```

Now, if you run the app and tap on the Increment button then you will see the following output.

ContentScreen: `_store changed`.

```
NumberListView: _store changed.  
LightBulbView: _store changed.
```

This means when you increment the number. All three views are getting re-evaluated and diffed.

Once again this does not mean that all three views are getting re-rendered. Only views that are changed will be re-rendered.

To address the issue, we can improve our implementation by passing only the specific dependencies needed by each subview. This ensures that each subview is independent and only depends on the data it requires. For instance:

NumberListView requires the count value.

LightBulbView requires the isOn state.

Here's the updated implementation of these subviews:

```
struct NumberListView: View {  
    let count: Int  
  
    var body: some View {  
        let _ = Self._printChanges() // Logs view re-evaluation  
        VStack {  
            Text("\(count)")  
        }  
    }  
}  
  
struct LightBulbView: View {  
    let isOn: Bool  
  
    var body: some View {  
        let _ = Self._printChanges() // Logs view re-evaluation  
        VStack {  
            Text(isOn ? "ON" : "OFF")  
        }  
    }  
}
```

The `Store` class is updated to include the `isOn` property, which tracks the on/off state:

```
@MainActor  
class Store: ObservableObject {  
    @Published var count: Int = 0  
    @Published var isOn: Bool = false  
}
```

The `ContentScreen` now passes only the required data to each subview:

```

struct ContentScreen: View {

    @EnvironmentObject private var store: Store

    var body: some View {
        let _ = Self._printChanges() // Logs view re-evaluation

        VStack {
            Text("\("\(store.count)")")

            Button("Add Number") {
                store.count += 1
            }

            Toggle(isOn: $store.isOn) {
                EmptyView()
            }
            .fixedSize()

            // Passing only required data to subviews
            NumberListView(count: store.count)
            LightBulbView(isOn: store.isOn)
        }
    }
}

```

This updated implementation improves the design by decoupling subviews, ensuring they rely only on the data they require, such as `NumberListView` depending solely on `count` and `LightBulbView` on `isOn`. This makes the subviews more reusable and easier to test. Additionally, the explicit passing of dependencies reduces reliance on `EnvironmentObject`, avoiding tight coupling and improving clarity.

Performance is also enhanced, as SwiftUI efficiently re-evaluates and updates only the affected parts of the UI, ensuring that changes in `store.count` or `store.isOn` do not trigger unnecessary re-renders elsewhere in the view hierarchy. This approach results in cleaner, more maintainable, and performance-optimized code.

It's important to understand that in SwiftUI, the natural flow of data is from parent views to child views. Typically, the parent view is indicated by a name ending with the suffix `Screen`, while child views are represented with the suffix `View`. The key principle is to have the parent view handle data loading and then pass only the necessary data down to its child views. This approach ensures clarity, reduces unnecessary dependencies, and promotes reusable, modular components within the view hierarchy.

ENVIRONMENT OBJECT AND NAVIGATION STACK

One of the powerful benefits of using `EnvironmentObject` in SwiftUI is that all screens within a `NavigationStack` automatically stay in sync with the shared data. This happens because all screens access the same global state, eliminating the need for manual updates.

In contrast, a common anti-pattern seen in the past is developers passing refresh flags between screens to ensure data synchronization after adding or updating an item. This approach not only adds unnecessary complexity but also indicates a misunderstanding of SwiftUI's declarative nature. With **EnvironmentObject**, such workarounds become unnecessary, as SwiftUI handles state updates seamlessly, ensuring a clean and efficient implementation.

This means that if your **EnvironmentObject** contains state used across 20 different screens within a **NavigationStack**, all those screens will automatically stay in sync without any additional effort on your part. SwiftUI ensures that all screens in the stack share and react to the same global state.

The only adjustment you need to make is to inject the **Store** at the **NavigationStack** level, instead of at an individual screen level like **ContentScreen**. Here's how you can do it:

```
NavigationStack {  
    ContentScreen()  
}  
.environmentObject(Store())
```

By injecting the **Store** at the **NavigationStack**, it becomes available to all screens in the navigation hierarchy, ensuring seamless state management and reducing boilerplate code.

Keep in mind that the same diffing rules apply to screens inside the **NavigationStack**. This means that if you have 20 screens in the stack, and all of them use **@EnvironmentObject**, each screen will be re-evaluated during a state change, even if they don't directly depend on or use any properties from the global state.

After re-evaluation, only the views that depend on the state change will get re-rendered.

Developers often worry about screens in a **NavigationStack** being re-evaluated due to **@EnvironmentObject**. However, the more pertinent question to ask is: how many screens are typically in your **NavigationStack** at any given time?

NEW @OBSERVABLE MACRO

In iOS 17, Apple introduced the Observation framework, offering a cleaner syntax and significant performance improvements. One key enhancement is that views that do not actively use the environment state properties are no longer re-evaluated, further optimizing SwiftUI's rendering process. This improvement ensures that only views dependent on specific state changes are updated, reducing unnecessary computations and enhancing app performance.

In order to update our current code to use Observation framework, we will start with the **Store**.

```
@MainActor  
@Observable  
class Store {  
    var count: Int = 0
```

```

    var isOn: Bool = false
}

```

We don't need to conform to `ObservableObject` anymore. We can simply use the `@Observable` macro. Also, if you noticed the `@Published` keyword has been removed. Both stored properties `count` and `isOn` are automatically published properties.

The way Environment injection is implemented will shift from using the `environmentObject` view modifier to the `environment` modifier. The updated approach is demonstrated below:

```

NavigationStack {
    ContentScreen()
}
.environment(Store())

```

Accessing the environment within a view also requires an update. Instead of using `@EnvironmentObject`, you will now use the `@Environment` property wrapper to refer to the injected store type. The updated code snippet is as follows:

```

@Environment(Store.self) private var store

```

This updated approach simplifies how environment values are injected and accessed, offering more flexibility and better type safety.

```

struct ContentScreen: View {

    @Environment(Store.self) private var store

    var body: some View {

        let _ = Self._printChanges()

        VStack {
            Text("\(store.count)")
            Button("Add Number") {
                store.count += 1
            }
            Toggle(isOn: $store.isOn) {
                EmptyView()
            }.fixedSize()

            NumberListView(count: store.count)
            LightBulbView(isOn: store.isOn)

            NavigationLink("Detail Screen") {
                DetailScreen()
            }
        }
    }
}

```

Unfortunately, the above code will not compile and give you the following error

Cannot find '\$store' in scope.

This error occurs because you cannot directly bind properties of an `@Environment` object to views. For example, using `$store.isOn` with a `Toggle` view will fail. This limitation arises from the fact that `@Environment` values are not automatically exposed as bindings.

To resolve this issue, you need to use the `@Bindable` property wrapper in your `Store` type. This allows you to bind the properties of the `Store` to views. Here's how you can achieve this:

```
struct ContentScreen: View {  
    @Environment(Store.self) private var store  
  
    var body: some View {  
        @Bindable var store = store  
  
        VStack {  
            Text("\(store.count)")  
            Button("Add Number") {  
                store.count += 1  
            }  
            Toggle(isOn: $store.isOn) {  
                EmptyView()  
            }.fixedSize()  
  
            NumberListView(count: store.count)  
            LightBulbView(isOn: store.isOn)  
  
            NavigationLink("Detail Screen") {  
                DetailScreen()  
            }  
        }  
    }  
}
```

By updating the `Store` type to use the `@Bindable` property wrapper, you can successfully bind the `Toggle` view to a property contained within the environment.

The `@Bindable` property wrapper enables properties in your environment object to be directly used as bindings within views. This makes it possible to connect UI components, like a `Toggle`, to the state managed by your environment object without additional boilerplate.

ENVIRONMENT USAGES IN REAL WORLD APPS

Using the `@Environment` to manage global state in iOS projects isn't always necessary—much like `Redux` isn't mandatory for every `React` project. However, it can be extremely helpful in specific scenarios. This is particularly true when you need to share state across screens or views that are deeply nested

within the view hierarchy. By leveraging the `@Environment`, you can avoid cumbersome state-passing through multiple layers, making your app architecture more streamlined and maintainable.

Using multiple `@Environment` objects to manage global state in an app can be a powerful way to organize and encapsulate state. By structuring these objects based on their bounded context or scope, rather than per screen, you can achieve a clean separation of concerns and maintainability. Here's a partial implementation of a `CartStore` that manages cart items for a user:

```
@Observable
class CartStore {

    let httpClient: HTTPClient
    var cart: Cart?

    init(httpClient: HTTPClient) {
        self.httpClient = httpClient
    }

    var total: Double {
        cart?.cartItems.reduce(0.0, { total, cartItem in
            total + (cartItem.product.price * Double(cartItem.quantity))
        }) ?? 0.0
    }

    func loadCart() async throws {

        let resource = Resource(url: Constants.Url.loadCart, modelType:
        CartResponse.self)
        let response = try await httpClient.load(resource)

        if let cart = response.cart, response.success {
            self.cart = cart
        } else {
            throw CartError.operationFailed(response.message ?? "")
        }
    }
}
```

`CartStore` and other stores are then injected into the root of the application.

```
@main
struct HelloMarketClientApp: App {

    @State private var productStore = ProductStore(httpClient: HTTPClient())
    @State private var cartStore = CartStore(httpClient: HTTPClient())

    var body: some Scene {
        WindowGroup {
            HomeScreen()
                .environment(productStore)
                .environment(cartStore)
                .environment(\.authenticationController, AuthenticationController)
        }
    }
}
```

```
}
```

This makes them available to all the views in the application. Below you can see the implementation of `CartScreen`, which uses `CartStore`.

```
struct CartScreen: View {

    @Environment(CartStore.self) private var cartStore
    @Environment(\.showMessage) private var showMessage

    var body: some View {
        List {
            if let cart = cartStore.cart {

                Button(action: {
                    // proceed to checkout
                }) {

                    Text("Proceed to checkout ^[(\\(cartStore.itemsCount) Item]
(inflect: true)))"
                        .bold()
                        .frame(maxWidth: .infinity)
                        .padding()
                        .background(Color.green)
                        .foregroundColor(.white)
                        .cornerRadius(8)
                }

                CartItemListView(cartItems: cart.cartItems)

            } else {
                ContentUnavailableView("No items in the cart.", systemImage:
"cart")
            }
        }

        .listStyle(.plain)
        .navigationTitle("Cart")

        .task {
            do {
                try await cartStore.loadCart()
            } catch {
                showMessage(error.localizedDescription)
            }
        }
    }
}
```

For mocking create a protocol `HTTPClientProtocol` and provide implementation of `MockedHTTPClient`.

Since the cart state is centrally managed by `CartStore`, accessing the cart items in the checkout screen becomes seamless. By retrieving `CartStore` from the environment, the `cartItems` property is

readily available for use. This eliminates the need to manually pass data between screens, simplifying the flow of information and reducing boilerplate code.

CONCLUSION

The `@Environment` and related tools in SwiftUI provide a robust mechanism for managing shared state in a clean and scalable manner. This deep dive has explored how to use `@EnvironmentObject`, `@Environment`, and the newer `@Observable` and `@Bindable` macros to achieve efficient and maintainable state management.

We've seen how injecting objects at different levels of the view hierarchy can accommodate various use cases, from global state shared across the app to context-specific state scoped to certain views. By carefully designing your environment objects based on bounded contexts and separating concerns, you can avoid common pitfalls like unnecessary re-evaluations or tightly coupled components.

The transition from `@EnvironmentObject` to the `Observation` framework in iOS 17 marks a significant improvement in SwiftUI's architecture. With features like automatic property publication and better type safety, the framework ensures more precise and efficient updates, further simplifying the development process.

It's essential to strike a balance when using environment objects – leveraging their power for deeply nested view hierarchies while avoiding overuse that could lead to unwieldy dependencies. By passing only the necessary data to subviews and utilizing SwiftUI's diffing mechanism, you can optimize performance and maintain a clear data flow.

Ultimately, using `@Environment` effectively can lead to modular, testable, and maintainable SwiftUI applications. With these tools in your development toolkit, you can confidently architect AI systems or any complex application that requires shared state management, ensuring scalability and elegance in your design.

SWIFTDATA ARCHITECTURE

SwiftData, announced at WWDC 2023, represents a major step forward in data persistence for SwiftUI applications. Built from the ground up with Swift-native syntax and a declarative approach, SwiftData is designed to replace Core Data with a solution that feels more intuitive, modern, and seamlessly integrated into the SwiftUI ecosystem.

As with any new framework, the big question is how you should architect your app around it. Should you fully embrace SwiftData's built-in conveniences like `@Model` and `@Query`, or should you abstract them behind protocols for added flexibility? Where should your business logic reside? And how can you structure your code so that it remains testable, maintainable, and easy to reason about?

In this article, I'll walk you through the patterns and practices I've developed for building real-world SwiftData applications. The main reason for adding this chapter is to show you that how the type of application we are building, affects the architecture we choose.

We need to answer questions like if we can use stores/services based architecture discussed in the last few chapters or do we have to come up with a different architecture when working with SwiftData.

Let's get started.

ARCHITECTURE

Over the years, I've experimented with many different patterns for building SwiftData applications. Yet I always find myself asking the same question: How can I make this simpler while still keeping the code scalable, testable, and easy to maintain?

In this article, I'll walk you through building a SwiftData application using a budget tracking app as an example. I chose a budget tracker because most people are already familiar with the concepts of budgeting and expenses, which allows us to focus on the architecture rather than the domain itself.

Let's begin by creating a model to represent a Budget. Here's the implementation:

```
import SwiftData

@Model
class Budget {
    var name: String
    var limit: Double

    init(name: String, limit: Double) {
        self.name = name
        self.limit = limit
    }
}
```

The Budget model is straightforward—it contains only two properties: `name` and `limit`.

Next, let's explore how to persist a new budget to the database using the SwiftData framework. The implementation of the BudgetListScreen is shown below:

```
struct BudgetListScreen: View {

    @State private var name: String = ""
    @State private var limit: Double?

    @Environment(\.modelContext) private var context
    @Query private var budgets: [Budget] = []

    var body: some View {
        Form {
            TextField("Name", text: $name)
            TextField("Limit", value: $limit, format: .currency(code: "USD"))
            Button("Save") {

                // validate the name and limit
                guard let limit = limit else { return }

                let budget = Budget(name: name, limit: limit)
                context.insert(budget)

            }

            if !budgets.isEmpty {
                Section("Budgets") {
                    List(budgets) { budget in
                        Text(budget.name)
                    }
                }
            } else {
                ContentUnavailableView("No budgets available", systemImage:
"chart.pie.fill")
            }
        }
    }
}

#Preview {
    BudgetListScreen()
        .modelContainer(for: Budget.self, inMemory: true)
}
```

There are several key aspects to note about the implementation above. First, for Xcode Previews, we've injected a `modelContainer` with `inMemory` explicitly set to `true`. This ensures that preview data is not persisted to the actual database, allowing for safe and isolated testing within the preview environment.

When the user taps the button, a new `Budget` object is instantiated, its values are assigned, and it is inserted into the model context. There's no need to explicitly call the `save()` function on the model context, as SwiftData automatically persists changes in response to certain app events—such as navigation or when the app moves to the background.

SwiftData also introduces the `@Query` macro, which enables seamless tracking and fetching of records from the database. It functions similarly to the `@FetchRequest` property wrapper in Core Data, which internally relies on `NSFetchedResultsController` to monitor changes and update the UI accordingly. When a new budget is added, the `@Query` macro automatically fetches the updated list of budgets and reflects the changes on the screen.

Currently, our application doesn't implement any business rules. So far, we've focused solely on persisting budgets and retrieving them. In the next section, we'll explore how to incorporate domain logic and business rules into your SwiftData application.

BUSINESS RULES

To ensure that each budget has a unique name—meaning no two budgets can share the same name—we need to introduce a validation step. While SwiftData provides the `#Unique` macro for enforcing uniqueness, it doesn't throw an error or offer a clean way to handle violations when the rule is broken.

As a workaround, we can implement an `exists` function in the `Budget` model that checks whether a budget with the given name already exists in the database.

Although it's possible to place this logic directly in the `BudgetListScreen`, a better approach is to encapsulate it within the `Budget` class. This not only makes the code more reusable and easier to maintain but also enables unit testing of the logic independently of the UI layer.

The implementation is shown below:

```
@Model
class Budget {
    var name: String
    var limit: Double

    init(name: String, limit: Double) {
        self.name = name
        self.limit = limit
    }

    private func exists(context: ModelContext, name: String) throws -> Bool {

        let predicate = #Predicate<Budget> { budget in
            budget.name == name
        }

        let fetchDescriptor = FetchDescriptor<Budget>(predicate: predicate)
        let results: [Budget] = try context.fetch(fetchDescriptor)
        return !results.isEmpty
    }

    func save(context: ModelContext) throws {

        if try exists(context: context, name: name) {
            throw BudgetError.duplicateName
        }
    }
}
```

```

        context.insert(self)
    }
}

```

We've added our business logic directly inside the `Budget` model. At the moment, we have a single rule: preventing users from adding duplicate budget names to persistent storage. This rule is enforced through the `exists` function, as shown in the implementation above.

The `exists` function is marked as `private`, making it inaccessible from outside the `Budget` class. However, you can still unit test this logic indirectly by using the publicly available `save` function, which internally relies on `exists` to enforce the uniqueness rule.

This approach of embedding business rules directly within your model class keeps things clean and straightforward, without the need to wrap your objects in additional design patterns.

We can also validate the budget limit by ensuring it is greater than zero. The `save` function has been updated to enforce this rule.

```

func save(context: ModelContext) throws {
    guard limit > 0 else {
        throw BudgetError.invalidLimit
    }

    if try exists(context: context, name: name) {
        throw BudgetError.duplicateName
    }

    context.insert(self)
}

```

Business rules aren't limited to the `Budget` class alone—they can also apply to relationships defined within the model. In the example below, we've added a one-to-many relationship between `Budget` and `Expense`, meaning a single budget can be associated with multiple expenses.

```

@Model
class Budget {
    var name: String
    var limit: Double
    @Relationship(deleteRule: .cascade)
    var expenses: [Expense] = []

    // other code... }
}

@Model
class Expense {
    var name: String
    var amount: Double
    var quantity: Int
    var budget: Budget?

    var total: Double {
        amount * Double(quantity)
    }
}

```

```

    }

    init(name: String, amount: Double, quantity: Int = 1) {
        self.name = name
        self.amount = amount
        self.quantity = quantity
    }
}

```

Now, we can enhance our `Budget` class by adding the computed properties `spent` and `remaining`, which dynamically calculate the total amount spent and the remaining budget based on the associated expenses.

```

@Model
class Budget {
    var name: String
    var limit: Double
    @Relationship(deleteRule: .cascade)
    var expenses: [Expense] = []

    init(name: String, limit: Double) {
        self.name = name
        self.limit = limit
    }

    var spent: Double {
        expenses.reduce(0) { result, expense in
            result + expense.total
        }
    }

    var remaining: Double {
        limit - spent
    }
}

```

Properties like `total` in the `Expense` model, as well as `spent` and `remaining` in the `Budget` model, represent business logic. As such, it's important to write unit tests for them to ensure they behave as expected under various scenarios.

SHOULD WE CREATE SEPARATE LAYER FOR BUSINESS LOGIC?

I tend to avoid introducing extra layers into my application unless they're absolutely necessary. At this stage, I don't see a strong need for a separate business logic layer, as the `Budget` and `Expense` models are perfectly capable of handling their own logic.

That said, there are certainly cases where creating services, managers, or other abstractions makes sense—particularly when importing data from a JSON API and persisting it to storage. In those scenarios, separating concerns can help keep the codebase clean and maintainable.

Here's the outline of the **Importer**, a component responsible for fetching expenses from a JSON API. The imported data can then be persisted into the local database using the `SwiftData` framework.

```
struct Importer {  
    let httpClient: HTTPClient  
  
    func importExpenses() -> [Expense] {  
        // use HTTPClient to fetch the expenses from JSON API  
  
        // return the expenses  
        return []  
    }  
}
```

You may have noticed that `importExpenses` returns the actual `SwiftData` model rather than a DTO. Whether to return a model or a DTO depends on the specific use case, and I'll dive deeper into this decision in the [Handling JSON Responses](#) section of the article.

WHAT IF I WANT TO USE A DIFFERENT PERSISTENT STORAGE IN THE FUTURE?

Apple designed `SwiftData` to integrate seamlessly with `SwiftUI`, which is evident from the fact that the `@Query` macro can only be used inside `SwiftUI` views. While this tight integration offers convenience, it also introduces a major trade-off: coupling your views directly to `SwiftData`.

Now, imagine you've been building your application for a few months and later decide to switch to a different persistence solution like `GRDB` or `Realm`. Because your views rely on `SwiftData`-specific features like `@Model` and `@Query`, migrating becomes difficult. These constructs are unique to `SwiftData` and don't translate cleanly to other data persistence frameworks, making your architecture harder to adapt or extend over time.

A possible solution is to create a custom data access layer and define a protocol to abstract away the underlying data provider. In the implementation below, I've introduced a `DataAccess` protocol that includes `getBudgets` and `addBudget` functions. Currently, there's a single concrete implementation called `BudgetSwiftDataAccess`, but the architecture allows for easily adding alternative implementations in the future, such as for testing or switching to a different data source.

```
protocol DataAccess {
    func getBudgets() throws -> [BudgetPlain]
    func addBudget(name: String, limit: Double)
}

class BudgetSwiftDataAccess: DataAccess {

    var container: ModelContainer
    var context: ModelContext

    init(container: ModelContainer = ModelContainer.default()) {
        self.container = container
        self.context = container.mainContext
    }

    func getBudgets() throws -> [BudgetPlain] {
        let budgets = try context.fetch(FetchDescriptor<Budget>())
        return budgets.map(BudgetPlain.init)
    }

    func addBudget(name: String, limit: Double) {
        let budget = Budget(name: name, limit: limit)
        context.insert(budget)
    }
}
```

One important thing to note is that the `getBudgets` and `addBudget` functions in the `DataAccess` protocol operate on plain types. They are not tied to any specific persistence framework like `SwiftData`, `Core Data`, or `Realm`. The `BudgetPlain` type is a simple struct containing just the `name` and `limit` properties.

Each data layer is responsible for converting its specific model into a `BudgetPlain` instance. This ensures that the view layer interacts only with simple, framework-agnostic types, rather than being coupled to specialized models from `SwiftData`, `Core Data`, or `Realm`.

Next, you can access your data access layer in your view through the use of environment values. This is shown below:

```
struct BudgetListScreen: View {

    @State private var name: String = ""
    @State private var limit: Double?
    @State private var budgets: [BudgetPlain] = []
```

```
@Environment(\.dataAccess) private var dataAccess

// other code ...
}
```

So, the million-dollar question is: should you use the APIs provided by SwiftData, or should you always create data access layers? There is no right answer to this what-if scenario—it all comes down to trade-offs. If you choose to use SwiftData’s APIs such as `@Query`, `@Model`, `#Predicate`, and others, you gain tight integration with SwiftUI and a faster development experience. However, you also introduce coupling that makes it harder to switch to another persistence layer down the road.

On the other hand, if you build a more abstract architecture—using plain Swift models, protocol-based repositories, and custom query layers—you’ll gain flexibility and testability, but at the cost of added complexity and boilerplate.

In the end, it’s about choosing the right balance for your project’s scope, longevity, and team preferences.

QUERIES

SwiftData provides the `@Query` macro, which allows you to fetch records directly from persistent storage. Beyond just fetching, `@Query` also automatically triggers a view update whenever a record is added, updated, or deleted—as long as a property from that record is used within the view’s body.

This behavior is similar to how `NSFetchedResultsController` works in Core Data, and I wouldn’t be surprised if SwiftData uses it behind the scenes to achieve this reactivity.

The `@Query` macro is only available within SwiftUI views. In our implementation, we’ve used it inside the `BudgetListScreen`, as shown below:

```
@Query private var budgets: [Budget] = []
```

When developers see this code, their immediate reaction often centers around separation of concerns. The instinct is to push back—arguing that placing data access logic directly in the view is unacceptable.

However, as I mentioned earlier, in SwiftUI, the view also acts as the view model. Of course, that doesn’t mean you should start dumping all of your business logic into the view. But it is an appropriate place for presentation logic, data transformation, and mapping.

Additionally, separation of concerns still exists—it’s just structured differently. Instead of being divided across layers (like in MVC or MVVM), responsibilities are now organized by feature or view. For example, all logic related to listing budgets lives inside `BudgetListScreen`, while everything related to adding a new expense resides in `AddExpenseScreen`.

So, the principle of separation of concerns hasn’t been lost—it has simply shifted in alignment with SwiftUI’s component-driven architecture.

Now, you might be thinking about the massive views problem. If a view starts getting too large, there's nothing stopping you from breaking it down into smaller, reusable views. SwiftUI makes it easy to extract and compose views, helping you keep your codebase clean, modular, and maintainable.

Let's get back to the `@Query` macro. In most cases, it's recommended to use `@Query` directly inside the view for clarity and simplicity. However, if you prefer, you can also move the query definition into the model itself. This approach helps centralize query logic and can make your views cleaner. Here's an example demonstrating how to implement this:

```
@Model
class Budget {
    var name: String
    var limit: Double

    static var all: FetchDescriptor<Budget> {
        FetchDescriptor<Budget>()
    }

    // other code ...
}
```

And now, you can use it in the view as shown below:

```
struct BudgetListScreen: View {

    @State private var name: String = ""
    @State private var limit: Double?

    @Environment(\.modelContext) private var context

    @Query(Budget.all) private var budgets: [Budget] = []

    // other code
}
```

This approach also makes it easier to write unit tests for your queries and promotes reusability. If you find yourself needing the same query in multiple places, extracting it into a shared model function is helpful. That said, if you're reusing the exact same query and presentation, it may be even better to make the view itself reusable so it can be embedded in different parts of your application with minimal duplication.

But what about dynamic queries? How do you implement queries in SwiftData that depend on a runtime value? Unfortunately, there's no straightforward way to change the predicate once `@Query` has been defined.

One approach—demonstrated in Apple's sample code—involves creating a subview and passing the filter or sort order as a parameter. Inside the subview's initializer, you can then define the `@Query` using that dynamic value. This technique allows for more flexible, runtime-driven queries, as shown below:

```
struct BudgetListView: View {

    let sortOrder: SortOrder
```

```

@Query private var budgets: [Budget] = []

init(sortOrder: SortOrder) {
    self.sortOrder = sortOrder

    let sortDescriptor = SortDescriptor<Budget>(\.name, order: sortOrder
== .asc ? .forward: .reverse)
    _budgets = Query(sort: [sortDescriptor])
}

var body: some View {
    List(budgets) { budget in
        Text(budget.name)
    }
}

```

The `BudgetListView` encapsulates the logic for displaying sorted budgets. The `@Query` is constructed within the initializer based on the value passed in—such as `sortOrder` in this case. This approach demonstrates how you can implement dynamic queries in `SwiftData` by generating the query at runtime through subview initialization.

This technique does require you to break your views based on the dynamic parameter. Depending on the scenario, that can actually be a good thing. Since SwiftUI’s dependency graph is view-based, it will only re-evaluate the views whose input parameters—or dependencies—have changed. This can lead to more efficient UI updates and better performance, especially in complex view hierarchies.

SwiftData model testing will be covered in the Testing chapter.

PREVIEWS

As one Apple engineer put it during a [WWDC session](#), “*Previews are like tests.*” Previews allow us to quickly visualize the appearance of our user interface without needing to launch the simulator every time we make a change. As an iOS developer, I rely heavily on Xcode Previews and truly believe they make SwiftUI development faster, more efficient, and a lot more enjoyable.

The `BudgetListScreen` uses the following code to construct a preview:

```

#Preview {
    NavigationStack {
        BudgetListScreen()
    }.modelContainer(previewContainer)
}

```

An important detail to note is that for the `modelContainer` argument, we passed in the `previewContainer` property. This property is a specialized instance of `ModelContainer` designed specifically for use in previews. It provides an isolated, in-memory context that allows your `SwiftData` models to function properly within Xcode Previews without affecting your main app data.

```

@MainActor
var previewContainer: ModelContainer = {

    let container = try! ModelContainer(for: Budget.self, configurations:
ModelConfiguration(isStoredInMemoryOnly: true))

    for budget in SampleData.budgets {
        container.mainContext.insert(budget)
    }

    return container
}()

struct SampleData {

    static var budgets: [Budget] {
        return [Budget(name: "Groceries", limit: 400), Budget(name:
"Vacation", limit: 2000)]
    }

}

```

The `previewContainer` also gives you the flexibility to insert default values into the in-memory storage. These values are then used to populate and display meaningful content in the `BudgetListScreen`, making your previews more realistic and useful during development. There are some scenarios where you need to first fetch data using `@Query` and then pass an individual item to the screen. This is shown below:

```
BudgetDetailScreen(budget: budgets[0])
```

You may try to write the following preview:

```

#Preview {

    @Previewable @Query var budgets: [Budget] = []

    NavigationStack {
        BudgetDetailScreen(budget: budgets[0])
    } .modelContainer(previewContainer)
}

```

The `@Previewable` macro allows you to use dynamic properties inline in previews.

Unfortunately, the code above will throw an “index out of range” exception. This happens because the `previewContainer` has not been properly initialized before attempting to access the `budgets` collection.

One way to resolve this issue is to manually create a container view—similar to how we used to do it before `@Query`—and use `@Query` in the parent view, then pass the resulting data down to the child view. This approach ensures the data is available when the child view is rendered in the preview.

```

struct BudgetDetailScreenContainer: View {
    @Query(sort: \Budget.name, order: .forward) private var budgets: [Budget]

    var body: some View {
        BudgetDetailScreen(budget: budgets[0])
    }
}

#Preview { @MainActor in
    NavigationStack {
        BudgetDetailScreenContainer()
    }.modelContainer(previewContainer)
}

```

We created the BudgetDetailScreenContainer specifically to support previews. In the preview provider, we set up the model container with mock data. The BudgetDetailScreenContainer then uses @Query to fetch items from the in-memory database and passes the relevant data down to BudgetDetailScreen. This approach ensures that the preview has access to real data, enabling us to visualize the UI in a meaningful and realistic way.

Another option introduced in the WWDC 2024 session titled *What's New in SwiftData* is the use of PreviewModifier and PreviewTrait. These new tools allow you to inject model data directly into your previews—eliminating the need to create a custom parent container view just for previewing purposes.

The following example, taken from Apple's WWDC 2024 sample code, demonstrates how to use PreviewModifier and PreviewTrait to simplify preview setup:

```

import SwiftUI
import SwiftData

struct SampleData: PreviewModifier {
    static func makeSharedContext() throws -> ModelContainer {
        let config = ModelConfiguration(isStoredInMemoryOnly: true)
        let container = try ModelContainer(for: Trip.self, configurations:
config)
        Trip.makeSampleTrips(in: container)
        return container
    }

    func body(content: Content, context: ModelContainer) -> some View {
        content.modelContainer(context)
    }
}

extension PreviewTrait where T == Preview.ViewTraits {
    @MainActor static var sampleData: Self = .modifier(SampleData())
}

struct ContentView: View {
    @Query

```

```

    var trips: [Trip]

    var body: some View {
        ...
    }
}

#Preview(trait: .sampleData) {
    ContentView()
}

```

The `PreviewModifier` can also be used for previews of views that take a model as a parameter:

```

import SwiftUI
import SwiftData

#Preview(trait: .sampleData) {
    @Previewable @Query var trips: [Trip]
    BucketListItemView(trip: trips.first)
}

```

Moving forward, you should prefer using `PreviewModifier` and `PreviewTrait` to provide sample data to your previews. This approach leads to cleaner, more maintainable code by reducing the need for custom container views and boilerplate setup.

Xcode Previews are an essential tool in every SwiftUI developer's workflow, and with the enhancements introduced in WWDC 2024—such as `PreviewModifier` and `PreviewTrait`—creating rich, data-driven previews is now easier and more scalable than ever. These tools allow you to build realistic previews with minimal effort, making it simpler to catch UI issues early, prototype quickly, and deliver polished interfaces with confidence.

While custom container views still have their place in certain scenarios, embracing `PreviewModifier` and `PreviewTrait` will help streamline your preview setup, improve reusability, and keep your preview code clean and focused.

By making previews an integral part of your development process—and powering them with real, testable data—you'll not only speed up development but also improve the quality of your SwiftUI apps.

HANDLING JSON RESPONSES

When building SwiftData applications, there are times when you need to fetch data from a remote source and insert it into your local database. In my course, *Build Gardening App Using SwiftUI & SwiftData*, I demonstrate this by downloading a list of vegetables from an API and allowing the user to add them to their garden based on their interaction.

In scenarios like this, most developers instinctively create DTO (Data Transfer Object) types to represent the server's response. While this is often a good approach, the need for DTOs really depends on a few important factors.

For instance, what if you control the server and the response format? Or what if the server response is already flat and closely resembles your SwiftData model? In such cases, introducing a separate DTO layer may be unnecessary.

In this section, we'll explore different use cases to determine when a DTO is beneficial and when it's reasonable to skip it and map the response directly to your SwiftData model.

Consider a scenario where you're building a vegetable gardening application, and you receive the following response from your own server—one that you have complete control over:

```
[
  {
    "vegetableId": 1,
    "name": "Carrot",
    "body": "A root vegetable, usually orange in color, rich in beta-carotene."
  },
  {
    "vegetableId": 2,
    "name": "Spinach",
    "body": "A leafy green vegetable high in iron and vitamins."
  },
  {
    "vegetableId": 3,
    "name": "Tomato",
    "body": "A red, juicy fruit often used as a vegetable in cooking."
  }
]
```

The response is quite straightforward and closely aligns with our data model. In this case, we can map the response directly to a SwiftData model without introducing a separate DTO layer. This keeps the code simple and reduces unnecessary abstraction.

The implementation of the SwiftData model is shown below:

```
@Model
class Vegetable: Decodable {

    var vegetableId: Int
    var name: String
    var body: String

    private enum CodingKeys: String, CodingKey {
        case vegetableId, name, body
    }

    required init(from decoder: any Decoder) throws {

        let container = try decoder.container(keyedBy: CodingKeys.self)
        self.vegetableId = try container.decode(Int.self,
forKey: .vegetableId)
        self.name = try container.decode(String.self, forKey: .name)
        self.body = try container.decode(String.self, forKey: .body)
    }
}
```

```
}  
}
```

But what if the response is not controlled by you and it is nested, complicated, or both? In those cases, it's a good idea to implement a DTO and let the DTO handle all the mapping. Here is one example:

Below, you can find the response from the JSONPlaceholder `/users` endpoint.

```
[{  
  "id": 1,  
  "name": "Leanne Graham",  
  "username": "Bret",  
  "email": "Sincere@april.biz",  
  "address": {  
    "street": "Kulas Light",  
    "suite": "Apt. 556",  
    "city": "Gwenborough",  
    "zipcode": "92998-3874",  
    "geo": {  
      "lat": "-37.3159",  
      "lng": "81.1496"  
    }  
  },  
  "phone": "1-770-736-8031 x56442",  
  "website": "hildegard.org",  
  "company": {  
    "name": "Romaguera-Crona",  
    "catchPhrase": "Multi-layered client-server neural-net",  
    "bs": "harness real-time e-markets"  
  }  
}]
```

As you can see, the response is nested across multiple levels. While it's technically possible to map this response directly to our SwiftData `User` model, doing so would introduce a significant amount of boilerplate code into the model itself. Additionally, the structure of our model may differ from the structure of the actual API response.

A better approach is to use dedicated DTO (Data Transfer Object) types to handle the JSON decoding. These lightweight structures allow us to cleanly map the API response and then convert the data into our SwiftData model as needed.

The implementation of the DTO objects is shown below:

```
import Foundation  
  
struct GeoDTO: Codable {  
    let lat: String  
    let lng: String  
}  
  
struct AddressDTO: Codable {  
    let street: String  
    let suite: String
```

```

    let city: String
    let zipcode: String
    let geo: GeoDTO
}

struct CompanyDTO: Codable {
    let name: String
    let catchPhrase: String
    let bs: String
}

struct UserDTO: Codable {
    let id: Int
    let name: String
    let username: String
    let email: String
    let address: AddressDTO
    let phone: String
    let website: String
    let company: CompanyDTO
}

```

And the implementation of the SwiftData `User` model is shown below:

```

import Foundation
import SwiftData

@Model
class User {
    @Attribute(.unique) var id: Int
    var name: String
    var username: String
    var email: String
    var phone: String
    var website: String
    var address: String
    var company: String

    init(id: Int, name: String, username: String, email: String, phone:
String, website: String, address: String, company: String) {
        self.id = id
        self.name = name
        self.username = username
        self.email = email
        self.phone = phone
        self.website = website
        self.address = address
        self.company = company
    }

    static func fromDTO(_ dto: UserDTO) -> User {
        let addressString = "\(dto.address.street), \(dto.address.suite), \(
dto.address.city), \(dto.address.zipcode)"
        let companyString = dto.company.name
        return User(
            id: dto.id,

```

```

        name: dto.name,
        username: dto.username,
        email: dto.email,
        phone: dto.phone,
        website: dto.website,
        address: addressString,
        company: companyString
    )
}
}

```

As you can see, the decision to use a DTO depends heavily on the complexity and ownership of the data source. When the server response is flat and predictable—as in the case of our gardening app—you can safely map the data directly to your SwiftData model and avoid unnecessary layers.

However, when working with more complex or nested responses—especially from third-party APIs—it's often best to introduce DTOs. They help isolate the decoding logic, keep your models clean, and make your application more maintainable and testable.

Ultimately, there's no one-size-fits-all answer. The key is to evaluate each situation carefully and choose the approach that keeps your codebase simple, scalable, and easy to reason about.

CLOUDKIT

One of the key benefits of using SwiftData with SwiftUI is its seamless integration with CloudKit. In most cases, no additional configuration is needed—your local SQLite database will automatically sync with the user's private CloudKit database.

At the time of this writing, SwiftData only supports syncing with the user's private CloudKit database.

Most CloudKit-related issues are surfaced in the Xcode output window, which can help guide you in resolving them. The most common problems typically include not being signed into iCloud on the device or simulator, or failing to provide default or optional values for properties in your SwiftData models.

As shown in the example below, I've provided default values for all model properties and marked relationships as optional to ensure smooth syncing with CloudKit:

```

@Model
class Budget {

    var name: String = ""
    var limit: Double = 0.0
    @Relationship(deleteRule: .cascade)
    var expenses: [Expense]? = []
}

```

Additionally, if you're integrating with CloudKit, you cannot use unique constraints (e.g., `#Unique<Budget>([\.name])`) on your model's properties. CloudKit does not support enforcing uniqueness at the schema level, and including such constraints in your SwiftData model will result in sync failures.

The following code works perfectly fine on a single device with CloudKit integration. Unfortunately, when tested on multiple devices for CloudKit sync, it becomes clear that the changes are not being propagated and the view is not getting refreshed.

```
import SwiftUI
import SwiftData

struct BudgetDetailScreen: View {

    @Bindable var budget: Budget
    @Environment(\.modelContext) private var context

    var body: some View {

        VStack {

            // other code

            Form {
                Section("Budget") {
                    TextField("Budget name", text: $budget.name)
                    TextField("Budget limit", value: $budget.limit,
format: .currency(code: Locale.currencyCode))
                }

                Section("Expenses") {
                    List {

                        ForEach(budget.expenses) { expense in
                            ExpenseCellView(expense: expense)
                        }.onDelete(perform: deleteExpense)
                    }
                }

            }.navigationTitle(budget.name)
        }
    }
}
```

CloudKit sends silent push notifications to notify the app of changes, but these do not automatically trigger view updates. That is because the tracked entity, in this case `Budget`, has not technically changed. Since `Budget` is a reference type and still points to the same memory location, SwiftUI does not see it as modified, and therefore the view is not re-rendered. As a result, properties like `budget.expenses` are not re-evaluated when new data is synced from CloudKit.

One effective way to solve this issue is by tracking expenses directly using the `@Query` macro. Instead of accessing expenses through the `Budget` model, you can construct and execute a dynamic `@Query` to

fetch all relevant expenses. This allows SwiftUI to re-render when the underlying expense data changes, ensuring that your UI stays in sync with the latest CloudKit updates.

The implementation is shown below:

```
struct BudgetDetailScreen: View {

    @Bindable var budget: Budget
    @Environment(\.modelContext) private var context
    @Query private var expenses: [Expense] = []

    init(budget: Budget) {
        self.budget = budget
        let budgetId = self.budget.persistentModelID

        let predicate = #Predicate<Expense> {
            if let budget = $0.budget {
                return budget.persistentModelID == budgetId
            } else {
                return false
            }
        }

        _expenses = Query(filter: predicate)
    }
}
```

A custom initializer has been added to `BudgetDetailScreen`, which is responsible for constructing a dynamic `@Query`. This query fetches all expenses associated with the provided budget. By using `@Query`, SwiftUI is able to track changes to the underlying expense data, allowing the view to re-render correctly when updates such as CloudKit syncs occur.

The updated implementation is shown below:

```
Section("Expenses") {  
    List {  
  
        // use expenses instead of budget.expenses  
        ForEach(expenses) { expense in  
            ExpenseCellView(expense: expense)  
        }.onDelete(perform: deleteExpense)  
    }  
}
```

By shifting from `budget.expenses` to a dynamic `@Query`, we ensure that SwiftUI is properly notified when new data arrives, whether it is created locally or synced silently from CloudKit. This approach not only solves the view refresh issue but also aligns well with SwiftData's design, where `@Query` plays a central role in tracking changes and driving UI updates.

While CloudKit integration with SwiftData feels seamless on the surface, subtle nuances like view re-rendering and data observation require careful handling. Leveraging tools like dynamic `@Query` helps you build more robust, reactive interfaces that stay in sync across devices and platforms.

As SwiftData continues to evolve, we can expect improvements and more predictable syncing behavior. Until then, strategies like these can help you bridge the gap and deliver a smooth user experience with real-time data updates.

CONCLUSION

SwiftData brings a fresh, modern approach to data persistence for SwiftUI apps, streamlining everything from model definition to syncing with CloudKit. With features like `@Model`, `@Query`, and seamless integration into the SwiftUI lifecycle, SwiftData simplifies what used to be a complex part of iOS development.

Throughout this article, we explored a practical architecture for building SwiftData applications, starting with modeling your data, incorporating business rules, handling JSON responses, writing meaningful tests, and creating efficient real world previews. We also covered how to structure your code in a way that remains flexible, testable, and maintainable, whether you are building a side project or scaling a production app.

SwiftData works best when you embrace its strengths, but it is also important to be mindful of its limitations, especially around flexibility and framework coupling. Whether you choose to build tightly around SwiftData or abstract your persistence layer behind protocols, the key is making thoughtful decisions that align with your project's goals.

In the end, a well architected SwiftData app is one that balances simplicity with structure, leverages previews and unit tests to catch issues early, and keeps the business logic close to where it matters most, your models.

Thanks for reading. I hope this article helps you build better, cleaner, and more enjoyable SwiftUI applications using SwiftData.

GLOBAL SHEETS AND TOASTS

Sheets and toasts are essential parts of SwiftUI development. **Sheets** are presented over the current view and provide a way to display additional content without navigating away. They are often used for creating or editing items, showing detail screens, or presenting modal workflows.

Toasts, on the other hand, are lightweight overlays that briefly appear at the bottom or top of the screen to communicate short messages, such as confirmations, alerts, or errors. Unlike sheets, toasts do not block user interaction and automatically disappear after a short duration.

Together, sheets and toasts give developers flexible tools for building engaging and user-friendly interfaces in SwiftUI.

In this chapter, you will learn different patterns for displaying sheets and toast in a SwiftUI application.

BASIC SHEETS

SwiftUI provides a sheet view modifier that can easily be used to display sheets. The display of the sheet can be controlled by various parameters. The easiest one is the `Binding<Bool>`, which shows the sheet when the boolean value changes.

The implementation is shown below:

```
struct ContentView: View {  
    @State private var isPresented: Bool = false  
  
    var body: some View {  
        VStack {  
            Button("Show Sheet") {  
                isPresented = true  
            }  
            }.sheet(isPresented: $isPresented, content: {  
                Text("Sheet Content")  
            })  
        }  
    }  
}
```

You can also display a sheet based on an item, where item is `Binding<Identifiable?>`. This allows you to add a sheet dependency on an item that is initially null and when it is assigned you also get access to the unwrapped value of the item. This is shown below:

```
struct ContentView: View {  
    @State private var customer: Customer?  
  
    var body: some View {  
        VStack {
```

```

        Button("Show Sheet for Customer") {
            customer = Customer(name: "John Doe")
        }
    }
    .sheet(item: $customer, content: { customer in
        Text(customer.name)
    })
}
}

```

The sheet modifier also provides access to the dismiss closure, which is fired when the sheet is dismissed. The easiest way to dismiss a sheet is to access and use the dismiss function from the `@Environment(\.dismiss)`.

```

struct ContentView: View {

    @State private var isPresented: Bool = false

    var body: some View {
        VStack {
            Button("Show Sheet") {
                isPresented = true
            }

        }

        .sheet(isPresented: $isPresented, onDismiss: {
            // fires when the sheet is dismissed
        }, content: {
            Text("Sheet Content")
        })
        .padding()
    }
}

```

Sheets can be presented on top of each other too. Although this kind of user interaction is not encouraged but it is still possible. The code is pretty straightforward since, you will simply display a sheet in a normal way. This is shown in the implementation below:

```

struct CustomerDetailScreen: View {

    let customer: Customer
    @State private var isPresented: Bool = false

    var body: some View {
        VStack {
            Text(customer.name)
            Button("Show Address") {
                isPresented = true
            }
        }
        .sheet(isPresented: $isPresented) {
            Text("1200 Richmond Ave, Houston, TX")
        }
    }
}

```

```

struct ContentView: View {

    @State private var customer: Customer?

    var body: some View {
        VStack {

            Button("Show Sheet for Customer") {
                customer = Customer(name: "John Doe")
            }

            .sheet(item: $customer, content: { customer in
                CustomerDetailScreen(customer: customer)
            })

        }
    }
}

```

I know what you are thinking. Can I do sheet on top of sheet on top of sheet? Yes you can but please don't. This makes the user interface extremely hard to use and remember that your user also needs to dismiss all the sheets to get back to the underlying screen.

You can also control the size of the displayed sheet. This is done by adjusting the presentationDetents view modifier. You can choose from medium, large and even custom.

```

.sheet(isPresented: $showSheet, content: {
    Text("Sheet")
    .presentationDetents([.fraction(0.25), .medium, .large])
})

```

Another common requirement is to display different sheets from the same screen. Consider a scenario, where you can tap the "Add" button in the NavigationBar to open a sheet to add a new customer but you can also press and hold on an existing customer from a list to view different options.

One common approach is to use separate individual properties for each state associated with the view. Depending on the scenario, this approach is fine to start with but can quickly become out of hands when more sheets can be displayed from a single screen.

Take a look at the following implementation, where a screen manages five different state variables one for each sheet.

```

struct MultipleSheetsView: View {

    @State private var showAddCustomerSheet = false
    @State private var selectedCustomer: Customer?
    @State private var showLegalNoticeSheet = false
    @State private var showEditCustomerSheet = false
    @State private var showDeleteCustomerSheet = false

    var body: some View {

```

```

List(customers) { customer in
    Text(customer.name)
        .onTapGesture {
            selectedCustomer = customer
        }
}.toolbar {
    ToolbarItem(placement: .topBarTrailing) {
        Button("Add Customer") {
            showAddCustomerSheet = true
        }
    }
}
.sheet(isPresented: $showAddCustomerSheet) {
    Text("ShowAddCustomerSheet")
}
.sheet(item: $selectedCustomer) { customer in
    Text(customer.name)
}
.sheet(isPresented: $showLegalNoticeSheet) {
    Text("Show Legal Notice")
}
.sheet(isPresented: $showEditCustomerSheet) {
    Text("Edit Customer Sheet")
}
}
}

```

This works but it is not an ideal solution. Instead of managing 5 individual properties, we can group them in a single enum. Here is a simple enum, which group all the events together.

```

private enum Sheet: Identifiable {
    case addCustomer
    case customerDetails(Customer)

    var id: String {
        switch self {
            case .addCustomer:
                return "addCustomer"
            case .customerDetails(let customer):
                return customer.id.uuidString
        }
    }
}

```

Once the enum is declared an instance can be used directly inside the view. This instance is declared with @State property wrapper, as it will help trigger the view rendering once it has been assigned. The implementation is shown below:

```

@State private var activeSheet: Sheet?

var body: some View {
    List(customers) { customer in
        Text(customer.name)
            .onTapGesture {
                activeSheet = .customerDetails(customer)
            }
    }
}

```

```

        }
    }.toolbar {
        ToolbarItem(placement: .topBarTrailing) {
            Button("Add Customer") {
                activeSheet = .addCustomer
            }
        }
    }
}
.sheet(item: $activeSheet) { activeSheet in
    switch activeSheet {
        case .addCustomer:
            Text("Add Customer")
        case .customerDetails(let customer):
            Text("Customer Details \(customer.name)")
    }
}
}

```

Instead of using multiple sheet modifiers, we handled all the different cases with a single enum data type. This enum-based approach to displaying sheets can be adopted by a large number of apps, and in most cases, it will be sufficient to meet their needs.

But sometimes, we may want to handle sheets globally, in a central place. In the next section, we will handle the display of sheets in a single place and also implement an Environment value to provide a simple function to show the sheet.

GLOBAL SHEETS

The main idea behind global sheets is to have a single place where all the sheets in the app are defined. This central definition can be as simple as a straightforward enum, or even a hierarchical or nested enum. I recommend starting simple, as a basic enum is usually sufficient. Global sheets rely on a single sheet view modifier attached to the root view of the app. One limitation of this approach is that you cannot display a sheet on top of another sheet, since only one sheet modifier is used.

Displaying a sheet on top of another sheet is generally discouraged. However, if your app requires this behavior, you can skip the global sheets pattern for those specific cases and use regular sheets instead.

We will start by defining our SheetAction.

```

struct SheetAction {
    typealias Action = (Sheet) -> Void
    let action: Action

    func callAsFunction(_ sheet: Sheet) {
        action(sheet)
    }
}

```

SheetAction encapsulates a closure that accepts a Sheet enum as input. By exposing a callAsFunction method, it enables a clean, function-like syntax, allowing sheets to be presented from within SwiftUI views with a simple function call.

Our next step is to extend the EnvironmentValues with the new showSheet modifier. This is implemented below:

```
extension EnvironmentValues {  
    @Entry var showSheet = SheetAction { _ in }  
}
```

The @Entry macro has made our life easier and saved us from writing several additional lines of code. Now all we need to do is to inject the showSheet Environment value and the sheet modifier to the root view of the app. The preview code is shown below:

```
#Preview {  
  
    @Previewable @State var selectedSheet: Sheet? = nil  
  
    ContentView()  
        .environment(\.showSheet, SheetAction(action: { sheet in  
            selectedSheet = sheet  
        })).sheet(item: $selectedSheet) { sheet in  
            sheet.destination  
        }  
}
```

And here is the code you will implement in the App file of your iOS app.

```
@main  
struct GlobalSheetsApp: App {  
  
    @State var selectedSheet: Sheet? = nil  
  
    var body: some Scene {  
        WindowGroup {  
            ContentView()  
                .environment(\.showSheet, SheetAction(action: { sheet in  
                    selectedSheet = sheet  
                })).sheet(item: $selectedSheet) { sheet in  
                    sheet.destination  
                }  
        }  
    }  
}
```

In the code above, the sheet depends on the selectedSheet binding. This means when the binding changes, the sheet is triggered. The showSheet environment value is responsible for firing the SheetAction. The usage of the showSheet function is shown below:

```
struct ContentView: View {  
  
    @Environment(\.showSheet) private var showSheet  
  
    var body: some View {  
        VStack {  
            Button("Show Settings") {  
                showSheet(.settings)  
            }  
        }  
    }  
}
```

```

        }

        Button("Show Contact") {
            showSheet(.contact("John Doe"))
        }
    }
    .padding()
}

```

Nice and simple! You can even extend the `showSheet` function to support an `onDismiss` closure or other configuration options.

As always, start simple. You may not need a global sheets pattern at all, especially if your app has only a few sheets. When you reach a tipping point and need a centralized approach, consider adopting the global sheets pattern.

TOASTS

Toasts are special messages that briefly appear on the screen to provide feedback or notify the user about an action. Unlike alerts or modals, they don't interrupt the user's workflow, they automatically disappear after a short time.

When designing a view-based reusable API, I like to start from the end by first imagining what the API should look like in use. Once the usage feels clean and intuitive, I work backwards to implement the supporting code.

For global toast messages, we'll follow the same approach as global sheets, keeping the API consistent and easy to adopt. The usage example is shown below:

```

struct ContentView: View {
    @Environment(\.showToast) private var showToast

    var body: some View {
        VStack {

            Button("Success") {
                showToast(.success("Success"), placement: .bottom)
            }

            Button("Error") {
                showToast(.error("Operation failed."), placement: .top)
            }

            Button("Info") {
                showToast(.info("Informational message"), placement: .top,
disappearAfter: 5.0)
            }

        }
    }
}

```

```
}
```

The `showToast` function is exposed via an environment value and is responsible for presenting toast messages. It accepts an enum that specifies the message type—such as success, error, or info. You can also customize where the toast appears and how long it stays visible. Both `placement` and `disappearAfter` have sensible defaults, so you only need to provide them when you want to change the behavior.

Now that we’ve sketched the public API, let’s implement it from the inside out. We’ll start by defining `ToastType`, which captures the different kinds of toasts and provides the data needed to render them: background color, icon, and message.

```
enum ToastType {
    case success(String)
    case error(String)
    case info(String)

    var backgroundColor: Color {
        // return color here
    }

    var icon: Image {
        // return correct icon here
    }

    var message: String {
        switch self {
            case .success(let msg), .error(let msg), .info(let msg):
                return msg
        }
    }
}
```

Next, we’ll implement a `ToastView`, the actual view that appears on screen when a toast is shown. This is where we can get creative and focus on design details—making the toast visually appealing, polished, and pleasant to interact with.

```
struct ToastView: View {

    let type: ToastType

    var body: some View {
        HStack(alignment: .center, spacing: 10) {
            type.icon
                .foregroundColor(.white)
            Text(type.message)
                .foregroundColor(.white)
                .font(.subheadline)
                .multilineTextAlignment(.leading)
        }
        .padding()
        .background(type.backgroundColor)
        .cornerRadius(12)
        .shadow(radius: 4)
        .padding(.horizontal, 16)
    }
}
```

```

    }
}

```

Next, we will add the `ShowToastAction`. This is where we define the action and its `callAsFunction` convenience. The pattern should feel familiar because we used the same approach for the `showSheet` environment value earlier.

```

struct ShowToastAction {
    typealias Action = (ToastType, ToastPlacement, TimeInterval) -> Void
    let action: Action

    func callAsFunction(_ type: ToastType, placement: ToastPlacement = .top,
disappearAfter: TimeInterval = 2.0) {
        action(type, placement, disappearAfter)
    }
}

extension EnvironmentValues {
    @Entry var showToast = ShowToastAction(action: { _, _, _ in })
}

```

Next, we will do one thing differently for `showToast`: create a custom view modifier called `withToast`. This lets us keep all toast handling inside the modifier, including routing different toast types, presenting the view, and scheduling or canceling dismissal tasks. The modifier becomes the single place that wires everything together.

You can attach `withToast` to the root view of the app to provide toast services to all descendant views. Views presented in sheets are not covered automatically, so you will need to apply `withToast` to the sheet's content as well.

The implementation below shows the `withToast` modifier:

```

struct ToastModifier: ViewModifier {

    @State private var type: ToastType?
    @State private var placement: ToastPlacement = .top
    @State private var disappearAfter: TimeInterval = 2.0
    @State private var dismissTask: DispatchWorkItem?

    func body(content: Content) -> some View {
        content
        .environment(\.showToast, ShowToastAction(action: { type,
placement, disappearAfter in
            withAnimation(.easeInOut) {
                self.type = type
                self.placement = placement
                self.disappearAfter = disappearAfter
            }

            // cancel previous dismissal task if any
            dismissTask?.cancel()

            // schedule a new dismissal

```

```

        let task = DispatchWorkItem {
            withAnimation(.easeInOut) {
                self.type = nil
            }
        }

        self.dismissTask = task
        DispatchQueue.main.asyncAfter(deadline: .now() +
disappearAfter, execute: task)

    )))
    .overlay(alignment: placement == .top ? .top: .bottom) {
        if let type {
            ToastView(type: type)
                .transition(.move(edge: placement
== .top ? .top: .bottom).combined(with: .opacity))
                .padding(.top, 50)
        }
    }
}

```

There are a few key things happening inside the modifier. First, if a toast is triggered multiple times, the existing task is canceled and replaced with a new one. This ensures only the most recent toast is scheduled to appear. The task itself is managed using `DispatchQueue`, which lets us specify the delay in seconds before it is executed.

The `overlay` is responsible for rendering the actual `ToastView`. It depends on the `type` state variable, so whenever `type` changes, the `ToastView` is refreshed to match the new toast. Also, after the specified time interval, `type` is set to `nil`, which hides the `ToastView`.

The final step is to add a `View` extension that wraps the modifier. This keeps your code clean and makes attaching toast behavior as simple as calling a single function on any view.

```

extension View {
    func withToast() -> some View {
        modifier(ToastModifier())
    }
}

```

That's it! I know it was a bit of work, but that's often the case when building a reusable API designed to be used across your entire app. The effort pays off in the long run by giving you a clean, consistent, and easy-to-use way to show toast messages anywhere in your project.

CONCLUSION

Sheets and toasts may seem like small pieces of the SwiftUI toolkit, but together they play a critical role in shaping the user experience of your app. Sheets give you a structured way to present focused content, from creating and editing items to handling entire workflows, while toasts provide lightweight, non-intrusive feedback that keeps users informed without breaking their flow.

In this chapter, we explored how to use SwiftUI's built-in sheet modifiers, how to simplify multiple sheets with an enum-based approach, and when a global sheets pattern can make sense. We also walked through the design and implementation of a reusable toast system, giving your app a flexible, consistent way to communicate success, errors, and other messages.

As with most patterns in SwiftUI, the key takeaway is balance: start simple, and only adopt more advanced patterns like global sheets or global toasts when your app truly needs them. By mastering these techniques, you'll be equipped to create SwiftUI applications that feel polished, intuitive, and enjoyable to use.

VALIDATION

There is a well known saying in software development: garbage in, garbage out. If you allow bad input into your app, bad data will eventually find its way into your database. And once that happens, cleaning it up later is almost always harder than doing the right thing upfront.

Validation is often treated as an afterthought. We focus on building features, polishing the UI, and getting things shipped. But validation is one of those quiet responsibilities that has a huge impact on the long term health of your application. When it is done well, users rarely notice it. When it is missing, the problems show up everywhere.

In SwiftUI, validation fits naturally into the way the framework works. Because views are driven by state, validation can be expressed clearly and predictably, without turning your code into a maze of flags and conditionals.

In this chapter, we will look at practical ways to perform validation in SwiftUI. We will start with simple, view level checks and gradually move toward more reusable and scalable approaches. The goal is to help you write validation code that is easy to reason about, easy to maintain, and does its job quietly in the background while your app stays focused on delivering a great experience.

SIMPLE VALIDATION

Validation often depends on the complexity of the screen, and in many cases, you can reduce the need for extensive validation by using the appropriate user interface elements. Choosing the right UI components not only minimizes extra coding but also makes the interface more intuitive for users.

Consider an example where the user needs to input their state of residence. A quick way to build this screen might be to use a `TextField` for state input, as shown below:

```
struct RegisterLicenseScreen: View {  
  
    @State private var name: String = ""  
    @State private var state: String = ""  
  
    var body: some View {  
        Form {  
            TextField("Name", text: $name)  
            TextField("State", text: $state)  
        }  
    }  
}
```

Although the screen is functional, it allows the user to enter anything for the state. This means that someone from Texas might input "Texas," "Taxes," "TX," "tx," or other variations. These inconsistent entries are then stored in the database, making it difficult to query users based on state due to the lack of standardization.

While you could add validation to restrict the input to “Texas” only, a simpler and more effective solution is to use a more appropriate user interface element—such as a picker—to ensure consistent data entry. This is demonstrated below:

```
struct RegisterLicenseScreen: View {

    @State private var name: String = ""

    let states = // array of US states

    @State private var state: String = ""

    var body: some View {
        Form {
            TextField("Name", text: $name)
            Picker("State", selection: $state) {
                ForEach(states, id: \.self) {
                    Text($0)
                }
            }
        }
    }
}
```

As you can see, we’ve replaced the `TextField` for the state with a `Picker`. The `Picker` prevents users from entering custom input—they must choose from the predefined options. This not only improves the user experience but also eliminates the possibility of incorrect or inconsistent data entry, which can lead to issues later when querying or analyzing the data.

It’s also important to keep things simple. I’ve always believed that unnecessary complexity slows progress—and in some cases, can even kill a project. When designing user interfaces, always ask yourself: what’s the minimum amount of code needed to clearly communicate that something is wrong on the screen?

In the example below, we don’t even display any explicit validation message, yet the user clearly understands why the login button is disabled.

```
struct LoginScreen: View {

    @State private var username: String = ""
    @State private var password: String = ""

    private var isValid: Bool {
        !username.isEmptyOrWhiteSpace && !password.isEmptyOrWhiteSpace
    }

    var body: some View {
        Form {
            TextField("User name", text: $username)
            TextField("Password", text: $password)
            Button("Login") {

```

```

        }.disabled(!isFormValid)
    }
}

```

The `isFormValid` property represents the validation logic of the view. It leverages the `isEmptyOrWhiteSpace` property defined in a `String` extension. This setup also provides an opportunity to write unit tests specifically for the validation logic, ensuring it behaves as expected.

```

import Testing

struct StringIsEmptyOrWhiteSpaceTests {

    @Test
    func returnsTrue_forEmptyString() {
        let value = ""
        #expect(value.isEmptyOrWhiteSpace)
    }

    @Test
    func returnsTrue_forWhitespaceOnly() {
        let value = "   "
        #expect(value.isEmptyOrWhiteSpace)
    }

    @Test
    func returnsTrue_forNewlineOnly() {
        let value = "\n\t "
        #expect(value.isEmptyOrWhiteSpace)
    }

    // other tests
}

```

The login and registration screens can usually benefit from just using the simple validation. But as the screen complexity grows, you will need to adapt different techniques. In the next section, you will learn how to display errors on the screen.

VALIDATION SUMMARY

Back when I was working with ASP.NET, there was a control called `ValidationSummary`. It was ideal for situations where you needed to present a list of failed validation messages to the user. This concept can be easily implemented in SwiftUI.

You can start by adding a `validate` function inside the view. This function will perform validation on the user input and populate the `validationErrors` state accordingly.

```

private func validate() -> Bool {
    validationErrors = [] // Clear previous errors

    // Validate each field
    if fullName.trimmingCharacters(in: .whitespaces).isEmpty {
        validationErrors.append("Full Name is required.")
    }
}

```

```

    }
    if street.trimmingCharacters(in: .whitespaces).isEmpty {
        validationErrors.append("Street is required.")
    }
    if city.trimmingCharacters(in: .whitespaces).isEmpty {
        validationErrors.append("City is required.")
    }
    if state.isEmpty {
        validationErrors.append("State is required.")
    }

    return validationErrors.isEmpty
}

```

You can trigger the `validate` function at the appropriate time—typically when the user attempts to submit a form or perform an action. All validation errors are collected in the `validationErrors` array and passed to a custom `ValidationSummary` view for display. This is demonstrated in the implementation below:

```

struct ValidationSummary: View {
    let validationErrors: [String]

    var body: some View {
        Section(header: Text("Validation Summary").foregroundColor(.red)) {
            ForEach(validationErrors, id: \.self) { error in
                Text(error)
                    .foregroundColor(.red)
            }
        }
    }
}

```

`ValidationSummary` is an excellent choice when you want to present a concise list of validation messages to the user. It promotes reusability, keeps your view logic clean, and separates concerns by offloading the responsibility of error display to a dedicated view.

This pattern is particularly useful in forms with multiple fields, where inline validation can clutter the UI. By grouping all validation feedback in one place, you improve the user experience while maintaining clear and testable code.

Whether you're validating a simple form or a complex multi-step process, combining a `validate` function with a `ValidationSummary` view gives you a scalable and maintainable approach to form validation in SwiftUI.

VALIDATION USING PROPERTY WRAPPERS

I always find it interesting how knowledge of different frameworks and languages can help you come up with better designs. The ASP.NET MVC framework exposes several validation attributes that allow you to validate incoming HTTP request data.

One of the usage is shown below:

```
public class Movie
{
    public int Id { get; set; }

    [Required]
    [StringLength(100)]
    public string Title { get; set; } = null!;

    [Required]
    [StringLength(1000)]
    public string Description { get; set; } = null!;

    [Range(0, 999.99)]
    public decimal Price { get; set; }
}
```

Required, **StringLength**, and **Range** are all built-in validation attributes provided by the ASP.NET framework. Properties decorated with these attributes must conform to their respective validation rules. We can implement similar techniques in Swift using custom property wrappers. One of the main benefits of this approach is that it allows you to easily extend your library of custom validators. For instance, if you later need validators for patterns like regular expressions or date ranges, you can implement and integrate them with minimal effort.

Because the validation logic is encapsulated within the property wrapper itself, it also becomes straightforward to write unit tests for your validators. This gives you peace of mind, knowing that your validation logic works as intended.

Let's take a look at an example that demonstrates how to validate a simple form.

```
struct MovieLoginScreen: View {

    @State private var username: String = ""
    @State private var email: String = ""
    @State private var password: String = ""

    var body: some View {

        Form {

            TextField("Username", text: $username)
            TextField("Email", text: $email)
            SecureField("Password", text: $password)

            Section {
                Button("Login") {
                    // Handle login action
                }
            }
        }
    }
}
```

```
}
```

The `MovieLoginScreen` currently defines `username`, `email`, and `password` as individual `@State` variables within the view. The first step toward improving this design is to extract these properties into their own struct. This approach makes it easier to apply custom property wrappers to each field, allowing for a cleaner, more modular validation strategy.

```
extension MovieLoginScreen {  
    struct MovieLoginState {  
        var username: String = ""  
        var password: String = ""  
        var email: String = ""  
  
        func validate() -> [String] {  
            // validate the fields  
            return []  
        }  
    }  
}
```

`MovieLoginState` is implemented as a simple struct that wraps the `username`, `password`, and `email` properties. Some might argue that `MovieLoginState` qualifies as a view model—and if that mental model helps you, feel free to think of it that way. However, it's important to note that when most developers refer to a view model, they typically mean a class that conforms to `ObservableObject` (or uses the `@Observable` macro) and manages dependencies such as `HTTPClient` and other services.

In contrast, `MovieLoginState` is a value type—a plain struct—and functionally equivalent to defining `username`, `password`, and `email` as individual `@State` variables within the view.

Here is the implementation of the `@Required` property wrapper:

```
@propertyWrapper  
struct Required {  
    private var value: String  
    private let message: String  
  
    var wrappedValue: String {  
        get { value }  
        set { value = newValue }  
    }  
  
    var projectedValue: String? {  
        return value.isEmpty ? message : nil  
    }  
  
    init(wrappedValue: String, _ message: String) {  
        print(wrappedValue)  
        self.value = wrappedValue  
        self.message = message  
    }  
}
```

The `@Required` property wrapper is fairly straightforward. The `value` property holds the actual value to which the wrapper is applied, while the `message` property defines the error message that will be displayed to the user if validation fails.

Now that we've implemented the `@Required` wrapper, we can start using it in our application to validate form inputs more cleanly and declaratively.

```
struct MovieLoginState {  
  
    @Required("Username is required")  
    var username: String = ""  
    @Required("Password is required")  
    var password: String = ""  
  
    @Required("Email is required")  
    var email: String = ""  
  
    func validate() -> [String] {  
        var errors: [String] = []  
  
        if let usernameError = $username {  
            errors.append(usernameError)  
        }  
  
        if let passwordError = $password {  
            errors.append(passwordError)  
        }  
  
        if let emailError = $email {  
            errors.append(emailError)  
        }  
  
        return errors  
    }  
}
```

Each property that requires validation is decorated with the appropriate validation rule. At the moment, only the `@Required` property wrapper is implemented, but the system is designed to be easily extendable—allowing support for additional wrappers such as `@Regex`, `@DateRange`, and more.

The `validate()` function triggers the validation logic defined within each property wrapper and collects any resulting error messages into a custom `errors` array. After all validations have been executed, the function returns the array of errors for display in the UI.

Here's an example of how it's used:

```
struct MovieLoginScreen: View {  
  
    @State private var movieLoginState = MovieLoginState()  
    @State private var errors: [String] = []  
  
    var body: some View {
```

```

Form {
    TextField("Username", text: $movieLoginState.username)
    TextField("Email", text: $movieLoginState.email)
    SecureField("Password", text: $movieLoginState.password)

    Section {
        Button("Login") {
            errors = movieLoginState.validate()
            if errors.isEmpty {
                // perform login
            }
        }
    }
}
}
}

```

We've removed the individual `@State` properties and replaced them with a single instance of `MovieLoginState`. The `TextField` bindings have also been updated to reference the properties within `MovieLoginState`.

When the button is tapped, it triggers the `validate()` function on the `MovieLoginState` struct, which returns any validation errors. These errors can then be displayed using the `ValidationSummary` view.

One of the key advantages of this approach is its flexibility—it allows you to easily add new custom validation property wrappers as needed. For example, below is the implementation of the `@RegularExpression` property wrapper, which enables validation of values against specific patterns.

```

@propertyWrapper
struct RegularExpression {

    private var value: String
    private let pattern: String
    private let message: String

    var wrappedValue: String {
        get { value }
        set { value = newValue }
    }

    var projectedValue: String? {
        // Validate the value against the regular expression pattern
        if !matchesPattern(value) {
            return message
        }
        return nil
    }

    // Helper method to validate the value against the pattern
    private func matchesPattern(_ value: String) -> Bool {

```

```

        guard let regex = try? NSRegularExpression(pattern: pattern) else {
            return false // Invalid regex pattern
        }
        let range = NSRange(location: 0, length: value.utf16.count)
        return regex.firstMatch(in: value, options: [], range: range) != nil
    }

    init(wrappedValue: String, pattern: String, message: String) {
        self.value = wrappedValue
        self.pattern = pattern
        self.message = message
    }
}

```

This technique of using property wrappers makes it easy to expand your set of validators based on your specific needs. You can even take it a step further by creating a dedicated Swift package to host all your custom validators. This allows you to import the package into future projects and reuse your validation logic without duplicating code.

In this section, you learned how to implement custom property wrappers to seamlessly add validation to your SwiftUI applications. However, one limitation of this approach is that Swift only allows a single property wrapper per property. As a result, if you want to validate that a `username` is both non-empty and matches a specific pattern, you can't directly apply both `@Required` and `@RegularExpression` wrappers to the same property.

In the next section, we'll explore how to implement a `@Validate` property wrapper that enables the use of multiple validation rules on a single property—making your validation logic even more flexible and powerful.

IMPLEMENTING VALIDATE PROPERTY WRAPPER

Before starting the actual implementation of the `@Validate` property wrapper, let's first visualize how it will be used in code. I often use this technique of reverse-engineering the API design based on the calling site.

Here are a few examples of how the `@Validate` property wrapper can be used:

```

@Validate(.required("Email is required"))
var email: String = ""

@Validate(.required("Email is required"), .regularExpression(.email, "Email is
in incorrect format.))
var email: String = ""

```

In the first example, the `email` property uses a single validation rule: `.required`. In the second example, the `email` property is not only required but must also conform to a regular expression pattern. This design allows you to pass as many validation rules as needed, giving you the flexibility to compose validations based on your specific requirements.

Moreover, you can build an extensive library of reusable validation rules that can be shared across this project and future ones.

Now that we've explored the calling site, let's define the `ValidationRule` enum. Below is an example that includes two basic rules:

```
enum ValidationRule {
    case required(String)
    case regularExpression(RegexPattern, String) // pattern and the message
}
```

If you need additional rules in the future, you can simply extend the `ValidationRule` enum by adding new cases. Next you will implement the `Validate` property wrapper.

```
@propertyWrapper
struct Validate {
    private var value: String
    private let rules: [ValidationRule]
    private(set) var errorMessages: [String] = [] // Holds validation errors

    var wrappedValue: String {
        get { value }
        set {
            value = newValue
            validate()
        }
    }

    var projectedValue: [String] { // Projected value for accessing errors
        errorMessages
    }

    init(wrappedValue: String, _ rules: ValidationRule...) {
        self.value = wrappedValue
        self.rules = rules
        validate()
    }

    // MARK: - Validation Logic
    private mutating func validate() {
        errorMessages.removeAll()

        for rule in rules {
            switch rule {
                case .required(let message):
                    if
value.trimmingCharacters(in: .whitespacesAndNewlines).isEmpty {
                        errorMessages.append(message)
                    }
                case .regularExpression(let pattern, let message):
                    let range = NSRange(location: 0, length:
value.utf16.count)

                    if pattern.regex?.firstMatch(in: value, options: [],
range: range) == nil {
```

```
}  
}  
}  
}  
} } } } }
```

The `@Validate` property wrapper allows you to pass in an array of validation rules. In the `validate` function, each rule is evaluated to perform custom validation. The validation errors are stored in the `errorMessagees` array and returned as the result of the `validate` function.

You can even utilize your `ValidationSummary` view to display error messages to the user

Now let's put our new `@Validate` property wrapper to work in a real screen.

We are going to start by creating some state that represents the data we want to collect. In this case, it is a simple `Customer` type with an `email` property. Notice how we decorate the property with `@validate` and attach multiple rules to it.

```
struct Customer {  
    @Validate(  
        .required("Email is required."),  
        .regularExpression(.email, "Email should be in correct format.")  
    )  
    var email: String = ""  
}
```

Next, we set up the bindings in our view and display validation messages. For showing errors, I created a view modifier called `withValidation`. Its job is simple: show the error message right next to the control being validated.

And just like everything else in SwiftUI, the UI part is flexible. This is only one way to display validation. If you prefer a message below the field, a red border, an icon, or a summary at the top of the screen, go for it.

```
struct ContentView: View {

    @State private var customer = Customer()

    var body: some View {
        VStack {
            TextField("Email", text: $customer.email)
                .textFieldStyle(.roundedBorder)
                .withValidation(customer.$email)
        }
        .padding()
    }
}
```

The result is exactly what you would expect. As soon as the user leaves the field empty or types an invalid email, the message shows up right next to the text field. Clear, immediate feedback, without cluttering your view with validation logic.

The best part is that this pattern scales nicely. Once your validation rules and helpers are in place, you can reuse them across screens, across features, and even across projects. At that point, you are not just validating input anymore. You are building a small validation framework you can drop into any app when you need it

CONCLUSION

Validation is one of those things that quietly makes or breaks an app. When it is done well, users barely notice it. When it is done poorly, frustration shows up immediately. In this chapter, we walked through several ways to approach validation in SwiftUI, starting simple and gradually moving toward more reusable and scalable solutions.

We began with the most overlooked form of validation: the UI itself. Choosing the right control, like using a `Picker` instead of a `TextField`, can eliminate entire classes of invalid input before the user even has a chance to make a mistake. That kind of decision pays off immediately and keeps your code simpler.

From there, we added lightweight inline validation using computed properties such as `isFormValid`. Disabling actions until the form is complete is a small change, but it sets clear expectations and helps users move forward with confidence instead of being surprised by errors later.

As forms grew more complex, we introduced the `validationSummary` view, inspired by ASP.NET. Centralizing validation messages in one place keeps your UI clean and makes it easier to reason about what is wrong and why. Your views stay focused on layout, and your validation logic stays organized.

We then pushed validation closer to the data by using property wrappers like `@Required` and `@RegularExpression`. This shift is important. Validation starts to feel like a first class concern instead of something sprinkled throughout the view code.

Finally, we brought everything together with the `@Validate` property wrapper. This approach gives you flexibility without sacrificing readability. Multiple rules can be applied to a single property, your intent is clear, and the result is code that is easier to test and easier to maintain.

There is no single correct way to handle validation. Simple screens can get by with smart UI choices and a few inline checks. More complex flows benefit from structured and reusable validation systems. What matters is that you now have a solid set of tools and the judgment to choose the right one for the job.

Most importantly, validation is not about punishing users for getting things wrong. It is about guiding them toward success. When you get that balance right, users have a smoother experience and you avoid dealing with bad data later. Everyone wins.

NAVIGATION

When SwiftUI launched in 2019, it introduced a simple navigation system built around `NavigationView`. While `NavigationView` handled basic navigation well, it quickly showed limitations in more advanced scenarios—such as dynamic or programmatic routing, managing centralized routes, and handling complex navigation flows.

To address these issues and provide greater flexibility for developers, `NavigationStack` was introduced in the iOS 16 operating system. `NavigationStack` offers a modern, data-driven approach to navigation, enabling programmatic routing, centralized route management, and dynamic navigation flows that were difficult to achieve with `NavigationView`.

In this chapter, you will learn about different Navigation patterns in SwiftUI and how you can adopt those patterns to simplify your app navigation.

BASIC LIST NAVIGATION

One of the most common patterns in a SwiftUI app is navigating from a list screen to a detail screen. SwiftUI provides multiple ways to accomplish this, but the most common approach is to use a `NavigationLink` inside a `NavigationStack`.

With this technique, the `label` defines the view presented in the list, while the `destination` specifies the detail view to display when the link is selected. The example below demonstrates this pattern:

```
struct CustomerListScreen: View {  
    let customers = [Customer(name: "John Doe"), Customer(name: "Mary Doe")]  
  
    var body: some View {  
        NavigationStack {  
            List(customers) { customer in  
                NavigationLink {  
                    CustomerDetailScreen(customer: customer)  
                } label: {  
                    Text(customer.name)  
                }  
            }  
        }  
    }  
}
```

One important detail to keep in mind is that the initializer of `CustomerDetailScreen` is invoked for every customer in the list. In most cases this is not a concern. However, if you place expensive operations inside a view initializer, which is discouraged in SwiftUI, it can negatively impact performance.

One way to avoid repeated initializer calls is to use the `navigationDestination` view modifier to handle navigation. This modifier was introduced in iOS 16 together with `NavigationStack`. Using this approach, the previous implementation can be rewritten as follows:

```
struct CustomerListScreen: View {

    let customers = [Customer(name: "John Doe"), Customer(name: "Mary Doe")]

    var body: some View {
        NavigationStack {
            List(customers) { customer in
                NavigationLink(value: customer) {
                    Text(customer.name)
                }
            }.navigationDestination(for: Customer.self) { customer in
                CustomerDetailScreen(customer: customer)
            }
        }
    }
}
```

Instead of using the `NavigationLink` initializer that takes a `destination` and `label`, we use the version that accepts a `value` conforming to the `Hashable` protocol along with a `label`. When the user taps the `NavigationLink`, the corresponding `navigationDestination` for that value is triggered. This allows a single view to define multiple `navigationDestination` modifiers, each handling a different type.

When a `navigationDestination` is triggered, the associated value is passed to the destination view, in this case `CustomerDetailScreen`.

So, which method should you use? For a common list-to-detail navigation scenario, I prefer the first approach because it is simpler and easier to understand. This does not necessarily mean the first approach is better than the second. It is simply a matter of choosing the option that feels more straightforward for the situation.

DECOUPLE NAVIGATION FROM VIEWS

In the previous implementations we used the `List` view to display customers. But what happens when you want to build reusable views? How can you decouple navigation from child views. For example, we are currently using a `List` to display customers, but later we might extract that logic into a `CustomerListView`. How can we avoid coupling the navigation to a `CustomerListView`.

The implementation is quite simple, we just have to remember to use `navigationDestination` on the parent level and not on the child as shown below:

```
struct DecoupleViewsFromNavigation: View {

    let customers = [Customer(name: "John Doe"), Customer(name: "Mary Doe")]

    var body: some View {
        NavigationStack {
```

```

        CustomerListView(customers: customers)
            .navigationDestination(for: Customer.self) { customer in
                CustomerDetailScreen(customer: customer)
            }
        }
    }
}

```

```

struct CustomerListView: View {
    let customers: [Customer]

    var body: some View {
        List(customers) { customer in
            NavigationLink(value: customer) {
                Text(customer.name)
            }
        }
    }
}

```

With this refactor, navigation is decoupled from `CustomerListView`. The view now focuses only on displaying the list of customers, while the parent controls navigation. This lets different screens reuse `CustomerListView` and provide their own destination, making the component flexible and independent of any specific navigation flow.

In this section we covered the basics of `List` based navigation. Sometimes navigation depends on the result of an asynchronous event, such as a network response or a sign in action. In the next section we will explore dynamic and programmatic navigation in SwiftUI.

DYNAMIC/PROGRAMMATIC NAVIGATION IN SWIFTUI

`NavigationStack` supports programmatic navigation. You can change the current screen by adding or removing values in its path. This is especially useful when the next step depends on the result of a time consuming task, such as an asynchronous operation.

Start by defining your routes as a `Hashable` enum. The implementation below shows this approach.

```

enum Route: Hashable {
    case doctor
    case patient
}

```

For larger apps you can organize routes with nested enums to create a clear hierarchy, such as screens grouped under a selected category. We will cover that pattern soon.

In your view, keep routes as an array and bind it to `NavigationStack` using the `path` property. To navigate to the Doctor screen after an asynchronous result, append the appropriate case to your routes enum. SwiftUI will match that value and present the destination you defined in `navigationDestination`.

The implementation below shows this approach.

```
struct ProgrammaticNavigationDoctorsAndPatientsScreen: View {

    @State private var routes: [Route] = []

    var body: some View {
        NavigationStack(path: $routes) {
            VStack {
                Button("Doctors") {
                    // perform some time consuming operation
                    Task {
                        try! await Task.sleep(for: .seconds(2.0))
                        routes.append(.doctor)
                    }
                }

                Button("Patients") {
                    // perform some time consuming operation
                    Task {
                        try! await Task.sleep(for: .seconds(2.0))
                        routes.append(.patient)
                    }
                }
            }.navigationDestination(for: Route.self) { route in
                switch route {
                case .doctor:
                    Text("Doctor Screen")
                case .patient:
                    Text("Patient Screen")
                }
            }
        }
    }
}
```

The `routes` array is local state inside a single view. Once you move to a different screen, that view is no longer active, so you cannot access `routes` to drive navigation.

A simple solution is to expose routing through the environment. Inject a router at the app root, and every screen can reach it. To keep views reusable, read the environment only in screen level views, not in small subviews.

The first step is to create a `Router` type that you can inject. You can implement it as an `ObservableObject` or with the `@Observable` macro. The implementation of `Router` is shown below.

```
@Observable
class Router {
    var routes: [Route] = []
}
```

Inject `Router` at the root of your app so it is available to all views through the environment. Access it in a screen with `@Environment(Router.self)`.

To provide a path binding to `NavigationStack`, first create a local bindable projection of the router using `@Bindable`, then pass `$router.path` to the stack's `path` parameter. The implementation below shows this setup.

```
struct ProgrammaticNavigationDoctorsUsingEnvironment: View {
    @Environment(Router.self) private var router

    var body: some View {
        @Bindable var router = router

        NavigationStack(path: $router.routes) {
            VStack {
                Button("Doctors") {
                    // perform some time consuming operation
                    Task {
                        try! await Task.sleep(for: .seconds(2.0))
                        router.routes.append(.doctor)
                    }
                }

                Button("Patients") {
                    // perform some time consuming operation
                    Task {
                        try! await Task.sleep(for: .seconds(2.0))
                        router.routes.append(.patient)
                    }
                }
            }.navigationDestination(for: Route.self) { route in
                switch route {
                case .doctor:
                    Text("Doctor Screen")
                case .patient:
                    Text("Patient Screen")
                }
            }
        }
    }
}
```

Since `Router` is in the environment, other views especially container views can access it and append routes, which triggers navigation.

If your routes need more hierarchy, introduce nested enums. For example, model a top level `Route` that contains cases like `doctor(DoctorRoute)` or `patient(PatientRoute)`. The implementation below demonstrates this pattern.

```
enum Route: Hashable {
    case doctor(DoctorRoute)
    case patient(PatientRoute)
```

```

}

enum DoctorRoute: Hashable {
    case list
    case details(UUID)
    case schedule(doctorId: UUID, date: Date)
}

enum PatientRoute: Hashable {
    case list
    case details(id: UUID)
}

```

Each route can provide its own destination view. Add a `destination` computed property to `Route`, `DoctorRoute`, and `PatientRoute`, and mark it with `@ViewBuilder` so each case returns the correct SwiftUI view.

This keeps navigation logic close to the route definitions and keeps container views simple.

```

enum Route: Hashable {
    case doctor(DoctorRoute)
    case patient(PatientRoute)

    @ViewBuilder
    var destination: some View {
        switch self {
            case .doctor(let doctorRoute):
                doctorRoute.destination
            case .patient(let patientRoute):
                patientRoute.destination
        }
    }
}

enum DoctorRoute: Hashable {
    case list
    case details(UUID)
    case schedule(doctorID: UUID, date: Date)

    @ViewBuilder
    var destination: some View {
        switch self {
            case .list:
                DoctorListScreen()
            case .details(let id):
                DoctorDetailScreen(doctorID: id)
            case .schedule(let doctorID, let date):
                DoctorScheduleScreen(doctorID: doctorID, date: date)
        }
    }
}

```

Some developers prefer to have the `Router` enum conform to the `View` protocol instead of exposing a `destination` property. Both techniques are correct; in the end, it is simply a matter of taste. Personally I like to be explicit, so I always add a `destination` property to `Router`.

This design is easy to extend. When you need a new screen, add a new case to the `Route` enumeration and provide its `destination`. The rest of the code stays the same. If you use nested enums like `DoctorRoute` or `PatientRoute`, follow the same pattern.

We also need to update the `navigationDestination` view modifier to use the new destination property of the route. This is shown in the implementation below:

```
.navigationDestination(for: Route.self) { route in
    route.destination
}
```

Now you can invoke your routes using the following code:

```
Button("Doctors") {
    // perform some time consuming operation
    Task {
        try! await Task.sleep(for: .seconds(2.0))
        router.routes.append(.doctor(.list))
    }
}
```

The router approach is perfectly fine and is the most common pattern used among iOS developers. However, I am always exploring alternative approaches, often inspired by similar frameworks. In the next section, we will look at how navigation in React can help us design a different kind of system in SwiftUI.

IMPLEMENTING NAVIGATION HOOK USING ENVIRONMENT VALUES

SwiftUI, React, Flutter, and Jetpack Compose are similar in nature, so it is natural to look for better or different solutions in other frameworks. For example, in React you can use the `navigate` hook a hook is just a function and perform navigation like this:

```
navigate("/session-timed-out");
```

I find this a clear and concise way to navigate between pages. How can we bring the same concept to SwiftUI?

SwiftUI does not use the term hook, but as noted above a hook is just a function, and we can expose one in any way we choose. One technique is to use environment values to expose a `navigate` function so it is available to all views.

We will start by creating a `NavigateAction`.

```
struct NavigateAction {
    typealias Action = (Route) -> ()
}
```

```

    let action: Action
    func callAsFunction(_ route: Route) {
        action(route)
    }
}

```

`NavigateAction` contains an `action`, which is a closure that takes a `Route` and returns nothing. We also implement `callAsFunction`, which enables function like calls on an instance. Instead of `navigate.action(.home)`, you can write `navigate(.home)`.

Next, add the new `navigate` value to the environment using the `@Entry` macro.

```

extension EnvironmentValues {
    @Entry var navigate = NavigateAction { _ in }
}

```

The `@Entry` macro lets us skip writing repetitive code when creating a new environment value. Without `@Entry`, we would have written something like the following:

```

extension EnvironmentValues {
    var navigate: (NavigateAction) {
        get { self[NavigateEnvironmentKey.self] }
        set { self[NavigateEnvironmentKey.self] = newValue }
    }
}

```

You can right click a macro and select **Expand Macro** to see the generated code. The next step is to inject the environment value at the root view and handle the routes.

The implementation is shown below:

```

struct ProgrammaticNavigationUsingNavigateHook: View {

    @State private var routes: [Route] = []

    var body: some View {
        NavigationStack(path: $routes) {
            HomeScreen()
                .navigationDestination(for: Route.self) { route in
                    route.destination
                }
        }.environment(\.navigate, NavigateAction(action: { route in
            routes.append(route)
        })))
    }
}

```

The `navigate` environment value is injected into the `NavigationStack`, which serves as the root of the application. When the `navigate` function is called, the `NavigateAction` receives the route and appends it to the `routes` array. This updates the binding passed to `NavigationStack` and triggers the `navigationDestination` modifier, which returns the appropriate view.

Since `navigate` is part of the environment, it is easily accessible inside any view. The example below shows it in use:

```
struct HomeScreen: View {  
    @Environment(\.navigate) private var navigate  
  
    var body: some View {  
        VStack {  
            Button("Doctor List Screen") {  
                navigate(.doctor(.list))  
            }  
  
            Button("Patient Detail Screen") {  
                navigate(.patient(.details(UUID())))  
            }  
        }  
    }  
}
```

Although this approach works for the current scenario, it does not work correctly if the application uses multiple tabs. Each tab must maintain its own navigation history. If your app has three tabs, you should use three `NavigationStack` instances, one per tab. This ensures that the history of one tab does not interfere with the others and that each tab works independently.

MULTI-TAB NAVIGATION IN SWIFTUI USING MULTIPLE ARRAYS

Multi tab apps introduce a new set of navigation problems. Each tab must maintain its own history. Our current approach does not support this because all routes are stored in a single `routes` array.

There are several ways to support multi tab navigation. A simple option is to give the `Router` a separate array for each tab. When the user is on a tab, append the new route to that tab's array. This keeps histories isolated and avoids cross tab interference.

We will start by creating our `Router`.

```
@MainActor  
@Observable  
class MultiTabRouter {  
    var patientRoutes: [PatientRoute] = []  
    var doctorRoutes: [DoctorRoute] = []  
}
```

In `MultiTabRouter` we define two arrays. The `patientRoutes` array stores routes related to patients, and the `doctorRoutes` array stores routes related to doctors.

Next we configure our tab views. Each tab hosts its own `NavigationStack`. Since we have two tabs one for patients and one for doctors we bind the corresponding routes array to the stack's `path` so each tab maintains its own history. The implementation below shows this setup.

```
struct MultiTabWithSeparateRoutesScreen: View {

    @State private var selectedTab: AppTab = .patients
    @Environment(MultiTabRouter.self) private var router

    var body: some View {

        @Bindable var router = router

        TabView(selection: $selectedTab, content: {

            Tab("Patients", systemImage: "heart", value: AppTab.patients) {
                NavigationStack(path: $router.patientRoutes) {
                    PatientScreen()
                    .navigationDestination(for: PatientRoute.self) { route
in
                        route.destination
                    }
                }
            }

            Tab("Doctors", systemImage: "cross", value: AppTab.doctors) {
                NavigationStack(path: $router.doctorRoutes) {
                    DoctorScreen()
                    .navigationDestination(for: DoctorRoute.self) { route
in
                        route.destination
                    }
                }
            }

        })
    }
}
```

As you can see, each `NavigationStack` manages only the routes it owns. Each stack binds its `path` to the matching array and sets a corresponding `navigationDestination`: the patient tab uses `navigationDestination(for: PatientRoute.self)`, and the doctor tab uses `navigationDestination(for: DoctorRoute.self)`.

The usage is shown below:

```
struct PatientScreen: View {

    @Environment(MultiTabRouter.self) private var router

    var body: some View {
```

```

        Button("Go to Patient Detail Screen") {
            router.patientRoutes.append(.details(UUID()))
        }
    }
}

```

Although this technique works, it has some drawbacks. First, you must remember to add each route to the correct array. If you place a route in the wrong array, navigation will not fire, which you should catch during testing. You do write end to end tests, right?

Second, you cannot load a doctor view from the patients tab or a patient view from the doctors tab. Each array is scoped to its own route enum, `PatientRoute` for patients and `DoctorRoute` for doctors.

Third, as you add more tabs, you must add more arrays. Managing four or five separate arrays quickly becomes cumbersome.

In the next section, we will solve these problems by combining `NavigationPath` with our custom environment values hooks to create a unified and reusable navigation flow across tabs.

MULTI-TAB NAVIGATION IN SWIFTUI USING NAVIGATIONPATH AND NAVIGATE HOOK

The primary issue we were facing in our last implementation was that each tab required an individual routes array to keep track of the history for that tab. This can become hard to maintain as our application add more tabs. One way to solve this problem is to use a dictionary instead. The dictionary key can serve as the tab and the value can contain routes associated with the tab.

```

@State private var routes: [AppTab: NavigationPath] = [: ]

```

`NavigationPath` holds the navigation history for a `NavigationStack`. It stores a sequence of `Hashable` values that describe where the user has been and where to go next. You append values to push, remove values to pop, or replace the entire path to reset.

Compared with an array of `Route`, `NavigationPath` is type erased so it can hold different route types in one place, it is easy to reset by assigning a new `NavigationPath` instance, it works naturally with SwiftUI state bindings, and it can be saved and restored when the elements are codable.

We also need to implement a helper function that can return the correct `NavigationPath` binding based on the tab. This is shown in the implementation below:

```

private func binding(for tab: AppTab) -> Binding<NavigationPath> {
    Binding(
        get: { routes[tab, default: NavigationPath()] },
        set: { routes[tab] = $0 }
    )
}

```

Finally, we can update our `TabView` to use the new routes array and the binding function. This is shown below:

```

    TabView(selection: $selectedTab) {

        Tab("Patients", systemImage: "heart", value: AppTab.patients) {
            NavigationStack(path: binding(for: .patients)) {
                PatientScreen()
                .navigationDestination(for: Route.self) { route in
                    route.destination
                }
            }
        }

        Tab("Doctors", systemImage: "cross", value: AppTab.doctors) {
            NavigationStack(path: binding(for: .doctors)) {
                DoctorScreen()
                .navigationDestination(for: Route.self) { route in
                    route.destination
                }
            }
        }

    }.environment(\.navigate, NavigateAction(action: { route in
        var navigationPath = routes[selectedTab, default:
NavigationPath()]
        navigationPath.append(route)
        routes[selectedTab] = navigationPath }))

```

One of the major benefits of this approach is that you can navigate between `PatientRoute` and `DoctorRoute`. For example, if you are on the patient tab and need to show a doctor list or a doctor detail screen, you can do so without friction.

This works because `NavigationPath` stores the sequence of routes independently of any single feature, so the path is not tied to `PatientRoute` or `DoctorRoute` alone.

The usage is shown below:

```

struct PatientScreen: View {

    @Environment(\.navigate) private var navigate

    var body: some View {
        Button("Go to Patient Detail Screen") {
            navigate(.patient(.details(UUID())))
        }

        Button("Go to Doctor Detail Screen") {
            navigate(.doctor(.details(UUID())))
        }
    }
}

```

In the `TabView` above we created individual tabs. That is perfectly fine, but if you want to make the code more concise you can conform your `AppTab` enum to `CaseIterable` and add computed properties for `title`, `icon`, and `destination`. You can then loop over `AppTab.allCases` inside `TabView` to simplify the code.

The implementation is shown below:

```

    TabView(selection: $selectedTab) {
        ForEach(AppTab.allCases) { tab in
            Tab(tab.title, systemImage: tab.icon, value: tab) {
                NavigationStack(path: binding(for: tab)) {
                    tab.destination
                        .navigationDestination(for: Route.self) { route in
                            route.destination
                        }
                }
            }
        }
    }

    }.environment(\.navigate, NavigateAction(action: { route in
        var navigationPath = routes[selectedTab, default:
NavigationPath()]
        navigationPath.append(route)
        routes[selectedTab] = navigationPath
    })))

```

Now, you can easily add more tabs to the `AppTab` enum without worrying about creating separate `NavigationStack` instances, since the `ForEach` loop ensures that a new `NavigationStack` is initialized for each `AppTab` case.

I always like to start with the simplest approach I can think of. In this scenario, that might mean creating separate `NavigationStack` instances manually. Later, when the need arises, I can always go back and refactor. Of course, you always need to account for the cost of change. If making the change later would be significantly more expensive, then it's better to design it to be more flexible from the start.

POP TO ROOT

Consider a scenario where a user is three to five levels deep in the hierarchy and needs to return to the home screen. This action is commonly called pop to root. How can we take the user directly to the home screen without replaying the back animation several times?

We can extend your `NavigateAction` to support pop to root. The implementation below shows how.

```

struct NavigateAction {
    typealias Action = (Route) -> ()
    typealias PopToRoot = () -> Void

    let action: Action
    let popToRoot: PopToRoot

    func callAsFunction(_ route: Route) {
        action(route)
    }
}

extension EnvironmentValues {
    @Entry var navigate = NavigateAction { _ in } popToRoot: { }
}

```

We added an additional closure to handle `popToRoot`. If you prefer, you can also rename `Action` to `PushAction` and `action` to `push`, which reads better alongside the term “pop.”

Jumping back to the root is straightforward: reset the `NavigationPath`. The implementation below shows how.

```
.environment(\.navigate, NavigateAction(action: { route in
    var navigationPath = routes[selectedTab, default:
NavigationPath()]
    navigationPath.append(route)
    routes[selectedTab] = navigationPath

    }, popToRoot: {
        routes[selectedTab] = NavigationPath()
    })))
```

Now you can invoke `popToRoot` function, wherever needed as follows:

```
Button("Pop to Root") {
    navigate.popToRoot()
}
```

This will reset the current tab’s navigation to the root screen. Keep in mind that pop to root is a special case and not required in every iOS app. Add it only if your flow truly needs it; otherwise, you are introducing unnecessary complexity.

CONCLUSION

SwiftUI has moved from the simple `NavigationView` model to the more capable `NavigationStack`, which gives you programmatic control, support for value based destinations, and a clear way to separate navigation concerns from presentation. You saw the core list to detail pattern, learned why placing `navigationDestination` on the parent keeps child views reusable, and practiced driving navigation from asynchronous work by appending route values. You modeled routes as `Hashable` enums, added nested enums for hierarchy, and placed destination building close to the route with a `destination` property. You also exposed a small navigate hook through environment values so any screen can request navigation without knowing how it is performed.

For multi tab apps you compared two approaches. Separate arrays per tab work but do not scale well and prevent cross feature navigation. Using a dictionary of `NavigationPath` values keyed by tab keeps each tab’s history independent while still allowing you to move between `PatientRoute` and `DoctorRoute` when needed. Along the way you saw how to reset a stack by replacing the path, how to avoid heavy work in view initializers, and why navigation state belongs at the container level.

With these patterns you can design navigation that is explicit, testable, and easy to extend. As you continue, consider adding deep linking, state restoration with the codable representation of `NavigationPath`, and a review of `NavigationSplitView` for multi column layouts.

UIKIT TO SWIFTUI

If you are starting a brand new project for Apple platforms today, SwiftUI should be your first choice. Since its introduction at WWDC 2019, SwiftUI has evolved into a mature and reliable framework for building beautiful, accessible, and performant apps across all Apple devices. Its declarative syntax, combined with a powerful state driven architecture, allows developers to build user interfaces at incredible speed with far less boilerplate. When paired with the architectural principles outlined in this book, SwiftUI enables code that is both elegant and easy to reason about.

However, most developers are not starting from a blank canvas. The reality of professional software development is that we often inherit large, battle tested UIKit (or even AppKit) codebases that have grown over many years. These legacy projects cannot simply be rewritten overnight, nor should they be. Instead, teams must learn how to gradually integrate SwiftUI into existing UIKit applications, adopting modern patterns without sacrificing stability.

In this chapter, we will explore strategies for introducing SwiftUI into UIKit projects, bridging the gap between the two frameworks, and understanding how they can coexist. You will learn how to embed SwiftUI views inside existing UIKit screens, use UIKit components within SwiftUI, and migrate piece by piece with confidence. By the end, you will see that moving from UIKit to SwiftUI does not have to be an all or nothing decision, it can be a smooth, deliberate evolution toward a more modern, declarative future.

WHY MIGRATE?

SwiftUI is not just the new way to build apps, it represents a simpler and more modern way to think about user interfaces. But migration should never be done just because something is new. It should be done because it helps you move faster, write cleaner code, and maintain your app more easily over time.

Faster Development

SwiftUI lets you describe what your interface should look like instead of writing step by step instructions for how to build it. You spend less time wiring up constraints and outlets and more time focusing on what really matters—the user experience. With live previews in Xcode, you can see changes instantly, which makes iteration fast and fun.

Simpler State Management

In UIKit, keeping the interface in sync with data can quickly become messy. You often have to handle notifications, delegates, or manual updates. In SwiftUI, when your data changes, your view automatically updates. This reactive model removes a whole class of bugs and makes your code easier to reason about.

Shared Code Across Devices

SwiftUI works across iPhone, iPad, Mac, Apple Watch, and Apple TV. You can reuse the same components on multiple devices with very little change. This saves time and gives your app a consistent look and feel everywhere.

Cleaner Architecture

UIKit view controllers often become large and hard to manage because they handle everything—logic, layout, and navigation. SwiftUI encourages smaller, focused views and separates presentation from data. Your code becomes easier to test, reuse, and understand.

Faster Design Iteration

Xcode's live preview makes design work a joy. You can experiment with layouts, colors, and typography in real time without building and running the entire app. This short feedback loop makes a big difference in how fast teams can deliver.

Accessibility and Localization

SwiftUI automatically supports features like dynamic type, dark mode, and VoiceOver. It also makes localization simpler. This means your app can reach more users without a lot of extra code.

The Future of Apple Development

Apple continues to expand SwiftUI every year. While UIKit is still fully supported, most new frameworks and controls are designed with SwiftUI in mind. Adopting SwiftUI gradually helps keep your app current and prepares your team for what is coming next.

Easier Maintenance

SwiftUI views are smaller and more predictable than UIKit code. They have fewer side effects and are easier to test. Over time, that leads to fewer bugs, faster refactors, and happier developers.

You Do Not Need to Rewrite Everything

Migration does not mean starting over. You can bring SwiftUI into your existing UIKit app one screen at a time. This lets you take advantage of SwiftUI's benefits without losing the stability of your existing code.

WHEN SWIFTUI MAY NOT BE THE RIGHT MOVE YET

SwiftUI is a powerful and modern framework, but it is not a silver bullet. Migration should be driven by clear benefits, not by pressure to adopt something new. There are real situations where staying with UIKit, at least temporarily, is the most responsible decision.

One common case is when your app relies on complex, highly tuned custom animations that are already stable in UIKit. If those animations are central to the user experience and have been refined over time, rewriting them in SwiftUI may introduce subtle regressions with little immediate payoff. Until SwiftUI offers a clear advantage for that specific interaction, stability often matters more than modernization.

Another challenge arises with third party dependencies. Many mature apps depend on UIKit based libraries for things like charts, maps, video playback, or custom controls. If these libraries do not provide reliable SwiftUI wrappers, forcing them into SwiftUI can lead to awkward bridging code, increased maintenance, and hard to diagnose bugs. In these cases, UIKit remains the most predictable integration point.

Team readiness is also a critical factor. SwiftUI requires a different mental model, especially around state, data flow, and view lifecycle. If your team is still early in that learning curve and working under tight deadlines, a large migration can slow delivery and increase stress. It is often better to introduce SwiftUI gradually through small, low risk screens while the team builds confidence.

There are also moments in a product's lifecycle where migration simply is not the priority. If the app is in a stabilization phase, responding to customer issues, or preparing for a major release, the risk of a UI rewrite may outweigh the benefits. Migration is most effective when it aligns with natural moments of change, such as feature redesigns or platform updates.

In all of these cases, delaying SwiftUI adoption is not a failure. It is a strategic choice. SwiftUI will continue to improve, and your existing UIKit code is not suddenly obsolete. The goal is steady progress, not churn. When the timing is right, SwiftUI will be there to help you move forward with confidence.

MIGRATION STRATEGY

Migrating from UIKit to SwiftUI should never be treated as a massive rewrite. The most effective strategy is to approach it as a gradual evolution guided by value, not novelty. Begin by preparing your codebase for coexistence. Extract business rules, data loading, and formatting logic into plain Swift services or stores that can be shared between UIKit and SwiftUI. Establish a simple design system in SwiftUI for typography, colors, and spacing so new components look consistent from the start. Strengthen your tests around shared logic and critical user flows before you make any visual changes.

Once the foundation is ready, start with a single pilot screen. Choose something of medium complexity that is already due for a design update or feature enhancement. Replace that screen with a SwiftUI version or embed a SwiftUI subview using `UIHostingController`. Pass data into the view through initializers and return results through closures or shared stores. This first migration will help the team understand how data flows between the two worlds and what adjustments are needed for navigation and testing.

After validating your approach, expand SwiftUI use gradually. All new features should be built in SwiftUI by default, while existing UIKit screens can remain untouched unless they require major updates. Continue to wrap essential UIKit views with `UIViewRepresentable` or `UIViewControllerRepresentable` so they can be reused in SwiftUI. As your SwiftUI codebase grows, unify the look and feel through shared modifiers and environment values, ensuring a consistent experience across both frameworks.

When enough of your app has been migrated, focus on consolidating. Remove duplicated state by making your stores the single source of truth. Keep navigation behavior consistent whether a screen is

built with SwiftUI or UIKit, and strengthen integration tests to cover flows that cross the boundary between the two. During this stage, performance profiling is important—pay attention to large lists, animations, and image-heavy screens, optimizing where needed.

Finally, when the SwiftUI portion of your app becomes dominant, consider transitioning the root navigation to SwiftUI. This can happen naturally as legacy UIKit screens are retired during redesigns or new product initiatives. By this point, your app will already be SwiftUI-first, using UIKit only where it provides a specific advantage.

Throughout the process, track meaningful metrics like delivery time, defect rate, and component reuse to ensure migration brings real benefits. Keep communication open by maintaining a living migration guide in your repository and encouraging short weekly check-ins to share lessons and blockers. Success is not measured by how quickly you replace UIKit, but by how confidently your team can deliver new features with SwiftUI—faster, cleaner, and with fewer regressions—while users continue to enjoy a seamless and consistent experience.

A Practical Migration Checklist

Before moving on to implementation, it helps to pause and sanity check your approach. You do not need everything to be perfect, but you should be intentional.

Use this checklist as a guide:

- Extract shared business logic into plain Swift types that can be used by both UIKit and SwiftUI
- Pick a single pilot screen that provides clear value. This can be the settings screen.
- Establish basic SwiftUI design primitives for colors, typography, and spacing
- Keep navigation behavior consistent across UIKit and SwiftUI screens
- Measure outcomes such as delivery speed, defect rate, and component reuse

If you can confidently check off most of these items, you are ready to begin migrating. If not, slow down and address the gaps first. A small investment here prevents painful rewrites later.

EMBED SWIFTUI IN UIKIT

As discussed earlier, migration should be done in small steps. If you think that a UIKit screen is not really going to change that much and will not benefit from using SwiftUI then leave it as it is and move to the next screen. That being said some teams still try to perform a complete migration of the app from UIKit to SwiftUI and that is perfectly fine too, when done in small phases.

Sheet and Push Navigation

One of the screens that usually becomes the first to be updated to SwiftUI is the “**Settings**” screen. This becomes the automatic first choice because even if you mess up the settings screen, most apps will not have disastrous consequences.

Consider an example, where you are displaying a list of customers using UIKit framework. The screen also consist of a settings button as a `BarButtonItem` in the navigation bar. When the user taps on the settings button, it opens the settings screen in a sheet. Everything works as expected, but now the requirement is to update the settings screen to SwiftUI.

Once, you have implemented a settings screen in SwiftUI, you can present it from UIKit controller by using the `UIHostingController`. `UIHostingController` is a bridge between SwiftUI and UIKit. It allows you to embed any SwiftUI view inside a UIKit-based app (for example, inside a `UIViewController`, `UITableViewCell`, or even a `UINavigationController` stack).

Think of it as a container that takes a SwiftUI view and makes it behave like a normal UIKit `UIViewController`. The implementation is shown below:

```
@objc private func openSettings() {  
    let settingsScreenHC = UIHostingController(rootView: SettingsScreen())  
    present(settingsScreenHC, animated: true)  
}
```

Now, when you tap on the settings button in a UIKit view, it will open the settings screen, implemented in SwiftUI. This is the simplest form of SwiftUI view embedding in UIKit application.

In some cases, you want to add the SwiftUI view in the UIKit navigation stack. Consider a scenario, where a user taps on the customer which performs a push navigation and takes the user to the detail screen. We can implement the detail screen in SwiftUI and push it in the `NavigationStack`.

```
// MARK: - Table Delegate  
override func tableView(_ tableView: UITableView, didSelectRowAt  
indexPath: IndexPath) {  
  
    let customer = customers[indexPath.row]  
  
    let detail = UIHostingController(rootView:  
CustomerDetailScreen(customer: customer))  
        navigationController?.pushViewController(detail, animated:  
true)  
}
```

Now, when you perform the push navigation, a SwiftUI control becomes the navigation destination. In the later part of this section, we will learn how the navigation strategies work in a mixed app.

Embed SwiftUI Views in UIKit

Sometimes you don't want to present a complete SwiftUI view in a UIKit app but you want to embed a SwiftUI view as a subview in a UIKit view. Think of all the times, you have implemented amazing looking views in SwiftUI. Wouldn't it be great if you can use the same views in a UIKit application?

For this example, I will be using a RatingView I implemented in SwiftUI. The rating view displays stars and a user can tap on the star to fill it, indicating the selected rating. How can we display this RatingView in our UIKit application.

The RatingView was initially implemented for a SwiftUI application so it does provide the @Binding support for the caller. This means the caller can pass a @State property to the RatingView and when the RatingView updates the rating (by user tapping on the stars) then the caller will get the new rating through the SwiftUI publishing mechanism. Just to give you an idea, a small snippet from RatingView is shown below:

```
struct RatingView: View {  
    @Binding var rating: Int?  
  
    // other code here  
}
```

In order to display the RatingView in a UIKit view, we will take help from UIHostingController. The implementation of CustomerDetailsViewController is shown below:

```
class CustomerDetailsViewController: UIViewController {  
    override func viewDidLoad() {  
        super.viewDidLoad()  
  
        // add the ratings view  
        let hostingController = UIHostingController(rootView:  
RatingView(rating: .constant(4)))  
  
        guard let ratingView = hostingController.view else { return }  
        ratingView.translatesAutoresizingMaskIntoConstraints = false  
        self.addChild(hostingController)  
  
        self.view.addSubview(ratingView)  
  
        // add constraints for the ratingView  
    }  
}
```

If you run the application, you will see the RatingView displayed inside a UIView. Unfortunately, when you tap on it nothing will happen. This is because in the UIHostingController we passed the RatingView with a constant value of 4. This means it will always show 4 stars. So, now the question is that how can we make it interactive and get the new rating when the user taps on it. Also, keep in mind that you are not allowed to change the implementation of RatingView because this view is used throughout your application.

Handling SwiftUI View Events Using Container Views

There are several different ways of handling SwiftUI view events in a UIKit view. If you are still using ObservableObject protocol and not @Observable macro then one of the options is to wrap the RatingView in a container view. This is needed, as the container view can redraw the RatingView, when the value changes. Then in the UIKit controller, you can subscribe to rating value changes through events generated by the Combine framework.

We will start by defining Rating and RatingViewController. The implementation is shown below:

```
class Rating: ObservableObject {
    @Published var value: Int?
}

struct RatingViewContainer: View {
    @ObservedObject var rating: Rating

    var body: some View {
        RatingView(rating: $rating.value)
    }
}
```

If you are using Xcode 26 then you will need to import Combine in order to work with ObservableObject, otherwise you will be greeted with errors.

You might be wondering why not just create Binding in the view controller and pass it down to the RatingView, instead of going through the trouble of making Rating (Observable Object) and the RatingViewContainer. The binding technique will allow you to capture the events generated by the RatingView but it would reflect those changes in the user interface. The issue is that the RatingView has no reason to render again. This is the main reason we have introduced the RatingViewContainer. The @ObservedObject property wrapper forces the RatingViewContainer to re-evaluate and then re-render when the rating value changes.

The implementation of CustomerDetailsViewController is shown below:

```
class CustomerDetailsViewController: UIViewController {

    var rating: Rating = Rating()
    var cancellables: AnyCancellable?

    override func viewDidLoad() {
        super.viewDidLoad()

        cancellables = rating.$value.sink { [weak self] rating in
            if let rating {
                self?.ratingLabel.text = "\(rating)"
            }
        }
    }
}
```

```

        let hostingController = UIHostingController(rootView:
RatingViewController(rating: rating))

        guard let ratingViewC = hostingController.view else { return }
        ratingView.translatesAutoresizingMaskIntoConstraints = false

        self.addChild(hostingController)
        self.view.addSubview(ratingViewC)
        self.view.addSubview(ratingLabel)

        hostingController.didMove(toParent: self)

        // add constraints here
    }
}

```

After creating an instance of Rating (Observable Object), we setup the subscription to the value. This is possible because value is a @Published property, which makes it a publisher of events. The sink event will be fired, whenever the rating value is updated.

In the UIHostingController, we don't pass the RatingView but RatingViewController with the rating instance as the dependency to the RatingViewController. Finally, the RatingViewController SwiftUI view is accessed through the hosting controller's view property and then injected into the UIKit view hierarchy.

Now, if you tap on the rating in RatingView, not only it updates the user interface and displays the correct stars but also fires the event and pass the rating back to caller, where it is displayed in a UIKit view.

All this communication required a bit of setup, when using the ObservableObject protocol. In the next section, you will learn how you can use the new @Observable macro to solve the same problem with less steps.

Handling SwiftUI View Events Using @Observable Macro

If you are targeting iOS 17 or later, the @Observable macro is now the preferred way to model observable state for SwiftUI. It replaces much of the boilerplate required by ObservableObject and @Published, while offering tighter integration with SwiftUI's rendering and observation system.

If your app needs to support earlier iOS versions, ObservableObject remains a valid and widely used approach. The concepts are the same, but the tools differ. In a mixed codebase, it is common to see both approaches used side by side, with @Observable adopted gradually as your deployment target moves forward.

Apple introduced the Observation framework to make state changes more explicit and predictable. Instead of broadcasting changes through publishers, SwiftUI can now track exactly which properties are read and written during view evaluation. This leads to fewer unnecessary view updates and simpler mental models for how data flows through your UI.

In our case, we can use `@Observable` to simplify the communication between UIKit and SwiftUI without introducing container views or Combine subscriptions.

Defining Observable State with `@Observable`

We begin by defining the `Rating` class using the `@Observable` macro:

```
@Observable
class Rating {
    var value: Int?
}
```

There is no need to conform to `ObservableObject` or mark properties with `@Published`. SwiftUI automatically tracks changes to the `value` property and re-renders any views that depend on it.

This class becomes the single source of truth for the rating, shared between UIKit and SwiftUI.

Passing State into a SwiftUI View from UIKit

The `RatingView` already expects a `Binding<Int?>`, and we do not want to change its implementation. Instead, we create a binding inside the UIKit view controller that bridges UIKit state updates to SwiftUI and vice versa.

```
override func viewDidLoad() {
    super.viewDidLoad()

    let binding = Binding<Int?>(
        get: { [weak self] in
            self?.rating.value
        },
        set: { [weak self] newValue in
            self?.rating.value = newValue
            self?.ratingLabel.text = newValue.map { "\($0)" }
        }
    )

    let hostingController = UIHostingController(
        rootView: RatingView(rating: binding)
    )

    // add hostingController.view to the UIKit view hierarchy
}
```

When the user taps on a star in the SwiftUI `RatingView`, the binding setter is called. This updates the shared `Rating` instance and immediately reflects the change in the UIKit label. At the same time, SwiftUI automatically re-renders the view because it observes the updated state.

There is no Combine pipeline, no container view, and no manual re-rendering logic.

Why This Approach Works Better

The key difference between this approach and `ObservableObject` is intent. `ObservableObject` was designed as a general purpose publishing mechanism that SwiftUI later adopted. `@Observable` is designed specifically for SwiftUI's observation model.

SwiftUI can precisely track which properties are accessed during view evaluation and only update the views that actually depend on them. This results in simpler code and more predictable updates, especially in hybrid UIKit and SwiftUI applications.

Because the state lives outside the view and is shared, UIKit does not need to know anything about SwiftUI's observation system. It simply reads and writes values as it always has.

Practical Guidelines

As a rule of thumb:

- Use `@Observable` for new SwiftUI code targeting iOS 17 and later
- Continue using `ObservableObject` when supporting earlier iOS versions
- Avoid mixing `@Observable` and `ObservableObject` for the same piece of state
- Keep your state models simple and free of UI-specific logic

In a mixed UIKit and SwiftUI app, `@Observable` shines because it allows SwiftUI views to react to shared state changes without forcing UIKit to adopt SwiftUI-specific concepts or frameworks.

By leveraging the Observation framework, you can reduce boilerplate, clarify data flow, and keep both UIKit and SwiftUI perfectly in sync. This makes it easier to modernize your app incrementally while maintaining stability and confidence throughout the migration process.

SwiftUI Views as UIKit TableView Cells

`UITableView` has long been one of the most fundamental and versatile components in iOS development, providing a powerful way to display data in a scrolling, structured format. As SwiftUI continues to mature, Apple has made it increasingly easy to integrate its declarative views into existing UIKit codebases.

Starting in iOS 16, UIKit introduced the `UIHostingConfiguration` API, a lightweight bridge that allows developers to embed SwiftUI views directly inside UIKit elements such as `UITableViewCell` and `UICollectionViewCell`. Unlike the older approach of wrapping SwiftUI views inside a `UIHostingController`, `UIHostingConfiguration` automatically manages view lifecycle, sizing, and reuse, making the integration seamless and efficient.

This means you can now take advantage of SwiftUI's expressive, reactive UI syntax while maintaining the structure and flexibility of a traditional `UITableView`. The example below demonstrates how to use a SwiftUI view as the content of a table view cell:

The implementation below shows you to load a SwiftUI view as a `UITableView` cell in UIKit.

```

override func tableView(_ tableView: UITableView, cellForRowAt indexPath:
IndexPath) -> UITableViewCell {

    let cell = tableView.dequeueReusableCell(
        withIdentifier: "CustomerTableViewCell",
        for: indexPath
    )

    let customer = customers[indexPath.row]

    cell.contentConfiguration = UIHostingConfiguration {
        // CustomerRow is the SwiftUI view
        CustomerRow(name: customer.name, city: customer.city)
    }

    return cell
}

```

When you assign a `UIHostingConfiguration` to a table view cell, UIKit automatically manages the SwiftUI view's lifecycle. This means you do not need to create or dispose of the view manually. As the cell scrolls on and off the screen, UIKit efficiently reuses the underlying SwiftUI content without unnecessary re-rendering. The system also handles layout sizing automatically, allowing the SwiftUI view to determine its own height and adapt to content changes in real time.

Because `UIHostingConfiguration` integrates directly with UIKit's reuse mechanism, it performs well even with large lists. This makes it ideal for introducing SwiftUI views into existing UIKit based applications incrementally, without rewriting the entire user interface. You can start small by embedding a few SwiftUI views inside your table cells and gradually adopt more SwiftUI components as your project evolves.

LOADING UIKIT VIEWS INTO A SWIFTUI APP

UIKit has been around for a very long time and a lot of successful apps have been implemented using the UIKit framework. This also means that developers have written really nice custom views for their UIKit applications. In this section you will learn how to import those views into your SwiftUI applications, without having to rewrite them from scratch.

Consider a scenario, that you have implemented a star ratings view in UIKit. How can we use this rating, implemented in UIKit in a SwiftUI application?

Luckily SwiftUI provides the `UIViewRepresentable` protocol to wrap any `UIView` so it can participate in SwiftUI's layout and state system. Think of it as an adapter. Your UIKit view remains exactly as it is, while the wrapper exposes SwiftUI bindings and forwards updates both ways.

At a high level you will:

1. Create a SwiftUI wrapper that conforms to `UIViewRepresentable`.
2. Build the UIKit view in `makeUIView`.

3. Keep it in sync with SwiftUI state in `updateUIView`.
4. Use a Coordinator when you need to send events from UIKit back to SwiftUI.

Below is a minimal wrapper around a `SimpleStarRating` control that exposes a SwiftUI binding:

```
struct StarRatingView: UIViewRepresentable {
    @Binding var rating: Int

    func makeUIView(context: Context) -> SimpleStarRating {
        let view = SimpleStarRating()
        view.addTarget(context.coordinator, action:
#selector(Coordinator.valueChanged(_:)), for: .valueChanged)
        return view
    }

    func updateUIView(_ uiView: SimpleStarRating, context: Context) {
        if uiView.rating != rating {
            uiView.rating = rating
        }
    }

    func makeCoordinator() -> Coordinator {
        Coordinator(rating: $rating)
    }

    final class Coordinator: NSObject {
        var rating: Binding<Int>
        init(rating: Binding<Int>) { self.rating = rating }
        @objc func valueChanged(_ sender: SimpleStarRating) {
            rating.wrappedValue = sender.rating
        }
    }

    func sizeThatFits(_ proposal: ProposedViewSize, uiView: SimpleStarRating,
context: Context) -> CGSize? {
        uiView.systemLayoutSizeFitting(UITableView.layoutFittingCompressedSize)
    }
}
```

Use it like any other SwiftUI view:

```
struct ContentView: View {

    @State private var rating: Int = 2

    var body: some View {
        StarRatingView(rating: $rating)
    }
}
```

Important sizing note

Many UIKit controls do not report an intrinsic size. If your control stretches to fill the screen, give it an intrinsic size or tell SwiftUI to use a compact size. You can do this by overriding `intrinsicContentSize` in the UIKit control or by adding `.fixedSize()` and an explicit `.frame` on the SwiftUI side. This prevents unexpected stretching and keeps the control as compact as its content.

When to use a Coordinator

Use a Coordinator when your UIKit view needs to call back into SwiftUI. Typical examples are target-action, delegate callbacks, or gesture events. The Coordinator holds bindings or closures and forwards events to update SwiftUI state.

Best practices

- Keep UIKit logic inside the UIKit view. The wrapper should remain thin.
- Avoid tight coupling. Expose simple properties and actions that map cleanly to SwiftUI bindings.
- Consider accessibility. Forward accessibility labels and values so VoiceOver reads the control correctly.
- Test in isolation first. Verify the UIKit view works by itself, then add the wrapper.

With this approach, you can reuse mature UIKit components in SwiftUI while keeping a single source of truth for state and a clean separation

EMBEDDING UIKIT CONTROLLERS IN SWIFTUI

In the previous section, we looked at how to load UIKit views inside SwiftUI. Sometimes you need more than a view. You may want to present a full UIKit controller, such as a camera, a mail composer, or a custom screen that relies on delegates and navigation. SwiftUI makes this possible through `UIViewControllerRepresentable`, which wraps a UIKit controller so it can appear and behave like a native SwiftUI view.

In this section, you will learn when to choose a controller wrapper, how `makeUIViewController` and `updateUIViewController` work together, and how to use a Coordinator to pass events and data back to SwiftUI.

Consider a scenario where you want to pick a photo from the user's library. SwiftUI does provide a native built-in view to select photos but for this demonstration we are going to ignore it.

The first step is to implement a struct that conforms to `UIViewControllerRepresentable`. This involves providing the implementations of `makeUIViewController` and `updateUIViewController` methods. The `makeUIViewController` is responsible for initializing and returning the UIKit controller and `updateUIViewController` is where SwiftUI applies new state to existing controller, `UIImagePickerController`.

We will also need to utilize the UIImagePickerControllerDelegate since we need to take action when the image is selected by the user. This responsibility falls on the shoulders of the Coordinator, which will conform to the UIImagePickerControllerDelegate and handle all the events.

The complete implementation is shown below:

```
struct ImagePicker: UIViewControllerRepresentable {
    enum Source {
        case photoLibrary
        case camera
    }

    var source: Source = .photoLibrary
    var onImagePicked: (UIImage) -> Void
    @Environment(\.dismiss) private var dismiss

    func makeUIViewController(context: Context) -> UIImagePickerController {
        let picker = UIImagePickerController()
        picker.delegate = context.coordinator
        switch source {
        case .photoLibrary:
            picker.sourceType = .photoLibrary
        case .camera:
            picker.sourceType = .camera
        }
        picker.allowsEditing = false // set true if you want crop UI
        return picker
    }

    func updateUIViewController(_ viewController: UIImagePickerController,
    context: Context) {
        // no update needed for this controller
    }

    func makeCoordinator() -> Coordinator {
        Coordinator(parent: self)
    }

    final class Coordinator: NSObject, UIImagePickerControllerDelegate,
    UINavigationControllerDelegate {
        let parent: ImagePicker
        init(parent: ImagePicker) { self.parent = parent }

        func imagePickerController(_ picker: UIImagePickerController,
                                   didFinishPickingMediaWithInfo info:
    [UIImagePickerController.InfoKey : Any]) {
            // prefer edited image if allowsEditing was true
            let image = (info[.editedImage] ?? info[.originalImage]) as?
    UIImage
            if let image { parent.onImagePicked(image) }
            parent.dismiss()
        }

        func imagePickerControllerDidCancel(_ picker: UIImagePickerController)
    {
        parent.dismiss()
    }
}
```

```

    }
  }
}

```

The ImagePicker also exposes Source enum, which allows the user to select a photo from the library or take a new photo using the camera. Once, the image is selected, it is passed back to the caller using a closure. Once the ImagePicker is implemented, you can use it in different views. This is shown in the implementation below:

```

struct PhotoCaptureView: View {
  @State private var showPicker = false
  @State private var useCamera = false
  @State private var image: UIImage?

  var body: some View {
    VStack(spacing: 16) {
      if let image {
        Image(uiImage: image)
          .resizable()
          .scaledToFit()
          .frame(height: 240)
      }

      HStack {
        Button("Pick Photo") {
          useCamera = false
          showPicker = true
        }
        Button("Take Photo") {
          if UIImagePickerController.isSourceTypeAvailable(.camera)

in
            useCamera = true
            showPicker = true
          else {
            // handle no camera available if needed
          }
        }
      }
    }
  }
  .sheet(isPresented: $showPicker) {
    ImagePicker(source: useCamera ? .camera : .photoLibrary) { picked
      image = picked
    }
  }
  .padding()
}

```

UIViewControllerRepresentable bridges the SwiftUI world and a UIKit controller so you can present system pickers and custom controllers without rewriting them. With this pattern in place you can reuse

other UIKit controllers in the same way. In the next section we will look at different navigation strategies in a mixed app.

CONCLUSION

Moving from UIKit to SwiftUI does not have to feel scary or disruptive. As you saw in this chapter, the two frameworks are meant to live together. You can embed SwiftUI views inside UIKit, bring UIKit views and controllers into SwiftUI, and even mix both in the same flow without breaking your app or your team's momentum.

The most important idea is intention. Migrate when it makes sense. Start small. Pick screens or components that clearly benefit from SwiftUI's declarative style and simpler state handling. Keep your business logic shared and your state in one place so UIKit and SwiftUI stay in sync. Over time, SwiftUI naturally becomes the default, while UIKit slowly fades into a supporting role.

UIKit to SwiftUI is not an all or nothing decision. It is a gradual and practical evolution. By treating migration as a journey instead of a rewrite, you protect your users, your deadlines, and your sanity. You get to modernize your app without throwing away years of solid work.

At the end of the day, success is not about removing UIKit as fast as possible. It is about shipping features with confidence, understanding your UI state clearly, and building screens that stay predictable as your app grows. SwiftUI helps you move forward. UIKit gives you stability while you get there. Together, they let you evolve at your own pace.

COMPOSING AND CUSTOMIZING VIEWS

In the **Container and Presenter pattern**, we learned how to split a feature into two clear responsibilities. One view owns state and presentation logic, often called the container or screen. The other view focuses on rendering UI, often called the presenter. This separation lets you build complex features by composing smaller views that are easier to reason about and easier to test. The container owns data flow and side effects, while the presenter stays pure and declarative.

View composition goes one step further. It becomes the foundation for building high performance, maintainable SwiftUI apps. In this chapter, you will learn how to apply composition principles to simplify your codebase, reduce duplication, and create reusable components that scale with your app.

COMPOSABLE LIST CELL VIEWS

Throughout this book, I tried to use real world examples from apps I have actually built. This chapter is inspired by a gardening app I published called My Veggie Garden. One of the screens in the app is responsible for showing a list of vegetables. The simplest way to build it is to put the row UI directly inside the `List`.

```
List(greenPlaceStore.vegetables) { vegetable in
    HStack {
        AsyncImage(url: vegetable.thumbnailImage) { img in
            img.resizable()
                .frame(width: 75, height: 75)
                .clipShape(RoundedRectangle(cornerRadius: 16.0,
style: .continuous))
        } placeholder: {
            ProgressView("Loading...")
        }
        Text(vegetable.name)
    }
}
```

This works, but it starts to hurt as soon as you reuse the same row style across multiple screens. You end up copying the same layout rules everywhere, spacing drifts over time, and making one small UI tweak becomes a hunt across the codebase.

A simple refactor is to extract the row into its own view.

```
struct VegetableListScreen: View {
    @Environment(GreenPlaceStore.self) private var greenPlaceStore
    var body: some View {
```

```

List(greenPlaceStore.vegetables) { vegetable in
    VegetableRow(vegetable: vegetable)
}.task {
    do {
        try await greenPlaceStore.loadVegetables()
    } catch {
        print(error.localizedDescription)
    }
}
}
}

```

This is the simplest kind of composition and it pays off immediately. Your list reads better, the row becomes reusable, and your UI rules now live in one place. The visualization of the VegetableRow is shown below:



Hot Pepper

Customizing a row without duplicating everything

Now imagine a new requirement: each row needs to display a circular progress indicator alongside the vegetable name and image. This is a very common situation as apps evolve and new features are layered on top of existing screens.

One straightforward approach is to create a second row view that includes this additional UI. This keeps the original row untouched and makes the new requirement explicit, at the cost of introducing another row implementation that looks very similar to the first.

```

struct VegetableProgressRow: View {
    let vegetable: Vegetable

    var body: some View {
        HStack {
            AsyncImage(url: vegetable.thumbnailImage) { img in
                img.resizable()
                .frame(width: 75, height: 75)
                .clipShape(RoundedRectangle(cornerRadius: 16.0,
style: .continuous))
            } placeholder: {
                ProgressView("Loading...")
            }
            Text(vegetable.name)
            Spacer()
        }
    }
}

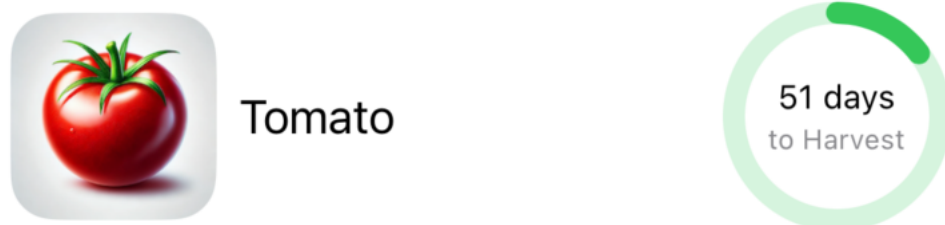
```

```

        CircularProgressView(progress: Double.random(in: 0...1),
daysRemaining: Int.random(in: 0...90))
    }
}

```

The result is shown below:



This works, but it looks almost identical to `VegetableRow`. The only difference is the progress view.

Then the next request arrives. Show the vegetable description. Do you create a third row view?

It depends.

I know that answer is not satisfying, but it is the honest one.

Creating separate views for each variation gives you maximum flexibility. Each view can evolve independently without worrying about breaking others. The downside is more code and more duplication.

Creating a generic row view reduces duplication, but it introduces a new risk. Over time, the generic row starts collecting edge cases. One screen wants an animated image. Another wants a badge in the top left. Another wants a ribbon. You start adding conditions, and the generic row turns into a pile of special rules.

If you find yourself adding flags like `showsBadge`, `isCompact`, `useAlternateLayout`, or piling up `if` statements that change the row's structure, it is usually time to extract a dedicated view. Generic rows work best when the layout stays stable and only small pieces swap in and out.

There is no one size fits all solution. A good rule of thumb is this:

If the variation changes the layout, create a separate view. If the variation keeps the same layout and only swaps small pieces, a generic row can be a great fit. In the next section, we will build a generic list row that supports multiple variations without forcing you to create a new row type every time.

GENERIC LIST ROW

The idea behind a generic list row comes from UIKit. `UITableViewCell` gave us a practical default structure with a title, subtitle, image, and accessory views. We can take that same mental model and build something similar in SwiftUI.

We will start with a basic row that supports a title, optional subtitle, and optional image.

```
struct ListRow: View {

    let title: String
    var subtitle: String?
    var imageURL: URL?

    init(title: String, subtitle: String? = nil, imageURL: URL? = nil) {
        self.title = title
        self.subtitle = subtitle
        self.imageURL = imageURL
    }

    var body: some View {
        HStack {

            if let imageURL {
                AsyncImage(url: imageURL) { img in
                    img.resizable()
                        .frame(width: 75, height: 75)
                        .clipShape(RoundedRectangle(cornerRadius: 16.0, style:
.continuous))
                } placeholder: {
                    ProgressView("Loading...")
                }
            }
            VStack(alignment: .leading) {
                Text(title)
                if let subtitle {
                    Text(subtitle)
                }
            }

        }.padding()
    }
}
```

The subtitle and imageURL are marked as optionals giving extra flexibility to the callers to only pass them when appropriate. The usage is shown below:

```
ListRow(title: "Carrot", subtitle: "A versatile root vegetable commonly
orange, but also available in purple, red, yellow, and white varieties.",
imageURL: URL(string: "https://azamsharp.com/images/carrot.png")!)
```



Carrot

A versatile root vegetable commonly orange, but also available in purple, red, yellow, and white varieties.

Adding leading and trailing accessories

Next, we make `ListRow` flexible by allowing callers to inject leading and trailing accessory views. This is where view builders shine.

```
struct ListRow<LeadingAccessoryView: View, TrailingAccessoryView: View>: View
{
    let title: String
    var subtitle: String?
    var imageURL: URL?
    var leadingAccessoryView: LeadingAccessoryView
    var trailingAccessoryView: TrailingAccessoryView

    init(title: String, subtitle: String? = nil, imageURL: URL? = nil,
    @ViewBuilder leadingAccessoryView: () -> LeadingAccessoryView =
    { EmptyView() }, @ViewBuilder trailingAccessoryView: () ->
    TrailingAccessoryView = { EmptyView() }) {
        self.title = title
        self.subtitle = subtitle
        self.imageURL = imageURL
        self.leadingAccessoryView = leadingAccessoryView()
        self.trailingAccessoryView = trailingAccessoryView()
    }

    // Body is identical to the previous ListRow implementation.
    // Only the stored properties and initializer changed to support accessories.
    var body: some View {

    }
}
```

To support customization, we updated the `ListRow` definition to use generic view types. Both `LeadingAccessoryView` and `TrailingAccessoryView` are constrained to conform to `View`, allowing callers to inject any SwiftUI view they need.

Separate generic types are used for the leading and trailing accessories so each side can accept a different kind of view. This keeps the API flexible without forcing both accessories to share the same type.

The initializer provides `EmptyView` as the default for both accessories. This means callers only need to supply the views they care about, while the rest of the layout remains unchanged.

Finally, the body of `ListRow` is updated to render these accessory views alongside the existing title, subtitle, and image, completing the customization without altering the overall structure of the row.

```
var body: some View {
    HStack {

        leadingAccessoryView

        if let imageURL {
            AsyncImage(url: imageURL) { img in
                img.resizable()
                    .frame(width: 75, height: 75)
                    .clipShape(RoundedRectangle(cornerRadius: 16.0, style:
                .continuous))
            } placeholder: {
                ProgressView("Loading...")
            }
        }

        VStack(alignment: .leading) {
            Text(title)
            if let subtitle {
                Text(subtitle)
            }
        }

        trailingAccessoryView

    }.padding()
}
```

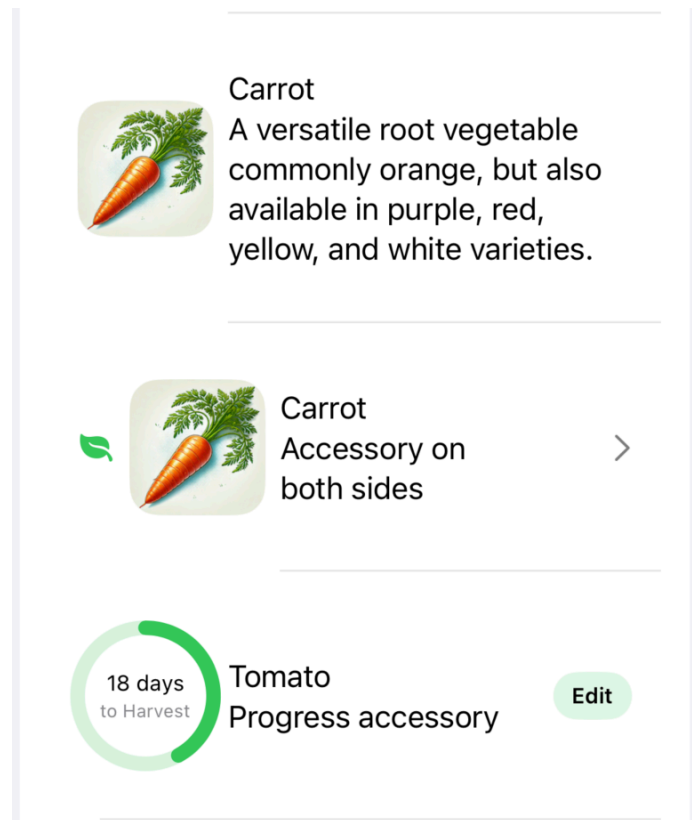
The usage is shown below:

```
ListRow(title: "Carrot", imageURL: nil, leadingAccessoryView: {
    CircularProgressView(progress: Double.random(in: 0...1),
daysRemaining: Int.random(in: 0...90))
}, trailingAccessoryView: {
    Text("TrailingView")
})
```

`ListRow` gives you the flexibility to configure a single row in many different ways. In this case, it replaces three or four separate implementations of the vegetable row view while keeping the layout consistent across screens. If needed, you can take this idea even further by allowing callers to inject additional elements such as custom content or background views.

To validate a generic view like this, previews become especially important. You can use ChatGPT or any other AI tool to quickly generate multiple variations of `ListRow` for previews. Seeing different combinations side by side makes it much easier to reason about spacing, alignment, and edge cases.

Few of the `ListRow` variations generated by ChatGPT are shown below:



At this point, `ListRow` is focused purely on presentation. It does not yet expose any way to respond to user interactions. In real apps, rows often need to react to events such as selection or long press. In the next section, we will extend `ListRow` to support these interactions in a clean and flexible way.

Adding events to `ListRow`

Events in `ListRow` are exposed as closures. They are optional because not every row needs to respond to user interactions. In the example below, an `onSelect` event is added to handle row selection. Using the same approach, you can introduce additional events such as `onLongPress` or other interactions as needed, without changing how the row is rendered.

```
struct ListRow<LeadingAccessoryView: View, TrailingAccessoryView: View>: View
{
    let title: String
    var subtitle: String?
    var imageURL: URL?
    var leadingAccessoryView: LeadingAccessoryView
    var trailingAccessoryView: TrailingAccessoryView
    var onSelect: (() -> Void)?

    init(title: String, subtitle: String? = nil, imageURL: URL? = nil,
    @ViewBuilder leadingAccessoryView: () -> LeadingAccessoryView =
    { EmptyView() }, @ViewBuilder trailingAccessoryView: () ->
    TrailingAccessoryView = { EmptyView() }, onSelect: (() -> Void)? = nil ) {
        self.title = title
        self.subtitle = subtitle
        self.imageURL = imageURL
        self.leadingAccessoryView = leadingAccessoryView()
        self.trailingAccessoryView = trailingAccessoryView()
        self.onSelect = onSelect
    }
}
```

```
// Body is the same as the accessory-enabled ListRow,
// plus tap/long-press handlers to trigger onSelect.
}
```

Using closures works well when a view exposes only a few events. However, as the number of supported interactions grows, the initializer can quickly become crowded. An alternative approach is to group related interactions into an enum.

With this design, `ListRow` exposes a single closure that receives a `ListRowEvent` value describing the interaction that occurred. This centralizes event handling, keeps the initializer from expanding unnecessarily, and can make the calling site easier to read and reason about.

Below is an example of introducing a `ListRowEvent` enum to represent the supported interactions.

```

struct ListRow<LeadingAccessoryView: View, TrailingAccessoryView: View>: View
{
    let title: String
    var subtitle: String?
    var imageURL: URL?
    var leadingAccessoryView: LeadingAccessoryView
    var trailingAccessoryView: TrailingAccessoryView
    var onEvent: ((ListRowEvent) -> Void)? = nil

    enum ListRowEvent {
        case onSelect
        case onLongPress
    }

    init(title: String, subtitle: String? = nil, imageURL: URL? = nil,
    @ViewBuilder leadingAccessoryView: () -> LeadingAccessoryView =
    { EmptyView() }, @ViewBuilder trailingAccessoryView: () ->
    TrailingAccessoryView = { EmptyView() }, onEvent: ((ListRowEvent) -> Void)? =
    nil) {
        self.title = title
        self.subtitle = subtitle
        self.imageURL = imageURL
        self.leadingAccessoryView = leadingAccessoryView()
        self.trailingAccessoryView = trailingAccessoryView()
        self.onEvent = onEvent
    }

    var body: some View {
        HStack {

            self.leadingAccessoryView

            if let imageURL {
                AsyncImage(url: imageURL) { img in
                    img.resizable()
                        .frame(width: 75, height: 75)
                }
            }

            VStack {
                Text(title)
                if let subtitle {
                    Text(subtitle)
                }
            }

            self.trailingAccessoryView
        }
    }
}

```

```

        .clipShape(RoundedRectangle(cornerRadius: 16.0, style:
        .continuous))
    } placeholder: {
        ProgressView("Loading...")
    }
}

VStack(alignment: .leading) {
    Text(title)
    if let subtitle {
        Text(subtitle)
    }
}

Spacer()

self.trailingAccessoryView
}
.contentShape(Rectangle())
.padding()
.onTapGesture {
    onEvent?(.onSelect)
}
.onLongPressGesture {
    onEvent?(.onLongPress)
}
}
}
}

```

The calling site is shown below:

```

ListRow(title: "Carrot", imageURL: nil, leadingAccessoryView: {
    CircularProgressView(progress: 0.75, daysRemaining: 12)
}, trailingAccessoryView: {
    Text("TrailingView")
}, onEvent: { event in
    switch event {
        case .onSelect:
            print("onSelect")
        case .onLongPress:
            print("onLongPress")
    }
})

```

The choice between exposing events as individual closures or grouping them into an enum depends on the situation. I usually reach for an enum when a view needs to support more than two events, but this is not a hard rule. What matters most is keeping the calling site clear and the view easy to evolve. If the API starts to feel awkward to use or explain, that is often a sign it needs to change. Try both patterns and see which one fits your needs best.

In this section, you learned how to build a generic, reusable view that can adapt to multiple scenarios. I have used this technique in several of my apps. In some cases, it significantly reduced duplication and kept the codebase clean. In other cases, it introduced unnecessary complexity and became harder to maintain over time.

The key is to stay alert to edge cases and watch how the view evolves. As soon as variations begin to drift apart in structure or layout, it is usually better to extract a dedicated view rather than continue adding special cases to a generic one. Generic views are powerful, but only as long as they remain focused and predictable.

VIEW EXTENSIONS AND MODIFIERS

Both view extensions and view modifiers help you abstract commonly used SwiftUI code. They allow you to reuse styles and behavior without repeating the same modifiers across your views. Used correctly, they reduce duplication, improve readability, and make your UI easier to evolve.

Although they look similar at the call site, view extensions and view modifiers solve different problems. In this section, we will start with view extensions and then transition into view modifiers.

View Extensions

View extensions are exactly what they sound like. They are extensions on `View` that return a modified version of the view. View extensions are ideal when you want to reuse **purely visual styling**.

A key characteristic of view extensions is that they do not have access to view state. They cannot store `@State`, accept bindings, or read from the environment. Their job is simple: take a view and apply a consistent set of visual rules.

Consider the following UI.

```
VStack(alignment: .leading, spacing: 8) {
    Text("Total Balance")
        .font(.caption)
        .foregroundColor(.secondary)

    Text("$3,420")
        .font(.title.bold())
}
.padding()
.background(Color(.secondarySystemBackground))
.clipShape(RoundedRectangle(cornerRadius: 16, style: .continuous))
.shadow(color: .black.opacity(0.05), radius: 6, y: 2)
```

This view looks fine and works exactly as expected. The problem appears later. As soon as you need the same card style in another part of the app, most people copy and paste the same modifiers. Over time, this leads to duplication scattered across the codebase.

When a design change comes in, such as adjusting the corner radius or shadow, you now have to hunt down every copy and update it manually. This is fragile and easy to get wrong.

A better approach is to give these styles a permanent home by extracting them into a view extension.

```
extension View {
    func cardStyle() -> some View {
        self
            .padding()
            .background(Color(.secondarySystemBackground))
            .rounded(16)
            .shadow(color: .black.opacity(0.05), radius: 6, y: 2)
    }
}
```

Now the calling site becomes much simpler.

```
VStack(alignment: .leading, spacing: 8) {
    Text("Total Balance")
    Text("$3,420")
        .font(.title.bold())
}
.cardStyle()
```

This is a significant improvement. All styling logic lives in one place, and any future changes to the card style only need to be made once. The view itself is easier to read because it focuses on structure and content instead of decoration.

I personally do not start with view extensions right away. I usually write styles inline first. Once I notice the same pattern appearing in multiple places, that is my signal to extract it into an extension. If you do this consistently, you will naturally build a small library of reusable styles that reflect your app's design language.

View extensions are an excellent choice when all you need is reusable styling. They are simple, expressive, and easy to reason about. However, as soon as you need access to state or environment values, view extensions are no longer enough.

That is where view modifiers come in. In the next section, we will look at how view modifiers differ from view extensions and when they are the right tool for the job.

View Modifiers

One of the most important differences between view extensions and view modifiers is that a view modifier is its own type. Because of that, a modifier can store state using `@State`, accept bindings with `@Binding`, and read values from the environment using `@Environment`.

This makes view modifiers a great fit when you need reusable **behavior**, not just reusable styling.

In the example below, we implement a `Watermark` view modifier. It can be applied to any view and displays the current time as an overlay. Tapping the watermark toggles between a compact and an expanded format.

```
struct Watermark: ViewModifier {

    @State private var expanded: Bool = false

    func body(content: Content) -> some View {
        content
            .overlay(alignment: .topTrailing) {
                TimelineView(.periodic(from: .now, by: 1)) { timeline in
                    Text("\((timeline.date.formatted(date:
expanded ? .complete: .omitted, time: expanded ? .complete: .standard)))")
                        .font(.caption)
                        .padding(8)
                        .background(.black.opacity(0.5))
                        .foregroundColor(.white)
                        .onTapGesture {
                            expanded.toggle()
                        }
                }
            }
    }
}
```

To make the modifier easy to apply, we expose it through a small convenience extension.

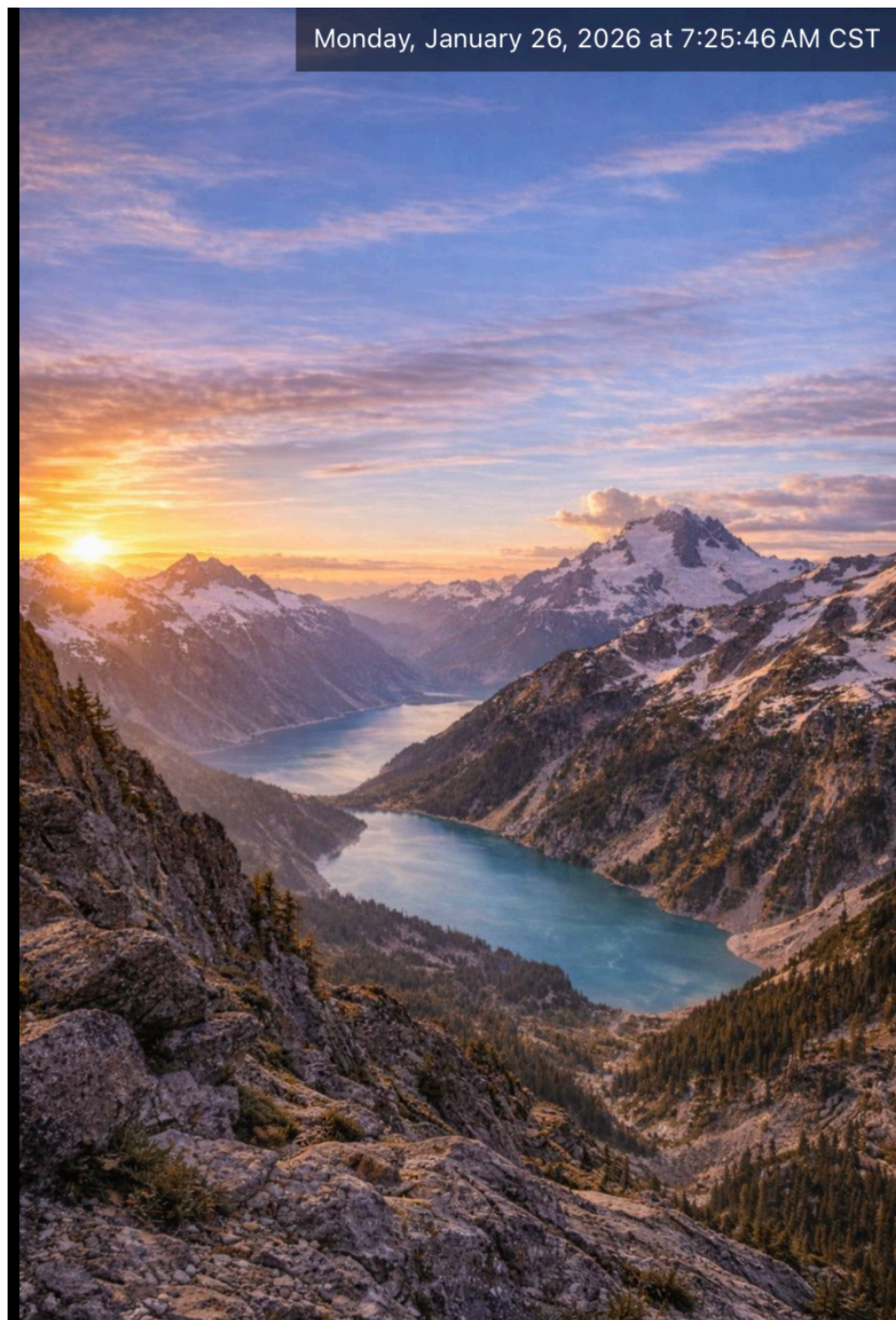
```
extension View {

    func watermark() -> some View {
        modifier(Watermark())
    }
}
```

Usage is straightforward.

```
Image("mountains")
    .resizable()
    .aspectRatio(contentMode: .fit)
    .watermark()
```

The result is shown below:



The result is an image with a live updating timestamp in the top right corner. Tapping the watermark switches between compact and expanded modes.

This example highlights why view modifiers exist. The behavior requires local state, user interaction, and a timer. None of that would be possible with a simple view extension.

Advanced modifiers and environment injection

View modifiers become even more powerful when they inject custom values into the SwiftUI environment. A common example is a toast system that allows any view in the hierarchy to trigger a toast without knowing how it is presented.

Below is a simplified example of a toast modifier that manages toast state and injects a `showToast` action into the environment.

```
struct ToastModifier: ViewModifier {  
  
    @State private var type: ToastType?  
    @State private var placement: ToastPlacement = .bottom  
    @State private var dismissTask: DispatchWorkItem?
```

```

    func body(content: Content) -> some View {
        content
        .environment(\.showToast, ShowToastAction(action: { type,
placement in
    }
    })
    }
}

```

This modifier can then be applied at the root of the app, making the toast action available everywhere.

```

struct PlatzApp: App {
    var body: some Scene {
        WindowGroup {
            ZStack {
                // ...
            }
            .withToast()
        }
    }
}

```

Any child view can now trigger a toast without depending on a specific implementation or presentation layer. The modifier owns the behavior, while the views remain simple and declarative.

When to reach for a view modifier

Just like view extensions, you do not need to start with a view modifier on day one. Inline code is often the fastest way to explore an idea. Once you notice repeated behavior or shared state logic, that is your signal to extract a modifier.

I have created view modifiers for toasts, navigation coordination, global sheets, and other cross cutting UI concerns. Used thoughtfully, view modifiers eliminate duplication, centralize behavior, and allow teams to move faster with fewer bugs.

If a modifier proves useful across multiple apps, it is worth investing a little extra time to polish it and publish it as a Swift package. That effort compounds over time. Work you extract today often becomes infrastructure you rely on for years.

BUTTON STYLES

In SwiftUI, buttons come with few default styles. This includes plain, bordered, borderless, borderedProminent and few more. But sometimes, you want to create your own custom styles to match with the look and feel of your app. This can be achieved in variety of different ways. Below you can see that we have applied SwiftUI view modifiers to the button to implement a shadow.

```

Button("Button with Shadow") {
    // do something
}

```

```

    } .padding(.horizontal, 20)
      .padding(.vertical, 10)
      .background(Color.accentColor)
      .foregroundColor(.white)
      .clipShape(RoundedRectangle(cornerRadius: 10, style: .continuous))
      .shadow(color: .black.opacity(0.25), radius: 6, x: 0, y: 3)

```

The above code will apply shadow to the button but it does not handle the opacity or perform animation when the button is pressed, since we don't have access to any property that indicates the pressed state of the button. Another problem is that we will need to copy/paste the code again and again, when we to apply the same styles again.

One solution is to move the code to view extension or view modifier. This way is can be reused in other places. But a better solution is to expose our shadow style as a custom ButtonStyle. The main reason is that then it will show up as an additional style of the button view and you will also get automatic access to the label and isPressed event for customization.

You will start by conforming to the ButtonStyle protocol. The implementation is shown below:

```

struct ShadowButtonStyle: ButtonStyle {

    func makeBody(configuration: Configuration) -> some View {
        configuration.label
            .padding(.horizontal, 20)
            .padding(.vertical, 10)
            .background(Color.accentColor)
            .foregroundColor(.white)
            .clipShape(RoundedRectangle(cornerRadius: 10,
style: .continuous))
            .shadow(color: .black.opacity(0.25), radius: 6, x: 0, y:
3)
            .scaleEffect(configuration.isPressed ? 0.95 : 1.0)
            .opacity(configuration.isPressed ? 0.8 : 1.0)
            .animation(.easeOut(duration: 0.15), value:
configuration.isPressed)
    }
}

```

The only function you need to implement is the makeBody function. The makeBody function is responsible for configuring the style for the button control. As you can see the configuration object is automatically passed to the makeBody function. The configuration object contains the label and the isPressed property, which can be used for customizing the button style.

We can also extend the ButtonStyle with a constraint on ShadowButtonStyle.

```

extension ButtonStyle where Self == ShadowButtonStyle {
    static var shadow: ShadowButtonStyle { ShadowButtonStyle() }
}

```

Now, you use your brand new shadow style in a button view. This is shown below:

```
Button("Button with Shadow") {  
    // do something  
}.buttonStyle(.shadow)
```

By defining a custom button style, you make your design reusable, consistent, and easier to maintain. Instead of repeating the same view modifiers throughout your codebase, you can now apply your brand's look with a single, expressive line. More importantly, by leveraging the `ButtonStyle` protocol, you gain access to the button's pressed state, allowing you to add subtle animations and tactile feedback that make your interface feel alive. Custom styles like `ShadowButtonStyle` not only simplify your code but also help create a cohesive and polished user experience across your entire app.

FORMATTING

One of the great things about working with SwiftUI is that it has built-in formatting capabilities, allowing you to easily display numbers, dates, and measurements in a localized and user-friendly way without extra code. In this section, we will take a look at how SwiftUI's formatting system works in practice. You will learn how to format numbers, dates, and measurements using the built-in `FormatStyle` API, explore how localization happens automatically based on the user's region settings, and even create your own custom format styles for specialized use cases. By the end, you will be able to present data in a way that feels natural and polished across different languages and locales, all without leaving SwiftUI's declarative world.

Basic Number Formatting

When displaying numbers in your app, presentation matters. A simple value like `1234.5678` can look clean and professional or messy and confusing, depending on how it's formatted. SwiftUI makes number formatting easy through the built-in `FormatStyle` API, allowing you to control precision, grouping, and style – all while automatically respecting the user's locale.

Let's start with a simple example:

```
Text(1234.5678, format: .number)
```

This displays the number using the user's regional conventions. For instance, users in the United States will see `1,234.568`, while users in France will see `1 234,568`. Notice how SwiftUI automatically uses the correct grouping and decimal separators for each region.

You can also control how many digits appear after the decimal point using `.precision(.fractionLength())`.

```
Text(1234.5678, format: .number.precision(.fractionLength(2)))
```

This displays the number as `1,234.57` – rounded to two decimal places. You can specify a fixed length or a range:

```
Text(1234.5678, format: .number.precision(.fractionLength(0..2)))
```

In this case, SwiftUI displays between 0 and 2 fraction digits depending on the value, which helps prevent unnecessary zeros.

Large numbers are easier to read with separators. SwiftUI enables grouping automatically, but you can adjust the behavior if needed:

```
Text(1000000, format: .number.grouping(.automatic))
```

This results in 1,000,000 for U.S. users or 1 000 000 for French users. SwiftUI again takes care of the regional differences, so you don't need to worry about localization details.

You can also control rounding by combining precision with rounding rules. For example, if you want to always round down:

```
Text(1234.5678,
format: .number.precision(.fractionLength(2)).rounded(rule: .down))
```

This ensures your number never rounds up, which can be important for showing prices, quantities, or capacity values.

Formatting numbers isn't just about aesthetics. It is about trust and clarity. When users see data displayed in a familiar format, it feels more natural and reliable. SwiftUI's declarative format styles allow you to focus on meaning rather than implementation, and because the system automatically adapts to locale, your app is ready for a global audience with almost no additional work.

Percentages

Percentages are everywhere, from progress indicators and discount labels to interest rates and completion stats. SwiftUI makes working with percentages simple by providing a built-in `.percent` format style. With just a single line of code, you can transform a decimal value into a clean and localized percentage.

Let's start with a simple example:

```
Text(0.85, format: .percent)
```

This automatically multiplies the value by 100 and displays it as 85%. The output is localized as well. Users in different regions may see spacing or symbols formatted differently, but SwiftUI manages that for you.

By default, percentages are rounded intelligently, but you can control the number of digits after the decimal point using `.precision(.fractionLength())`. For instance, if you are displaying accuracy rates or statistical data, you might want more precision:

This displays 85.6%, keeping one decimal place. You can also specify a range to avoid unnecessary zeros:

```
Text(0.80, format: .percent.precision(.fractionLength(0...2)))
```

When the value is exactly 0.8, SwiftUI displays 80% instead of 80.00%, keeping the interface clean and readable.

Just like number formatting, percentages automatically adapt to the user's region. Some languages include a space before the percent symbol, while others do not. SwiftUI manages these differences for you so that your interface always feels natural to the user.

Percentages are a small but important part of presenting information clearly. Whether showing task completion, investment growth, or battery level, well formatted percentages make data easier to understand. By using SwiftUI's `.percent` style, you avoid manual string formatting and allow the system to take care of localization, rounding, and presentation for you.

Measurement Formatting

Numbers often represent physical quantities such as distance, weight, or temperature. Showing those values with the right unit and in a format that feels natural to the user's region is essential. SwiftUI makes this easy through its built-in support for `Measurement` and the `.measurement` format style.

Let's start with a simple example:

```
Text(Measurement(value: 25, unit: UnitTemperature.celsius),
      format: .measurement(width: .narrow))
```

This displays **25°C**. If you change the locale to the United States, the same code automatically converts the temperature to Fahrenheit and displays **77°F**. SwiftUI reads the user's regional settings and adjusts both the unit and format automatically.

The `.measurement(width:)` modifier controls how the unit appears. You can choose from `.narrow`, `.abbreviated`, or `.wide`.

```
VStack(alignment: .leading) {
    Text(Measurement(value: 1500, unit: UnitLength.meters),
          format: .measurement(width: .narrow))           // 1.5 km
    Text(Measurement(value: 1500, unit: UnitLength.meters),
          format: .measurement(width: .abbreviated))      // 1.5 km
    Text(Measurement(value: 1500, unit: UnitLength.meters),
          format: .measurement(width: .wide))             // 1.5 kilometers
}
```

This allows you to present measurements in a way that fits the design of your interface. For example, compact units may work better in tables or charts, while wide descriptions are more readable in text based views.

SwiftUI automatically adapts units to the user's region. For example:

- **Meters** become **yards** for users in the United States
- Celsius becomes Fahrenheit

- Kilograms become pounds

This makes your app instantly ready for international audiences without extra logic or conversion code.

Displaying measurements correctly is one of the most subtle details that makes an app feel local and trustworthy. Users notice when an app speaks their language and uses their familiar units. With SwiftUI's measurement formatting, you get automatic localization, built-in conversions, and flexible presentation options – all while keeping your code clean and declarative.

Time and Duration Formatting

Time is one of the most common and important forms of data you will present in your app. Whether you are showing a workout duration, a countdown timer, or the length of a podcast, SwiftUI provides a clear and localized way to display time values through its `.time` and `.timer` format styles.

Let's begin with a simple example.

```
Text(Duration.seconds(120), format: .time(pattern: .minuteSecond))
```

This displays **2:00**, representing two minutes and zero seconds. SwiftUI automatically interprets the value as seconds and formats it according to the pattern you choose.

You can use different patterns depending on what you want to show.

```
VStack(alignment: .leading) {  
    Text(Duration.seconds(65), format: .time(pattern: .minuteSecond))  
    // 1:05  
    Text(Duration.seconds(3675), format: .time(pattern: .hourMinute))  
    // 1:01  
    Text(Duration.seconds(3675),  
    format: .time(pattern: .hourMinuteSecond)) // 1:01:15  
}
```

The `.time` format automatically adapts to the user's locale. For example, some regions use a colon as a separator, while others use a period. SwiftUI manages this automatically.

Displaying time correctly is about more than numbers. It reflects how users experience your app – in moments, progress, and duration. By using SwiftUI's `.time` and `.timer` formats, you ensure that time feels natural and familiar everywhere your app runs. You avoid manual formatting logic, keep your code simple, and deliver an experience that feels local and precise.

Formatting currencies

We cannot talk about number formatting without mentioning currency. I've always enjoyed building personal finance apps, and one thing they all have in common is the need to display amounts in the user's local currency. SwiftUI makes this incredibly easy through the `.currency` format option in the `Text` view.

For example:

```
Text(45.54, format: .currency(code: locale.currency?.identifier ?? "USD"))
```

You simply pass the appropriate currency code, and SwiftUI takes care of formatting it correctly based on the locale.

To experiment with different currencies, you can inject various locale values into the SwiftUI environment. This is especially helpful for testing how your interface adapts to users in different regions. For instance, the following example sets the preview locale to **French**, which uses euros:

```
FormattingDemo()  
    .environment(\.locale, Locale(identifier: "fr_FR"))
```

Here's the implementation of the `FormattingDemo` view:

```
struct FormattingDemo: View {  
    // get the locale  
    @Environment(\.locale) private var locale  
  
    var body: some View {  
        Text(45.54, format: .currency(code: locale.currency?.identifier ??  
"USD"))  
    }  
}
```

Inside the view, we access the current locale and use its currency identifier as the format code. If the identifier cannot be resolved—for example, when running in Xcode previews—we fall back to **USD** as a default.

This approach makes it effortless to display localized currencies, whether you're showing balances, prices, or transactions. SwiftUI handles the formatting rules for you, ensuring that users always see amounts in a format that feels natural for their region.

Custom Number Styles

While SwiftUI provides a rich set of built-in format styles for numbers, currencies, percentages, and measurements, there are times when you may want something more specific. For example, you might want to display large numbers in a compact form such as 1.2K or 3.4M, or show a rating as a string of stars. SwiftUI's formatting system is flexible enough to let you create your own custom format styles that integrate seamlessly with the `Text` view.

You can create a custom number style by conforming to the `FormatStyle` protocol. A format style defines how a value is converted into a string.

Here is a simple example that formats large numbers into compact forms:

```
struct CompactNumberFormat: FormatStyle {  
    func format(_ value: Double) -> String {  
        switch value {  
        case 1_000_000...:
```

```

        return String(format: "%.1fM", value / 1_000_000)
    case 1_000...:
        return String(format: "%.1fK", value / 1_000)
    default:
        return String(format: "%.0f", value)
    }
}

```

You can now use this custom style just like any other SwiftUI format style:

```
Text(CompactNumberFormat().format(12500))
```

This displays 12.5K, making large numbers easier to read without taking up extra space.

You can even take this further by making your format style generic so that it works directly within SwiftUI's `Text` interpolation syntax:

```

extension FormatStyle where Self == CompactNumberFormat {
    static var compact: CompactNumberFormat { CompactNumberFormat() }
}

```

Now you can write:

```
Text(12500, format: .compact)
```

This approach keeps your views clean and declarative while reusing the same logic across the entire app.

Custom number styles are not limited to compact numbers. You can create specialized formatters for your own app domain:

- **Star Ratings:** Convert a floating point rating like 4 into "★★★★☆".
- **File Sizes:** Convert bytes into KB, MB, or GB.
- **Temperature Ranges:** Display combined minimum and maximum values like "18–25°C".

Here is a simple rating formatter example:

```

struct StarRatingFormat: FormatStyle {
    func format(_ value: Double) -> String {
        let fullStars = Int(value)
        let halfStar = value - Double(fullStars) >= 0.5
        return String(repeating: "★", count: fullStars) + (halfStar ? "☆" :
    "" )
    }
}

```

We can also make it easier for callers by implementing a designated format style for the `StarRatingFormat`. This is shown below:

```
extension FormatStyle where Self == StarRatingFormat {  
    static var starRating: StarRatingFormat { StarRatingFormat() }  
}
```

The usage is shown below:

```
Text(4, format: .starRating) // -> ★★★★★
```

Custom format styles help you express domain specific logic in a consistent and reusable way. Instead of scattering string manipulation code throughout your views, you can centralize formatting behavior and apply it declaratively anywhere in SwiftUI. This keeps your code clean, expressive, and easy to maintain, while giving you full control over how values are displayed.

All of these techniques may look like UI concerns on the surface, but they have direct architectural impact. Thoughtful composition reduces coupling, slows down UI entropy, and makes refactoring safer as features evolve.

In SwiftUI, composition is one of the few tools that improves both developer velocity and long-term maintainability at the same time. When views are small, focused, and extensible by design, architectural changes tend to stay localized instead of rippling across the codebase.

CONCLUSION

Composing and customizing views is the craft that turns small pieces into a coherent whole. By separating containers from presenters you keep state, effects, and dependencies out of your visual layer, which stays simple and declarative. That clarity pays off as features grow. Each presenter remains focused on layout and data display, while each container manages flow and logic with intention.

You saw how a focused cell view can quickly multiply when requirements change. The answer is not more nearly identical views, but a single composable building block that accepts configuration and well placed extension points. The flexible `ListRow` shows how titles, subtitles, images, and accessory slots create a wide range of layouts without duplication. When needs diverge in meaningful ways, create a new view instead of bending one thing to do everything.

You also learned when to reach for a view extension versus a view modifier. Use extensions for visual polish that carries no state. Use modifiers when behavior or local state is involved, such as timers, toggles, or interactive overlays. This keeps responsibilities crisp and makes reuse feel effortless.

Formatting in turn transforms raw values into language that users trust. With `FormatStyle`, you present numbers, percentages, measurements, times, and currencies that match a user's region and expectations. Most of the work is declarative and automatic, and you can still define custom styles when your domain demands it.

Finally, button styles bring consistency and personality to your interface. By wrapping visual design and interactivity into a reusable `ButtonStyle`, you can express your brand's identity across the app with a

single line of code. Styles like `ShadowButtonStyle` encapsulate padding, color, animation, and feedback in one clean definition, making your buttons both visually consistent and responsive.

Together, these techniques including composition, extensions, modifiers, formatting, and button styles form a foundation for building SwiftUI apps that are modular, elegant, and easy to maintain.

TESTING

I get paid for code that works, not for tests, so my philosophy is to test as little as possible to reach a given level of confidence (I suspect this level of confidence is high compared to industry standards, but that could just be hubris). If I don't typically make a kind of mistake (like setting the wrong variables in a constructor), I don't test for it. I do tend to make sense of test errors, so I'm extra careful when I have logic with complicated conditionals. When coding on a team, I modify my strategy to carefully test code that we, collectively, tend to get wrong.

- Kent Beck

The primary purpose of writing tests is to ensure that software behaves as expected. Tests provide confidence that changes made in one part of the system won't break functionality in other areas.

Of course, not every application requires tests. For simple apps with straightforward domains, manual testing may be sufficient. However, in most professional environments, developers work with complex domains governed by critical business rules. These rules often form the foundation of how a company operates and generates revenue, making reliable testing essential.

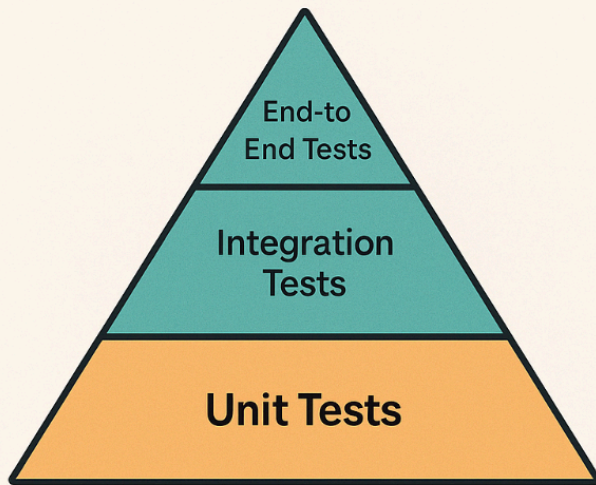
In the SwiftUI community, I have often seen developers reshape their entire app architecture just to make it testable. While this sometimes succeeds in enabling testing, it often comes at a steep cost. The resulting architecture becomes complex, difficult to understand, and harder to reason about.

In this chapter, I will explore different testing techniques and share how developers can write effective tests that deliver the highest return on their investment.

TESTING PYRAMID

The testing pyramid is a visual concept that illustrates the ideal balance of different types of tests in a software project. It emphasizes that not all tests are equal in purpose, speed, or cost. At its foundation are unit tests, which are small, fast, and focused on verifying individual components or functions. Above them are integration tests, which check how different parts of the system work together. At the top are end-to-end tests, which simulate real user interactions and ensure the entire system behaves correctly from the user interface to the backend.

The Testing Pyramid



The idea behind the pyramid is to have a large number of fast and reliable unit tests, fewer integration tests, and even fewer but high value end-to-end tests. This layered approach ensures that your test suite remains efficient and trustworthy. A strong base of unit tests gives quick feedback on core logic, integration tests provide confidence in how modules communicate, and end-to-end tests verify that everything works together in the real world. Maintaining this balance helps teams achieve speed, reliability, and confidence without overwhelming maintenance costs.

The concept was popularized by Mike Cohn in *Succeeding with Agile*, who introduced the pyramid as a response to teams over-relying on fragile UI tests. Many organizations make the mistake of inverting the pyramid—writing too few unit tests and too many end-to-end tests—which leads to slow, brittle suites that break frequently. The testing pyramid serves as a reminder that fast, focused tests form the foundation of quality software.

1. Unit Tests

Unit tests sit at the base of the pyramid and form the foundation of your testing strategy.

Purpose: To verify that individual pieces of code—such as functions, methods, or small classes—work correctly in isolation.

Characteristics: Unit tests are fast to run, easy to write, and highly reliable because they avoid external dependencies. They typically run in memory without involving the file system, network, or database.

Example: Testing that a `calculateTotal()` function correctly adds up item prices or that a `LoanCalculator` returns the right interest rate based on the given credit score.

Goal: Catch small bugs early, provide immediate feedback during development, and ensure that your core logic behaves exactly as expected before integrating with other parts of the system.

Since unit tests execute quickly, they can run on every build or code change, forming the first line of defense against regressions. A solid unit test suite gives developers confidence to refactor code without fear of breaking functionality.

2. Integration Tests

Integration tests occupy the middle layer of the pyramid.

Purpose: To ensure that multiple modules or services in your application work correctly together.

Characteristics: They are slower than unit tests because they often involve communication with databases, APIs, or file systems, but they remain faster and easier to maintain than end-to-end tests. Integration tests provide valuable feedback about how data flows across system boundaries.

Example: Verifying that your `OrderService` correctly interacts with the `PaymentGateway`, or that your `UserRepository` retrieves and saves user data properly using your database client.

Goal: Validate the boundaries and communication between components, ensuring that integrations behave correctly under realistic conditions.

Integration tests help reveal issues that unit tests cannot catch—such as schema mismatches, serialization errors, or configuration problems. They are most effective when they use real implementations for the system under test and lightweight fakes or test doubles for external dependencies that are beyond your control.

3. End-to-end Tests (E2E)

End-to-end tests sit at the top of the pyramid and simulate real user behavior.

Purpose: To verify that the entire application works as intended, from the user interface to the backend and external services.

Characteristics: These are the slowest, most complex, and most expensive tests to write and maintain. They require a stable environment, realistic data, and careful orchestration of setup and teardown steps. However, they provide the highest level of confidence because they mirror how users interact with the system.

Example: Automating a scenario where a user logs in, adds items to a cart, and completes a purchase using a test credit card.

Goal: Confirm that the system functions correctly as a whole and that all critical user journeys are reliable.

Because of their cost, end-to-end tests should focus only on core workflows that represent business value, such as completing a payment, submitting a form, or updating user settings. These are the tests that assure stakeholders the app works from a user's perspective.

Maintaining the Right Balance

In summary, the pyramid encourages many unit tests, some integration tests, and a few end-to-end tests. This balance keeps your test suite fast, maintainable, and reliable while providing comprehensive coverage.

A well-structured pyramid also fits naturally into continuous integration and deployment workflows. Unit tests give instant feedback on every code change, integration tests ensure your system remains stable, and end-to-end tests act as the final safety net before release.

The testing pyramid is not a rigid rule but a guiding principle. The goal is to build a testing strategy that provides maximum confidence with minimal friction. When implemented effectively, it transforms testing from a chore into a powerful feedback mechanism that supports continuous delivery and long-term code quality.

NOT ALL TESTS ARE CREATED EQUAL

The first and most important thing to understand is that not all tests are created equal. Just as not all of your app features are equally important, some tests carry far more weight than others. It is easy to fall into the trap of treating all code with the same level of attention, but in reality, testing is most valuable when it is tied to the priorities of the business.

Imagine you are building an application for a bank. One of the most critical business rules is to charge overdraft fees when an account has insufficient funds. Banks generate billions of dollars in revenue from fees alone. This is not just another feature on a requirements list—it is a core driver of revenue for the business. That is why, as a developer, you must write thorough and high quality tests to ensure that overdraft fee calculations work exactly as intended. Even a small error in this area could translate into millions in lost revenue, regulatory fines, or damage to customer trust. Few features in a banking system are as directly tied to the financial health of the institution as this one.

Now, compare this with other features that may exist in the same banking application. For instance, the app might include functionality for rendering marketing email templates. If an email template has a formatting issue or the spacing looks slightly off, the business impact is negligible. The bank will not lose millions of dollars because a font size was wrong in a message. Customers might find it slightly unpolished, but it is not the kind of error that triggers emergency calls in the middle of the night.

The same reasoning applies to something like displaying a customer's profile picture. A missing or distorted image may frustrate a handful of users, but it does not stop the bank from collecting fees or processing transactions. From a business perspective, the risk is low. In these cases, light testing is often enough to provide basic confidence without sinking resources into exhaustive test coverage.

Logging is another good example. Many applications log events for developers or operations teams. While this information can be useful during debugging, the act of logging itself rarely makes or breaks the business. If you are only writing log entries for internal use, you may not need to test it at all, since it is an implementation detail. However, context always matters. If you are building a logging framework—where logging itself is the core product—then the situation changes completely. In that case, you must thoroughly test the public API because the correctness of the framework is the very value you are delivering to customers.

This raises an important question: how do you decide which features are critical and deserve the most rigorous testing? As a developer, you may not always have the full picture of what drives revenue, reduces risk, or enables growth. This is where collaboration becomes essential. Product managers, business analysts, and domain experts play a vital role in clarifying which rules are central to the business model and which features are merely supportive. By working closely with these stakeholders, you can prioritize your testing strategy around what truly matters.

The point is not that certain features are unimportant, but that their importance is relative. Time and effort spent on testing should align with how much the feature matters to the business. Features that directly generate revenue or enforce critical business rules should receive the highest level of scrutiny,

with comprehensive and carefully designed tests. Features that are cosmetic, ancillary, or primarily developer-facing do not require the same rigor.

The takeaway is clear: always align your testing strategy with business priorities. Good testing is not about chasing one hundred percent coverage or treating every line of code equally. It is about protecting the parts of the system that matter most. By focusing your efforts where the business impact is greatest—and by leaning on the knowledge of product managers and domain experts—you maximize the return on the time you spend writing tests and ensure that your software delivers lasting value.

TEST BEHAVIOR NOT IMPLEMENTATION DETAILS

One of the most common mistakes developers make is testing implementation details instead of testing the behavior of the application. This usually leads to brittle tests that break as soon as internal code changes, even though the visible behavior of the system remains correct.

The real trigger for writing a test is not the introduction of a new class or function. It is the arrival of a new requirement. Just because you added another class does not mean you should immediately write a test. Classes, functions, and internal wiring are implementation details, and those details often change over time. Your tests should always be anchored in business requirements, not in code structures.

Here are a few examples of behaviors derived from requirements:

- When a customer withdraws funds and has insufficient balance, the system charges an overdraft fee.
- When the number of stocks specified by the customer reaches the set limit, they are automatically submitted for trade at the given price.

The behavior stems from the requirement of the project. Tests that check implementation details instead of behavior tend to be fragile and can easily break during refactoring, even when the user facing functionality remains unchanged. During refactoring you should not modify your test. If you must modify your test then it is a clear sign that you are testing the implementation details, instead of the behavior.

Consider a scenario, where you are testing that after a user performs a payment, they are given with a payment receipt. For this feature, you may start with defining a protocol because you will have to mock the `PaymentGateway` instead of invoking a real one. The implementation of the `PaymentReceipt` and the `PaymentGatewayProtocol` is shown below:

```
struct PaymentReceipt: Codable, Equatable {
    let id: UUID
    let amount: Double
    let date: Date
    let status: String
}

protocol PaymentGatewayProtocol {
    func performPayment(amount: Double) async throws -> PaymentReceipt
}
```

Next, implement a mock of the `PaymentGateway` to return a simulated response instead of calling the real gateway, which could incur charges.

This is shown below:

```
class PaymentGatewayMock: PaymentGatewayProtocol {

    var isPerformPaymentCalled: Bool = false

    func performPayment(amount: Double) async throws -> PaymentReceipt {

        isPerformPaymentCalled = true

        // Simulate successful payment
        return PaymentReceipt(
            id: UUID(),
            amount: amount,
            date: Date(),
            status: "Success"
        )
    }
}
```

`CartStore` uses the `PaymentGateway` to process payments and then updates its published `receipt` property. Because `receipt` is published, SwiftUI observes changes and refreshes the interface as needed.

```
@Observable
class CartStore {

    var receipt: PaymentReceipt?
    let paymentGateway: PaymentGatewayProtocol

    init(paymentGateway: PaymentGatewayProtocol = PaymentGateway()) {
        self.paymentGateway = paymentGateway
    }

    func processPayment(amount: Double) async throws {
        receipt = try await paymentGateway.performPayment(amount: amount)
    }
}
```

The next step is to write an integration test that ensures `CartStore` calls `PaymentGateway` correctly. Because this test will run many times a day in continuous integration, we should not call the real payment gateway, which could incur charges and add instability. Instead, use the mock implementation to simulate responses. Here is one way to structure the test.

```
@Test func test_performPayment_is_called_on_gateway() async throws {

    let mockGateway = PaymentGatewayMock()
    let cartStore = CartStore(paymentGateway: mockGateway)
    _ = try await cartStore.processPayment(amount: 100.0)

    #expect(mockGateway.isPerformPaymentCalled == true)
```

```
}
```

The test creates a `PaymentGatewayMock`, injects it into `CartStore`, calls `processPayment`, and then checks whether `isPerformPaymentCalled` is true on the mock.

This is a classic case of testing an implementation detail. If you later refactor the gateway and rename `performPayment` to `processPayment`, the test will fail unless you also rename the flag and every assertion that depends on it. The behavior has not changed, the user still receives a receipt, but the test breaks because it is tied to internal wiring rather than outcomes.

A better approach is to test the behavior. Call `processPayment` and assert that a valid `PaymentReceipt` is returned and that any observable state, such as a published receipt on `CartStore`, is updated as expected.

The behavior focused version of the test is shown below.

```
@Test func test_performPayment_returns_receipt() async throws {
    let mockGateway = PaymentGatewayMock()
    let cartStore = CartStore(paymentGateway: mockGateway)
    try await cartStore.processPayment(amount: 100.0)

    #expect(cartStore.receipt?.amount == 100.0)
    #expect(cartStore.receipt?.status == "Success")
}
```

In the above test, even if you change the underlying implementation details it will not fail as long as the expected behavior is the same.

Next time you are writing a test, ask what outcome the requirement promises and assert that outcome from the outside, not that a specific method ran inside.

SWIFTUI PREVIEWS ARE LIKE TESTS

One of the best features of working with SwiftUI is the ability to use Previews. Similar to the *hot reload* functionality found in React and Flutter, Previews allow developers to instantly visualize the user interface without running the entire app. This tight feedback loop dramatically improves productivity, especially when fine-tuning layouts, animations, or view hierarchies.

Apple engineers often describe Previews as being “like tests.” While this might sound exaggerated, the idea is that a well-configured Preview helps verify how a view behaves under different conditions—just as a test verifies logic. For example, you can render the same view in multiple states, such as loading, success, and error, and confirm visually that it behaves as intended.

```
#Preview("Error State") {
    LoginScreen(errorMessage: "Invalid password.")
}

#Preview("Loading State") {
    ProductListScreen(state: .loading)
}
```

```
}
```

By setting up multiple Previews for different scenarios, you can catch layout issues, color contrast problems, and state management bugs early, long before you write formal tests or run the app on a device. This makes Previews an excellent companion to unit and UI testing, though they do not replace automated tests. Previews validate appearance and presentation, while tests validate behavior and correctness.

Previews can also simulate different environments such as light or dark mode, larger dynamic type sizes, or even different device sizes:

```
#Preview("Dark Mode – iPad") {  
    ContentView()  
        .environment(\.colorScheme, .dark)  
        .previewDevice("iPad Pro (12.9 inch)")  
}
```

For more advanced scenarios, you can inject mock data or dependencies directly into the Preview. This approach helps visualize real world situations without relying on live network calls or databases:

```
#Preview("Sample Order") {  
    OrderDetailView(order: .preview)  
}
```

Although Previews technically work with UIKit, they truly shine when used with SwiftUI because of its declarative nature. They offer instant visual feedback that keeps your design and development process tightly aligned. In practice, treating Previews as visual tests that verify your interface across different states and devices can save hours of debugging and greatly improve the quality of your app's user experience.

TESTING PRESENTATION LOGIC

Presentation logic refers to the logic contained within a SwiftUI view. It is not the same as domain logic because it does not enforce business rules. Instead, presentation logic determines how state is displayed to the user. It includes tasks such as formatting values, enabling or disabling buttons, showing or hiding loading indicators and error messages, selecting appropriate labels and icons for different statuses, handling simple navigation, and mapping domain models into display-friendly data.

Apple does not provide a built-in API to unit test presentation logic inside views. This limitation is one of the main reasons many developers turn to the MVVM pattern. However, as discussed earlier in the MVVM chapter, that approach comes with its own set of challenges and trade-offs.

While Apple's frameworks do not support direct testing of SwiftUI views, third-party libraries such as **ViewInspector** can be used to access views and their properties in unit tests.

So, how can we test presentation logic in SwiftUI without creating dedicated view models for every screen? In many cases, the logic is so simple that it does not warrant individual tests. Instead, **Xcode previews** can serve as an effective way to verify expected behavior visually.

Of course, automated tests should be part of a continuous integration pipeline and should not rely on manual verification. But in practice, not every piece of presentation logic is worth the time investment to test. For trivial cases, relying on previews is often a perfectly reasonable choice.

Here are a few examples of presentation logic tests that provide little to no return on investment:

```
func testTextHasCorrectModifiers() {
    let view = Text("Hello").font(.title).foregroundColor(.blue)
    // Asserting modifier chain or hierarchy is useless
    // because it doesn't affect business logic and can easily change.
}
```

Changing `.font(.title)` to `.font(.headline)` would break the test but not the app's behavior. This kind of test is fragile and provides no real confidence.

```
func testButtonAnimatesOnTap() {
    // Trying to assert that a button animates is pointless.
    // You're testing SwiftUI's built-in animation engine, not your logic.
}
```

The goal is not to verify Apple's framework internals. The animation existing is part of the UI design, not business behavior.

You might never have written tests like these, but I've seen many teams spend weeks maintaining thousands of them – tests that fail constantly but reveal nothing meaningful about app behavior. Instead of investing in such tests, write more domain tests and end-to-end tests.

Code is a liability, not an asset. So goal of software engineer is delivering the maximum amount of desired functionality at the cost of the least amount of code complexity, even as desired functionality evolves over time.

- Greg Brockman

So this begs the question that what kind of presentation logic test are worth implementing. Let's take a look at few scenarios, where testing presentation logic may yield a better result.

Validation Logic Tests:

Consider a scenario where a user is registering for a new account using their email and password. Both the email and password fields are required and must be provided. Additionally, the email must be in a valid format, and the password must be at least 10 characters long.

One possible implementation of this validation logic is shown below:

```
struct RegisterScreen: View {
    @State private var email: String = ""
```

```

@State private var password: String = ""

private var isFormValid: Bool {
    let trimmedEmail =
email.trimmingCharacters(in: .whitespacesAndNewlines)
    let trimmedPassword =
password.trimmingCharacters(in: .whitespacesAndNewlines)

    let emailRegex = #"^[A-Z0-9a-z._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}$"#
    let emailPredicate = NSPredicate(format: "SELF MATCHES %@",
emailRegex)

    return !trimmedEmail.isEmpty
        && !trimmedPassword.isEmpty
        && trimmedPassword.count >= 10
        && emailPredicate.evaluate(with: trimmedEmail)
}
var body: some View {
    Form {
        TextField("Email", text: $email)
        TextField("Password", text: $password)
        Button("Register") {

        }.disabled(!isFormValid)
    }
}

```

Even though the code above works and enforces the required constraints, it is somewhat messy and difficult to read. It also makes unit testing challenging in its current form. A better approach is to extract the validation rules into well-defined extensions on `String`, using computed properties and helper functions. The view can then simply call these helpers to perform validation.

This approach keeps the view clean and focused on presentation, while making the validation logic easy to understand and test in isolation. Below, you can see how the presentation logic is moved into extensions.

```

extension String {

    var isEmptyOrWhitespace: Bool {
        self.trimmingCharacters(in: .whitespacesAndNewlines).isEmpty
    }

    var isValidPassword: Bool {
        self.trimmingCharacters(in: .whitespacesAndNewlines).count >= 10
    }

    var isEmail: Bool {
        let emailRegex = #"^[A-Z0-9a-z._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}$"#
        return NSPredicate(format: "SELF MATCHES %@", emailRegex)
            .evaluate(with: self)
    }
}

```

By moving the validation logic into a String extension, each rule becomes a small and focused unit of behavior that can be tested independently. This not only makes the code more readable but also promotes reusability and reliability. Instead of testing everything through a single view level property like `isFormValid`, we can now write precise unit tests for individual rules, such as checking if a string is blank, verifying password length after trimming, or ensuring proper email formatting.

The following example shows how to write clear and expressive tests for these new extension methods using the Swift Testing framework. Each test focuses on one specific aspect of string validation, which makes it easier to identify and fix issues if the logic changes in the future.

```
import Testing
@testable import TestingPresentationLogic

@Suite("String Extensions")
struct StringExtensionsTests {

    // MARK: - isEmptyOrWhitespace

    @Test("Empty string and whitespace-only are considered empty")
    func isEmptyOrWhitespace_basic() {
        #expect("").isEmptyOrWhitespace
        #expect(" \n\t ").isEmptyOrWhitespace
        #expect(!"a".isEmptyOrWhitespace)
        #expect(!" a ".isEmptyOrWhitespace)
    }

    @Test("Password counts after trimming")
    func isValidPassword_trimming() {
        #expect(" 1234567890 ".isValidPassword) // 10 after trim
        #expect(!" 123456789 ".isValidPassword) // 9 after trim
        #expect(!"".isValidPassword)
        #expect(!" ".isValidPassword)
    }

    // MARK: - isEmail

    @Test("Valid emails (simple, subdomain, uppercase)")
    func isEmail_validSamples() {
        #expect("user@example.com".isEmail)
        #expect("user@mail.example.co.uk".isEmail)
        #expect("USER+tag@EXAMPLE.COM".isEmail)
    }

    @Test("Invalid emails (missing parts, bad dots, spaces)")
    func isEmail_invalidSamples() {
        #expect(!"user.example.com".isEmail) // no @
        #expect(!"user@example".isEmail) // no TLD
        #expect(!"user@example..com".isEmail) // double dot
        #expect(!" user@example.com".isEmail) // leading space
        #expect(!"user@example.com ".isEmail) // trailing space
        #expect(!"@example.com".isEmail) // empty local
    }
}
```

```

        #expect(!"user@.com".isEmail)           // empty domain label
        #expect(!"user@com".isEmail)           // no dot in domain
    }
}

```

Each test function is concise, descriptive, and self-contained.

The test names and messages clearly communicate what behavior is being verified—making the suite easy to read and maintain.

With this setup:

- **Validation rules are isolated**, so failures are immediately traceable.
- **The code is easy to extend**, for example, by adding new checks such as requiring special characters or uppercase letters in passwords.
- **The tests serve as living documentation**, showing how each rule behaves with both valid and invalid input.

This approach keeps your SwiftUI views clean and your validation logic predictable, maintainable, and confidently tested.

Sorting Logic Tests:

Consider a scenario where you have some sorting logic defined inside your view. Sorting logic can represent either business logic or presentation logic, depending on how it is used and whether it relies on domain rules. In this example, we will treat it as presentation logic, since it does not interact with or depend on any business rules.

An example implementation of such logic is shown below:

```

struct SortingPresentingLogic: View {

    @State private var sortOption: SortOption = .ascending

    @State private var customers = [
        "John Doe",
        "Emily Smith",
        "Michael Brown",
        "Sophia Johnson",
        "David Wilson"
    ]

    var body: some View {
        List {
            Button(sortOption == .ascending ? "Sort Descending": "Sort
Ascending") {
                // Toggle sort option
                sortOption = (sortOption
== .ascending) ? .descending : .ascending
            }
        }
    }
}

```

```

        ForEach(customers, id: \.self) { customer in
            Text(customer)
        }
    }
    .onChange(of: sortOption, initial: true) {
        switch sortOption {
        case .ascending:
            customers.sort(by: <)
        case .descending:
            customers.sort(by: >)
        }
    }
}

```

As you can see, we used Swift's built-in `sort` function to perform the actual sorting of the `customers` array. This means that if we decide to write unit tests for this particular scenario, we would essentially be testing the Swift standard library itself. That would not be a good use of our limited time and resources.

However, for the sake of argument, let's assume the sorting logic is more complex, as shown in the implementation above. In that case, we cannot easily test the logic if it lives inside the view's code. To make it testable, we should extract that logic into a separate struct and then write unit tests specifically for that struct.

This is shown in the implementation below:

```

struct CustomerSorter {
    func sort(_ customers: [String], by option: SortOption) -> [String] {
        switch option {
        case .ascending:
            customers.sorted(by: <)
        case .descending:
            customers.sorted(by: >)
        }
    }
}

```

`CustomerSorter` is a simple value type struct that exposes the `sort` function which can be used in the view for sorting purposes. The view code can be updated to use `CustomerSorter` as shown below:

```

List {
    Button(sortOption == .ascending ? "Sort Descending": "Sort
Ascending") {
        // Toggle sort option
        sortOption = (sortOption
== .ascending) ? .descending : .ascending
    }

    ForEach(customers, id: \.self) { customer in
        Text(customer)
    }
}

```

```

        .onChange(of: sortOption, initial: true) {
            customers = sorter.sort(customers, by: sortOption)
        }

```

Since the sorting logic is now encapsulated inside the `CustomerSorter`, this allows us to easily write unit tests for it. This is shown in the following implementation.

```

struct CustomerSorterTests {

    let sorter = CustomerSorter()
    let customers = ["John", "Alice", "Bob"]

    @Test("Sorts customers in ascending order")
    func testSortAscending() {
        let result = sorter.sort(customers, by: .ascending)
        #expect(result == ["Alice", "Bob", "John"])
    }

    @Test("Sorts customers in descending order")
    func testSortDescending() {
        let result = sorter.sort(customers, by: .descending)
        #expect(result == ["John", "Bob", "Alice"])
    }

    @Test("Returns empty array when input is empty")
    func testEmptyArray() {
        let result = sorter.sort([], by: .ascending)
        #expect(result.isEmpty)
    }

    @Test("Handles already sorted input correctly")
    func testAlreadySorted() {
        let result = sorter.sort(["A", "B", "C"], by: .ascending)
        #expect(result == ["A", "B", "C"])
    }
}

```

When I show this technique to iOS developers, one of the first questions I hear is, “Isn’t `CustomerSorter` just a view model with a different name?” It’s a fair question, and it’s easy to see why the confusion arises.

But `CustomerSorter` is not a view model. It is not a class, it does not conform to `ObservableObject`, and it is not a source of truth. In other words, it has no references to the networking layer, data access layer, or any other external dependencies.

`CustomerSorter` is simply a struct, a plain value type no different from an `Int`, `Double`, or `Bool`. Its only purpose is to encapsulate a small piece of presentation logic: sorting.

We saw this idea earlier in the Validation chapter, where we wrapped validation logic inside the `MovieLoginState` struct. The same principle applies here: isolate presentation logic into lightweight, testable units.

Extracting logic into small, focused structs or extensions makes your code more predictable and testable. Not all presentation logic deserves tests, but when it does, isolating it from the view allows you to write clean, expressive tests that add real confidence.

In the end, whether you write unit tests for presentation logic depends on the complexity of the feature. If the logic involves multiple parameters or edge cases, unit tests can provide valuable confidence that everything works as intended. But if the logic is simple and self-evident, like basic validation or sorting, it is perfectly fine to skip testing and focus your effort elsewhere.

TESTING SWIFTDATA MODELS

SwiftData models often contain business logic that defines how data behaves within your application. These rules might include enforcing unique constraints, validating values, or calculating derived properties based on related data. Because this logic resides in the model layer, it is essential to verify that it works correctly through well designed unit tests.

When testing SwiftData models, the goal is to verify your own business rules, not the behavior of the SwiftData framework itself. Writing effective tests means focusing on scenarios that provide real value and confidence, such as preventing duplicate entries, enforcing limits, or computing totals accurately.

Tests are code too, and just like any other code, they come with a maintenance cost. Poorly written or redundant tests add complexity instead of reducing it. By focusing only on tests that validate meaningful business logic, you ensure that your codebase remains reliable, efficient, and easy to maintain over time.

Setting Up the Container

Before writing unit tests for your SwiftData models, you should set up a dedicated model container specifically for testing. Unit tests need to run in isolation, so data created in one test should never affect another. To achieve this, use an **in-memory model container** instead of a persistent store. This approach ensures that each test starts with a clean database and runs faster since nothing is written to disk.

Take a look at the following code, which sets up the model container for the test fixture.

```
struct TestingSwiftDataModelsTests {  
  
    private let context: ModelContext!  
  
    @MainActor  
    init() {  
        let container = try! ModelContainer(for: Budget.self, configurations:  
ModelConfiguration(isStoredInMemoryOnly: true))  
        context = container.mainContext  
    }  
  
    @Test func user_save_budget_successfully() throws {  
        // your test here!  
    }  
}
```

```
}
```

A new instance of `ModelContainer` is created in the initializer, ensuring that each test receives a fresh `ModelContext` and remains isolated from the side effects of other tests. This ensures that your tests run independently and produce reliable results.

If needed, you can also insert seed data into the in memory database before running your tests. This is useful when your tests depend on existing user data such as budgets or expenses to validate calculations or other domain logic.

Writing SwiftData Model Tests

When working with SwiftData, your models often contain important business logic such as enforcing rules, validating data, or performing calculations. These are the parts worth testing. The goal of writing unit tests for SwiftData models is not to verify that the framework itself works, but to ensure that your own domain logic behaves correctly.

Take a look at the following unit test:

```
@Test func user_save_budget_successfully() throws {  
    let budget = Budget(name: "Groceries", limit: 500)  
    context.insert(budget)  
  
    // fetch the newly saved budget  
  
    let fetchDescriptor = FetchDescriptor<Budget>()  
    let budgets = try context.fetch(fetchDescriptor)  
  
    let savedBudget = budgets[0]  
    #expect(savedBudget.name == "Groceries")  
    #expect(savedBudget.limit == 500)  
}
```

The test creates a new `Budget` instance and saves it to an in memory SwiftData container. It then verifies the save by fetching the model from storage.

Although this test may pass, it doesn't offer much value because it merely confirms that SwiftData works as intended rather than validating your own business logic.

Let's look at a test that delivers far more meaningful insight and ensures our rules are being enforced correctly.

```
@Test func throw_exception_when_inserting_budgets_with_duplicate_name() throws  
{  
    let budget = Budget(name: "Vacation", limit: 100)  
    try budget.save(context: context)  
  
    #expect(throws: BudgetError.duplicateName, "Duplicate name exception was  
not thrown.", performing: {
```

```

        let anotherBudget = Budget(name: "Vacation", limit: 500)
        try anotherBudget.save(context: context)

        // also check that in the database there is only one instance
        let budgets = try context.fetch(Budget.all)
        #expect(budgets.count == 1)
    })
}

```

The test above ensures that an exception is thrown when a user attempts to save a budget with a duplicate name. It also verifies that the storage contains only a single budget record, confirming that duplicates are not persisted.

In a similar fashion, you can write high-quality tests that provide value to your codebase and protect you against regressions.

```

@Test func calculate_spent_amount_for_budget() {
    let budget = Budget(name: "Vacation", limit: 500)

    budget.expenses.append(Expense(name: "Rental car", amount: 200))
    budget.expenses.append(Expense(name: "Airfare", amount: 120))

    #expect(budget.spent == 320)
}

```

In the above test, we validated that the `spent` property on the `Budget` instance works as expected. You can apply the same approach to test the `remaining` property and ensure it returns the correct value based on the budget's limit and associated expenses.

By focusing on tests that validate your business logic—like enforcing unique budget names or correctly calculating totals—you're not just testing code, you're safeguarding the behavior of your application. These kinds of tests are resilient, easy to understand, and provide long-term value as your app evolves.

Rather than testing what frameworks already guarantee, invest your time in writing unit tests that cover what your app is responsible for. This ensures your business rules remain intact, your logic remains sound, and your codebase remains trustworthy.

As you continue building with SwiftData, remember: it's not about testing everything—it's about testing the right things.

INTEGRATION TESTING

Integration testing sits between unit testing and end-to-end testing. While unit tests focus on verifying small isolated pieces of logic, integration tests ensure that different parts of the system work correctly together. They help catch issues that appear when multiple components such as services, models, or network layers interact with each other.

The main goal of integration testing is to confirm that the boundaries between components are well defined and behave as expected. For example, you might test how your `OrderService` interacts with a `PaymentGateway`, or how your data layer communicates with an API client. Unlike unit tests, integration tests often rely on real implementations rather than mocks, though you may still replace unmanaged dependencies such as external APIs with test doubles or sandbox environments.

Integration tests provide a good balance between speed and realism. They run faster than end-to-end tests but still give meaningful assurance that the system behaves correctly across multiple layers. When done effectively, they help you identify problems early before they reach the user interface while keeping your test suite maintainable and efficient.

Consider a scenario where you are building an app that allows users to calculate their APR (interest) rate based on their credit score. The user enters their social security number, which is then used to fetch their credit score from a third party credit score service. Once the score is retrieved from this unmanaged dependency, your company's APR service calculates the appropriate interest rate for the client. Finally, the calculated APR is displayed on the screen.

In this scenario, our `APRService` communicates with an unmanaged dependency, `CreditScoreService`. The communication between the two needs to be verified. Since `CreditScoreService` is a third party service, we will have to mock it to verify that `APRService` correctly calls `CreditScoreService`.

Below you can find the implementation of an integration test written using the new Swift Testing framework. The test verifies that internally `APRService` calls the required functions on `CreditCardService`.

```
@Test
func apr_service_calls_get_credit_score_on_credit_score_service() async
throws {

    let validSSN = "123-45-6789"
    let expectedAPR = 3.2

    var mockCreditScoreService = MockCreditScoreService()

    try await confirmation("APRService did not call GetCreditScore on
CreditScoreService", expectedCount: 1) { confirmation in
        // you need to make sure that
mockCreditScoreService.getCreditScore is fired

        mockCreditScoreService.onGetCreditScore = { ssn in
            confirmation()
            return CreditScore(score: 500, lastUpdated: "02/02/2025",
reportedBy: "Experian")
        }

        let aprService = APRService(creditScoreService:
mockCreditScoreService)
        let actualAPR = try await aprService.getAPR(ssn: validSSN)
        #expect(actualAPR == expectedAPR)
    }
}
```

The new confirmation function sets up the expectation. The expected count makes sure that the confirmation function is called exactly once. Inside the confirmation function, we call the mocked version of the `CreditScoreService` service. On receiving the credit score, we call the confirmation function, hence fulfilling the expectation.

If `APRService` internally does not call the `getCreditScore` function on `CreditScoreService` then the test will fail. The test will also fail, if the `getCreditScore` function was called more than one time. This is due to the fact that in our expectation we indicated that the expectation only needs to be fulfilled once.

You should only mock unmanaged dependencies. For managed dependencies you should use the real instances.

To wrap up, treat integration tests as living contracts at the seams of your system. Keep external boundaries deterministic by replacing unmanaged dependencies with stable fakes or protocol backed stubs, or point to a sandbox when you need to exercise the real stack. Assert what matters to the domain, such as a single call to the credit score service and a correct APR outcome, and avoid assertions that mirror private implementation details. Keep each test focused and fast, seed it with realistic data, and run this suite in CI after unit tests but before any end-to-end pack. As contracts or data shapes evolve, these tests will fail early and loudly, giving you confidence that the parts still fit together as the system grows.

END-TO-END TESTING

End-to-end testing is the best kind of testing against regression. It truly tests the app from the user's point of view, touching every single layer and abstraction in your application from the UI all the way down to the database or network. Unfortunately, most developers skip end-to-end testing because it is complicated to write, slow to run, and often difficult to maintain as the app evolves.

However, when done right, end-to-end tests provide the highest level of confidence that your app actually works in the real world. They catch integration issues that unit or integration tests might miss such as misconfigured dependencies, broken navigation flows, or race conditions between asynchronous operations.

Modern tools like `XCTest`, `XCUITest`, `Playwright`, and `Cypress` have made it easier to automate these tests even for mobile applications. The key is to write a few high value end-to-end tests that simulate the most critical user journeys for example signing in, completing a checkout, or creating a new record instead of trying to automate everything.

Think of end-to-end testing as your final safety net. Unit tests verify your logic, integration tests check your contracts, but end-to-end tests ensure your users do not wake up to a broken experience after your next deploy.

Writing End-to-End Tests

Let's revisit the APR calculation app introduced earlier. In that example, we tested how the `APRService` interacts with the `CreditScoreService` to compute the correct interest rate. In end-to-end testing, we take a broader view. Instead of testing these components in isolation, we verify that a real user can open the app, enter their social security number, trigger the APR calculation, and see the correct result displayed on the screen. This ensures that all layers from the user interface to the network and data flow work together seamlessly in a real world scenario.

The key idea is that we should not mock any dependencies in this test, since the goal of end-to-end testing is to verify the system as a whole. However, this does not mean we will call the production endpoints of third party services. In practice, these unmanaged dependencies usually provide a dedicated QA or sandbox environment that can safely be used for testing without affecting real data.

Unmanaged dependencies in testing are those that lie outside your direct control. These typically include third-party services such as payment gateways, external APIs, or cloud storage providers.

You can use a factory to return the credit score service based on the environment you are running on.

Here is a simple `CreditScoreFactory` implementation.

```
enum CreditScoreServiceFactory {  
  
    case qa  
    case production  
  
    static var current: CreditScoreServiceFactory {  
        ProcessInfo.processInfo.arguments.contains("QA") ? .qa: .production  
    }  
  
    var service: CreditScoreServiceProtocol {  
        switch self {  
            case .qa:  
                return QACreditScoreService()  
            case .production:  
                return CreditScoreService()  
        }  
    }  
}
```

There are several ways to configure different environments in Xcode. The example above demonstrates a simple setup, but you can also explore more advanced options such as Xcode schemes, `.xcconfig` files, and other configuration management techniques.

The user interacts directly with the `APRService`, which in turn communicates with the `CreditScoreService` to retrieve the user's credit score. In SwiftUI, dependencies like these are commonly injected into views using the `Environment` or custom `EnvironmentValues`. This approach keeps the view clean and testable while allowing the service to be easily swapped out based on the current environment. The following example shows how this can be implemented:

```
extension EnvironmentValues {
```

```

    @Entry var aprService = APRService(creditScoreService:
CreditScoreServiceFactory.current.service)
}

```

Inside the view, we can access the injected `APRService` from the environment and call its `getAPR` function to calculate the interest rate. The example below demonstrates how this is done:

```

struct ContentView: View {

    @Environment(\.aprService) private var aprService

    @State private var ssn: String = ""
    @State private var apr: Double?
    @State private var message: String?

    var body: some View {
        Form {
            TextField("Enter ssn", text: $ssn)
                .accessibilityIdentifier("ssnTextField")
            Button("Calculate APR") {
                Task {
                    do {
                        apr = try await aprService.getAPR(ssn: ssn)
                    } catch {
                        message = error.localizedDescription
                    }
                }
            }
        }
    }
}

```

The user interface is simple. Once the APR is calculated, the result is displayed directly on the screen (not shown in the snippet above).

An end-to-end test follows the same sequence of actions a real user would take to calculate the interest rate. This includes launching the app, tapping the text field, entering a social security number, and pressing the **Calculate APR** button. This is what makes end-to-end tests so valuable, as they validate the complete user flow and confirm that all layers of the application work together as expected.

The following example shows an end-to-end test that verifies a successful APR calculation and ensures the result is correctly displayed to the user.

```

final class OuluBankR1UITests: XCTestCase {

    var app: XCUIApplication!

    override func setUp() {
        app = XCUIApplication()
        app.launchArguments = ["QA"]
        app.launch()
    }

    func test_user_can_calculate_apr_successfully_with_valid_ssn() {

```

```

let ssnTextField = app.textFields["ssnTextField"]
ssnTextField.tap()

ssnTextField.typeText("123-45-6789")

let calculateAPRButton = app.buttons["calculateAPRButton"]
calculateAPRButton.tap()

let aprText = app.staticTexts["aprText"]
let aprTextExists = aprText.waitForExistence(timeout: 5.0)

XCTAssertTrue(aprTextExists)
XCTAssertEqual(aprText.label, "3.2", "Expected APR to be 3.2 but got \(aprText.label)")
}
}

```

This UI test is written using the XCTest framework, since at the time of writing the Swift Testing library supports only unit tests and not UI tests.

The test begins by setting the launch arguments, which are passed to the SwiftUI app to determine the correct credit score service to use. Notice how the test is named – it clearly describes the expected behavior in plain language. A descriptive test name helps anyone on your team quickly understand the test’s purpose without needing to read the implementation.

Also note that the test compares the result against a specific value (3.2). This is appropriate when the environment is deterministic and always returns the same value. However, if your environment involves nondeterministic factors such as a live credit score service, changing rounding rules, or locale-dependent formatting, it’s better to assert on behavior rather than an exact value – for example, checking that the APR is displayed, that it follows the correct format, or that it falls within an acceptable range.

Xcode also provides a recording tool that allows you to capture your interactions with the app’s user interface and automatically generate the corresponding Swift code for your UI tests. This feature is especially useful for getting started quickly, as it helps you create the basic structure of a test without writing every command manually. You can then refine the generated code to make it more readable, maintainable, and aligned with your project’s testing conventions.

Page Object Pattern

Even though the test above works, if you plan to write several tests that start from the same screen you will benefit from using the Page Object pattern, also known as the Robot pattern. The Page Object pattern encapsulates a screen’s elements and the actions you can perform on them, which keeps your tests readable and easier to maintain.

Start by creating a class that represents the screen you are testing. In this case, call it `APRRequestScreen`. Then add properties for the UI elements and methods for the actions they support, such as taps and text entry. A partial implementation of `APRRequestScreen` is shown below.

```

class APRRequestScreen {

    let app: XCUIApplication

    init(app: XCUIApplication) {
        self.app = app
        self.app.launchArguments = ["UITEST"]
        self.app.launch()
    }

    private lazy var ssnTextField: XCUIElement = {
        let textField = app.textFields["ssnTextField"]
        return textField
    }()

    private lazy var calculateAPRButton: XCUIElement = {
        let calculateAPRButton = app.buttons["calculateAPRButton"]
        return calculateAPRButton
    }()

    private lazy var aprText: XCUIElement = {
        let aprText = app.staticTexts["aprText"]
        return aprText
    }()

    func enterSSN(_ text: String) -> Self {
        ssnTextField.tap()
        ssnTextField.typeText(text)
        return self
    }
}

```

You can name your properties however you prefer, but I like to use the element type as a suffix—for example, `ssnTextField` or `calculateAPRButton`. This naming style makes it easy to identify what kind of UI element each property represents.

All the properties are also marked as `lazy` because locating elements by their accessibility identifier is a relatively expensive operation. The `XCUIApplication` instance must traverse the entire view hierarchy to find matching nodes, which can impact performance if done repeatedly. By marking these properties as `lazy`, the lookup happens only once when the element is first accessed, avoiding unnecessary searches and making the tests more efficient.

The usage of the `APRRequestScreen`, following Page Object Pattern is shown below:

```

func test_user_can_calculate_apr_successfully_with_valid_ssn() {

    screen
        .enterSSN("123-45-6789")
        .tapCalculateAPRButton()

    XCTAssertTrue(screen.aprTextExists())
    let aprText = screen.getAPRText()
    XCTAssertEqual(aprText, "3.2", "Expected APR was 3.2 but got \(
aprText)")
}

```

```
}
```

Below is another example that demonstrates how to write a test using the Page Object pattern. This test verifies that an appropriate error message is displayed when a credit score cannot be found for the given social security number.

```
func test_displays_error_message_when_credit_score_was_not_found_for_ssn() {  
    let expectedErrorMessage = "No credit score was found for the provided  
SSN."  
  
    screen  
        .enterSSN("211-11-1111")  
        .tapCalculateAPRButton()  
  
    XCTAssertTrue(screen.messageTextExists(), "Message text should not be  
empty.")  
    XCTAssertEqual(screen.getMessageText(), expectedErrorMessage)  
}
```

Although Page Object Pattern requires some initial work to setup, it does pay dividends in the future, specially if you are writing a decent number of end-to-end tests. This leads us to the next question as how many end-to-end tests we should aim for.

How many End-to-End is Enough?

In an ideal world, we would have many end-to-end tests covering every possible scenario we could imagine. But we do not live in that ideal world. End-to-end tests are complex to write and take longer to run. When I say they are slow, I do not mean that as a negative thing—the slowness comes from the amount of real work they perform.

End-to-end tests simulate how the software is actually used by the user. They touch every layer of the application, from the user interface to the data stores, network calls, and local storage. Each of these layers adds time to the execution, but that time is well spent because it gives us confidence that the system works as a complete unit, just as our users will experience it.

In his book [Unit Testing Principles, Practices, and Patterns](#), Vladimir Khorikov recommends writing end-to-end tests for the most critical workflows in your application. He emphasizes creating comprehensive “**happy path**” tests that validate the main user journey, along with additional tests that focus on important edge cases. This balanced strategy ensures high confidence in the system’s behavior while keeping test maintenance manageable.

Here’s an example of a happy path end-to-end test:

As a user, I should be able to log in with valid credentials, browse the list of available products, add multiple items to my shopping cart, and successfully complete the checkout process using my credit card.

This test verifies the primary flow of the application from start to finish, covering authentication, navigation, data retrieval, and payment, ensuring that the core experience works smoothly when everything functions as expected.

And here is an example of edge case.

As a user, if I try to checkout but one of the items in my cart has gone out of stock, the system should prevent the checkout process, display an appropriate error message, and allow me to remove or replace the unavailable item before proceeding.

This type of test ensures that the system behaves correctly under unexpected or exceptional conditions, complementing the long happy path test. Together, these tests provide high confidence that both the main workflow and critical edge scenarios function as intended without requiring exhaustive end-to-end coverage.

As with most testing strategies, the goal is not to test everything through end-to-end tests, but to test the right things. End-to-end tests are valuable because they simulate real user interactions across the full stack, but they are also slow, brittle, and expensive to maintain. The best approach is to reserve them for verifying the most critical user journeys that directly impact revenue, customer trust, or business goals, and complement them with faster unit and integration tests for detailed coverage.

By combining a few well chosen end-to-end tests with a strong foundation of unit and integration tests, you can achieve both confidence and speed in your test suite. The Page Object Pattern plays an important role in making these end-to-end tests more maintainable and readable as your application grows.

VIEW MODEL TESTS DOES NOT TEST THE UI

Before we move on, there is one common misunderstanding worth clearing up, because I see it come up again and again in real projects.

There is an important clarification that deserves attention, even though it does not perfectly fit into the main flow of this chapter. Many developers assume that by following the MVVM pattern, writing tests for their view models automatically means that the user interface is being tested too. The same misconception often appears with the MV pattern. Testing logic inside a view or a view model only verifies the presentation logic – it does not guarantee that the actual user interface behaves correctly.

In a SwiftUI app built with MVVM, the view model holds the presentation logic, and it is completely valid to write tests for that layer. Those tests confirm how the view model transforms and manages data, but they stop short of verifying whether the interface renders correctly or responds properly to user actions.

To make this distinction clear, consider a simple example: a counter app where tapping a button increases a counter value, and the updated number appears on the screen.

The complete code for the app is shown below:

```

@Observable
class CounterViewModel {

    private(set) var value: Int = 0

    func increment() {
        value += 1
    }
}

struct ContentView: View {

    @State private var counterVM = CounterViewModel()

    var body: some View {
        VStack {
            Text("\(counterVM.value)")
            Button("Increment") {
                counterVM.increment()
            }
        }
    }
}

```

There is nothing complex about this code. It is simply a counter app. In fact, a view model is not needed here because the counter value can be managed using a private local `@State` variable inside the view. With that in mind, let us focus on writing a unit test for the increment function. The implementation is shown below.

```

@MainActor
struct LearnTests {

    @Test func increment_counter_view_model_value() async throws {
        let counterVM = CounterViewModel()
        counterVM.increment()
        #expect(counterVM.value == 1)
    }
}

```

This unit test correctly verifies that the `value` property on `CounterViewModel` increases when `increment` is called. It confirms that the presentation logic exposed by the view model is working as expected. But it does not test anything about the user interface. To see this in action, change the view code to the version below and notice that the test still passes even if the screen no longer reflects the state correctly.

```

struct ContentView: View {

    var body: some View {

```

```
}    EmptyView()
```

I have removed everything from the view, and it now only renders an `EmptyView`. Go ahead and run the unit test again. The test will still pass. This is expected because the test is not interacting with any user interface elements—it only validates the presentation logic inside the view model.

If you want to verify your user interface, you need to write a proper UI test. Do not rely on your view models to confirm UI behavior, as that will only give you a false sense of confidence.

For a scenario this simple, I usually do not write tests for the presentation logic and instead rely on previews and manual verification.

I honestly did not plan to include this section, but I have met many developers over the years who firmly believe that testing a view model is the same as testing the user interface. There is nothing wrong with writing tests for your view model logic, but if your goal is to verify the user interface, then you must write an actual UI test.

WRITING TESTS USING AI

Over the years, I've had countless conversations with developers. Lately, with the rise of AI tools, one thing keeps coming up: people are using AI to write their unit tests.

I find that interesting.

A long time ago, I watched a talk where the speaker shared his experience working in a very complicated domain. He explained how he used tests not just to verify code, but to understand the business rules. That idea stuck with me.

I've used the same approach at almost every company I've worked at. These were not simple systems. The domains were complex, and most of the real challenge was not writing code, but understanding how the business actually worked.

Writing unit tests in these environments is not busy work. It's a learning tool. Tests force you to ask the right questions. What should happen? What should never happen? What edge cases matter? Those answers live in the business rules, not in the code.

This is where AI needs to be used carefully.

AI can help with the mechanics of testing. It can scaffold test files, generate repetitive cases, clean up setup code, and even suggest edge cases you might have missed. Used this way, it saves time without taking away understanding.

But AI should not be the author of your tests.

If you ask AI to write all your tests from scratch, you skip the most important part of the process: learning the business rules. You end up with shallow tests that mirror implementation details or, worse, lock in incorrect assumptions about how the system should behave.

Use AI to write the plumbing. You write the meaning.

You should take every opportunity to learn the business rules of your project. You'll quickly realize that programming is the easy part. Understanding the business is the hard part.

And that part cannot be automated.

CONCLUSION

Great tests are not about volume, they are about confidence. Start from the business rules that move the needle, express them as behavior, and choose the lightest test that proves the outcome. Let unit tests lock down pure logic, let integration tests guard the seams, and keep a small suite of high value end-to-end flows for true user confidence. Mock only unmanaged dependencies, prefer real collaborators for the rest, and avoid tying assertions to private wiring.

Keep tests readable, fast, and independent. Name them by behavior, seed them with realistic data, and run them continuously in CI. When a requirement changes, update the tests that describe that requirement, not a web of incidental checks. Periodically prune flaky or low value tests and refactor test code just like production code.

If you remember only this, you will be fine: test what matters, test behavior, and use the right level of test for the job. Do this and your codebase will ship faster, break less, and stay easier to change as the product grows.

PERFORMANCE AND DEBUGGING

I have seen the same pattern over and over. A SwiftUI app starts out feeling light and effortless. Then features pile up. A list gets bigger. More state flows through the screen. Images start loading. Animations get added. Nothing looks obviously wrong, but the app starts to feel slightly annoyed with you. Scrolling is not as smooth. Taps feel delayed. Sometimes state resets and you cannot explain why.

This chapter is about getting that smooth feeling back. Not by micro optimizing every line, but by fixing the usual suspects. We will talk about stable identity, smaller views, smarter data fetching, caching images and requests, moving work off the main actor, and using the newer Observation system so only the views that need to update actually update. You will also learn a few debugging tricks that make SwiftUI less mysterious when it re-evaluates your views.

VIEW IDENTITY

SwiftUI represents your user interface as a tree of views. Every node in that tree has an identity. SwiftUI uses this identity during its diffing process to decide which views can be reused and which need to be recreated when state changes.

When identities remain stable, SwiftUI can efficiently update only the parts of the UI that actually changed. This preserves local state, keeps animations smooth, and avoids unnecessary work. When identities change unexpectedly, SwiftUI assumes it is dealing with a brand new view. The old view is discarded, its state is lost, and everything starts over.

Consider the following example that displays a list of customers:

```
struct Customer {
    let name: String
    let address: String
    let phoneNumber: String
    let isPremium: Bool
}

List(customers) { customer in
    CustomerRow(customer: customer)
}
```

If you try to compile this, you'll get an error:

Initializer 'init(_:rowContent:)' requires that 'Customer' conform to 'Identifiable'

SwiftUI needs a stable way to tell one row apart from another. Without identity, it cannot reliably match old views to new ones during updates.

You can fix this by conforming to `Identifiable`:

```

struct Customer: Identifiable {
    let id: UUID
    var name: String
    var address: String
    var phoneNumber: String
    var isPremium: Bool
}

```

Now each `Customer` has an identity, and SwiftUI can efficiently diff the list. When one customer changes, SwiftUI updates only that row instead of rebuilding the entire list.

One important detail is often missed here. The identity must be stable across updates. If you recreate your customer models from scratch on every refresh and generate new UUIDs each time, SwiftUI will still treat every row as new. In real apps, prefer identifiers that come from your database or server. Generate a UUID only once and persist it.

This concept is not unique to SwiftUI. React requires a `key` for list elements for the same reason. Both frameworks rely on identity to reconcile UI efficiently.

The danger of `.id()`

SwiftUI also lets you manually assign identity using the `.id()` modifier. Used carefully, this can be helpful. Used incorrectly, it can destroy performance.

```

List(customers) { customer in
    CustomerRow(customer: customer)
        .id(UUID())
}

```

At first glance, this looks harmless. In reality, it is one of the fastest ways to make a SwiftUI list perform poorly.

`UUID()` generates a new value every time the view updates. That means SwiftUI can never match the new row with the old one. Every update forces SwiftUI to tear down and rebuild all rows from scratch.

The consequences are easy to spot:

- All `@State` inside `CustomerRow` resets
- Animations restart
- Toggles jump back to defaults
- Scrolling performance degrades
- Flickering appears during updates

Only use `.id()` when you intentionally want to force SwiftUI to recreate a view.

Identity and if/else

Identity issues are not limited to lists. Conditional views are another common source of accidental resets.

```
struct CustomerBadge: View {
    let isPremium: Bool

    var body: some View {

        if isPremium {
            Text("Premium")
        } else {
            Text("Standard")
        }
    }
}
```

Even though this looks like one logical UI element, SwiftUI sees two different branches at the same position, which can lead to state loss if identity is not preserved. When `isPremium` flips, SwiftUI removes one view and inserts another. Any local state inside those views is lost.

If you want state to survive, keep the identity stable and change the content instead:

```
struct CustomerBadge: View {
    let isPremium: Bool

    var body: some View {

        Text(isPremium ? "Premium" : "Standard")
            .foregroundColor(isPremium ? .yellow : .secondary)
    }
}
```

Or wrap both branches in a container that preserves identity.

```
ZStack {
    if customer.isPremium {
        Text("Premium")
            .foregroundColor(.yellow)
    } else {
        Text("Standard")
            .foregroundColor(.secondary)
    }
}
```

Wrapping conditional content in a stable container preserves the container's identity, even though the conditional views inside it may still be replaced.

The rule of thumb is simple. If SwiftUI sees a different view type *at the same position in the hierarchy*, it assumes a different identity.

Takeaway

Stable identity is foundational to SwiftUI performance. Lists, conditionals, animations, and state all depend on it.

Use identifiers that come from your data model. Avoid generating new IDs during updates. Be mindful of conditional view structures that swap entire views. When identity stays stable, SwiftUI can do what it does best: update only what changed and leave everything else alone.

USING SELF._PRINTCHANGES() TO UNDERSTAND SWIFTUI UPDATES

One of the most confusing things about SwiftUI performance is that nothing *looks* wrong. The UI updates. The app works. But something feels off. A list scrolls a little heavier than it should. A view updates more often than you expect. And you are left guessing which part of the UI is actually doing work.

`Self._printChanges()` is how you stop guessing.

You drop it inside a view's `body`, and SwiftUI prints a message to the console every time that view is re-evaluated, along with why SwiftUI thinks it needs to update.

```
struct CustomerRow: View {  
    let customer: Customer  
  
    var body: some View {  
        let _ = Self._printChanges()  
  
        HStack {  
            Text(customer.name)  
            Spacer()  
            Text(customer.status)  
        }  
    }  
}
```

Now, every time `CustomerRow` is re-evaluated, you will see a log in the console. This immediately answers a few important questions:

- Is this view updating more often than I expected
- Is a change higher up the tree causing this view to re-evaluate
- Are list rows rebuilding during scrolling
- Did a small state change invalidate a much larger part of the UI

Where this is most useful

`Self._printChanges()` is most effective when you use it at different levels of your view hierarchy.

Try placing it in:

- The screen or container view
- A list row
- A small subview inside the row

When you trigger a state change, you can see exactly how far the update propagates. If everything logs at once, you have a broad invalidation problem. If only the small subview logs, SwiftUI is doing exactly what you want.

How to think about the output

Seeing logs does not mean SwiftUI is doing something wrong. SwiftUI re-evaluates view bodies often, and that part is cheap. What matters is how much of the tree is involved and what work happens during those re-evaluations.

If a view that does not depend on the changed state is re-evaluating, that is usually a signal that identity, state ownership, or observation boundaries need attention.

A simple rule of thumb

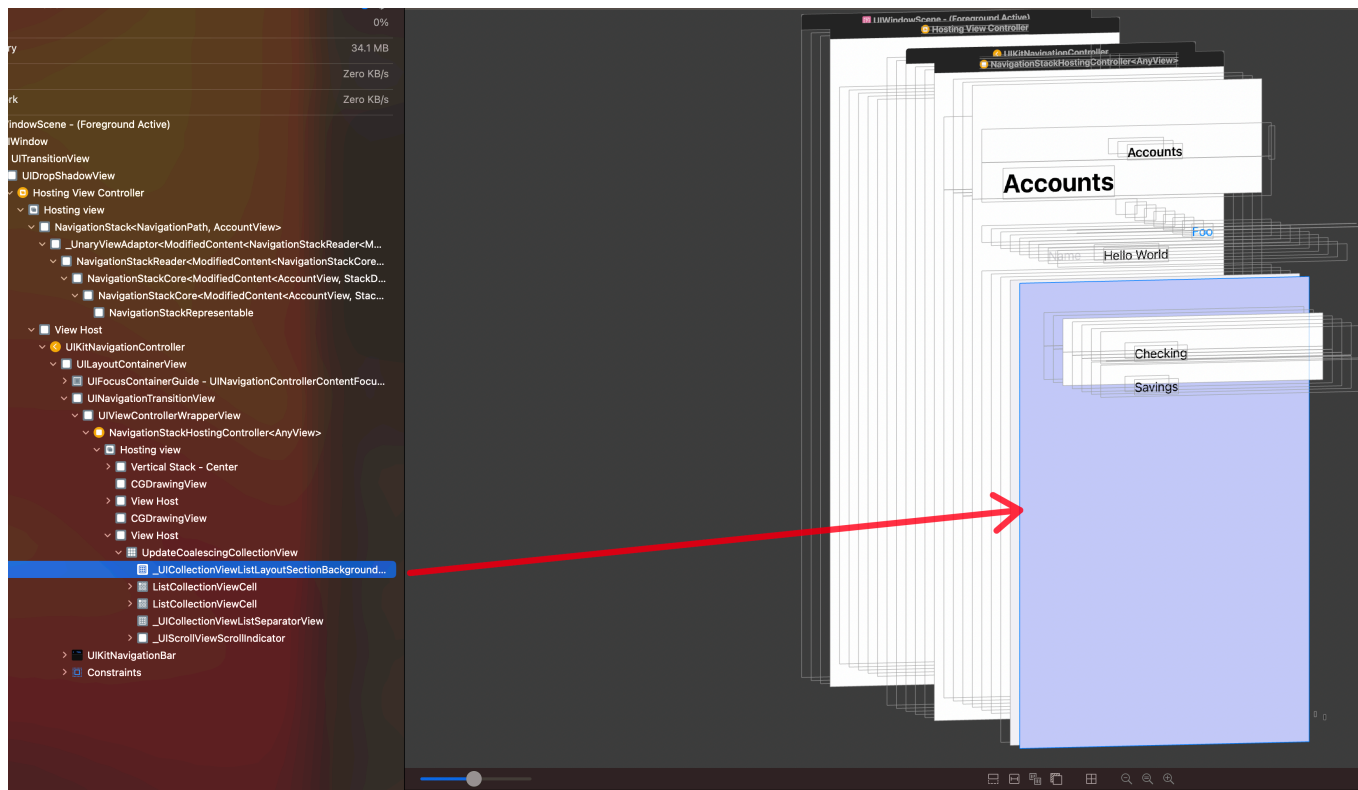
Use `Self._printChanges()` as a flashlight, not a hammer.

Turn it on when something feels off. Use it to confirm your mental model of how state flows through your UI. Then remove it once you understand what is happening. **It is a debugging tool, not something that belongs in production code.**

When you combine `Self._printChanges()` with stable identity, focused views, and precise observation, SwiftUI performance becomes far less mysterious and much easier to reason about.

VIEWING THE SWIFTUI VIEW HIERARCHY

Sometimes performance issues are not about the code you wrote. They are about the views SwiftUI actually created.



The image above comes from Xcode's **Debug View Hierarchy**. On screen, this UI looks simple: a navigation stack with a list of accounts. Nothing unusual. But when you pause the app and inspect the hierarchy, a very different picture appears.

On the left, you see the underlying UIKit view tree. On the right, Xcode shows a 3D exploded view of the same UI. What appears to be a single list is actually made up of many layers: hosting views, containers, background views, separators, and layout helpers.

This is normal. SwiftUI renders into UIKit. But seeing it makes the cost visible.

What this image is really showing

The highlighted section points to a `UICollectionView` backing a SwiftUI `List`. Each row becomes a real UIKit cell. Each separator is its own view. Each background and modifier adds another layer.

Individually, none of this is a problem. But when this structure is repeated dozens or hundreds of times in a scrolling list, small inefficiencies add up.

This is why:

- Applying shadows or overlays at the wrong level hurts scrolling
- Unstable identity causes rows to be rebuilt instead of reused
- “Simple” modifiers inside lists can feel surprisingly expensive

Nothing here is broken. SwiftUI is doing exactly what it is supposed to do. The hierarchy just shows you where the weight actually lives.

How to use this tool effectively

You do not need to understand every UIKit class in the tree. That is not the goal.

Instead, look for patterns:

- How deep is the hierarchy for a single row
- How many layers repeat for each item
- Which modifiers introduce extra wrapper views
- Whether large containers are being recreated when they should be reused

This is especially powerful when paired with `self._printChanges()`. First, you identify which views are re-evaluating. Then, you inspect what those views expand into at runtime.

When to reach for the view hierarchy

If a screen feels heavier than it should and the code looks innocent, this tool is often the fastest way to understand why. It gives you a visual explanation that code alone cannot.

SwiftUI is declarative, but performance is physical. Viewing the hierarchy lets you see that physical reality.

VIEW COMPOSITION

One of the simplest and most effective ways to improve SwiftUI performance is to break large views into smaller ones and pass only the data each view actually needs. This allows SwiftUI to localize updates and avoid unnecessary work when state changes.

SwiftUI re-evaluates view bodies frequently. That is normal and cheap. What really matters is how much work happens during that reevaluation and how much of the view hierarchy is affected. Smaller, focused views make it easier for SwiftUI to update only what changed and leave everything else alone.

Let's start with a simple example that displays a list of customers in `CustomerListScreen`.

```
struct CustomerListScreen: View {
    @State private var customers: [Customer] = []

    var body: some View {
        List(customers) { customer in
            VStack {
                Text(customer.name)
                Text(customer.address)
                Text(customer.phoneNumber)
            }
        }.task {
            customers = // get customers from API
        }
    }
}
```

```
}
```

Customers can be fetched from an API or from any other persistence layer and displayed on the screen. The code works and the task is achieved, but we can make it better.

Instead of letting `CustomerListScreen` handle both data ownership and presentation, we can extract the customer list into a smaller, reusable view called `CustomerListView`.

It is important to understand the intent behind these two views. `CustomerListScreen` is not meant to be reusable. It acts as a container or screen and may eventually host navigation logic, loading states, alerts, or other UI elements. `CustomerListView`, on the other hand, is designed purely for reuse. If another screen needs to display a list of customers, it can simply reuse this view without duplicating layout code.

The refactoring looks like this:

```
struct CustomerListView: View {
    let customers: [Customer]

    var body: some View {
        VStack {
            Text("Customers (\(customers.count))")
                .font(.title)

            List(customers) { customer in
                CustomerRow(customer: customer)
                    .padding(.vertical, 6)
            }
        }
    }
}
```

We went one step further and extracted each row into its own `CustomerRow` component. This keeps `CustomerListView` lightweight. SwiftUI may still re-evaluate it, but most of the work lives in `CustomerRow`, and updates stay localized to the row that actually changed. This makes performance more predictable and the code easier to maintain.

You can apply the same idea inside the row itself. In the example below, the progress indicator is extracted into its own view and receives only the data it needs.

```
struct CustomerRow: View {

    let customer: Customer

    var body: some View {
        HStack(spacing: 12) {
            CircularProgressView(progress: customer.progress)
                .frame(width: 44, height: 44)

            VStack(alignment: .leading, spacing: 4) {
                Text(customer.name)
            }
        }
    }
}
```

```

        .font(.headline)

        Text("\(Int(customer.progress * 100))% complete")
        .font(.subheadline)
        .foregroundColor(.secondary)
    }

    Spacer()
}
.padding(.vertical, 8)
}
}

```

Because `CircularProgressView` depends only on the `progress` value, SwiftUI can keep the rest of the row structurally identical and localize rendering work to the subview whose inputs changed.

Breaking views into smaller, focused components allows SwiftUI to avoid invalidating and recomputing parts of the view hierarchy whose inputs have not changed. This is an important point.

For example, if you add a `@State` counter to `CustomerListScreen` and update it, only the views that actually read that counter will be affected. `CustomerListView`, `CustomerRow`, and `CircularProgressView` will remain untouched because none of them depend on that state.

As your app grows, these compositional boundaries become even more valuable. They make views easier to reuse, easier to test, and easier to evolve without unintended side effects. Performance becomes a natural side effect of good design rather than something you have to chase after the fact.

@FETCHREQUEST AND @QUERY

When building SwiftUI applications that use Core Data or SwiftData, Apple provides two property wrappers that make data access simple and efficient. These are `@FetchRequest` for Core Data and `@Query` for SwiftData.

Some developers choose not to use these property wrappers because they are available only inside SwiftUI views. Instead, they prefer manual data fetching through `NSManagedObjectContext`, `NSFetchedResultsController` or `ModelContext`. This often happens due to a misunderstanding of how these wrappers work. In reality, `@FetchRequest` and `@Query` are carefully designed to work perfectly with SwiftUI's data flow and are highly optimized for performance and reactivity.

Although these property wrappers can only be used inside views, you can still design your architecture in a clean and reusable way. As explained in the SwiftData Architecture chapter, you can extract the construction of the query into your model layer or a dedicated data service. This allows you to test your query logic separately, reuse it across different screens, and keep your SwiftUI views focused purely on presentation.

It is also important to understand that while SwiftData is a new framework, it is built on top of Core Data. Under the surface, SwiftData is built on Core Data primitives and integrates deeply with Core Data's

change tracking mechanisms. This enables automatic updates whenever data in the store changes, ensuring that your user interface always stays consistent with the underlying data.

For performance optimization, SwiftData also provides additional tools for controlling how much data is loaded into memory. When creating a custom `FetchDescriptor`, you can specify a `fetchLimit` to restrict the number of records returned from the persistent store. This is very useful when dealing with large datasets, where you only need to show a subset of records such as the most recent items or a preview list. For example:

```
let descriptor = FetchDescriptor<Task>(
    sortBy: [SortDescriptor(\.createdAt, order: .reverse)],
    fetchLimit: 20
)
let recentTasks = try modelContext.fetch(descriptor)
```

Using a `fetchLimit` reduces memory usage and improves rendering performance, particularly for lists and grids. You can also use `fetchOffset` to implement simple pagination, loading additional data only when it is actually needed.

One thing to watch out for is doing too much work in the view itself. Avoid heavy sorting, filtering, or data transformation inside the view body. Let the query do the work so SwiftUI only has to render results, not compute them.

In summary, `@FetchRequest` and `@Query` are more than convenience wrappers. They are foundational tools for SwiftUI's data-driven design. When combined with clear architectural boundaries, limited fetch sizes, and focused queries, they allow your app to stay responsive and predictable even as your data grows.

Efficient data fetching is not about avoiding SwiftUI features. It is about using them as they were designed.

Debugging SwiftData/Core Data

SwiftData and Core Data abstract away the complexity of the underlying persistence layer by giving us an easy to work with API. This lets us focus on business rules and domain logic instead of writing SQL by hand, assuming you are using the default SQLite store.

That convenience comes with a tradeoff.

Because the framework hides the details, it can be difficult to understand what is actually happening behind the scenes. If a query is slow, how do you know what SQL is being generated? If something looks wrong in your data, how do you inspect the database directly?

Fortunately, Core Data gives us excellent debugging tools, and since SwiftData is built on top of Core Data, the same techniques apply.

It all starts by enabling Core Data debugging flags in your scheme.

```
-com.apple.CoreData.SQLDebug 1
-com.apple.CoreData.Logging.stderr 1
-com.apple.CoreData.ConcurrencyDebug 1
```

Once these flags are enabled and you run the app on a simulator or a real device, Xcode will begin printing detailed information to the output console. Early in the logs, you will see the full path to the SQLite database file used by your app.

For example:

```
/Users/azamsharp/Library/Developer/CoreSimulator/Devices/08C13BE6-
C7DC-4638-96E8-6E6D712413BE/data/Containers/Data/Application/
A2F5034B-8B1F-4BFD-93FC-A8B173C47FB3/Library/Application Support/default.store
```

You can open this file using any SQLite visualization tool. Popular options include Base, Beekeeper Studio, or even the SQLite command line tool that ships with macOS.

In addition to the database path, the console will also show every SQL statement executed by SwiftData or Core Data. For example:

```
CoreData: sql: CREATE TABLE ZBUDGET ( Z_PK INTEGER PRIMARY KEY, Z_ENT INTEGER,
Z_OPT INTEGER, ZLIMIT FLOAT, ZNAME VARCHAR )
CoreData: sql: CREATE TABLE ZEXPENSE ( Z_PK INTEGER PRIMARY KEY, Z_ENT
INTEGER, Z_OPT INTEGER, ZQUANTITY INTEGER, ZBUDGET INTEGER, ZPRICE
CoreData: sql: SELECT 0, t0.Z_PK, t0.Z_OPT, t0.ZLIMIT, t0.ZNAME FROM ZBUDGET
t0 ORDER BY t0.Z_PK
```

Seeing the actual SQL is incredibly valuable. It helps you understand how your models are mapped, how relationships are represented, and whether queries are doing more work than expected.

When you run into performance issues, unexpected results, or data corruption, this level of visibility can make the difference between guessing and truly understanding what is going on.

Since SwiftData uses CoreData behind the scenes, this will also work for SwiftData.

Once you add those flags and run on the simulator or real device, you will start receiving detailed debugging information in your output window. At the start of the output you will find the path to your database. You can use this path to open this database in any SQLite visualization tool you want.

IMAGE CACHING

SwiftUI provides `AsyncImage` to download and display images. One important limitation is that `AsyncImage` does not cache what it downloads. As a result, the same image may be fetched repeatedly, especially in lists and grids, adding unnecessary load to your servers and slowing down scrolling.

The good news is that creating a lightweight image caching solution is straightforward. A simple approach is to build an `ImageLoader` service that uses your existing networking layer to fetch image data and stores the result in an `NSCache`.

```
struct ImageLoader {

    let httpClient: HTTPClient
    private static let cache = NSCache<NSString, UIImage>()

    init(httpClient: HTTPClient = HTTPClient()) {
        self.httpClient = httpClient
    }

    func fetchImage(_ url: URL?) async throws -> UIImage? {

        guard let url = url else {
            throw NetworkError.badUrl
        }

        // check in cache
        if let cachedImage = Self.cache.object(forKey: url.absoluteString as
NSString) {
            return cachedImage
        } else {
            // fetch the image
            let resource = Resource(url: url, modelType: Data.self)
            let data = try await httpClient.load(resource)

            guard let image = UIImage(data: data) else {
                throw NetworkError.unsupportedImage
            }

            // store the image in the cache
            Self.cache.setObject(image, forKey: url.absoluteString as
NSString)
            return image
        }
    }
}
```

When an image is requested, the loader first checks the cache. If the image is present, it is returned immediately. If not, the image is downloaded, decoded, and stored in the cache for future use. This reduces network traffic and makes scrolling feel much smoother.

Keep in mind that `NSCache` is memory based. The system may evict cached images at any time under memory pressure, so caching should always be treated as an optimization, not a guarantee.

You can now build a simple SwiftUI view that uses this loader:

```
struct URLImage: View {

    let url: URL?
```

```

@State private var image: UIImage?
@Environment(\.imageLoader) private var imageLoader

var body: some View {
    ZStack {
        if let image {
            Image(uiImage: image)
                .resizable()
        } else {
            Image(systemName: "heart")
        }
    }.task(id: url) {
        image = try? await imageLoader.fetchImage(url)
    }
}

```

The `URLImage` view accesses the shared `ImageLoader` through the environment and loads the image asynchronously. Using `.task(id: url)` ensures the image reloads correctly if the URL changes.

Usage is simple:

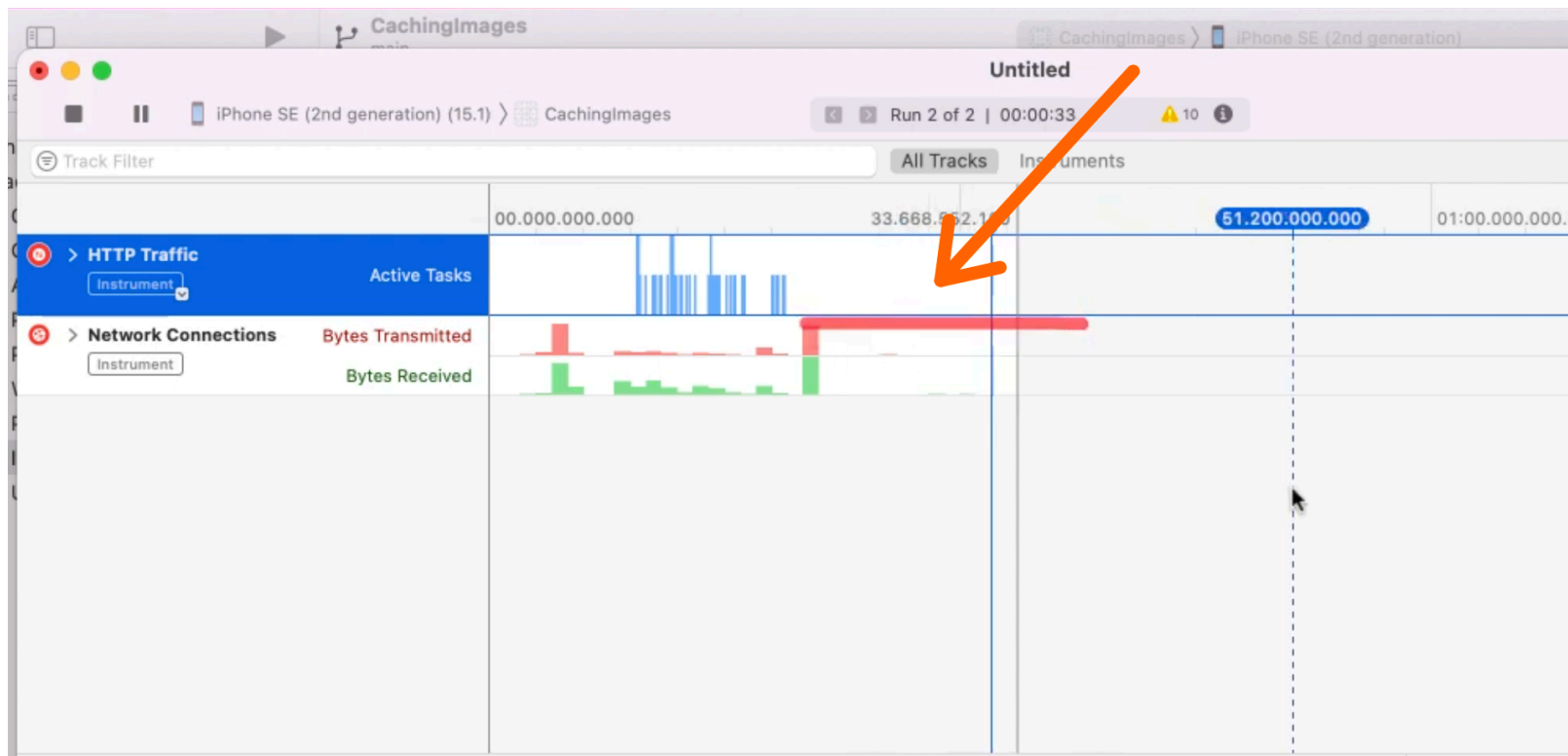
```
URLImage(url: vegetable.thumbnailImage)
```

Image caching is a common requirement in almost every iOS app. While building your own solution is valuable for learning and gives you full control, there are also mature third-party libraries such as Kingfisher and Nuke. These libraries provide memory and disk caching, prefetching, progressive loading, and automatic cache cleanup.

Whether you choose a third-party library or a custom implementation, understanding how image caching works is essential. Done correctly, it reduces network usage, improves scrolling performance, and makes your SwiftUI apps feel fast and responsive.

Below is a screenshot from Instruments. The blue bars show the moments when images were downloaded for the first time as the List appeared on screen. After that, when the user scrolls up and down, those **same images** are not downloaded again. They come straight from the cache. That is why

the network activity stays quiet where the arrow is pointing.



REQUEST CACHING

Beyond caching images, you can significantly improve app performance by caching network requests. Request caching prevents your app from downloading the same data repeatedly when it has not changed on the server. This reduces bandwidth usage, improves perceived performance, and makes SwiftUI apps feel more responsive, especially in data-heavy screens such as dashboards, feeds, and product lists.

In a SwiftUI app, request caching typically lives below the view layer. It belongs in your networking client or data service, not in your views. Views should not need to know whether data came from the network or a cache. They should simply render the result.

Only `GET` requests should be cached. `GET` requests are idempotent, meaning they can be safely repeated without causing side effects. Requests that modify data, such as `POST`, `PUT`, or `DELETE`, should never be cached.

There are several complementary techniques for implementing request caching, both on the client and server side.

ETag Validation

One of the most common and effective techniques is ETag-based caching. An ETag is a unique identifier that represents a specific version of a resource.

The flow works like this:

- The client makes a `GET` request.
- The server responds with the data and includes an `ETag` header.

- The client stores the response along with the ETag.
- On subsequent requests, the client sends the stored ETag using the `If-None-Match` header.
- If the resource has not changed, the server responds with `304 Not Modified`.
- The client reuses its cached data.
- If the resource has changed, the server returns the updated data along with a new ETag.

This approach avoids downloading unchanged data and dramatically reduces response times for frequently accessed resources.

HTTP Cache Headers

Another layer of caching is controlled through standard HTTP headers such as `Cache-Control` and `Expires`.

`Cache-Control` defines how long a response can be cached and under what conditions. For example, `max-age` specifies how long the response is considered fresh, while `public` and `private` control where it may be cached.

`Expires` provides an explicit expiration date after which a cached response should be considered stale.

These headers are typically set by the server, but your networking layer can respect or override them depending on your app's caching strategy.

Offline and Persistent Caching

For apps that need to function offline or handle unreliable networks, cached responses can be persisted locally using `SwiftData`, `Core Data`, or another lightweight database. This allows users to view previously fetched data without an internet connection and synchronize again once connectivity is restored.

Persistent caching is especially useful for content that changes infrequently, such as configuration data, reference lists, or previously viewed items.

Not all caching strategies live on the client. Some optimizations happen entirely on the server.

Micro Caching

Micro caching is a server-side technique commonly used by high-traffic systems. While it is not something you implement directly in a SwiftUI app, understanding it helps explain why repeated network requests sometimes feel faster than expected.

Instead of caching responses for minutes or hours, the server caches responses for very short periods, typically four to six seconds. During that window, identical incoming requests are served directly from

the cache, avoiding repeated database queries or expensive computations. Once the cache expires, the next request refreshes the data and starts a new short caching cycle.

This approach works particularly well for endpoints that experience traffic spikes but change frequently, such as trending content, dashboards, or live pricing feeds. Micro caching smooths out bursts of traffic while still delivering data that feels close to real time.

Micro caching is implemented entirely on the server. From the client's perspective, nothing changes. Requests simply return faster and more consistently. Your SwiftUI code does not need to be aware of it, but it benefits automatically through reduced latency and more predictable response times.

KEEP THE MAIN ACTOR FREE OF INTENSIVE WORK

I like to think of the Main Actor as the reception area of a restaurant. Customers arrive, place their orders, and wait comfortably. Once the food is ready, the receptionist calls their name and hands it over.

Now imagine if that same receptionist were also responsible for cooking the food.

After taking an order, they run into the kitchen, start cooking, and only return once the meal is ready. Meanwhile, more customers arrive. No one is at the counter. The line grows. Frustration builds. The restaurant slowly descends into chaos.

This is exactly what happens when intensive or blocking work runs on the Main Actor.

The Main Actor corresponds to the main thread in Swift. Its responsibility is to handle user interface work: drawing views, responding to touches, running animations, and performing layout. If you burden it with long-running tasks such as file processing, heavy computation, or network requests, your app will start to feel sluggish, jittery, or frozen.

A simple rule of thumb helps here:

If the work can block user interaction, it does not belong on the Main Actor.

Just like a restaurant works best when the kitchen handles cooking, your app works best when background tasks handle heavy lifting and the Main Actor stays focused on the user.

Task View Modifier

One of the techniques is to use the `.task` view modifier. This modifier allows you to start asynchronous work as soon as a view appears, without blocking the main actor. The task runs in the background, while the main actor continues to handle user interactions and render updates smoothly. Take a look at the implementation below, where we have fetched a list of products from the server inside the `.task` modifier and then updated the local state represented by `products` property.

```
struct ProductListScreen: View {  
    @State private var products: [Product] = []  
    @Environment(ProductStore.self) private var productStore
```

```

var body: some View {
    List(products) { product in
        Text(product.name)
    }
    .task {
        do {
            products = try await productStore.loadProducts()
        } catch {
            print("Failed to load products: \(error)")
        }
    }
}
}

```

One important thing to note about the task view modifier is that it is attached to the view's lifecycle. This means, if the view goes away then the task will be automatically cancelled. View can go away in several different ways like navigating away or removing it from the hierarchy. This means that you need to be careful about what you are doing in the task modifier. If you are downloading data from an API and the user leaves the view then the task will automatically be cancelled, but maybe that was not your intention and you wanted to make sure that the data gets downloaded.

One way to solve this problem is perform the task operation on the root view. The root view will be in the NavigationStack for the life of the app, unless you explicitly remove it. This means even if you navigate away from the view to a different screen, the task started on the root view will still continue.

```

@main
struct App: App {
    var body: some Scene {
        WindowGroup {
            ProductListScreen()
                .task {
                    // perform work here.
                }
        }
    }
}

```

Of course, this does not protect you if the user decided to start the task and then moves the app in the background. This will suspend the app and your task will be suspended. A suspended app cannot make forward progress, so your task may never finish. If you truly need work to continue while the app is in the background, move that work to a background friendly API and treat the view as a trigger, not the owner.

If you wish to continue performing the task, like downloading data from an API while the app is in the background then check out background-capable API such as URLSession with a background configuration or BGTaskScheduler.

Task

Apart from task view modifier, which is tied to the lifecycle of the SwiftUI view, Swift language also provides few different alternatives in the form of Task closure and Task.detached.

Task closure is used to create async context outside the async functions. One example can be a SwiftUI button tap gesture. If you want to perform a long running task when a button is tapped then you can wrap an async call inside the Task closure. Unlike the task view modifier, the work that you will perform inside the Task closure is not automatically cancelled when the view disappears or removed from the hierarchy. It will only stop if you cancel it yourself or the app is suspended/terminated by the system (e.g., when going to background, the work pauses).

If you want to control when the task gets cancelled then store the task and cancel it when the view goes away.

```
struct DetailScreen: View {
    @State private var work: Task<Void, Never>?

    var body: some View {
        Button("Start Work") {
            work = Task {
                do {
                    try await longRunningTask()
                } catch is CancellationError {
                    // handle cancellation if needed
                }
            }
        }
        .onDisappear {
            work?.cancel()
            work = nil
        }
    }
}
```

Inside the longRunningTask function, make it cancellation-friendly.

```
func longRunningTask() async throws {
    for step in 0..<10 {
        try Task.checkCancellation()
        // do a chunk of work
        try await Task.sleep(nanoseconds: 100_000_000)
    }
}
```

Task.checkCancellation is a suspending function that checks whether the current task has been cancelled or not. If the task has been cancelled, it immediately throws a CancellationError. If not, it just returns and execution continues.

Because SwiftUI views are main actor isolated, any Task you create from them also starts on the main actor. When an async call inside that task is awaited, Swift suspends the task and returns control to the main thread, keeping the user interface responsive while the operation continues in the background.

Detached Tasks

Detached tasks are tasks that are not bound to any existing actor or task hierarchy. Unlike regular `Task { ... }` calls, which inherit the actor context (often the main actor in SwiftUI) and cancellation state from their parent, a detached task runs completely independently.

They are useful for performing long-running or computationally intensive background operations, such as applying filters to images, processing large datasets, or downloading large files. Because detached tasks are independent, they are not cancelled automatically when the view that created them disappears, nor when their parent task is cancelled. This makes them a fire-and-forget mechanism, where you explicitly manage their lifetime and cancellation.

```
Task.detached(priority: .background) {  
    do {  
        let result = try await processLargeImage()  
        await MainActor.run {  
            image = result  
        }  
    } catch {  
        print("Processing failed: \(error)")  
    }  
}
```

In this example, the detached task runs outside the main actor, which means any UI updates must be wrapped in `await MainActor.run { ... }`.

Detached tasks are ideal for offloading heavy work that doesn't depend on the UI or the view's lifecycle. However, they should be used sparingly and carefully – always ensure you handle cancellation, actor isolation, and UI updates explicitly.

OBSERVABLEOBJECT TO @OBSERVABLE

The `ObservableObject` protocol was introduced with the first release of SwiftUI in 2019. Its goal was simple and powerful: define a source of truth that views can observe and react to. When a property marked with `@Published` changes, SwiftUI automatically updates the views that depend on that object, keeping the user interface in sync with the underlying data.

This model works, but it comes with an important limitation that becomes more visible as apps grow.

Whenever **any** `@Published` property changes, the *entire object* is marked as changed. SwiftUI does not know which specific property was updated. As a result, all views observing that object are reevaluated, even if they only read a property that did not change. This can lead to unnecessary reevaluation and extra work during SwiftUI's diffing process.

Let's look at a simple example.

Suppose we have a `UserStore` that conforms to `ObservableObject` and exposes two published properties: `name` and `age`.

```
class UserStore: ObservableObject {
    @Published var name: String = ""
    @Published var age: Int = 0
}
```

The store is created in a parent view and passed down to two child views. One displays the user's name, and the other displays the age.

```
struct ContentView: View {

    @StateObject private var userStore = UserStore()

    var body: some View {
        VStack(spacing: 20) {

            TextField("Enter Name", text: $userStore.name)
                .textFieldStyle(.roundedBorder)
                .padding()

            Stepper("Age: \(userStore.age)", value: $userStore.age)
                .padding()

            UserNameView(userStore: userStore)
            UserAgeView(userStore: userStore)
        }
        .padding()
    }
}
```

Even though `UserNameView` only depends on `name`, changing `age` still triggers its reevaluation. The same is true in reverse. SwiftUI has no way to know which property actually matters to which view.

This behavior is correct given how `ObservableObject` works, but it is not always efficient.

Enter the Observation Framework

In iOS 17, Apple introduced the Observation framework along with the `@Observable` macro. This represents a major improvement in how SwiftUI tracks changes.

Instead of observing the object as a whole, SwiftUI now tracks **individual property access**. When a property inside an `@Observable` type changes, only the views that actually read that property are reevaluated. Unrelated views are left alone.

`ObservableObject` tells SwiftUI *something changed*.

`@Observable` tells SwiftUI *this changed*.

Here is the same store rewritten using `@Observable`.

```
@Observable
class UserStore {
    var name: String = ""
    var age: Int = 0
}
```

The change is small, but the behavior is very different.

In the parent view, you no longer need `@StateObject`. A simple `@State` is enough.

```
struct ContentView: View {

    @State private var userStore = UserStore()

    var body: some View {
        VStack {
            Text(userStore.name)
            Text("\(userStore.age)")
        }
    }
}
```

In child views, you also remove `@ObservedObject`. The store is passed as a plain value.

```
struct UserAgeView: View {

    let userStore: UserStore

    var body: some View {
        let _ = Self._printChanges() // Logs reevaluation
        VStack {
            Text("User Age View")
            Text("\(userStore.age)")
        }.padding()
    }
}
```

Now, changing `name` does not cause `UserAgeView` to re-evaluate, and changing `age` does not affect `UserNameView`. SwiftUI updates only the views that depend on the changed property.

Binding with `@Observable` and `@Bindable`

When working with `@Observable` types, bindings are handled using the `@Bindable` property wrapper.

Because SwiftUI tracks property-level access, you no longer bind to the object itself. Instead, you bind directly to the properties you want to edit.

Here is an example of a view that edits the user's name:

```
struct EditUserView: View {

    @Bindable var userStore: UserStore

    var body: some View {
```

```

        TextField("Name", text: $userStore.name)
            .textFieldStyle(.roundedBorder)
            .padding()
    }
}

```

`@Bindable` creates bindings to individual properties on an `@Observable` type. SwiftUI automatically tracks which properties are read and written, ensuring updates stay precise and localized.

`@Bindable` is only needed when a view mutates observable state. Read-only views can continue to receive the store as a plain value.

In many cases, this replaces common uses of `@ObservedObject` and `@Binding` when working with observable reference types.

Migration Notes

You do not need to migrate everything at once. `ObservableObject` and `@Observable` can safely coexist in the same app. Existing code will continue to work, and you can adopt the new model incrementally where it provides the most value.

Apple's direction is clear, though. `ObservableObject` now lives in the Combine framework rather than SwiftUI, and Apple recommends using the Observation framework for new code.

Why This Matters

The shift from `ObservableObject` to `@Observable` removes one of the long-standing performance pain points in SwiftUI: broad change tracking. By observing properties individually, SwiftUI can keep reevaluation precise, updates local, and rendering efficient.

This change also simplifies architecture. In many common cases, you no longer need to constantly juggle `@ObservedObject`, `@EnvironmentObject`, and `@StateObject`. A simple `@State` or `@Bindable` reference is often enough.

More importantly, this model aligns naturally with SwiftUI's declarative design. Views express what data they read, and SwiftUI updates exactly those views when that data changes. Nothing more, nothing less.

As SwiftUI continues to evolve, adopting the Observation framework and the `@Observable` macro will help your apps stay responsive, easier to reason about, and ready for what comes next. This is not just a syntax change. It is a shift toward more precise reactivity and a cleaner SwiftUI architecture.

CONCLUSION

SwiftUI performance problems rarely come from a single bad line of code. They emerge gradually as an app grows and small decisions begin to compound. A slightly unstable identity here. A little too much work on the main actor there. A view that knows more state than it needs. Individually harmless.

Together, they are what make an app feel heavy.

The goal of this chapter was not to turn you into a performance obsessive. It was to give you a mental model.

When you understand how SwiftUI thinks about identity, you stop fighting state resets and animation glitches. When you compose views intentionally, updates stay local instead of rippling through the entire hierarchy. When you fetch and cache data thoughtfully, the UI feels fast even when the network is slow. When you keep the main actor focused on UI work, interactions stay smooth and predictable. And when you adopt the Observation framework, SwiftUI finally updates only what actually changed.

Debugging tools like `self._printChanges()` and the View Hierarchy inspector turn SwiftUI from a black box into something you can reason about. You no longer guess why a view is updating. You can see it. You can verify it. You can fix it with confidence.

The most important takeaway is this: performance in SwiftUI is a side effect of good architecture. Stable identity, clear data flow, focused views, and precise observation naturally lead to apps that feel fast and reliable. You do not optimize SwiftUI by hacking around it. You optimize SwiftUI by aligning with how it was designed to work.

And sometimes, even when you do all of that, something still feels off.

That is where profiling comes in. In the next chapter, we move beyond mental models and intuition and start measuring what actually happens at runtime. You will learn how to use Instruments and other profiling tools to spot bottlenecks, confirm assumptions, and make informed performance decisions based on real data rather than guesswork.

PROFILING SWIFTUI

In the previous chapter, you explored several practical techniques to improve the performance of your SwiftUI applications. You learned how view identity affects rendering, how breaking large views into smaller ones reduces unnecessary updates, and how smarter data flow can prevent unwanted redraws. You also saw a few debugging techniques that help make SwiftUI's behavior easier to understand.

In this chapter, we move beyond debugging and focus on profiling. Instead of relying on assumptions about where bottlenecks might exist, we will use real tools to measure what is actually happening inside the application and use that data to guide our decisions.

NETWORK PROFILING

Although networking is not specific to SwiftUI, it is an essential part of almost every modern iOS application. Downloading images, fetching JSON responses, and retrieving remote resources are all expensive operations. They depend on server latency, device conditions, and the quality of the user's network connection. It is important to remember that users are not always on fast and reliable WiFi. Many are on cellular networks, in low signal areas, or switching between connections. If your application repeatedly downloads the same resources without caching them properly, you are not only slowing down the user experience but also placing unnecessary load on your backend services.

In this section, we will focus on two common networking bottlenecks that frequently impact performance: image caching and JSON response caching. We will start with image caching because it is one of the most visible performance issues in list based interfaces.

Image Caching

AsyncImage was introduced at WWDC 2021 and was widely welcomed by the SwiftUI community. Before its introduction, developers often had to implement their own asynchronous image loading solutions. AsyncImage made it straightforward to load and display an image from a remote URL with minimal code. However, there is an important limitation that many developers overlook. AsyncImage does not provide built in image caching. This means that if the same image appears multiple times in a scrolling list, or if a user scrolls up and down repeatedly, the image may be downloaded again and again.

Consider the following example, which uses AsyncImage inside a List to display product images:

```
List(products) { product in
    HStack(alignment: .top, spacing: 12) {
        if let imageUrl = product.images.first {
            AsyncImage(url: imageUrl) { phase in
                switch phase {
                case .empty:
                    ProgressView()
```

```

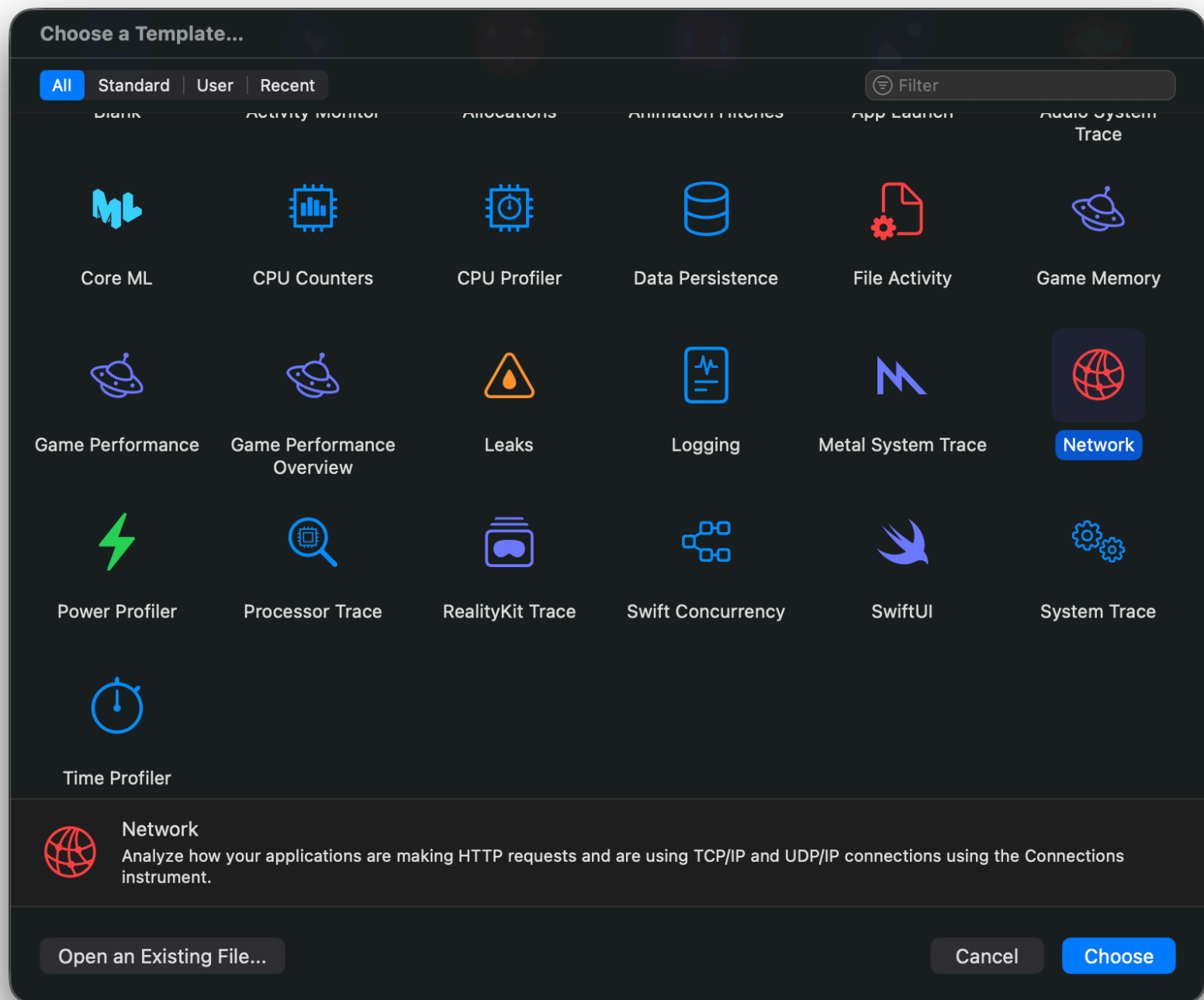
        .frame(width: 60, height: 60)
    case .success(let image):
        image
            .resizable()
            .scaledToFill()
            .frame(width: 60, height: 60)
            .clipped()
            .cornerRadius(8)
    case .failure:
        Image(systemName: "photo")
            .frame(width: 60, height: 60)
    @unknown default:
        EmptyView()
}

}
} else {
    Image(systemName: "photo")
        .frame(width: 60, height: 60)
}
}

```

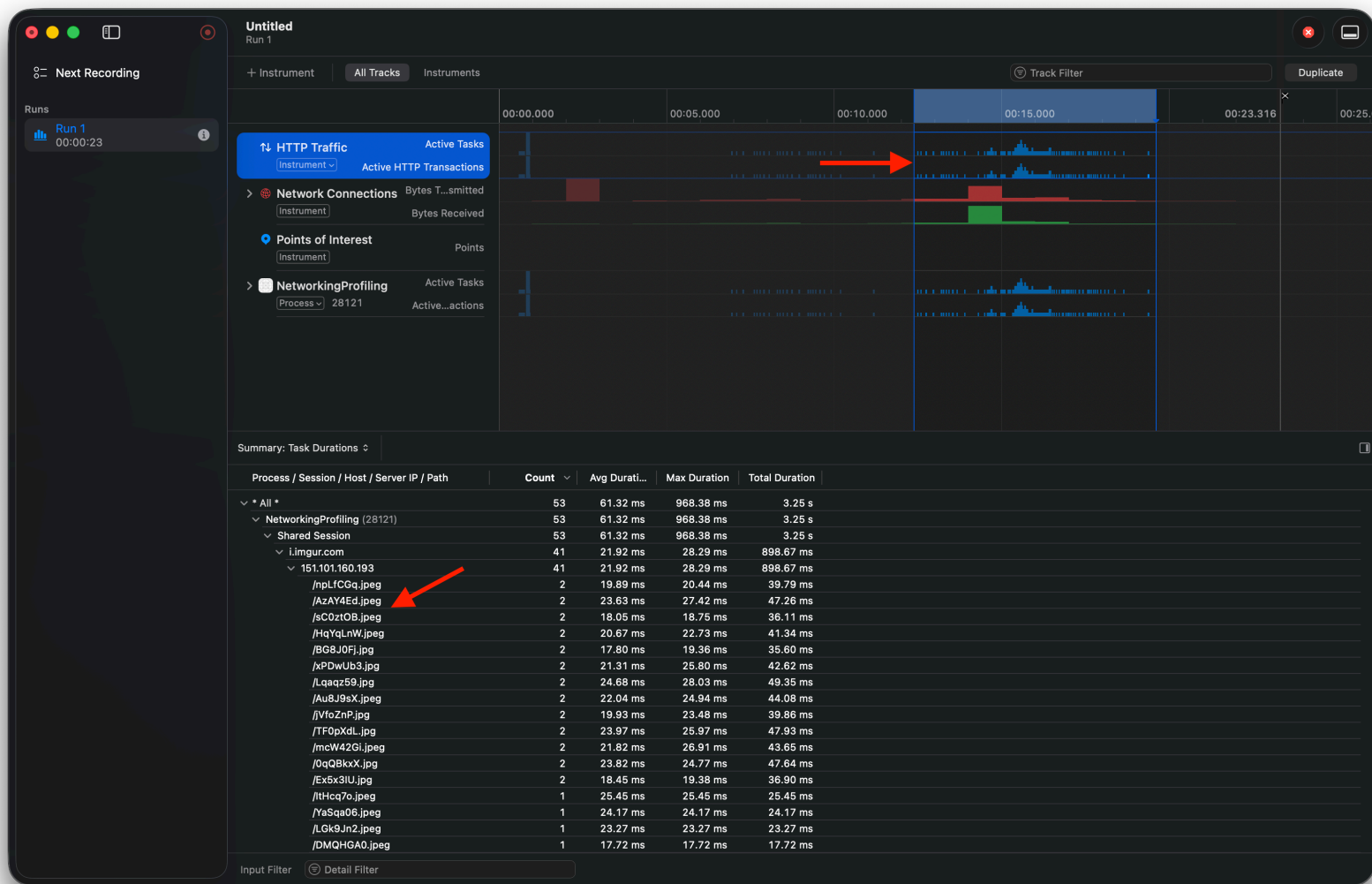
At first glance, this implementation looks clean and perfectly acceptable. However, if you scroll the list repeatedly, the same images will be downloaded multiple times. This results in increased network usage, visible scrolling hiccups, and unnecessary server load. The issue becomes even more pronounced when dealing with larger images or slower network connections.

To confirm this behavior, we can profile the application using Instruments. When performing network profiling, it is important to use a physical device rather than the simulator, as many Instruments templates do not provide accurate results when running in the simulator. Launch Instruments, select the **Network** template, connect a real device, and choose your application as the launch target. You can press "Command I" from Xcode to launch Instruments.



Once recording begins, interact with the application as a user would. Scroll the product list up and down and allow images to load. After stopping the recording, Instruments will display a timeline of network activity. You will likely observe spikes in HTTP traffic corresponding to image downloads. If you continue scrolling, those spikes will appear repeatedly, indicating that the same resources are being

requested multiple times.



Notice that we did not assume AsyncImage was inefficient. We measured the behavior. Profiling confirmed repeated downloads.

Fortunately, this problem is relatively easy to fix. You can either implement your own caching layer or use a well established library. For this example, we will use Kingfisher, which provides SwiftUI compatible views and automatic image caching. After adding Kingfisher through Swift Package Manager, you can replace AsyncImage with KfImage:

```
List(products) { product in
    HStack(alignment: .top, spacing: 12) {
        if let imageUrl = product.images.first {
            KfImage(imageUrl)
                .placeholder {
                    ProgressView()
                        .frame(width: 60, height: 60)
                }
                .resizable()
        } else {
            Image(systemName: "photo")
                .frame(width: 60, height: 60)
        }
    }
    .padding(.vertical, 4)
}
```

If you are interested in implementing your own caching image view then check out the video [Caching Images in SwiftUI](#).

After making this change, run the application through Instruments again. During the initial scroll, you will still observe network spikes as images are downloaded for the first time. However, when scrolling back through previously loaded content, those spikes should no longer appear. The images are now served from cache, resulting in smoother scrolling, reduced network traffic, and lower server load.

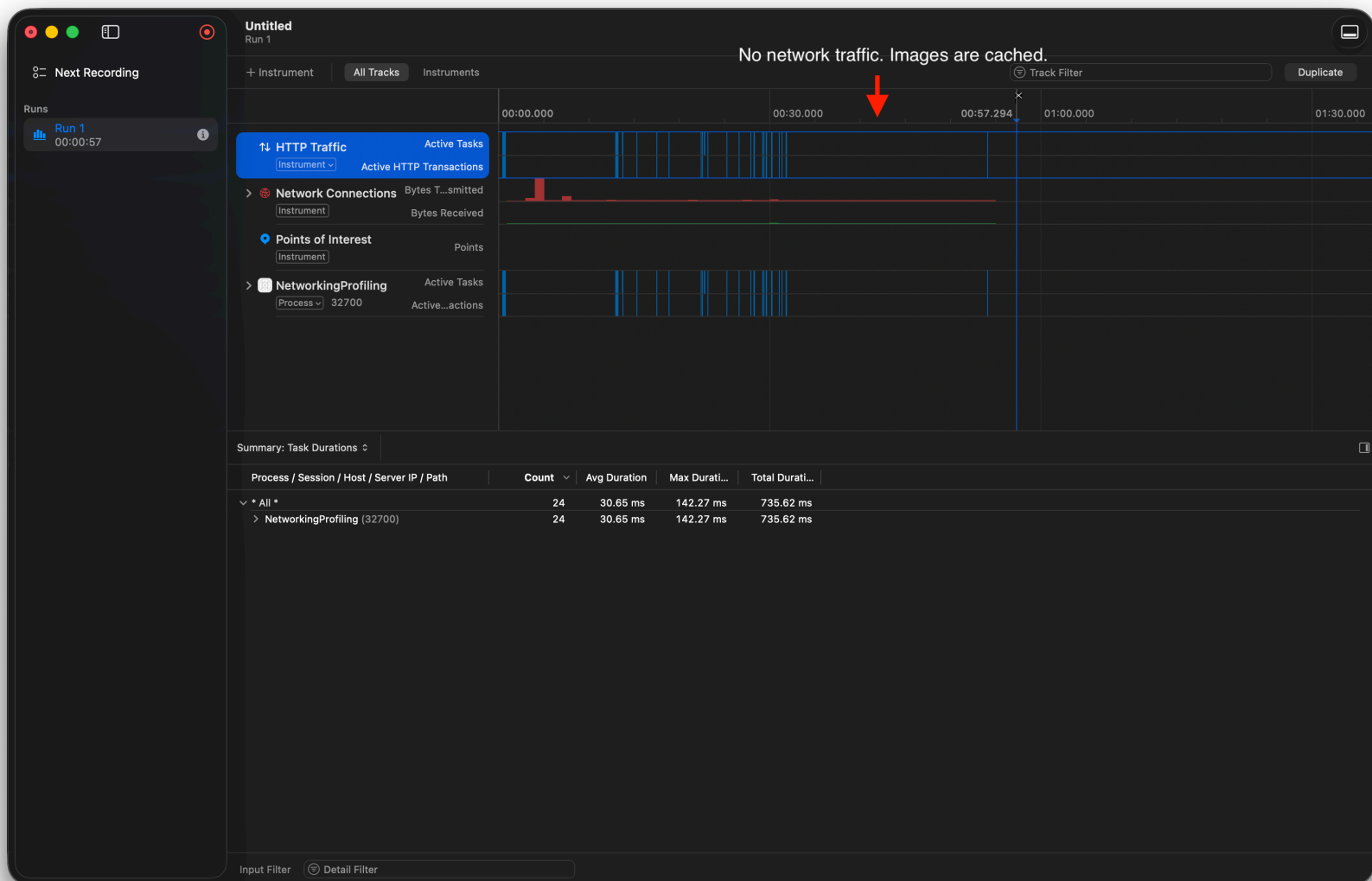


Image caching is what I consider a quick win. It does not require significant architectural changes, yet it can dramatically improve perceived performance. Small optimizations like this often make the difference between an app that feels slightly sluggish and one that feels polished and responsive.

Caching Responses

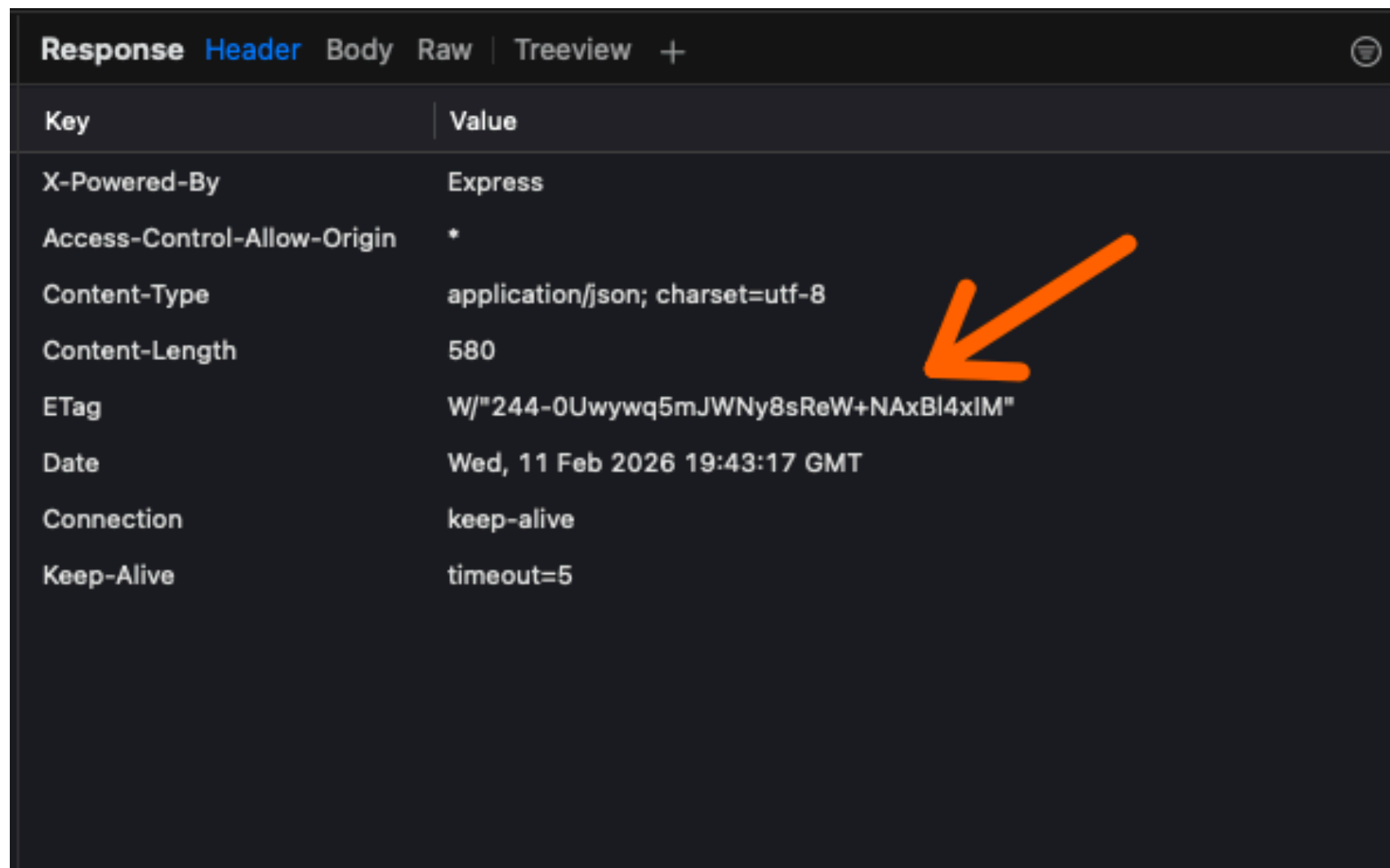
Apart from caching images, a lot of times it can also benefit to cache server's response. This is specially true if the response is not changing. This reminds me of an interesting tweet, where an iOS developer was trying to figure out a way to not fire task view modifier in SwiftUI when the user changes tabs. He came up with solutions to implement a custom view modifier, which only fires once and not when the view is displayed again. Those solutions worked but it created more responsibility on the client to call the correct view modifier.

A better approach would have been to solve this issue on the server by sending back the correct cache headers in the response. This not only eliminates the complexity on the client side but also provide the flexibility to the server to return different response headers based on the situation.

In this section, we are going to learn different techniques that will allow you to cache the data using server HTTP header tags.

One of the common ways for the server to tell the client, whether the response has changed or not is through the use of **ETag** (Entity Tag). ETag is a value sent by the server in response headers indicating if the response has changed or not. Server can generate ETag in many different ways but the most common is the hash of the response. Frameworks like ExpressJS sends ETag by default to the client.

This is shown in the screenshot below:



Response Header Body Raw Treeview +	
Key	Value
X-Powered-By	Express
Access-Control-Allow-Origin	*
Content-Type	application/json; charset=utf-8
Content-Length	580
ETag	W/"244-0Uwywq5mJWNy8sReW+NAXBI4xIM"
Date	Wed, 11 Feb 2026 19:43:17 GMT
Connection	keep-alive
Keep-Alive	timeout=5

The "W" at the start of the ETag represent that it is a weak ETag. A weak ETag is commonly used when sending back JSON responses, where it does not have to be re-computed because of spacing, formatting of the response has changed. A strong ETag on the other hand matches the response byte to byte and is commonly used in serving static files, supporting range requests, doing CDN-level caching and places where exact content matters.

Once the client receives the response with ETag, it makes sure to send the same ETag in headers using If-None-Match key. The If-None-Match request header is simply client's way of asking the server that if the response has changed or not. If the response is not changed then the server respond with 304 (Not Modified) and sends an empty body back to the client. If the response is changed then a new ETag is created on the server and sent back to the client.

Keep in mind that `URLSession` takes care of sending the same ETag back to the server and you don't have to do anything special for this interaction.

If you want to cache the response for few seconds then you can use `Cache-Control` header with `max-age` attribute set to the number of seconds.

```
res.set("Cache-Control", "private, max-age=5, must-revalidate")
```

`private` = only browser/device cache (not shared proxies)

`max-age=5` = fresh for 5 seconds

`must-revalidate` = once stale, client must revalidate (typically via ETag)

There are several different options for `Cache-Control`. To investigate further use [MDN documentation](#) for more details.

URLSession Caching and HTTP Cache Control

When developers talk about caching in iOS, the conversation often turns into custom solutions, in-memory dictionaries, disk persistence layers, or third party libraries. In reality, most of the time you already have a perfectly capable caching system built into the platform. It is called `URLSession`, and it follows the HTTP specification very closely.

By default, when you use `URLSession.shared` and call `data(from:)`, the request uses `.useProtocolCachePolicy`. This means the client does not decide how to cache. The server does. If your backend sends proper headers such as `Cache-Control`, `ETag`, or `Last-Modified`, `URLSession` will automatically store the response and reuse it according to those rules. You do not need to manually handle ETags. You do not need to compare timestamps. The system already does that work for you.

For example, if your server responds with:

```
Cache-Control: private, max-age=5  
ETag: W/"abc123"
```

The response is considered fresh for five seconds. During that window, repeated requests will return the cached response without hitting the network. After five seconds, the request is still sent, but now it includes an `If-None-Match` header with the previously received ETag. If the content has not changed, the server returns a `304 Not Modified` response. No body is downloaded. `URLSession` then serves the cached data. This is efficient, correct, and completely automatic.

You can also control cache behavior at the request level. For example, setting the cache policy to `.reloadIgnoringLocalCacheData` forces a network request every time. On the other hand, `.returnCacheDataDontLoad` will return cached data if available and fail if it is not, which can be useful in offline scenarios. These policies allow you to align networking behavior with your application's needs instead of writing your own caching layer.

If you want more control, you can configure a custom `URLCache` with specific memory and disk capacities and attach it to a custom `URLSessionConfiguration`. This is rarely needed for most applications, but it becomes useful in data-heavy apps or when you want predictable cache limits.

The most important takeaway is this: caching on iOS is primarily driven by server configuration. If your backend does not send proper cache headers, the client cannot magically invent a correct strategy. Before building a custom caching system, inspect the response headers. In many cases, the solution is not more Swift code. It is better HTTP configuration.

SWIFTUI VIEW RENDERING

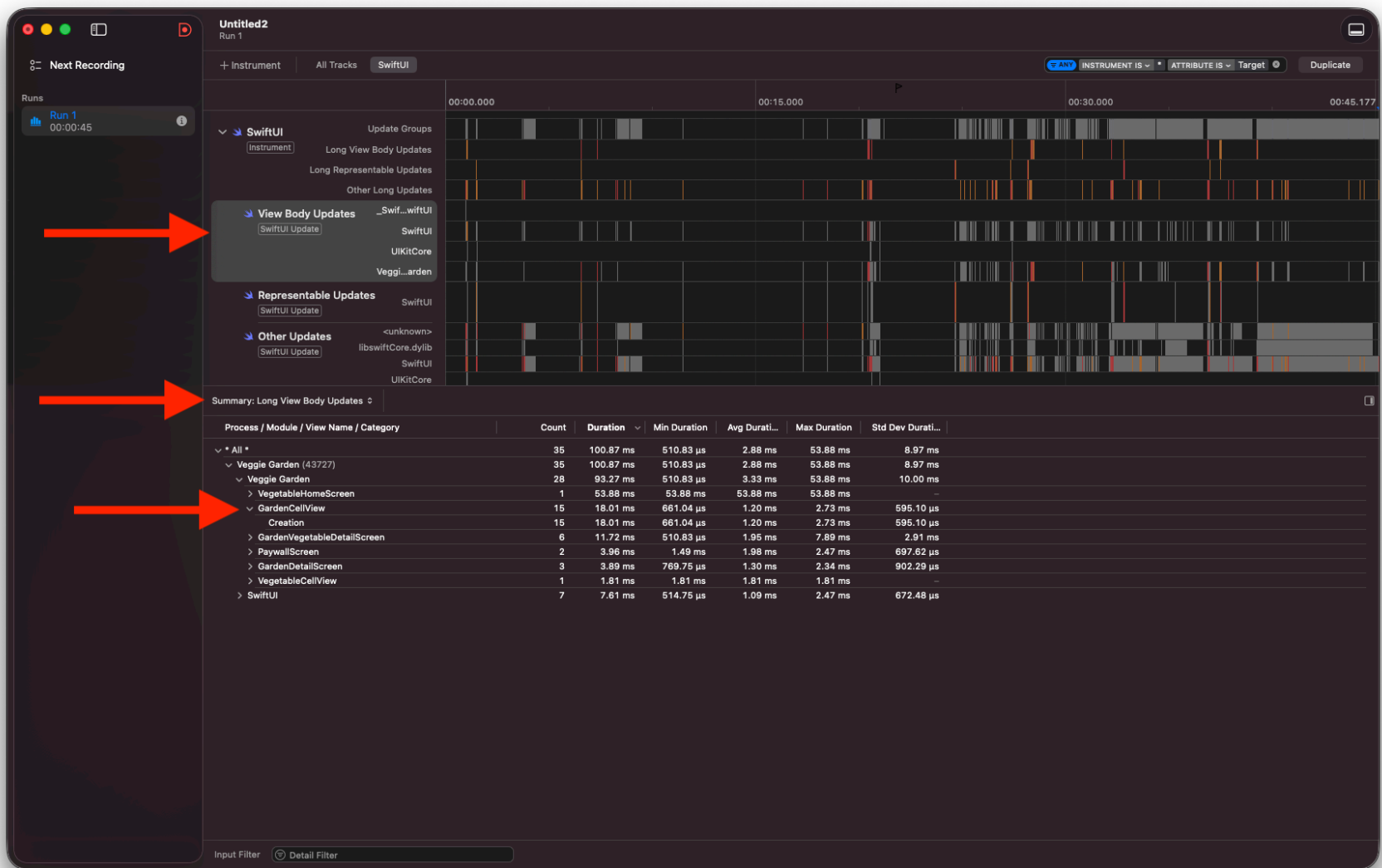
When developers begin profiling SwiftUI applications, many assume they are measuring drawing performance. That assumption is understandable, but it is usually incorrect. In SwiftUI, a view is not a long-living object sitting on screen waiting to be updated. A view is a lightweight description of the user interface for a specific state. Every time state changes, SwiftUI reevaluates the `body`, produces a new description of the interface, and then compares it with the previous version to determine what actually needs to change on screen. That comparison process, often referred to as diffing and reconciliation, is highly optimized. Reevaluating `body` is normal and expected behavior.

What surprises many developers is how frequently this reevaluation happens. A timer tick, a binding update, a network response, or even a parent view changing can all trigger another execution of `body`. Most of the time this is perfectly fine because SwiftUI is designed for this model. The framework expects `body` to run often, and it is optimized for that reality. The real problem begins when we place heavy computation inside `body`, or when view identity becomes unstable. At that point, frequent reevaluation stops being cheap and starts becoming expensive.

When profiling SwiftUI views, we are not measuring how fast pixels are drawn to the screen. We are measuring how often `body` runs, how much work happens during each run, and whether SwiftUI can efficiently reconcile the resulting hierarchy. Once you shift your perspective this way, Instruments becomes much easier to interpret. Instead of guessing why the application feels slow, you can identify exactly which views are being reevaluated and how much work they are performing.

I am going to run profiling on **My Veggie Garden**, an app to track planted vegetables in your garden but you can use any of your apps to perform the same action. Press “Command” I to launch Instruments and from the template choose “SwiftUI”.

Now press the recording button and make sure you are running the app on a physical phone. Navigate the app and perform some actions. For my gardening app, I planted a new vegetable and then harvested it. Once you stop the recording, the profiling tool start gathering data based on your interaction with the app. Below you can see the data gathered for my app.



The **View Body Updates** track shows you exactly when SwiftUI re evaluated a view's body. Every bar represents a render pass. When you are profiling performance, this track helps you answer a very important question: which views are actually doing work?

If you simply look at the raw updates, it can feel overwhelming. There are often many small body updates happening constantly. Instead of trying to analyze everything, select **Long View Body Updates**. This filters the data so you only see render passes that took longer than expected. Now your attention is focused only on the updates that matter.

In this example, it is clear that `GardenCellView` is contributing heavily to the long body updates. That immediately tells us something useful. The issue is not scattered across the entire hierarchy. It is concentrated in a specific view.

Now here is where Instruments becomes really powerful.

If you hover over `GardenCellView`, you will see a small arrow appear. Click that arrow and choose **Show Cause and Effect Graph**.



Instead of just seeing that the view updated, you now see *why* it updated.

The Cause and Effect Graph visually shows the chain of events that triggered the render. You can see which state changed, which parent view reacted, and how that change propagated down to `GardenCellView`. Sometimes it is an `@Observable` property. Sometimes it is a binding. Sometimes it is a parent view recomputing its body.

This is the difference between guessing and understanding.

Many developers look at a slow cell and assume the cell itself is poorly written. But often the real issue is not that the view is expensive. The real issue is that it is being invalidated too often because of state changes higher in the hierarchy.

The Cause and Effect Graph helps you step back and look at the architecture instead of just the symptom.

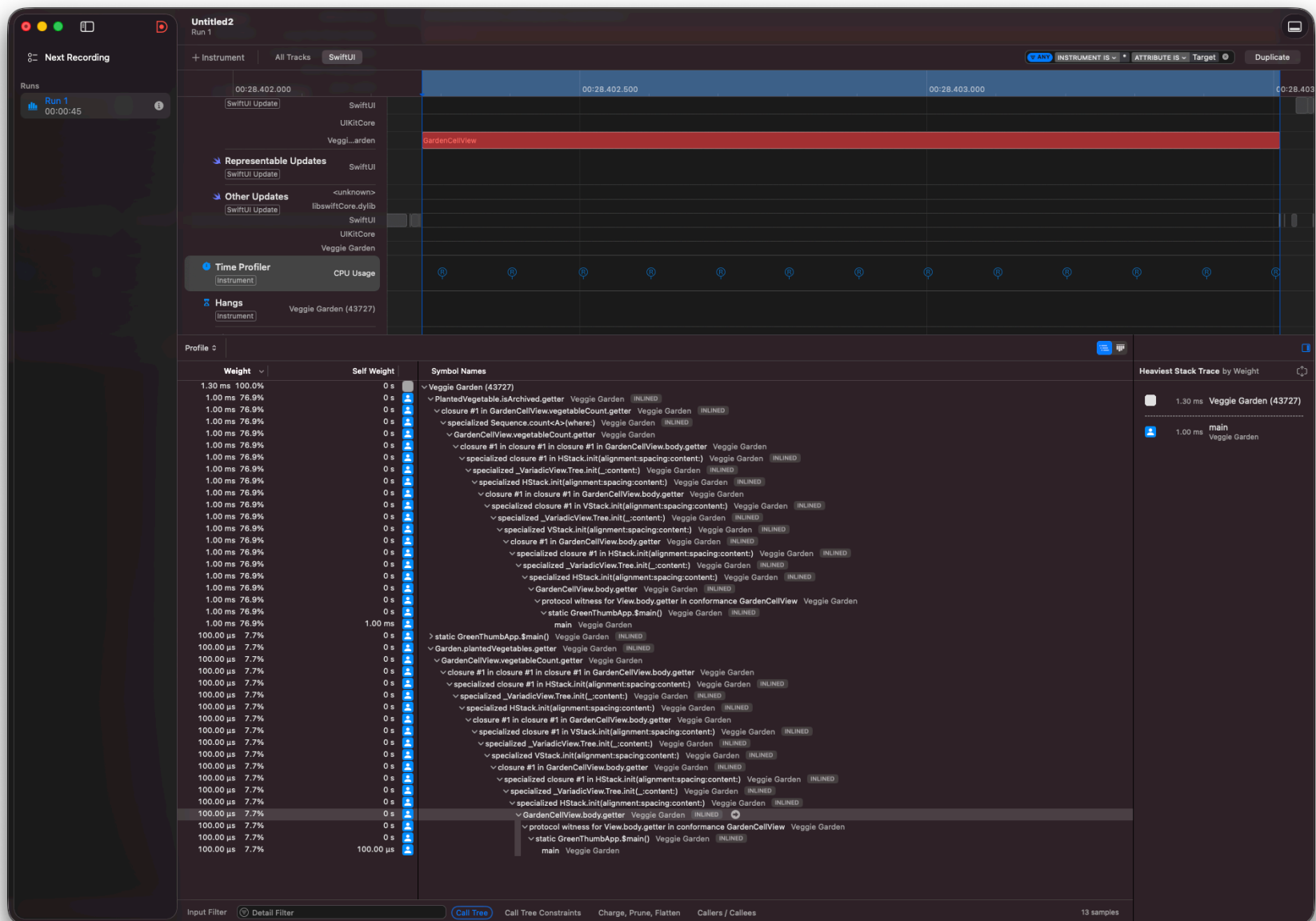
At this point, profiling stops being about chasing performance numbers and starts being about understanding data flow. And in SwiftUI, understanding data flow is everything.

After identifying `GardenCellView` as the source of long body updates, the next step is to switch to **Time Profiler**.

Now we are no longer asking how often the view updates. We are asking how expensive the work inside that view actually is.

Time Profiler

In the Time Profiler call tree, expand the stack under your app target. As you drill down, you will eventually reach `GardenCellView.body.getter`. From there, you can see the actual functions being executed during rendering.



Notice something interesting in this example. Many of the functions are marked as **inlined**. Swift has aggressively optimized the code. Instead of calling separate functions at runtime, the compiler has folded them directly into the body implementation.

This is important to understand.

Time Profiler does not show you “slow SwiftUI.” It shows you real CPU work. If your body contains sorting, filtering, heavy calculations, image processing, or repeated data transformations, those operations will appear here. If the compiler inlines them, they may appear deeper in the stack, but the CPU time will still be attributed correctly.

This is where you connect the dots.

The SwiftUI instrument told us that `GardenCellView` was updating frequently. The Cause and Effect Graph showed us why it was being invalidated. Time Profiler now shows us what it is actually doing during each render.

If the cost per render is small but the update frequency is high, the issue is architectural.
If the update frequency is reasonable but the cost per render is large, the issue is computational.

That distinction matters.

Many developers try to micro optimize layout or remove small modifiers, but the real performance wins often come from reducing unnecessary invalidations or moving expensive work outside the `body`. For example, precomputing derived values in an observable object, caching transformations, or isolating state so it does not propagate through the entire hierarchy.

Profiling is not about chasing numbers. It is about understanding the relationship between state changes, view invalidation, and CPU cost.

When you use the SwiftUI instrument and Time Profiler together, you get both sides of the equation. You see the architectural cause and the computational effect.

And once you see both, performance tuning becomes deliberate instead of reactive.

HANGS

Not every performance issue shows up as high CPU usage or excessive view updates. Sometimes the problem is much simpler and much more frustrating.

You tap a button and nothing happens for a second. Scrolling pauses unexpectedly. Animations stutter. The app does not crash, but it feels unresponsive.

When that happens, you are not dealing with a rendering issue. You are dealing with a blocked main thread.

SwiftUI, like UIKit, relies on the main thread to process user interaction and render updates. If you perform heavy synchronous work on the main thread, the system cannot respond to touches or redraw the interface until that work finishes. The result is what we call a hang.

This is where the **Hangs** instrument becomes extremely useful.

Unlike Time Profiler, which samples CPU activity, the Hangs instrument specifically detects when the main thread becomes unresponsive for a noticeable period of time. It does not care how much CPU you are using overall. It cares whether the UI is blocked.

Let's look at a simple example.

```
struct HangExampleView: View {
```

```

@State private var message = "Tap the button"

var body: some View {
    VStack(spacing: 20) {
        Text(message)
            .font(.headline)

        Button("Perform Heavy Work") {
            performHeavyWork()
        }
    }
    .padding()
}

func performHeavyWork() {
    // Simulate blocking computation
    for _ in 0..<200_000_000 {
        _ = UUID().uuidString
    }

    message = "Finished"
}
}

```

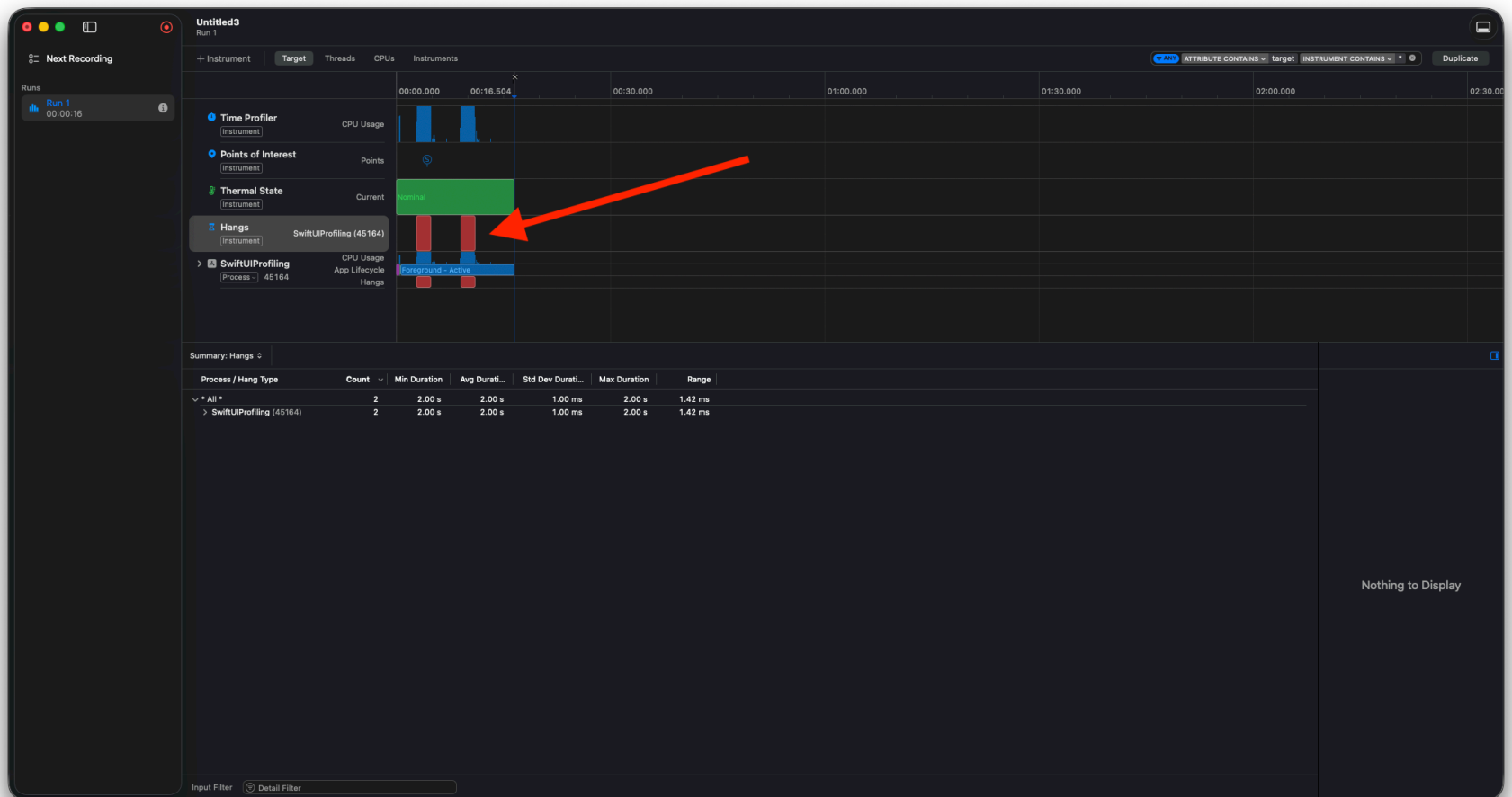
At first glance, this code does not look dangerous. But when you tap the button, the UI will freeze. The button does not animate. The text does not update. The interface becomes unresponsive until the loop completes.

Why? Because the heavy computation is running synchronously on the main thread.

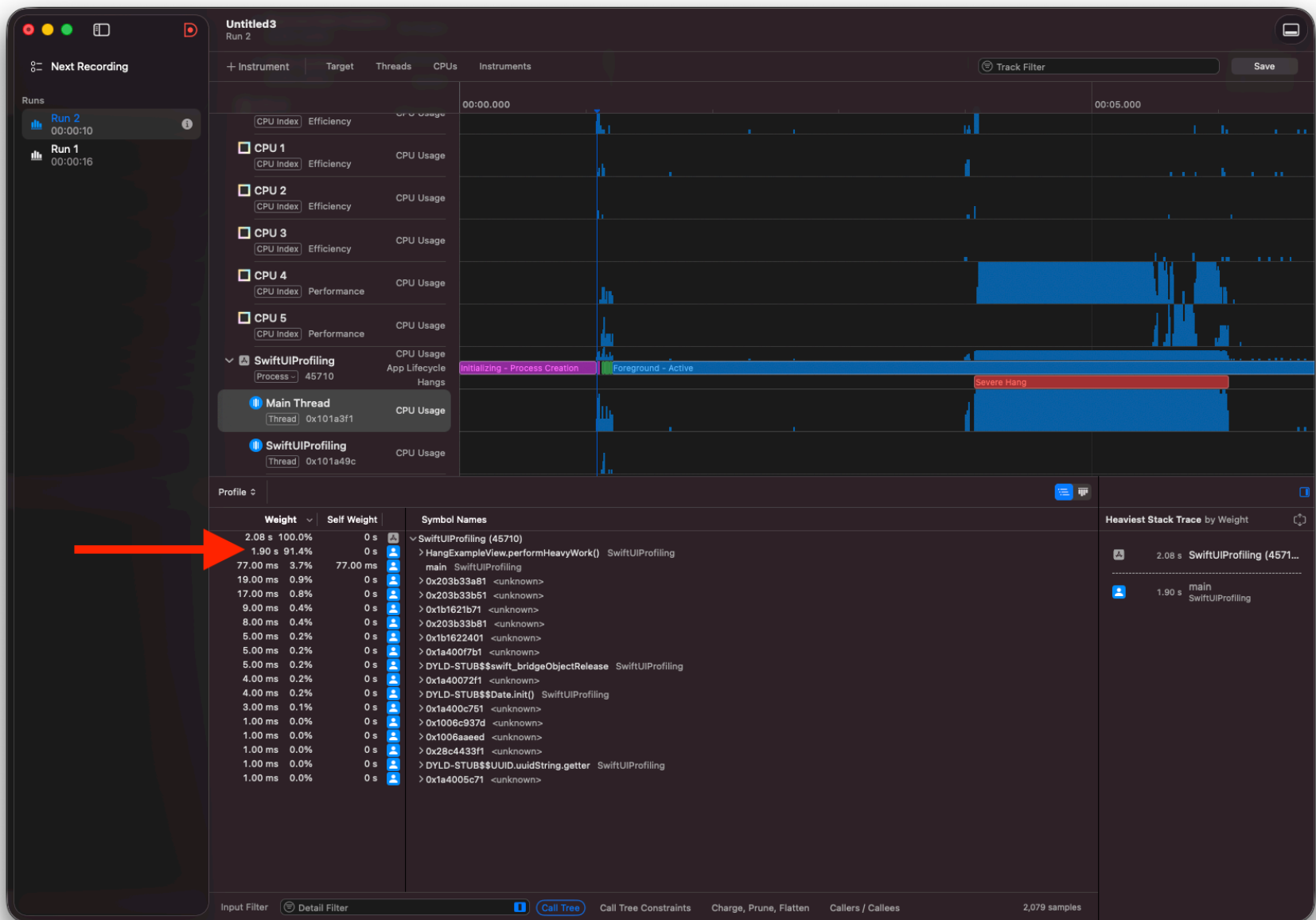
Now instead of guessing, profile it.

Launch Instruments, select the **Hangs** template, and run the app on a physical device. Start recording and tap the button. When the freeze occurs, Instruments will highlight a hang event. This is shown in the

screenshot below:



Each time you tap on the button, `performHeavyWork` function is triggered. Instruments marks it as a hang because we are performing intensive work on the main thread and it blocks the entire user interface. When the freeze occurs, Instruments highlights a hang event. This is shown in the screenshot below.



In the timeline, you can clearly see a red bar labeled **Severe Hang**. That red segment represents the period during which the main thread was blocked and the UI was unresponsive. This is not high CPU usage across multiple threads. This is a single thread being locked.

If you look at the lower pane, Instruments shows the call stack during the hang. Notice that `HangExampleView.performHeavyWork()` appears at the top of the stack. That is the exact function responsible for blocking the main thread. Instruments even shows the duration, in this case around two seconds. That is a very long time for the UI to be frozen.

This is an important moment.

Instead of saying "SwiftUI feels slow" or "the animation is glitchy," you now have precise evidence. The hang is not mysterious. It is not random. It is tied directly to synchronous work running on the main thread.

The fix is not to remove the work. The fix is to move it off the main thread.

With Swift concurrency, that becomes straightforward:

```
struct HangExampleView: View {  
    @State private var message = "Tap the button"
```

```

var body: some View {
    VStack(spacing: 20) {
        Text(message)
            .font(.headline)

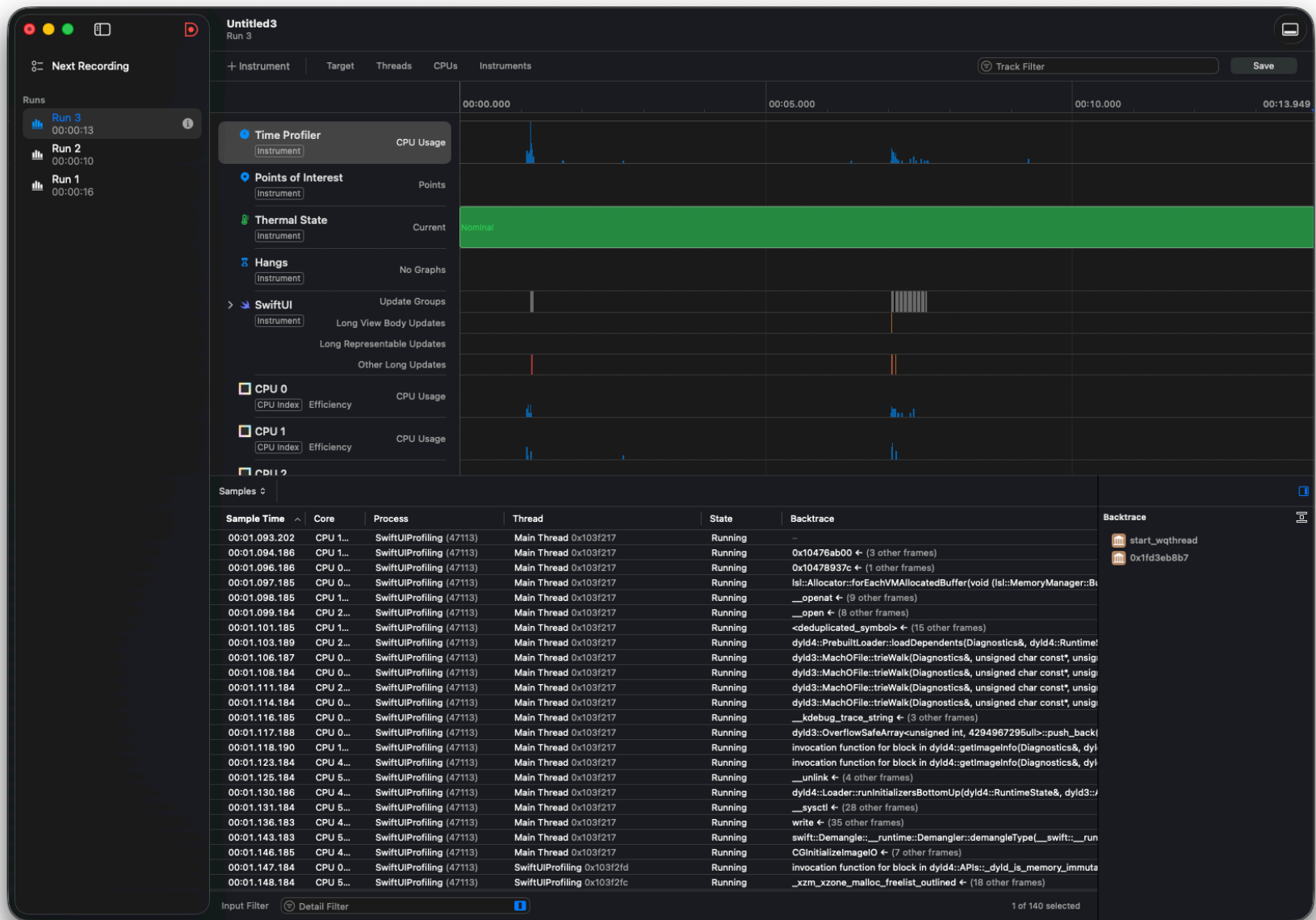
        Button("Perform Heavy Work") {
            Task {
                await performHeavyWork()
            }
        }
    }
    .padding()
}

func performHeavyWork() async {
    await Task.detached(priority: .userInitiated) {
        var count = 0
        for _ in 0..<200_000_000 {
            count += 1
        }

        await MainActor.run {
            message = "Finished"
        }
    }.value
}

```

Now the computation runs in the background. The main thread remains free to process user interaction and redraw the UI. If you profile this version with the Hangs instrument, no hang event will appear. This is shown in the screenshot below.



No more hangs. We have moved the expensive operation from the main actor to the background thread. This means our user interface will remain running without freezing and will provide a much better user experience.

SwiftUI is declarative, but it still runs on threads. If you decode JSON on the main thread, sort large arrays inside a button action, or perform database writes synchronously, you are blocking the UI. The framework is not the problem. The placement of the work is.

When you use the Hangs instrument together with the other profiling tools, you gain a complete picture of your app’s runtime behavior. You can see frequency, cost, and responsiveness. And once you can see all three, performance tuning becomes intentional instead of reactive.

CONCLUSION

Profiling SwiftUI is not about proving that something is slow. It is about understanding how your application behaves when a real person is using it. SwiftUI is not fragile. It is not secretly inefficient. It is designed for body to run often. That is normal. That is expected. The framework is optimized for this model.

Performance problems usually appear when we misunderstand that model. Either we allow state to ripple farther than intended, or we place expensive work in locations where it runs too frequently. When

that happens, the app begins to feel heavy. Not because SwiftUI is broken, but because our architecture needs refinement.

In this chapter you moved from guessing to measuring. The SwiftUI instrument showed you which views were updating and how often. The Cause and Effect Graph showed you why those updates were happening. Time Profiler revealed what the CPU was actually doing during each render. The Network instrument exposed repeated downloads and missing cache headers. The Hangs instrument reminded us that responsiveness matters just as much as raw performance.

Each tool answered a different question. Together they gave you clarity.

Most SwiftUI performance issues fall into two categories. Either too many updates are happening, which is architectural, or too much work is happening during each update, which is computational. Once you learn to separate frequency from cost, performance tuning becomes calm instead of reactive. You stop chasing numbers and start recognizing patterns.

Profiling is not something you do at the very end before shipping. It is part of building responsibly. It validates your assumptions. It strengthens your architectural decisions. It builds confidence in your design.

When you measure instead of assume, performance stops feeling mysterious. It becomes something you can reason about. And when you can reason about it, you can improve it intentionally.

That is how polished SwiftUI applications are built.

FIREBASE AND SUPABASE

If I remember correctly, I wrote my first full stack web application back in the late nineties. I had to install Internet Information Services on my own machine and then use classic ASP to connect to a SQL Server database. When ASP.NET came out, I jumped right in. I built applications with Web Forms, then moved to ASP.NET MVC, and honestly had a great time doing it.

Building your own backend teaches you a lot. You learn how authentication works, how data flows, how APIs are designed, and what really happens behind the scenes when a user taps a button in the UI. I have always believed that iOS developers, or any developers for that matter, should spend some time learning backend development. Even if you never plan to build a backend full time, the experience changes how you think about application architecture.

That said, building and maintaining a custom backend is not always realistic. Sometimes you do not have the time. Sometimes you do not have the resources. Sometimes the project simply does not justify the investment. In those cases, backend as a service platforms make a lot of sense.

This is where Firebase and Supabase come in.

Firebase and Supabase stand out as two of the most popular backend as a service platforms used by iOS developers today. They both offer authentication, databases, storage, messaging, notifications, edge functions, and more. They solve a lot of hard problems and they let you focus on building features instead of infrastructure.

In this chapter, I want to talk about the architectural decisions I made while working with these platforms. This is not an introduction to Firebase or Supabase. I am assuming you already know what they are and how to use them at a basic level. The examples in this chapter focus on Firebase, but the same ideas apply to Supabase as well since the platforms are conceptually very similar.

If you are using Supabase, think of Firebase Authentication as Supabase Auth, Firestore as Postgres via PostgREST, and snapshot listeners as Realtime subscriptions. The architectural decisions in this chapter remain the same even though the APIs differ.

FIREBASE APIS IN THE VIEW

One thing Firebase does very well is API design. The APIs are simple, approachable, and easy to use. For example, if you want to implement a registration flow using Firebase Authentication, you can do it with just a few lines of code.

```
try await Auth.auth().createUser(withEmail: email, password: password)
```

That call can be made from anywhere in your app as long as you import FirebaseAuth.

By default, Firebase Authentication lets you create a user using an email and password. That works fine for basic scenarios, but real applications almost always need to store additional information like a first name or last name.

A very common and practical approach is to first create the user using Firebase Authentication, then use the unique identifier generated by Firebase to store additional user data in Firestore. The user id becomes the bridge between authentication and your application data.

Below is an example of a register function implemented directly inside a RegistrationScreen view.

```
private func register() async {
    do {
        let result = try await Auth.auth().createUser(withEmail: email,
password: password)

        let user = User(uuid: result.user.uid, firstName: firstName,
lastName: lastName)

        try firestore.collection("users")
            .document(result.user.uid)
            .setData(from: user)
    } catch {
        message = "Registration failed: \(error.localizedDescription)"
    }
}
```

This is a perfectly reasonable approach.

The view is not making business decisions or enforcing domain rules. It is simply coordinating a short workflow. Create a user, build a model, and save it. In SwiftUI, views are allowed to orchestrate this kind of logic, especially when the flow is simple and tightly coupled to the screen.

At this stage, there is no need to introduce extra layers or abstractions. The intent is clear, the code is readable, and the responsibilities are balanced.

When Business Rules Enter the Picture

Let us look at another example. Suppose a user is adding a new budget and the budget name must be unique for that user. This is an important business rule. It is not presentation logic. It is domain logic.

Here is an implementation of saveBudget inside CreateBudgetScreen.

```
private func saveBudget() {

    guard let currentUser = Auth.auth().currentUser else { return }
    guard let limit = limit else { return }

    let budget = Budget(name: name, limit: limit)

    do {
```

```

        try firestore.collection("users")
            .document(currentUser.uid)
            .collection("budgets")
            .document(name.normalizedDocumentID())
            .setData(from: budget)
    } catch {
        print(error.localizedDescription)
    }
}

```

Notice something important here. The view is not explicitly checking for uniqueness. Instead, it encodes the rule by choosing a document structure that relies on Firestore's upsert behavior. Firestore stores documents in a tree structure:

```
users/{uuid}/budgets/{budget_id}
```

By using the normalized budget name as the document id, Firebase ensures uniqueness. If you try to save two budgets with the same name, Firestore performs an upsert and updates the existing document.

This is simple and effective.

The downside is reuse. If you need to save a budget from multiple screens, you now have to copy and paste this logic. That is usually the point where extracting a store starts to make sense.

ABSTRACTING FIREBASE THROUGH STORES

One of the main reasons to introduce an abstraction is to allow the underlying implementation to change without breaking the rest of your app. This is usually done through Swift protocols.

You could start with a BudgetStoreProtocol, then create BudgetStoreFirestore and BudgetStoreSupabase implementations. If you ever decide to switch from Firebase to Supabase, you swap the implementation and keep the rest of the app unchanged.

Should you always start with an abstraction? No.

I personally do not start with abstractions unless I need them. A protocol with only one concrete implementation does not buy you much. I prefer to let abstractions emerge naturally as the code evolves.

Authentication Store

Authentication is a great place to introduce a store.

AuthenticationStore acts as an interaction layer between SwiftUI views and Firebase Authentication. It handles login, registration, and listening to authentication state changes.

```

@Observable
class AuthenticationStore {

    var user: FirebaseAuth.User?
    private var handle: AuthStateDidChangeListenerHandle?
    private let auth: Auth

    init(auth: Auth = .auth()) {
        self.auth = auth
        handle = auth.addStateDidChangeListener { [weak self] _, user in
            self?.user = user
        }
    }

    func login(email: String, password: String) async throws {
        try await auth.signIn(withEmail: email, password: password)
    }

    func createUser(email: String, password: String) async throws {
        try await auth.createUser(withEmail: email, password: password)
    }

    deinit {
        if let handle {
            auth.removeStateDidChangeListener(handle)
        }
    }
}

```

AuthenticationStore depends on FirebaseAuth, which means we can later swap it with an emulator implementation.

Once created, we inject it into the environment.

```

@main
struct FirebaseDemoProjectApp: App {

    @State private var authenticationStore: AuthenticationStore?

    init() {
        FirebaseApp.configure()
        _authenticationStore = State(initialValue: AuthenticationStore())
    }

    var body: some Scene {
        WindowGroup {

            if let budgetStore {
                NavigationStack {
                    RootView()
                }
                .environment(authenticationStore)
            } else {
                ProgressView("Loading...")
            }
        }
    }
}

```

```

    }
  }
}

struct RootView: View {

    @Environment(AuthenticationStore.self) private var authenticationStore

    var body: some View {
        if authenticationStore.user == nil {
            LoginScreen()
        } else {
            BudgetListScreen()
        }
    }
}

```

The store is optional because Firebase must be configured before it is created. If you try to create it too early, you will get a runtime error.

Once injected, any view can access it through the environment.

Now, you can access `AuthenticationStore` through the `Environment` inside your views.

```

struct LoginScreen: View {

    @State private var email: String = ""
    @State private var password: String = ""

    @Environment(AuthenticationStore.self) private var authenticationStore

    private func login() async {
        do {
            try await authenticationStore.login(email: email, password:
password)
        } catch {
            print(error.localizedDescription)
        }
    }
}

```

Another benefit of having an `AuthenticationStore` is that it gives you a natural home for all authentication related behavior. Instead of spreading login, registration, sign out, and user state logic across multiple views, you centralize it in one place.

This makes reuse effortless. Any screen that needs to authenticate a user can rely on the same set of functions without duplicating code or rethinking the flow. Over time, as your app grows, this becomes even more valuable.

More importantly, `AuthenticationStore` becomes the place where your authentication business rules live. Things like how users sign in, when a session is considered valid, how errors are handled, or what should happen after authentication are all domain concerns, not view concerns. Keeping them inside

AuthenticationStore keeps your views focused on UI and keeps your authentication logic consistent across the entire application.

Budget Store

BudgetStore follows the same idea. Its job is to hide Firestore details and expose simple methods to the UI.

```
@Observable
class BudgetStore {

    private(set) var budgets: [Budget] = []
    var firestore: Firestore
    private var listener: ListenerRegistration?

    init(firestore: Firestore = Firestore.firestore()) {
        self.firestore = firestore
    }

    func createBudget(budget: Budget, for uid: String) throws {

        try firestore.collection("users")
            .document(uid)
            .collection("budgets")
            .document(budget.name.normalizedDocumentID())
            .setData(from: budget)
    }

    func startListening(uid: String) {

        listener?.remove()

        listener = firestore.collection("users")
            .document(uid)
            .collection("budgets")
            .addSnapshotListener { snapshot, error in
                guard let snapshot else { return }

                self.budgets = snapshot.documents.compactMap {
                    try? $0.data(as: Budget.self)
                }
            }
    }

    func stopListening() {
        listener?.remove()
        listener = nil
        budgets = []
    }

    deinit {
        listener?.remove()
    }
}
```

The main purpose of `BudgetStore` is to hide all the details of working with Firestore and expose simple, easy to use operations to SwiftUI views. Views should not care how data is fetched, saved, or kept in sync. They should only care about the data they need and the actions they can perform.

Where you create a `BudgetStore` depends entirely on how it is used. If a store is only needed by a single screen, you can create it as a `@State` property inside that container view. That keeps the scope tight and avoids unnecessary global state.

If the store is shared across multiple screens, it makes more sense to inject it at the root of the application using the environment. This allows any view in the hierarchy to access the same source of truth without having to manually pass it around.

I usually inject my stores at the root view so they are available throughout the entire view hierarchy. For tab based applications, another good option is to inject a specific store at the root of each tab. That way, only the tab and its child views have access to that store, which keeps responsibilities well scoped.

In the example below, I am using the first approach and injecting the store at the root, making it available to the entire view hierarchy.

```
@main
struct FirebaseDemoProjectApp: App {

    @State private var authenticationStore: AuthenticationStore?
    @State private var budgetStore: BudgetStore?

    init() {
        FirebaseApp.configure()
        _authenticationStore = State(initialValue: AuthenticationStore())
        _budgetStore = State(initialValue: BudgetStore(firestore:
Firestore.firestore()))
    }

    var body: some Scene {
        WindowGroup {

            if let budgetStore {
                NavigationStack {
                    RootView()
                }
                .environment(authenticationStore)
                .environment(budgetStore)
            } else {
                ProgressView("Loading...")
            }

        }
    }
}

struct RootView: View {

    @Environment(AuthenticationStore.self) private var authenticationStore
```

```

    var body: some View {
        if authenticationStore.user == nil {
            LoginScreen()
        } else {
            BudgetListScreen()
        }
    }
}

```

Once `BudgetStore` and `AuthenticationStore` are injected into the environment, they become available anywhere in the view hierarchy. You can access them from any view that needs them, without passing references around manually.

That said, it is a good practice to access stores only from container views, also called screens. Screens are responsible for coordinating data and actions. From there, data flows down into smaller, focused subviews through bindings or plain values. This keeps subviews simple, predictable, and easy to reuse.

Below is the implementation of `BudgetListScreen`. This screen is responsible for displaying the list of budgets. It pulls data from `BudgetStore` and reacts to authentication state changes through `AuthenticationStore`, making it a good example of a container view coordinating multiple stores.

```

struct BudgetListScreen: View {

    @Environment(BudgetStore.self) private var budgetStore
    @Environment(AuthenticationStore.self) private var authenticationStore
    @State private var presentAddBudget: Bool = false

    var body: some View {
        List(budgetStore.budgets) { budget in
            Text(budget.name)
        }.task(id: authenticationStore.user?.uid) {

            if let uid = authenticationStore.user?.uid {
                budgetStore.startListening(uid: uid)
            } else {
                budgetStore.stopListening()
            }
        }.toolbar {
            ToolbarItem(placement: .topBarTrailing) {
                Button("Add Budget") {
                    presentAddBudget = true
                }
            }
        }.sheet(isPresented: $presentAddBudget) {
            NavigationStack {
                CreateBudgetScreen()
            }
        }.onDisappear {
            budgetStore.stopListening()
        }
    }
}

```

We are using the `task` modifier with a dependency on the user identifier. This means the task is re-executed whenever the user id changes. As soon as the user is authenticated, the task runs and we attach a listener to that user's budgets.

From that point on, any change to the user's budgets whether a budget is added, updated, or removed will trigger an event from Firestore. We use that event to update `BudgetStore` with the most recent data, keeping the UI automatically in sync with the database.

Firestore also provides APIs for one time data fetching without attaching a listener. You can absolutely use those instead. The tradeoff is that once you move to one time fetches, you are now responsible for keeping the local state in sync with Firestore. With snapshot listeners, Firebase does that work for you, which is often the better choice for data that needs to stay live and up to date.

Inspired by Core Data's `FetchRequest` and SwiftData's `Query`, Firebase also provides the `@FirestoreQuery` property wrapper. It allows you to fetch documents directly inside a SwiftUI view, without manually writing listener or fetch code.

With `@FirestoreQuery`, Firebase handles querying and updates for you, and the view automatically refreshes whenever the underlying data changes. It can be a convenient option for simple screens where you want tight integration between the view and Firestore, similar to how data driven views work with Core Data and SwiftData.

Up to this point, all of the code we have written connects to the real Firebase services and performs actions against the live backend. That can work when you are building everything solo, because you are the only person creating and changing data.

But as soon as you are working with a team, it quickly turns into a hassle.

The biggest issue is that Firebase becomes a shared dependency. Everyone is pointing at the same backend. One developer might add test records, another might update them, and someone else might delete them. Your app might be working perfectly, but your data keeps changing underneath you. That makes debugging harder, testing inconsistent, and previews almost impossible to trust.

A much better approach is to run Firebase locally on your own machine.

Firebase provides an emulator that lets you run services like Authentication and Firestore locally. This gives you isolation. Your data is yours. Your tests are predictable. Your development environment becomes stable again.

In the next section, we will set up the Firebase emulator and configure the app to use it, so you can develop without depending on the live Firebase backend.

RUNNING THE EMULATOR

As your app grows and more people start working on it, relying on the live Firebase backend quickly becomes a problem. Even if everyone is careful, the backend is still shared state. Test users get created, documents are updated, data is deleted, and suddenly things stop behaving the way you expect.

This is where the Firebase Emulator really shines.

The Firebase Emulator lets you run Firebase services locally on your machine. Instead of talking to the cloud, your app talks to a local process. Authentication, Firestore, and other supported services behave just like the real thing, but everything stays completely isolated to your environment.

From a development and architectural standpoint, this is a huge win.

Why the Emulator Matters

Running Firebase locally gives you a few important benefits:

First, isolation. Your data is yours. No one else can accidentally modify it. You can experiment freely without worrying about breaking something for a teammate.

Second, speed. Local operations are fast. There is no network latency, which makes development smoother and previews more responsive when Firebase is involved.

Third, confidence. When something breaks, you know it is your code, not some unexpected data change coming from the cloud.

And finally, realism. Unlike mock data, the emulator behaves like real Firebase. Authentication flows work the same way. Firestore queries behave the same way. You are still testing real behavior, just without the risk.

Setting Up the Firebase Emulator

To use the Firebase Emulator, you first need the Firebase CLI installed on your machine. If you already use Firebase, chances are you already have it.

Once installed, you initialize emulators in your project directory:

```
firebase init emulators
```

During setup, Firebase will ask which services you want to emulate. For most iOS apps, Authentication and Firestore are the usual starting points.

After setup, you can start the emulators with:

```
firebase emulators:start
```

This launches local versions of Firebase services along with a web based dashboard. From there, you can see users, inspect Firestore documents, and watch requests in real time.

Connecting Your App to the Emulator

By default, Firebase SDKs connect to the live backend. To use the emulator, you explicitly tell Firebase to redirect traffic to localhost.

This configuration usually lives in your app startup code, right after `FirebaseApp.configure()`.

For example, connecting Firestore to the emulator:

```
let firestore = Firestore.firestore()
firestore.useEmulator(withHost: "localhost", port: 8080)
```

And for Firebase Authentication:

```
Auth.auth().useEmulator(withHost: "localhost", port: 9099)
```

Once this is in place, all Firebase calls go to your local machine instead of the cloud.

Emulator and Architecture

Architecturally, the emulator fits perfectly with the store based approach we have been using.

Your `AuthenticationStore` and `BudgetStore` do not need to know whether they are talking to the real backend or the emulator. That decision can be made at app startup. The rest of your app stays unchanged.

This is exactly what good architecture looks like. Behavior stays consistent. Only configuration changes.

Emulator vs Previews

It is important to draw a clear line here.

SwiftUI previews should ideally avoid Firebase altogether. That is where preview stores or environment values work best.

The Firebase Emulator is best suited for:

- Local development
- Manual testing
- Integration testing
- Multi screen flows that depend on real Firebase behavior

Previews are about UI. The emulator is about realistic behavior without shared state. They solve different problems, and using both together gives you the best development experience.

If you are serious about building apps with Firebase, running the emulator locally is not optional. It is one of those tools that once you start using it, you wonder how you ever worked without it.

It keeps your environment clean, your data predictable, and your team productive. More importantly, it lets you focus on architecture and behavior instead of constantly fighting shared backend state.

FIREBASE AND PREVIEWS

Firebase introduces some interesting challenges when it comes to SwiftUI previews. The main issue is that as soon as you use any Firebase service such as authentication, Firestore, or storage, Firebase expects the app to be fully configured. If Firebase is not configured correctly, previews will crash before you even see a single pixel.

Even if you do configure Firebase properly, you usually do not want your previews talking to the real, hosted Firebase services. That introduces a network dependency, which immediately makes previews slower and less reliable. On top of that, the data may be changing constantly because other team members or test devices are using the same backend. This makes previews unpredictable and, frankly, not very useful.

Previews are meant to be fast, isolated, and deterministic. You want to open a view and instantly see a stable snapshot of the UI, not wait on a network call or wonder why the data looks different every time you refresh.

Fortunately, there are several ways to approach this problem. In this chapter, we will walk through different techniques for making Firebase work nicely with SwiftUI previews. We will start with simple approaches and gradually move toward cleaner, more scalable solutions that keep preview specific behavior out of your production code while still giving you realistic data to work with during development.

Xcode Running for Previews Flag

Another option is to return hard coded data from `BudgetStore` when the app is running inside Xcode previews. Swift exposes an environment variable called `XCODE_RUNNING_FOR_PREVIEWS` that tells you whether the code is being executed as part of a preview.

You can check this value inside `BudgetStore` and, when running in preview mode, skip all Firebase related work and return in memory data instead. This allows your previews to render instantly without configuring Firebase or relying on any network calls.

The implementation below shows how `BudgetStore` can return a hard coded budget when running in Xcode previews.

[@Observable](#)

```

class BudgetStore {

    private(set) var budgets: [Budget] = []
    var firestore: Firestore
    // other code

    let isPreview: Bool

    init(firestore: Firestore = Firestore.firestore(), isPreview: Bool =
ProcessInfo.processInfo.environment["XCODE_RUNNING_FOR_PREVIEWS"] == "1") {
        self.firestore = firestore
        self.isPreview = isPreview

        if isPreview {
            budgets = [Budget(name: "Colorado Vacation", limit: 500)]
        }
    }
}

```

If you run your Xcode previews now, the screen will display the hard coded “Colorado Vacation” budget. From a functionality point of view, this works and it gets previews up and running quickly.

The downside is that this approach starts to leak preview specific logic into your production code. The `isPreview` flag does not stay limited to the initializer. It slowly finds its way into methods like `startListening`, `stopListening`, and `createBudget`, because those operations should only run when the app is not in preview mode.

At that point, your store is no longer just responsible for managing budgets. It is also carrying around preview related conditionals, which adds noise and makes the code harder to reason about. The implementation below shows how these preview checks begin to spread across the store.

```

func createBudget(budget: Budget, for uid: String) throws {

    if isPreview {
        budgets.append(budget)
        return
    }

    try firestore.collection("users")
        .document(uid)
        .collection("budgets")
        .document(budget.name.normalizedDocumentID())
        .setData(from: budget)
}

func startListening(uid: String) {

    guard !isPreview else { return }

    // rest of the code
}

func stopListening() {

```

```

        guard !isPreview else { return }

        // rest of the code
    }

```

As you can see, `isPreview` slowly starts creeping into almost every method in `BudgetStore`. Even though this approach works, I am not a big fan of it. It adds conditional logic that has nothing to do with the actual responsibility of the store and makes the production code noisier than it needs to be.

A cleaner alternative is to separate concerns entirely by introducing a protocol with two concrete implementations. One implementation is the real `BudgetStore` that talks to Firebase. The other is a `PreviewBudgetStore` that provides in memory, hard coded data for previews.

With this approach, production code stays focused on production behavior and preview code stays where it belongs. Each implementation has a single, clear responsibility. In the next section, we will walk through how to set this up and see why it scales much better as your app grows.

Budget Store Protocol and Facade Pattern

We will start by defining a `BudgetStoreProtocol`. This protocol acts as a contract that describes what a budget store is capable of doing, without caring about how it is implemented.

Both the real `BudgetStore` and the preview specific `PreviewBudgetStore` will conform to this protocol. As long as they honor the same contract, the rest of the app does not need to know or care whether the data is coming from Firebase or from in memory test data.

The implementation of `BudgetStoreProtocol` is shown below.

```

protocol BudgetStoreProtocol {
    var budgets: [Budget] { get }

    func createBudget(budget: Budget, for uid: String) throws
    func startListening(uid: String)
    func stopListening()
}

```

Now both `BudgetStore` and `PreviewBudgetStore` can conform to `BudgetStoreProtocol`. The existing `BudgetStore` does not need to change, since it already provides the required behavior.

The only new piece we need is a preview specific implementation. `PreviewBudgetStore` conforms to the same protocol but returns in memory, hard coded data instead of talking to Firebase. Its implementation is shown below.

```

@Observable
final class PreviewBudgetStore: BudgetStoreProtocol {

    var budgets: [Budget] = [
        Budget(name: "Colorado Vacation", limit: 500)
    ]
}

```

```

    func createBudget(budget: Budget, for uid: String) throws {
        budgets.append(budget)
    }

    func startListening(uid: String) {
        // implementation
    }

    func stopListening() {
        // implementation
    }
}

```

PreviewBudgetStore is a preview specific implementation of BudgetStoreProtocol. The big win here is that we can completely remove preview logic from the real BudgetStore. No more isPreview flags sprinkled across methods. Production code stays focused on production behavior, and preview code stays where it belongs.

That is a solid trade. Your BudgetStore becomes cleaner, easier to reason about, and easier to maintain. All preview behavior is now isolated inside PreviewBudgetStore, where it can evolve independently.

But this introduces a new problem.

So far, BudgetListScreen has been using the store like this:

```

@Environment(BudgetStoreProtocol.self) private var budgetStore

```

Think of the facade as a simple wrapper that has one job. It holds onto something that conforms to BudgetStoreProtocol and exposes the same operations to the rest of the app. The important part is that the facade itself is a concrete type, which means SwiftUI can inject it using @Environment without any issues.

So the protocol gives us flexibility, the preview store gives us clean previews, and the facade bridges the gap so SwiftUI stays happy.

The implementation is shown below.

```

@Observable
class BudgetStoreFacade {

    private let store: BudgetStoreProtocol

    init(_ store: BudgetStoreProtocol) {
        self.store = store
    }

    var budgets: [Budget] { store.budgets }

    func createBudget(budget: Budget, for uid: String) throws {
        try store.createBudget(budget: budget, for: uid)
    }
}

```

```

    }

    func startListening(uid: String) {
        store.startListening(uid: uid)
    }

    func stopListening() {
        store.stopListening()
    }
}

```

Now, we can use `BudgetStoreFacade` in our previews. This is shown below:

```

#Preview {
    NavigationStack {
        BudgetListScreen()
    }
    .environment(AuthenticationStore())
    .environment(BudgetStoreFacade(PreviewBudgetStore()))
}

```

Now our previews can use `PreviewBudgetStore` and return hard coded data without touching Firebase at all. That is a big deal.

Since `PreviewBudgetStore` works entirely in memory, there is no need to call `FirebaseApp.configure()`, no need for a network connection, and no risk of previews pulling real data that keeps changing. Your previews become fast, stable, and predictable, which is exactly what you want when you are working on UI.

The best part is that this keeps production and preview concerns separate. The real `BudgetStore` stays focused on Firebase and real data, while the preview store stays focused on giving you reliable sample data for design and layout work.

Right now, `AuthenticationStore` is still connected to the real Firebase services. That means our previews technically depend on Firebase authentication, which is not ideal.

In the upcoming sections, we will address this as well. We will refactor `AuthenticationStore` so it becomes completely independent from the real Firebase backend when running in previews, just like we did with `BudgetStore`. The goal is the same: fast, reliable previews that do not require network access or external services, while keeping the production code clean and focused on real behavior.

This technique works well, but it comes with one extra moving piece: the facade. We only need that facade because SwiftUI `@Environment` expects a concrete type and does not work with protocols.

So even though the protocol based approach gives us clean separation between production and preview stores, the facade exists mainly to satisfy SwiftUI's environment injection rules.

The good news is that SwiftUI gives us another option.

While `@Environment` does not support protocols, environment values do. With environment values, we can inject a protocol based store directly into the view hierarchy. That means we can drop the facade completely and keep the design even simpler.

In the next section, we will switch from `@Environment(BudgetStore.self)` to an environment value like `@Environment(\.budgetStore)`. This will let us inject either the real Firebase backed store or the preview store without adding an extra wrapper.

Environment Value for BudgetStore

One of the biggest differences between `@Environment` and environment values is flexibility. `@Environment` wants a concrete type. Environment values can hold a protocol, which means you can inject whatever implementation you want as long as it conforms to the contract.

This is exactly what we need.

Instead of forcing SwiftUI to work with a concrete `BudgetStore`, we expose a `BudgetStoreProtocol` as an environment value. Then we can inject the real Firebase backed store in production and inject `PreviewBudgetStore` in Xcode previews, without any facade and without any preview flags leaking into the production store.

We start by adding a new entry in `EnvironmentValues` using the `@Entry` macro:

```
extension EnvironmentValues {
    @Entry var budgetStore: BudgetStoreProtocol = BudgetStore()
}
```

Next, we inject the `PreviewBudgetStore` for Xcode previews. This is shown below:

```
#Preview {
    NavigationStack {
        BudgetListScreen()
    }
    .environment(AuthenticationStore())
    .environment(\.budgetStore, PreviewBudgetStore())
}
```

Now, we can use `budgetStore` in `BudgetListScreen`.

```
struct BudgetListScreen: View {

    @Environment(AuthenticationStore.self) private var authenticationStore
    @Environment(\.budgetStore) private var budgetStore

    // other code
}
```

This makes accessing `BudgetStore` much simpler, both in SwiftUI views and in Xcode previews. We did not have to create a facade. We simply leaned on what SwiftUI already provides and everything falls into place.

And honestly, that is the theme I keep coming back to with SwiftUI. Embrace simplicity. The framework already has a strong opinion about how data should flow. When you work with it instead of forcing extra layers, your code usually ends up cleaner.

Now, there is no single perfect solution here.

Sometimes using an `isPreview` flag is the quickest way to get previews working, especially early on. It is simple and it gets the job done. But as your store grows, those preview checks start spreading into every method and your production code gets noisy.

That is usually the point where the environment value approach starts to shine. It gives you better separation, keeps preview behavior out of production code, and makes it easy to swap implementations without adding `isPreview` conditions all over `BudgetStore`.

Authentication Store

Even though we created a preview friendly version of `BudgetStore`, we are still using the real `AuthenticationStore` in previews. And that creates a problem. If `AuthenticationStore` is talking to Firebase Auth, then previews now depend on Firebase being configured and available. That defeats the whole purpose of previews being fast and predictable.

Naturally, the first idea is to do what we did with budgets. Add an `isPreview` flag and return a fake user when running in previews.

```
@Observable
class AuthenticationStore {

    var user: FirebaseAuth.User?
    private var handle: AuthStateDidChangeListenerHandle?
    private let auth: Auth

    let isPreview: Bool

    init(auth: Auth = .auth(), isPreview: Bool =
ProcessInfo.processInfo.environment["XCODE_RUNNING_FOR_PREVIEWS"] == "1") {
        self.auth = auth
        self.isPreview = isPreview

        if isPreview {
            user = FirebaseAuth.User(/// we cannot create this user)
        }
    }
}
```

And this is where we hit a wall.

Firebase does not let you create a `FirebaseAuth.User` manually. That type is owned by Firebase. You only get an instance of it after a real authentication flow. So unlike budgets, where we can easily create hard coded models, we cannot create a fake Firebase user in code.

So if `AuthenticationStore` exposes `FirebaseAuth.User`, we are stuck. Previews will always need real Firebase Auth.

The fix is simple

Do not expose Firebase types from your store.

Instead, wrap the Firebase user in your own type that represents what your app actually needs. This gives you two huge benefits:

1. Your UI and the rest of your app stop depending on Firebase specific types.
2. You can create fake users in previews whenever you want.

Here is a simple model:

```
struct AuthenticatedUser {  
    let uid: String  
    let email: String  
}
```

I have only added a uid and email to the `AuthenticatedUser` type but you can add additional properties if needed.

Next, we need to update the `AuthenticationStore` to support `AuthenticatedUser` type. This is shown below:

```
@Observable  
class AuthenticationStore {  
  
    var user: AuthenticatedUser?  
    private var handle: AuthStateDidChangeListenerHandle?  
    private let auth: Auth  
  
    let isPreview: Bool  
  
    init(auth: Auth = .auth(), isPreview: Bool =  
        ProcessInfo.processInfo.environment["XCODE_RUNNING_FOR_PREVIEWS"] == "1") {  
        self.auth = auth  
        self.isPreview = true  
  
        if isPreview {  
            user = AuthenticatedUser(uid: "u123", email: "johndoe@gmail.com")  
            return  
        }  
  
        if let current = auth.currentUser {
```

```

        self.user = AuthenticatedUser(uid: current.uid, email:
current.email ?? "")
    } else {
        self.user = nil
    }

    handle = auth.addStateDidChangeListener { [weak self] _, firebaseUser
in

        guard let self else { return }

        if let firebaseUser {
            self.user = AuthenticatedUser(
                uid: firebaseUser.uid,
                email: firebaseUser.email ?? ""
            )
        } else {
            self.user = nil
        }
    }
}

func login(email: String, password: String) async throws {
    guard !isPreview else { return }
    try await auth.signIn(withEmail: email, password: password)
}

func createUser(email: String, password: String) async throws {
    guard !isPreview else { return }
    try await auth.createUser(withEmail: email, password: password)
}

deinit {
    if let handle {
        auth.removeStateDidChangeListener(handle)
    }
}
}

```

Now previews can have a user without doing anything with Firebase Auth.

Is using `XCODE_RUNNING_FOR_PREVIEWS` here okay?

It is a tradeoff, just like `BudgetStore`. You can always start with `isPreview` approach but if later you notice that `isPreview` checks are showing up everywhere inside `AuthenticationStore`, then it is a sign you should move to the same approach we used for budgets. Create a protocol, provide a `PreviewAuthenticationStore`, and inject it using an environment value.

But for now, this approach keeps things simple, and it keeps your previews moving fast.

CONCLUSION

Firebase and Supabase are powerful tools, but the real value they bring to your app depends on how you integrate them into your architecture.

Throughout this chapter, the goal was not to advocate for a specific backend as a service, nor to prescribe a single “correct” layering strategy. Instead, the focus was on understanding *where* backend concerns belong and *when* it makes sense to introduce abstractions. Used thoughtfully, Firebase and Supabase can dramatically reduce the cost of building and maintaining infrastructure, while still allowing your application architecture to remain clean, testable, and adaptable.

A few themes should stand out.

First, views are allowed to orchestrate simple workflows. Creating a user, saving a document, or triggering a short sequence of actions from a screen is perfectly acceptable when the logic is simple and tightly coupled to that UI. SwiftUI does not require you to push everything into view models or services on day one. Clarity and intent matter more than rigid layering.

Second, business rules deserve a stable home. As soon as rules emerge that must be reused, enforced consistently, or evolve over time, stores become a natural boundary. `AuthenticationStore` and `BudgetStore` are not about “following a pattern,” they exist to protect your domain logic from being scattered across views and duplicated in subtle, fragile ways.

Third, abstractions should emerge, not be forced. Starting with concrete implementations keeps your code easy to read and reason about. Protocols, facades, and environment values become valuable once you have a real reason for flexibility—previews, multiple backends, or test isolation—not simply because architecture diagrams say they should exist.

Fourth, local development matters. The Firebase Emulator is not a luxury. It is a practical tool that restores isolation, predictability, and confidence when working on a team. Combined with store-based architecture, it allows you to change configuration without changing behavior, which is exactly what good architecture aims to achieve.

Finally, SwiftUI previews are a design tool, not an integration environment. Previews should be fast, deterministic, and independent of networked services. Whether you use preview flags, protocol-based stores, or environment values, the guiding principle is the same: preview behavior should not leak into production code. When it does, that is your signal to step back and reconsider the boundary.

Firebase and Supabase solve hard infrastructure problems. SwiftUI solves UI composition and state propagation. Your architecture sits in between, translating one world into the other. When you respect the responsibilities of each layer and let solutions grow organically, you end up with code that is simpler, more flexible, and far easier to maintain over time.

In the end, good architecture is not about adding layers. It is about removing friction—between your UI and your data, between your team and the backend, and between the code you write today and the changes you will inevitably make tomorrow.

ARCHITECTURE IN THE AGE OF AI

There has been a great deal of noise lately around the idea that AI will replace programmers and that software developers will no longer be needed in the near future. Headlines make bold claims. Social media amplifies them. Venture capital backed companies promote them aggressively. When billions of dollars are invested in a technology, dramatic predictions naturally become part of the narrative. They capture attention. They attract funding. They create urgency.

But if we step back and look at the history of software development, every major leap in abstraction has sparked similar anxiety. When higher level languages replaced assembly, development became faster and more accessible. When frameworks matured, they eliminated enormous amounts of boilerplate. When visual tools emerged, they promised to simplify application development.

None of these shifts eliminated developers. They changed what developers focused on.

Instead of managing registers and memory directly, developers focused on business logic. Instead of wiring every UI component manually, they focused on user experience. Instead of building infrastructure from scratch, they focused on integration, scalability, and architecture.

AI represents another shift in abstraction. To understand its impact, we must look at it through that lens.

AI AS AN ABSTRACTION LAYER

Early programmers operated close to the hardware. Assembly required explicit management of memory and registers. High level languages abstracted away hardware complexity and allowed developers to express intent more clearly. Frameworks abstracted repetitive patterns. SwiftUI abstracted imperative UI construction into declarative data driven views.

Each abstraction removed friction and allowed developers to operate at a higher level.

AI is the next abstraction layer.

Instead of abstracting hardware or framework mechanics, AI abstracts implementation detail. Developers can describe what they want rather than explicitly writing how it should be constructed. A networking layer, a SwiftUI screen, a store implementation, or even a test suite can be generated in seconds. The interaction becomes conversational rather than mechanical.

But every abstraction changes where responsibility lives.

When hardware concerns were abstracted, developers focused on logic. When UIKit was abstracted by SwiftUI, developers focused on data flow and state ownership. Now that AI abstracts syntax and boilerplate, developers must focus more heavily on system boundaries, domain correctness, and architectural intent.

Abstractions do not eliminate responsibility. They relocate it.

When you operate at a higher layer, the mistakes you make are also higher level. A memory bug might crash a function. A poorly defined architectural boundary can destabilize an entire system. Similarly, when AI generates code rapidly, the primary risk is not syntax errors. It is architectural drift and domain misalignment.

AI allows us to move faster. But speed without clarity magnifies structural weaknesses. If the underlying design is thoughtful, AI becomes an accelerator. If the design is vague or inconsistent, AI amplifies that vagueness.

The real question is not whether AI writes code. It clearly does. The real question is whether developers are prepared to operate effectively at this new level of abstraction.

VIBE CODING

The term “vibe coding” was popularized by Andrej Karpathy in early 2025. He described a new style of development where you give in to the flow of AI assisted coding, describe what you want, accept the changes, run the application, and move forward. Error messages are pasted back into the model. Diffs are barely reviewed. The process feels conversational rather than mechanical.

I have personally vibe coded four of the five apps currently published under my name on the App Store. Each one took less than twenty four hours to build. If you asked me detailed questions about specific implementation details, I would not be able to explain every line confidently. The code produced by AI feels foreign in places. Yet the applications work. They pass review. They generate modest revenue.

For small, focused applications, vibe coding is incredibly powerful. It allows rapid validation of ideas. You can build a proof of concept in hours, deploy it, and see whether it gains traction. For experimentation and side projects, the reduction in friction is transformative.

However, this model does not scale cleanly into every context.

Enterprise applications operate under very different constraints. They integrate with legacy systems, comply with regulations, process sensitive data, and evolve over years rather than weekends. In that environment, code comprehension is not optional. It is foundational.

Vibe coding assumes that partial understanding is acceptable because the blast radius is small. In enterprise systems, the blast radius is large. A subtle misunderstanding in business logic can affect thousands or millions of users. A regulatory violation can have legal consequences. A miscalculated financial rule can cause real economic harm.

The difference is not about AI capability. It is about context and responsibility.

ENTERPRISE APPLICATIONS

Throughout my career, I have worked in domains including oil and gas, retail, finance, retirement systems, and healthcare. In each case, I learned the same lesson: writing code is rarely the hardest part. Understanding the business domain deeply enough to represent it correctly is where the real difficulty lies.

Enterprise complexity does not come from syntax or frameworks. It comes from rules, regulations, policies, edge cases, and evolving interpretations. When a user submits a medical insurance claim, eligibility must be verified, coverage limits evaluated, regulatory constraints honored, and policy exceptions applied. Once those rules are clearly understood, expressing them in code is straightforward. The real challenge is interpreting them correctly.

The same applies in financial systems. If a customer exceeds a credit limit, how is the penalty calculated? Does it trigger a fee? Does it revoke promotional rates? Does it alter risk classification? These are domain questions that must be answered before a single line of code is written.

Even something that appears simple, such as applying multiple coupons in an online store, can become nuanced. Are discounts cumulative or mutually exclusive? Does order matter? Are there time based or regional constraints? These decisions reflect business strategy, not programming mechanics.

AI can generate plausible implementations of these rules. But it does not validate the meaning behind them.

If a specification is ambiguous, incomplete, or contradictory, AI will still produce structured and syntactically correct code. However, syntactic correctness is not the same as semantic correctness.

Business requirements evolve. They contain hidden assumptions. They conflict with other policies. If flawed rules are provided as input, AI will implement those flaws consistently and at scale. The resulting system may look correct and may even pass generated tests, yet still violate the true intent of the business.

Tests do not eliminate this risk. If both the implementation and the tests are derived from the same misunderstanding, they will agree with each other. Consistency does not guarantee correctness.

In enterprise systems, correctness is about representing meaning accurately. That requires judgment, interpretation, and sometimes pushback. It requires someone to ask whether a rule makes sense, whether an edge case has been considered, and whether a policy aligns with regulatory constraints.

AI accelerates implementation. It does not replace understanding.

ARCHITECTURE MATTERS MORE IN THE AGE OF AI

This brings us back to architecture.

If AI can generate code quickly and efficiently, the bottleneck in software development shifts. It no longer lives primarily in implementation speed. It lives in design clarity.

Architecture defines boundaries. It defines ownership. It defines how data flows through a system. It determines which components are responsible for business rules and which are responsible for presentation. When those boundaries are unclear, AI will happily generate code that reinforces the confusion.

In the past, poor architecture revealed itself slowly. Features became harder to implement. Bugs multiplied. Refactoring became painful. With AI accelerating implementation, poorly designed systems can become messy at a much faster pace. AI amplifies whatever structure already exists.

If the structure is clean, AI multiplies productivity. If the structure is chaotic, AI multiplies the chaos.

When code generation becomes cheap, design decisions become expensive.

A well architected SwiftUI application with clear data flow, well defined stores, stateless services, and explicit ownership provides guardrails. AI can operate within those boundaries and generate predictable, isolated pieces of logic that fit naturally into the system.

Without that structure, AI will produce code that appears correct in isolation but destabilizes the system as a whole.

The future of iOS development will not be defined by how quickly we can generate code. It will be defined by how clearly we can design systems. AI removes friction, but it does not remove responsibility. It gives us speed, but it does not give us judgment.

As developers, our role is not shrinking. It is moving upward.

The more powerful our tools become, the more important architectural clarity becomes. In the age of AI, thoughtful system design is not optional. It is the foundation.

CONCLUSION

Artificial intelligence is not signaling the end of software development, nor is it rendering architectural thinking obsolete; rather, it represents the next meaningful shift in abstraction that changes how we interact with our tools and where we concentrate our effort. By dramatically reducing the friction of implementation, AI allows us to generate structures, scaffolding, and even entire features at a speed that would have seemed unrealistic only a few years ago. However, as with every major abstraction in the history of computing, increased speed and convenience come with a corresponding shift in responsibility. When we operate at a higher level, the consequences of our design decisions also become higher level, and the cost of unclear thinking becomes more significant.

In smaller, self contained applications, partial understanding may be acceptable because the blast radius is limited and the risks are manageable. In enterprise systems, where software intersects with financial transactions, medical records, regulatory requirements, and long term business strategy, partial

understanding is not a luxury we can afford. Business rules must still be interpreted carefully, edge cases must still be examined deliberately, and architectural boundaries must still be defined with precision. AI can generate plausible and syntactically correct code, but it cannot assume accountability for whether that code truly reflects the intent, constraints, and nuances of the domain it attempts to model.

As implementation becomes cheaper and faster, design clarity becomes more valuable. When the mechanical effort of writing code decreases, the intellectual effort of shaping systems increases in importance. Developers spend less time constructing boilerplate and more time defining ownership, validating business intent, and ensuring that data flows through well defined boundaries. The role of the developer does not shrink in this environment; it elevates. We move upward in the stack, focusing less on syntax and more on structure, less on mechanics and more on meaning.

AI acts as an amplifier. It amplifies clarity when clarity already exists, and it amplifies confusion when system boundaries are poorly defined. A well architected SwiftUI application with intentional state management and explicit ownership allows AI to accelerate productivity safely. A loosely structured system without clear responsibility invites AI to generate code that appears correct in isolation but destabilizes the system as a whole.

The defining challenge of this era is not whether AI can write code, because that question has already been answered. The real challenge is whether we are deliberate about the systems that contain that code and accountable for the decisions that shape it. Architecture has always been about making complexity manageable and change sustainable. In the age of AI, it becomes about ensuring that acceleration does not outpace understanding and that speed does not undermine stability.

The tools are becoming more powerful, but the need for judgment, clarity, and intentional design remains constant. In fact, it becomes more critical. When we embrace AI as an abstraction layer while maintaining ownership of our architectural decisions, we transform acceleration into leverage rather than fragility. That is why, in this new landscape, architecture does not diminish in importance. It becomes the foundation that allows progress to remain sustainable.

CONCLUSION

When SwiftUI was first introduced, many of us tried to fit it into architectural patterns that had served us well in the UIKit era. We reached for MVVM because it felt familiar. We added layers because layering felt safe. We recreated abstractions that had once solved real problems in imperative UI frameworks. Over time, however, it became clear that SwiftUI was not designed to be UIKit with different syntax. It introduced a fundamentally different mental model, one centered around state, declarative rendering, and frequent body reevaluation.

Throughout this book, we examined that mental model carefully. We explored where traditional patterns like MVVM align with SwiftUI and where they introduce unnecessary complexity. We looked at container and presenter approaches that better separate responsibilities without fighting the framework. We analyzed communication strategies inside observable stores and how to prevent state from rippling farther than intended. We examined navigation patterns, environment usage, validation strategies, SwiftData architecture, and integration with backend services such as Firebase and Supabase. Each topic was approached with the same question: does this design align with how SwiftUI actually behaves?

One of the recurring themes across these chapters is that SwiftUI rewards clarity. Views are not long living UI objects waiting to be mutated. They are lightweight descriptions of state at a moment in time. When state changes, SwiftUI reevaluates body and reconciles differences efficiently. This behavior is not a flaw. It is the core design. Performance issues begin not because SwiftUI runs body often, but because we place too much work inside body or allow state changes to propagate farther than necessary.

The distinction between architectural cost and computational cost is one of the most important lessons in this book. Sometimes performance problems arise because too many updates are happening. Other times they arise because each update is too expensive. Profiling tools such as the SwiftUI instrument, Time Profiler, Network profiling, and the Hangs instrument allow us to separate frequency from cost. Once you understand that distinction, performance tuning becomes deliberate rather than reactive. You stop guessing. You start measuring.

Testing follows a similar philosophy. Instead of designing architecture purely to make views testable, we focus on isolating behavior in stores and services where it naturally belongs. SwiftUI views remain declarative and focused on presentation, while logic becomes easier to verify in isolation. The goal is not to satisfy a pattern. The goal is to build confidence in behavior.

The same mindset applies to navigation, global sheets, validation, persistence, and backend integration. When architecture aligns with SwiftUI's declarative nature, complexity becomes manageable. When responsibilities are clearly separated and state is intentionally scoped, large applications remain understandable. SwiftUI does not remove complexity from software development, but it encourages us to structure that complexity around data flow rather than view mutation.

This book was never about promoting a single rigid pattern. It was about developing architectural judgment. Patterns are tools, not rules. They exist to solve specific problems. Understanding why a pattern works, when it helps, and when it adds friction is far more important than following it mechanically. SwiftUI continues to evolve, and new APIs, macros, and abstractions will emerge. What will remain stable are the underlying principles: understand state, control data flow, keep heavy work off the main thread, measure instead of assuming, and favor clarity over ceremony.

Ultimately, building polished SwiftUI applications is not about clever tricks or minimal lines of code. It is about thoughtful design decisions made with a clear understanding of how the framework operates. When you align architecture with SwiftUI's strengths instead of resisting them, your applications become more predictable, more testable, and more maintainable.

If there is one lasting takeaway, it is this: design with intention. Do not build around what feels familiar. Build around what the framework actually does. Measure behavior. Refine architecture. Keep your views simple. Let state drive the interface.

SwiftUI is powerful, but it rewards discipline. With the tools and principles covered in this book, you now have a structured way to approach architecture in a declarative world.

Now it is your turn to apply it.